Embedded C

Embedded C

From C fundamentals to registers, interrupts and drivers on the ATmega328P

Erik Pihl

QRTECH ACADEMY
20 October 2026

Embedded C
Copyright © 2026 QRTECH Academy.

The text and figures of this book, and of the course it is typeset from, are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). You may copy and redistribute them, and adapt them, for any non-commercial purpose, provided you give appropriate credit and share what you make under the same license. To view a copy of the license, visit `https://creativecommons.org/licenses/by-nc-sa/4.0/`. Third-party material shown in the figures, the screenshots of the Arduino IDE, Microchip Studio and Windows, and the board and component artwork in the circuit diagrams, remains subject to its owners' terms.

The code in this book may also be used under the MIT License, which is the license of all the code in the companion repository. The repository holds the lecture material, the worked demos and the simulator guide:

`https://github.com/qrtech-academy/embedded-c`

The C in this book is C11, compiled with `avr-gcc` for the ATmega328P on an Arduino Uno or Nano. Arduino, AVR, ATmega328P and Microchip Studio are trademarks of their respective owners.

Set in TeX Gyre Pagella and DejaVu Sans Mono with LuaLaTeX.

Typeset from the ten lectures of the course, their exercises, the simulator guide, and the two self-assessment papers. This edition: 20 October 2026.

Contents

Preface **ix**

1 C Fundamentals **1**
1.1 C Fundamentals for Experienced Programmers . . . 2
1.2 Review . . . 20
1.3 Exercises . . . 20

2 Hardware-Near Programming in C **23**
2.1 Registers and I/O . . . 24
2.2 Arduino via Microchip Studio . . . 32
2.3 Building and Flashing on Linux . . . 51
2.4 Review . . . 55
2.5 Exercises . . . 56

3 Interrupts **59**
3.1 Interrupts . . . 59
3.2 Review . . . 73
3.3 Exercises . . . 74

4 Analog-to-Digital Conversion **79**
4.1 Analog-to-Digital Conversion (ADC) . . . 79
4.2 Review . . . 89
4.3 Exercises . . . 90

5 Timers **93**
5.1 Timers . . . 93
5.2 Review . . . 101
5.3 Exercises . . . 102

6 Serial Communication (UART) **105**
6.1 Serial Communication (UART) . . . 105
6.2 Review . . . 124

6.3 Exercises . . . 125

7 EEPROM 129
7.1 EEPROM . . . 129
7.2 Review . . . 140
7.3 Exercises . . . 140

8 The Watchdog Timer 143
8.1 The Watchdog Timer . . . 143
8.2 Review . . . 155
8.3 Exercises . . . 156

9 Structs as Drivers 159
9.1 Structs as Drivers . . . 160
9.2 Review . . . 178
9.3 Exercises . . . 178

10 Encapsulation and Polymorphism in C 185
10.1 Encapsulation and Polymorphism in C . . . 186
10.2 Review . . . 244
10.3 Exercises . . . 245

A Running the Demos Without Hardware 247
A.1 What You Get, and What You Don't . . . 247
A.2 Install . . . 248
A.3 Build the Firmware . . . 248
A.4 Quick Path: Just Run It . . . 248
A.5 Debugging With avr-gdb . . . 249
A.6 Full Path: The Simulation Harness . . . 250
A.7 Demo Reference . . . 256
A.8 Gotchas . . . 258
A.9 Limits . . . 259

B Answers to the Exercises 261
Chapter 1: C Fundamentals . . . 261
Chapter 2: Hardware-Near Programming in C . . . 262
Chapter 3: Interrupts . . . 262
Chapter 4: Analog-to-Digital Conversion . . . 264
Chapter 5: Timers . . . 266
Chapter 6: Serial Communication (UART) . . . 268
Chapter 7: EEPROM . . . 268
Chapter 8: The Watchdog Timer . . . 269

Chapter 9: Structs as Drivers 270
Chapter 10: Encapsulation and Polymorphism in C 272

C The Self-Assessment Papers **275**

D Self-Assessment Paper A **279**
Register Reference 290

E Self-Assessment Paper B **295**
Register Reference 301

F Model Answers to Paper A **305**
Question 1: The language underneath 305
Question 2: Registers, and a program that does nothing 308
Question 3: An interrupt that never fires 310
Question 4: Converting and counting 312
Question 5: Talking to a PC 315
Question 6: Remembering, and recovering 318
Question 7: One driver, many instances 322
Question 8: Hiding things, and choosing at runtime 324

G Model Answers to Paper B **327**
Question 1: Saying what you mean 327
Question 2: Straight to the registers 331
Question 3: Reacting instead of polling 335
Question 4: Reading the world, and keeping time 338
Question 5: A channel to the host 341
Question 6: Surviving a power cut, and a hang 345
Question 7: A driver that scales 349
Question 8: Hiding the implementation 354

Preface

This book is for junior embedded developers, and for developers with some programming experience who want to learn embedded systems programming. It teaches embedded development in C on one real target, the ATmega328P, the microcontroller on the Arduino Uno and Nano, and it goes from the C language itself, through registers, interrupts and interrupt-driven peripheral drivers, to hand-written encapsulation and polymorphism.

The topics, in the order the book takes them:

- A C fundamentals refresher: structs, pointers, function pointers, dynamic memory, macros.
- Register-level I/O: bitwise operations, `DDRx`/`PORTx`/`PINx`.
- Interrupts: external interrupts and pin change interrupts.
- Analog-to-digital conversion (ADC).
- Timers: prescalers, Normal and CTC modes, non-blocking timing.
- Serial communication (UART).
- EEPROM: persisting state across resets.
- The watchdog timer.
- Structs as drivers: bundling state for peripherals with multiple instances.
- Encapsulation and polymorphism in C: opaque structs, function pointers in structs, and hand-written vtables.

Unlike a general C text, every chapter builds toward that one target, and every worked example is a real program, compiled and flashed to actual hardware rather than simulated. The book ends where *Modern Embedded C++*, a separate follow-on course and book, begins.

Along the way you will build circuits and write programs for them, not just read about them. You will:

- Wire and program real circuits (LEDs, buttons, potentiometers) on an Arduino Uno or Nano.
- Write drivers directly against the ATmega328P's registers and datasheet.
- Move from polling to interrupt-driven, non-blocking designs.
- Bundle a peripheral's state into a struct to support multiple instances.

- Refactor a driver from a plain struct into an opaque type, then into a hand-written interface with multiple implementations.

The examples and exercises stay close to typical embedded problems:

- Digital I/O and debounced button input.
- Timer-driven, non-blocking LED patterns.
- Serial reporting of sensor readings.
- Persisting state to EEPROM across resets.
- A GPIO driver evolving from a public struct, to an opaque type, to a vtable-based interface.

What you should be able to do at the end

Having worked through the book, you should be able to:

- Write and debug C code directly against a microcontroller's registers and datasheet.
- Move from blocking, polling-based designs to interrupt-driven, non-blocking ones.
- Keep interrupt handlers safe by flagging events in the ISR and handling them in `main()`.
- Bundle a peripheral's state into a struct to support several instances of the same peripheral.
- Split a driver across a header and source file, and use opacity to protect its invariants.
- Store and call through function pointers, including as struct members.
- Understand, and hand-implement, the vtable mechanism C++ uses for virtual dispatch.
- Move directly on to *Modern Embedded C++*.

What you are assumed to know

You are expected to be able to program already, in C or in another language. Chapter 1 is a fast-moving refresher on C for programmers, not a first course in programming: it covers the syntax quickly and slows down only where C differs from the languages you may know, above all at pointers. Nothing about embedded systems or electronics is assumed beyond knowing what an LED, a button and a resistor are. Everything else, from what a register is onwards, is taught from the beginning.

How the book is organised

The book is typeset from a ten-lecture course, and each chapter is one lecture. Its sections are the lecture's appendices, with the same numbered subsections, so a chapter here and a lecture directory in the repository can be read side by side: the lecture's Appendix A.7 is a chapter's first section, subsection 7.

Chapter	Lecture	Topic
1	`L01`	C fundamentals refresher
2	`L02`	Registers and I/O
3	`L03`	Interrupts
4	`L04`	Analog-to-digital conversion (ADC)
5	`L05`	Timers
6	`L06`	Serial communication (UART)
7	`L07`	EEPROM
8	`L08`	The watchdog timer
9	`L09`	Structs as drivers
10	`L10`	Encapsulation and polymorphism in C

Chapters 1 to 6 are the introductory arc: the language, digital I/O, reacting to events, reading the analog world, timing without blocking, and reporting back to a PC. Chapters 7 and 8 complete the survey of peripherals with the two the chip has exactly one of, and Chapters 9 and 10 are about driver design, built on everything before them.

Every chapter opens with what is in it, works through the material, closes with a review of what you should now be able to explain, and ends with its exercises. The appendices hold the guide to running the demos in a simulator, the answers to the exercises that have one right answer, and the two written papers described below.

What the exercises look like

The exercises come in **sets**, grouped the way the lecture groups them: questions about the concepts first, then programs. Every exercise is labelled with its kind:

Kind	What it asks of you
REFLECTION	Answer in prose: explain what the code does, why it is written the way it is, or why one design is better than another.
CALCULATION	Work a number out on paper: a voltage, a tick period, a register value.
CODE	Write a program, build it, and run it on the board, or in the simulator where there is no board.

The Reflection and Calculation exercises are answered in Appendix B, chapter by chapter, calculations with their working. **The Code exercises have no printed solution.** Each is a program that either does what the exercise describes on the board or does not, and the board is how you check it. Some Code exercises also ask a question about the design they make you write; the appendix answers those questions, but not with code. Try each exercise before you read its answer.

Two written papers, and what they are not

Nothing in the course this book comes from is marked. Assessment is the exercise set after every chapter, and the questions in every chapter's review.

Appendices C to G hold two three-hour papers with worked solutions, and they check something else: **what you can reconstruct on paper, with nothing in front of you.** Eight questions each, mixing theory with C you either read and repair or write from scratch, and no compiler, datasheet or board in the room to tell you which it is.

They exist purely so that you can test your own knowledge after working through the book. They gate nothing, they are not a qualification, and no part of the book requires them.

Take one after the last chapter. Both papers draw on all ten chapters, so sitting one partway through examines material you have not read yet.

What you need

Every worked example in this book runs on one board, and you need one to follow it the way it is written:

- **An Arduino Uno or Nano.** Either works: the same ATmega328P, the same registers, the same pin numbering. §2.1.2 compares the two.
- **A breadboard and a handful of parts**: LEDs with 220–470 Ω series resistors, push buttons with 10 kΩ pull-down resistors, a potentiometer for Chapter 4, and jumper wires. Chapter 6's last exercise adds a TMP36 temperature sensor.

And the software to build and flash programs for it, on either of two paths:

- **On Windows, Microchip Studio**, the IDE for writing and building the examples, with the Arduino IDE used once to borrow the command that flashes the board. §2.2 walks through it.
- **On Linux, or on Windows through WSL**, the AVR toolchain itself: `avr-gcc`, `avr-objcopy` and `avrdude`, used from the command line with a Makefile. §2.3 walks through it.

The companion repository holds the lectures, the worked demos and everything the book is typeset from. Clone it:

```
git clone https://github.com/qrtech-academy/embedded-c.git
```

Two commands at its root cover everything the book asks of it on Linux:

```
make build               # Build every demo, one project directory at a time.
make format-check        # Check the formatting of every C file.
```

Each demo directory also builds on its own, and flashes the board with `make flash`; Chapter 6 shows the layout they all share.

No hardware yet?

Every worked demo can also be run on your own computer, under the simavr simulator, including its serial output, button presses, EEPROM that survives a restart, and debugging with `avr-gdb`. Appendix A shows how. It is a way to keep moving until you have a board, not a replacement for one: a simulator will not teach you what a floating input, a missing pull-up or a badly seated jumper wire feels like. Get a board when you can.

CHAPTER 1

C Fundamentals

Enough C that from the next chapter on, only the hardware is new.

IN THIS CHAPTER

- Program structure, compiling, and data types: a fast pass over what's the same as languages you already know.
- Structs, enums, and typedefs: grouping data, naming states, and the `self`-pointer convention used throughout this book.
- Functions, `static`, and header guards: how a `.c`/`.h` pair is organized in this codebase.
- Pointers: out-parameters, arrays/strings, and the pointer-to-pointer idiom for replacing a caller's pointer.
- Dynamic memory allocation (`malloc`/`realloc`/`free`), and why embedded C mostly avoids it.
- Worked example: pulling all of the above together in one small program.

This chapter is a refresher on C for programmers who already know another language. Its aim is that you are comfortable enough with C's core syntax (types, structs, functions, pointers) not to be fighting the language once hardware programming starts in the next chapter. After it, you should be able to define and use a `struct`, an `enum` and a `typedef`, and explain why `self` is passed as a pointer; write a `.c`/`.h` pair with a correct header guard and a sensible static/public ordering; tell when a function parameter needs a single pointer (`int*`) and when a double pointer (`int**`); and use `malloc`/`realloc`/`free` correctly, including why a pointer should be nulled immediately after `free`.

Read §1.1 first: it is the chapter's core material, and its pointer subsection is the one to read slowly. Then work through the exercises in §1.3. They are programs for your own computer, not yet for the board, and all of them are Code exercises, with no printed solution; the questions some of them ask about the design they make you write are answered in Appendix B.

1.1 C Fundamentals for Experienced Programmers

A fast-moving refresher on C syntax for programmers who already know how to program in another language; not a full C course, but enough that later chapters can assume you're not fighting the language while learning hardware concepts. If a section looks obvious, skip ahead; the pointer section (§1.1.7) is the one worth actually reading.

1.1.1 Compiling and program shape

- C is compiled, not interpreted: source code → assembly → machine code. No runtime interpretation overhead, which is most of why it's the default choice for embedded systems.
- C is procedural: everything lives in functions; `main()` is the entry point.
- No safety net: no bounds checking, no garbage collection. The language trusts you.

A `Hello, world!` program is shown below:

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

- `#include <stdio.h>` pulls in declarations (a header), not an implementation; that's resolved at link time.
- `return 0` from `main()` signals success to the OS. C99+ makes it implicit if omitted, but write it anyway.
- A Doxygen comment above a function, written in the imperative mood and documenting its contract, is conventional throughout this codebase. For example, a function that computes voltage by Ohm's law can be documented as shown below:

```c
/**
 * @brief Compute voltage from resistance and current.
 *
 * @param[in] resistance_kohm Resistance in kOhm.
 * @param[in] current_ma Current in mA.
 *
 * @return Voltage in V.
 */
double compute_voltage(double resistance_kohm, double current_ma);
```

1.1.2 Data types & naming

Type	Meaning	Typical size
`void`	nothing (no return value)	–
`int`	signed integer	2–4 bytes
`float`	single-precision float	4 bytes
`double`	double-precision float (default for float literals)	8 bytes (4 with avr-gcc)
`char`	one character	1 byte
`char[]` / `char*`	string (array of `char`, NUL-terminated)	–
`bool` (`<stdbool.h>`)	`true` / `false`	1 byte

- Prefer `const` for anything that shouldn't change after initialization, following the same intent as `final`/`readonly` in other languages. It also lets the compiler catch mistakes and optimize more aggressively.

An exception is function declarations: do not qualify arguments passed by value with `const`, as it has no effect on the function interface and only reduces readability. In the corresponding function definition, however, pass-by-value arguments should be declared `const` for the reasons above.

- Naming convention: `snake_case` for variables/functions/constants, `UPPER_CASE` for macros and enum constants. Not camelCase/PascalCase; that's a Java/C#/Pascal/C++ convention, not a C one.
- **Integer division truncates**: `5 / 2 == 2`, not `2.5`. Force float division by making at least one operand a `double`/`float`, or cast: `(double)(a) / b`.
- `%` is modulus, integers only; using it on a float is a compiler error.

1.1.3 Structs, enums, and typedefs

Structs

- A `struct` groups related fields under one name, each accessed with `.` (or `->` if you're going through a pointer). No methods, no inheritance, no access control; just a memory layout, laid out in declaration order (the compiler may insert padding between fields for alignment).
- Compare to a plain data class/record in other languages, minus the behavior.
- The idiom used throughout this codebase for functions that operate on a struct: take a pointer to it as the first parameter, named `self`. It's the closest C gets to a method's implicit `this`, without the language actually having objects:

```
/**
 * @brief Electrical measurement structure.
 */
```

```c
struct measurement
{
    /** Voltage in V. */
    double voltage_v;

    /** Current in mA. */
    double current_ma;

    /** Resistance in kOhm. */
    double resistance_kohm;
};

/**
 * @brief Store a voltage/current reading and derive its resistance (Ohm's law).
 *
 * @param[out] self Pointer to the measurement to populate.
 * @param[in] voltage_v Voltage in V.
 * @param[in] current_ma Current in mA.
 */
static void measurement_init(struct measurement* self, const double voltage_v,
                             const double current_ma)
{
    self->voltage_v       = voltage_v;
    self->current_ma      = current_ma;
    self->resistance_kohm = voltage_v / current_ma;
}

struct measurement m;
measurement_init(&m, 5.0, 0.5);
printf("%g V\n", m.voltage_v);
```

- Passing a `struct` by value copies every field, so `self` is (almost) always a pointer:
  - A `const struct measurement*` when the function only reads it.
  - A plain `struct measurement*` when it writes through it, same as any other out-parameter (§1.1.7).

typedef

- `typedef` gives an existing type a new name, most often to avoid typing `struct measurement` everywhere. The convention in this codebase is a `_t` suffix on an anonymous struct:

```c
/**
 * @brief Electrical measurement structure.
```

```
 */
typedef struct
{
    /** Voltage in V. */
    double voltage_v;

    /** Current in mA. */
    double current_ma;

    /** Resistance in kOhm. */
    double resistance_kohm;
} measurement_t;

// Create measurement instance.
measurement_t m = { .voltage_v = 5.0, .current_ma = 0.5, .resistance_kohm = 10.0 ↩
    ↪ };
```

- `measurement_t` behaves exactly like `struct measurement` above; the `typedef` is purely a naming convenience, it doesn't change layout or behavior.

enum

- An `enum` names a set of related integer constants, so a `switch` reads as intent rather than magic numbers, and some compilers can warn you when a `switch` doesn't handle every named case:

```
/**
 * @brief Enumeration of sensor states.
 */
typedef enum
{
    SENSOR_STATE_IDLE,     ///< Sensor is idle.
    SENSOR_STATE_SAMPLING, ///< Sensor is sampling.
    SENSOR_STATE_ERROR     ///< Sensor is in an error state.
} sensor_state_t;

// Create variable holding the sensor state.
sensor_state_t state = SENSOR_STATE_IDLE;
```

- Unless you assign values explicitly, members count up from 0 in declaration order (`SENSOR_STATE_IDLE == 0`, `SENSOR_STATE_SAMPLING == 1`, ...).
- Assign explicit values when the numbers themselves matter, e.g. they're written to a hardware register.

- Prefer this over a block of `#define SENSOR_STATE_IDLE 0` macros: same cost at runtime, but the type shows up in a debugger and the compiler can help catch an unhandled `switch` case.

1.1.4 Scope, lifetime, and storage

Storage class	Scope	Lifetime
Local	Inside a function or block.	Stack-allocated; destroyed when the block exits.
`static` local	Inside a function or block.	Persists for the program's life.
Global	Outside any function.	Exists for the program's life; stored in RAM.

Rule of thumb:

- Minimize scope and lifetime.
- Mark file-scope functions and globals `static` when they shouldn't leak outside the `.c` file; it's the closest thing C has to "private," and it gives the compiler more room to inline and optimize.

1.1.5 Printing (`printf()`)

Specifier	Type
`%d` / `%i`	`int`
`%u`	unsigned int
`%f`	`double` (a `float` argument is promoted to `double`), fixed decimals; `%lf` means the same
`%g`	`double`, shortest representation
`%c`	single `char`
`%s`	string (`char*`)
`%p`	pointer address

```
printf("%s is %d years old!\n", name, age);
printf("Voltage: %.2f V\n", voltage);   // 2 decimal places
```

1.1.6 Functions

```
return_type function_name(param_type param_name, ...)
{
    ...
    return value;   // omit for void
}
```

- `static` before the return type = internal linkage (only visible in this `.c` file). Default to `static` for anything that isn't part of a module's public API.
- `static inline` on small, frequently-called functions hints the compiler to paste the body in at the call site instead of paying for a call/return. Modern compilers usually do this anyway, but it's idiomatic for small helpers.
- Declare functions (a prototype) before use, typically in a header, so callers don't need the full definition.

```c
/**
 * @brief Compute resistance from voltage and current (Ohm's law).
 *
 * @param[in] voltage Voltage in V.
 * @param[in] current Current in mA.
 *
 * @return Resistance in kOhm, or 0 if the current is 0.
 */
static inline double get_resistance(const double voltage, const double current)
{
    return 0.0 != current ? voltage / current : 0.0;
}
```

- Convention for ordering a `.c` file: forward-declare its `static` helpers once, right after the `#includes`, then define the public functions (in the same order the header declares them), and put the `static` helpers' own definitions at the bottom of the file. A reader hits the public API first and only descends into private implementation detail if they keep scrolling:

```c
#include "sensor.h"

/* Static functions: */
static double filter_reading(double raw_v);

double sensor_read(void)
{
    const double raw_v = adc_read();
    return filter_reading(raw_v);
}

/* raw_v filtering is an implementation detail, not part of sensor.h. */
static double filter_reading(const double raw_v)
{
    ...
}
```

- When a header can be included from more than one place (or pulled in indirectly through another header), wrap its contents in a header guard so the compiler never sees the same declarations twice:

```c
#ifndef SENSOR_H_
#define SENSOR_H_

double sensor_read(void);

#endif /* SENSOR_H_ */
```

- Convention: the guard macro is the file name in `UPPER_CASE`, with `.` replaced by `_` and a trailing `_` (`sensor.h` → `SENSOR_H_`), chosen so it's unlikely to collide with any other macro in the program. `#pragma once` does the same job in a single line and most modern compilers support it, but it isn't part of the C standard; traditional guards are the portable choice and what you'll see throughout this codebase when using C. In C++, `#pragma once` is also widely used.

1.1.7 Pointers (the one thing that's actually different)

> **Everything is pass-by-value in C.** Every function argument is copied. Passing a pointer still passes a copy of the pointer; it just happens to point to the same object as the caller's pointer. This lets a function modify the caller's object through the pointer, but assigning a new value to the pointer itself does not affect the caller.

If you're coming from a garbage-collected language, this is the section to actually read:

- A pointer holds an *address*, e.g. `int* ptr = &x;` makes `ptr` point at `x`:
  - `&x` means "address of x".
  - `*ptr` means "the value that `ptr` points to" (dereference).
- Pointers let a function modify the caller's object because, although the pointer itself is passed by value, it still points to that object.

```c
/**
 * @brief Assign 3 to the variable pointed to by x.
 *
 * @param[out] x Pointer to the variable to assign.
 */
static void assign(int* x)
{
    *x = 3; // Writes through the pointer into the caller's variable.
}
```

```c
int main(void)
{
    int a = 0;
    assign(&a);
    printf("a = %d\n", a); // a = 3.
}
```

This is the standard C idiom for "multiple out-parameters" or "modify in place," using no references, no boxing, just an address. You'll see this pattern constantly once we start writing to hardware registers:

- A pointer to a pointer (`int**`) lets a function replace *what the caller's pointer points to*, not just the value at that address; the same out-parameter idiom, one level deeper. You'll see this whenever a function needs to null out or reassign a pointer it was only given a copy of:

```c
#include <stddef.h>

static void set_to_null(int** ptr)
{
    *ptr = NULL; // Changes the caller's pointer itself, not just *ptr's target.
}

int main(void)
{
    int x = 5;
    int* p = &x;
    set_to_null(&p);
    // p is now NULL; x is untouched.
}
```

- Arrays and strings decay to a pointer to their first element automatically when passed to a function, no & needed:

```c
static void print_text(const char* s) { printf("%s", s); }
const char text[] = "no & needed here";
print_text(text);
```

- `const char*` means "pointer to data I won't modify through this pointer"; use it on any string/array parameter you're only reading, same spirit as marking a parameter `readonly`.
- A string is a `char` array ending in a `'\0'` (NUL) byte; there's no length field. `sizeof` gives you the buffer's declared capacity, not the content length, and only works on the actual array, not a pointer to it.

1.1.8 Function pointers

A function pointer holds the *address of a function* instead of a variable, callable through the pointer exactly like calling the function by name:

```c
#include <stdio.h>

static void greet1(void) { printf("Hello!\n"); }
static void greet2(void) { printf("World!\n"); }

int main(void)
{
    void (*fn)(void) = greet1; // fn now holds greet1's address.
    fn();                      // Calls greet1(), exactly like greet1() would.

    fn = greet2;               // fn now holds greet2's address.
    fn();                      // Calls greet2(), exactly like greet2() would.
}
```

- The declaration reads inside-out: `void (*fn)(void)` is "`fn` is a pointer to a function taking no arguments and returning `void`." The parentheses around `*fn` are required, `void *fn(void)` would instead declare a function returning `void*`.
- A function's name, used without calling it (no `()`), decays to its address the same way an array name decays to a pointer to its first element (§1.1.7): `fn = greet1;` needs no `&`.
- `typedef` cleans up the syntax the same way it does for structs (§1.1.3), and gives the pointer type a name that documents what it's for:

```c
typedef void (*callback_t)(void);

static void on_timeout(void) { printf("Timed out!\n"); }

static void run(callback_t cb) // cb holds a function's address, not a value.
{
    cb(); // Calls whichever function was passed in.
}

int main(void)
{
    run(on_timeout); // Prints "Timed out!".
}
```

- `run()` never mentions `on_timeout` by name, it just calls whatever `callback_t` it was given. That's the whole idea: the caller decides which function runs, the callee just knows the function's signature.

- This comes back twice, later in the book: Chapter 9's timer driver exercise (Exercise 9.6) stores one of these in a struct so a timer can run arbitrary code when it expires, and Chapter 10 builds an entire polymorphism system, structs whose behavior is decided at runtime rather than compile time, out of nothing but function pointers.

1.1.9 Reading input

You won't have a keyboard on a microcontroller, but the buffer-safety idiom here reappears when reading serial/UART data later, so it's worth knowing.

- `scanf("%s", buf)` is easy but unsafe: no bounds checking, and it stops at the first whitespace.
- `fgets(buf, sizeof(buf), stdin)` is the safe alternative; it takes an explicit buffer size and won't overflow, but it keeps the trailing `'\n'`, which callers usually strip:

```c
/**
 * @brief Read a line of input into a buffer, stripping the trailing newline.
 *
 * @param[out] buf Buffer to store the line in.
 * @param[in] buflen Capacity of the buffer, in bytes.
 */
static void readline(char* buf, const size_t buflen)
{
    // Read data from the terminal, store in the buffer.
    fgets(buf, (int)(buflen), stdin);

    // Replace '\n' characters with '\0'.
    for (size_t i = 0U; '\0' != buf[i]; i++)
    {
        if ('\n' == buf[i]) { buf[i] = '\0'; }
    }
}
```

- To get numbers out of text: read a line as a string, then convert with `atoi()` for `int` or `atof()` for `double` (both in `<stdlib.h>`). "Read raw bytes, then parse" is the same pattern you'll use for serial input later.

1.1.10 Control flow

C's control-flow syntax is close to every C-family language you already know, with a few local quirks:

- Conditions must be parenthesized: `if (x < 10)`.

- Braces are technically optional for a single statement, but this codebase always uses them, even for one-liners: it avoids the classic bug where a second statement is added later without adding braces to match, silently falling outside the `if`:

```
if (10 > x) { x = 25; }
else { printf("Out of range!\n"); }
```

- `while`, `for`, and `switch` behave as expected. `for (;;)` or `while (true)` is the idiomatic infinite loop.
- `switch` **falls through** unless you `break`, a common bug source if you're coming from a language that doesn't do this. Give each `case` its own `return`/`break`.
- &&, `||`, `!` for logic; == to compare, = to assign; mixing them up compiles and is a classic bug (most compilers will warn on it).
- **Yoda notation**: writing the constant on the left of a comparison, `if (0 == x)` rather than `if (x == 0)`, is a defensive habit against exactly that ==/= typo. Drop the second = with the constant on the left and `if (0 = x)` doesn't compile at all, you can't assign to a literal, so the compiler catches the mistake immediately instead of silently accepting an assignment that evaluates to `x`'s new value and is truthy for anything but `0`. This codebase writes comparisons this way throughout; `if (10 > x)` a few lines up is the same habit applied to >, even though inequalities don't carry the same typo risk, there's no way to accidentally assign with >.

1.1.11 Dynamic memory allocation (`malloc()`/`realloc()`/`free()`)

You almost certainly won't reach for this on a microcontroller: no OS to reclaim a leak, and a fragmented heap is a liability you can't always afford. It's here because `malloc()`/`realloc()`/`free()` are core C knowledge, and the double-pointer idiom below reappears in any hosted C code (test harnesses, host-side tooling, anything running under Linux rather than bare-metal).

- `malloc(size)` (from `<stdlib.h>`) asks the heap for `size` bytes and returns a `void*` (a generic pointer) to it, or a null pointer if the request can't be satisfied. Always check the return value, and cast it to the pointer type you actually want; not required by C, but required by C++, and it makes the intent explicit:

```
#include <stddef.h>
#include <stdlib.h>

int* data = (int*)(malloc(sizeof(int) * count));
if (NULL == data) { return 1; }
```

- `realloc(ptr, new_size)` resizes an existing allocation, possibly moving it to a new address. On success it frees the old block for you and returns the new address; on failure it

returns null and leaves the original block untouched. Capture the result in a temporary first, so a failed `realloc()` doesn't overwrite (and thereby leak) your only pointer to the still-valid original block:

```
int* copy = (int*)(realloc(data, sizeof(int) * new_count));
if (NULL == copy) { return 1; } // Data is untouched and still valid on failure.
data = copy;
```

- `free(ptr)` releases the block. Always set the pointer to `0`/`NULL` immediately after; a pointer that still holds a freed address is a *dangling pointer*; using it again is undefined behavior instead of a clean, obvious crash:

```
free(data);
data = NULL;
```

- For a function to null out the *caller's* pointer, not just its own local copy of it, it needs the address of that pointer: the pointer-to-pointer idiom from §1.1.7:

```
/**
 * @brief Free a dynamically allocated array and null out the caller's pointer.
 *
 * @param[in,out] data Address of the pointer to the array to free.
 */
static void array_delete(int** data)
{
    if ((NULL == data) || (NULL == *data)) { return; }
    free(*data);
    *data = NULL;
}
```

- Putting it together: `struct`, `self`, and dynamic allocation combine into the standard C shape for a growable array:

```
#include <stddef.h>
#include <stdlib.h>

/**
 * @brief Vector structure holding floating-point numbers.
 */
typedef struct
{
    /** Data field of floating-point numbers. */
    double* data;
```

```
    /** Number of elements in the data field. */
    size_t size;
} vector_t;

/**
 * @brief Free a vector's backing storage and reset it to empty.
 *
 * @param[in,out] self Pointer to the vector to clear.
 */
static void vector_clear(vector_t* self)
{
    free(self->data);
    self->data = NULL;
    self->size = 0U;
}

/**
 * @brief Grow or shrink a vector's backing storage.
 *
 * @param[in,out] self Pointer to the vector to resize.
 * @param[in] size New number of elements.
 *
 * @return 0 on success, 1 if the allocation failed (self is left untouched).
 */
static int vector_resize(vector_t* self, const size_t size)
{
    if (NULL == self) { return 1; }

    if (0U == size)
    {
        vector_clear(self);
        return 0;
    }

    double* copy = (double*)(realloc(self->data, sizeof(double) * size));
    if (NULL == copy) { return 1; }
    self->data = copy;
    self->size = size;
    return 0;
}
```

`vector_resize()`'s `NULL == self` check is worth keeping, unlike a similar check you'll see argued away later in this book (the `delay_ms()` of §3.1.4, where the pointer is only ever passed one hardcoded address from within the same file). The difference is what kind of

function this is: `delay_ms()` is a private, `static` helper with exactly one call site the compiler can see in full; `vector_resize()` is written as the kind of function a reusable module would export, called from code you don't control and can't audit by just reading the same file. Once "well-behaved caller" stops being something you can verify by inspection, checking a pointer parameter before dereferencing it stops being decorative and starts being the thing standing between a bad call and a crash.

1.1.12 Macros (`#define`)

You've already seen `#define` throughout this refresher: header guards (§1.1.6), `SENSOR_STATE_IDLE`-style constants (§1.1.3), and every `LED1`/`F_CPU`-style pin and clock constant this book leans on starting next chapter. It's worth understanding what it actually does before writing your own.

- `#define` runs in the **preprocessor**, a separate text-substitution pass that happens before the compiler ever sees your code. `#define TIMEOUT_MS 100` doesn't create a typed variable; it tells the preprocessor to replace every later occurrence of `TIMEOUT_MS` with the literal text `100`, verbatim, before compilation starts. The compiler never sees the name `TIMEOUT_MS` at all.
- Because it's pure text substitution, a macro has no type, no scope, and isn't visible to a debugger by name; it's gone by the time there's a program to debug. It also isn't namespaced to a file, a struct, or a function the way a variable is: once defined, `TIMEOUT_MS` means the same thing everywhere until a matching `#undef`, even in files that `#include` this one.

Object-like vs. function-like macros

- An **object-like macro** stands in for a value, `#define F_CPU 16000000UL` or `#define LED1 1U`, the pattern this book uses constantly for pin numbers and clock constants.
- A **function-like macro** takes parameters, syntactically like a function call, but is still pure text substitution: no type checking on the arguments, no single-evaluation guarantee:

```
#define SQUARE(x) (x * x)
```

Always parenthesize: the body, and every parameter

This is the single most important rule for writing a macro, and the reason a definition like `#define TICKS_NEEDED (TIMEOUT_MS / CLOCK_TIME_MS)` is written the way it is: the whole replacement is wrapped in parentheses. Skip that, and the macro's meaning changes depending on what surrounds it at the call site, because substitution happens *before* the usual rules of precedence get a chance to group anything:

```
#define TICKS_NEEDED TIMEOUT_MS / CLOCK_TIME_MS // No outer parentheses: fragile.

const uint16_t double_ticks = 2U * TICKS_NEEDED; // Expands to: 2U * TIMEOUT_MS / ↩
    ↪ CLOCK_TIME_MS.
// * and / share precedence and group left-to-right: (2U * TIMEOUT_MS) / ↩
    ↪ CLOCK_TIME_MS. Integer
// division truncates, so that equals 2U * (TIMEOUT_MS / CLOCK_TIME_MS) only if it↩
    ↪  divides evenly.
```

```
#define TICKS_NEEDED (TIMEOUT_MS / CLOCK_TIME_MS) // Parenthesized: safe ↩
    ↪ regardless of context.

const uint16_t double_ticks = 2 * TICKS_NEEDED;
// Expands to: 2 * (TIMEOUT_MS / CLOCK_TIME_MS), unambiguous no matter what ↩
    ↪ precedence surrounds
// the macro at its call site.
```

The `/`-only example above only survives without the parentheses when `TIMEOUT_MS` happens to be a multiple of `CLOCK_TIME_MS`, and that's luck, not safety; swap in a macro built from `+` or `-` and it breaks immediately:

```
#define TIMEOUT_PLUS_MARGIN TIMEOUT_MS + 10U   // No parentheses: broken.

const uint16_t scaled = 2U * TIMEOUT_PLUS_MARGIN;
// Expands to: 2U * TIMEOUT_MS + 10U, not 2U * (TIMEOUT_MS + 10U). Silently wrong,↩
    ↪  no compiler
// warning.
```

The same rule applies to every parameter of a function-like macro, not just the overall body; each individual use of a parameter needs its own parentheses too:

```
#define SQUARE(x) (x * x)     // Unsafe: parameter isn't parenthesized.
#define SQUARE(x) ((x) * (x)) // Safe: parameter is parenthesized everywhere it's ↩
    ↪ used.

SQUARE(a + b)
// Unsafe version expands to: (a + b * a + b), wrong.
// Safe version expands to:   ((a + b) * (a + b)), correct.
```

Macro parameters can be evaluated more than once

A function-like macro's parameter isn't evaluated once and reused the way a real function parameter is; the text you pass in gets substituted at every point the parameter name appears in the macro's body. Anything with a side effect duplicates that side effect:

```
#define SQUARE(x) ((x) * (x))

uint8_t i = 5U;
const uint8_t result = SQUARE(i++);   // Expands to ((i++) * (i++)): i is modified↩
    ↪  twice with
                                       // nothing to order the two, which is ↩
    ↪ undefined behavior.
                                       // A real function would increment i ↩
    ↪ exactly once.
```

Avoid passing an expression with side effects (`++`, `--`, assignment, a function call with side effects) into a function-like macro.

Multi-statement macros: wrap in `do { ... } while (0)`

A macro that expands to more than one statement breaks if it's ever used as the body of an `if` without braces, because only the first statement ends up inside the `if`:

```
#define LED_RESET LED_ON; LED_OFF;   // Two statements, no wrapper.

if (error) LED_RESET;   // Expands to: if (error) LED_ON; LED_OFF;
                        // LED_OFF now runs unconditionally, every time, ↩
    ↪ regardless of `error`.
```

Wrapping the body in `do { ... } while (0)` turns it back into a single statement syntactically, so it behaves correctly wherever a single statement is expected, including inside an `if` with no braces:

```
#define LED_RESET do { LED_ON; LED_OFF; } while (0)

if (error) LED_RESET; // Expands to a single do/while statement; behaves as one ↩
    ↪ block.
```

This codebase always braces its `if`s (§1.1.10), which sidesteps the specific failure shown above, but the `do { ... } while (0)` idiom is still worth knowing: you'll see it in other people's code, and it's the standard defense once you can't guarantee every call site remembers to brace its `if`.

Prefer `const`/`enum` over `#define` where you can

A macro has no type and isn't scoped, so where a real alternative exists, it's usually the better choice:

- A `const` variable is typed, scoped like any other variable, and visible to a debugger by name.
- An `enum` (§1.1.3) groups related named integer constants the same way, with the same debugger visibility, and lets some compilers warn on an unhandled `switch` case.

Embedded C keeps reaching for `#define` anyway, for one specific reason: a `const` variable in C is not a *compile-time constant expression* the way it is in C++. Even though its value can't change at runtime, the compiler doesn't treat it as a literal for contexts that require one, such as a `case` label, the size of a file-scope array, or a `#if` preprocessor check. A `#define` is a true compile-time constant and works in all three. An `enum` constant works in the first two, but not in `#if`: the preprocessor runs before the compiler and knows nothing about enums. It also pays off in generated code for something like `PORTB |= (1U << LED1)`: with `LED1` a compile-time constant, the compiler can usually fold the whole expression into a single bit-set instruction; with `LED1` a `const uint8_t` variable, nothing stops it from compiling, but the shift amount is only known at runtime, so the compiler may emit an actual runtime shift instead of folding it away. That's the real reason this book's pin numbers and bit positions are macros instead of `const uint8_t`s, not just convention.

1.1.13 Bringing it together

The pattern you'll see throughout this book, and one worth internalizing before we start writing to hardware registers next chapter, is small `static` functions, `const` everywhere possible, explicit types, no hidden allocations, and a Doxygen comment above every public function that isn't trivially self-explanatory:

```c
#include <stdio.h>


/**
 * @brief Compute resistance from voltage and current (Ohm's law).
 *
 * @param[in] voltage Voltage in V.
 * @param[in] current Current in mA.
 *
 * @return Resistance in kOhm.
 */
static inline double get_resistance(const double voltage, const double current)
{
    return voltage / current;
```

```
}

/**
 * @brief Print voltage, current, and resistance to the terminal.
 *
 * @param[in] voltage Voltage in V.
 * @param[in] current Current in mA.
 * @param[in] resistance Resistance in kOhm.
 */
static inline void print_quantities(const double voltage, const double current,
                                    const double resistance)
{
    printf("Voltage: %g V\n", voltage);
    printf("Current: %g mA\n", current);
    printf("Resistance: %g kOhm\n", resistance);
}

/**
 * @brief Application entry point.
 *
 * @return 0 on successful termination of the program.
 */
int main(void)
{
    const double voltage = 5.0;
    const double current = 0.5;
    const double resistance = get_resistance(voltage, current);
    print_quantities(voltage, current, resistance);
    return 0;
}
```

That style carries directly into embedded C, where you don't have a heap to hide behind anyway.

Practice exercises for this section are in §1.3.

1.2 Review

What you should be able to do now

After this chapter, you should:

- Be comfortable enough with C's core syntax (types, structs, functions, pointers) to not be fighting the language once hardware programming starts in the next chapter.
- Be able to define and use a `struct`, `enum`, and `typedef`, and explain why `self` is passed as a pointer.
- Be able to write a `.c`/`.h` pair with a correct header guard and a sensible static/public ordering.
- Understand the difference between a single pointer (`int*`) and a double pointer (`int**`) as a function parameter, and when each is needed.
- Be able to use `malloc`/`realloc`/`free` correctly, including why a pointer should be nulled immediately after `free`.

Questions to test yourself

You should be able to explain:

1. Why is `self` passed as a pointer rather than by value in this codebase's struct functions?
2. What's the difference between a single pointer (`int*`) and a double pointer (`int**`) as a function parameter, and when do you need the second?
3. Why must you capture `realloc`'s return value in a temporary before overwriting your original pointer?
4. Why should you set a pointer to `0`/`NULL` immediately after `free`?

Looking ahead

Chapter 2 is about hardware-near programming in C: registers, bitwise I/O, and driving/reading pins on the ATmega328P.

1.3 Exercises

How to work these. Every exercise in this chapter is a Code exercise, and every one is a program for your own computer, not for the board: they practise the C of §1.1 before any hardware is involved. Build and run them with the PC's own compiler, for example `gcc -std=c11 -Wall -Wextra -o main main.c` and then `./main`; the board and its toolchain arrive in Chapter 2. While you test Exercises 1.8 and 1.9, add `-fsanitize=address`, as the tip after the last exercise explains.
How to check your work. The Code exercises have no printed solution: the program's output tells you whether it does what the exercise describes. The questions some of them ask about

their design, such as why Exercise 1.7 needs a double pointer, are answered in Appendix B. Try each exercise before you read its answer.

EXERCISE SET 1

Structs, enums, and typedefs

Exercise 1.1 A temperature reading

CODE

Define a `struct` named `temp_reading` with fields `adc_value` (`unsigned int`), `voltage_v` (`double`), and `temperature_c` (`double`). Write a `static` function `temp_reading_init()` that takes a pointer to a `temp_reading` (named `self`, per the convention in §1.1.3) plus an ADC value, and stores it along with the voltage and temperature derived from it (you don't need the real TMP36 formula yet; a placeholder conversion is fine, the real one comes with Chapter 6's temperature capstone, Exercise 6.5).

Exercise 1.2 A typedef for the reading

CODE

Turn `struct temp_reading` into a `typedef`, `temp_reading_t`, following the `_t` convention from §1.1.3.

Exercise 1.3 An enum of states

CODE

Define a `typedef enum` named `state_t` with five states: `STATE_OFF`, `STATE_SLOW`, `STATE_MEDIUM`, `STATE_FAST`, `STATE_ON`. Write a `switch` over a `state_t` variable that prints a message for each state, then remove one `case` and see whether your compiler warns about the missing case (try building with `-Wswitch` if it doesn't by default). **Stretch:** write a function `state_next()` that returns the next state, wrapping from `STATE_ON` back to `STATE_OFF`; the core of the state machine you'll build in Chapter 3's last exercise, Exercise 3.8.

EXERCISE SET 2

Functions and header guards

Exercise 1.4 A header and source pair

CODE

Split `temp_reading_init()` from Exercise 1.1 into a `temp_reading.h`/`temp_reading.c` pair: declare the struct and prototype in the header, define the function in the source file, and add a proper header guard.

Exercise 1.5 A private helper CODE

Add one `static` helper function to `temp_reading.c` that isn't declared in `temp_reading.h`. For example, one that converts an ADC value to a voltage. Following the ordering convention from §1.1.6, where should its forward declaration go, and where should its definition go?

EXERCISE SET 3

Pointers

Exercise 1.6 Swap through pointers CODE

Write a function `swap()` that takes two `int*` parameters and swaps the values they point to. Call it from `main()` on two local variables and print both before and after.

Exercise 1.7 Reset and delete a reading CODE

Write a function `temp_reading_reset()` taking a `temp_reading_t*` that resets all fields to zero. Call it from `main()` and verify that the caller's struct changes.

Then write a function `temp_reading_delete()` that takes a `temp_reading_t**`, a pointer to a pointer, per §1.1.7 and §1.1.11, and sets the caller's pointer to `NULL`. Why does this need a double pointer instead of a single `temp_reading_t*`?

EXERCISE SET 4

Dynamic memory allocation

Exercise 1.8 An array of readings CODE

Write a function `temp_reading_array_new(size_t count)` that `malloc()`s an array of `count` `temp_reading_t` structs and returns a pointer to it (or `NULL` on failure). Pair it with `temp_reading_array_delete()`, taking a `temp_reading_t**`, that frees the array and nulls the caller's pointer.

Exercise 1.9 Resizing the array CODE

Extend the pair from Exercise 1.8 with `temp_reading_array_resize()`, using `realloc()`, following the pattern in §1.1.11. Make sure it behaves correctly when resizing down to zero elements. This is the shape you'd reach for to keep a rolling history of the last few readings, like the five-reading average in Chapter 6's temperature capstone (Exercise 6.5).

Tip: compile with `-fsanitize=address` (GCC/Clang) while testing Exercises 1.8 and 1.9; it catches use-after-free and leaks immediately, which are easy to introduce by hand here.

CHAPTER 2

Hardware-Near Programming in C

No OS between you and the pins, only registers and the bits in them.

In this chapter

- Introduction: what's different about programming a microcontroller versus a normal computer.
- The board: Arduino Uno/Nano / ATmega328P, I/O ports and PIN↔PORT mapping.
- Bitwise operations: OR, AND, NOT, shifting, how register-level I/O actually works.
- Driving an output pin and reading an input pin (`DDRx`, `PORTx`, `PINx`).
- Two worked examples: LED follows a button; two LEDs blink while a button is held.

This chapter is where the book meets the hardware. It explains why embedded systems are programmed close to the hardware, in C, without an OS underneath, and it teaches the one skill everything after it is built on: reading and writing bitwise operations (`|`, `&`, `~`, `<<`, `>>`) to set, clear, and read individual bits in a register, so that you can configure a pin as an input or an output and drive or read it correctly.

Read §2.1 first; it is the chapter's core material. The two sections after it are for reference, and one of them is where you set up the toolchain the rest of the book uses: §2.2 walks through flashing a Microchip Studio build onto the board via the Arduino bootloader and `avrdude`, useful if you don't have a dedicated ISP programmer, and §2.3 is the same idea from native Linux: installing `avr-gcc`/`avrdude`, compiling from the command line, and flashing over serial, including WSL2's USB-passthrough step. Then work through the exercises in §2.5. The Reflection exercise is answered in Appendix B, and so are the values the expressions of Exercise 2.1 should produce; the Code exercises have no printed solution.

2.1 Registers and I/O

2.1.1 What's different about embedded

- A microcontroller (MCU) is a complete computer with no screen or keyboard; I/O happens through pins wired to sensors and actuators (buttons, LEDs, motors, sensors).
- Your code is still compiled to machine code, but there's no OS between you and the hardware; you write directly to memory-mapped registers.
- C is the default choice here: predictable, tiny memory footprint, close to the metal, and still far more productive than assembly. A very skilled assembly programmer can occasionally beat a modern compiler, but rarely by much, and it's almost never worth it.

2.1.2 The board: Arduino Uno/Nano

Either board works for this book: same **ATmega328P** chip, same registers, same PIN↔PORT mapping for D0–D13 and A0–A5. The Nano just adds two extra analog-only pins (A6, A7) and drops the barrel power jack.

The ATmega328P is an 8-bit AVR microcontroller with three I/O ports, each covering a group of pins:

Board pin	Register port
D0–D7	`PORTD0`–`PORTD7`
D8–D13	`PORTB0`–`PORTB5`
A0–A5	`PORTC0`–`PORTC5`

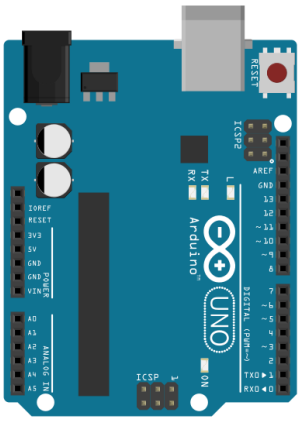


Figure 2.1 The Arduino Uno: digital pins D0–D13 along the top edge, analog pins A0–A5 along the bottom.

Here's the Nano for direct comparison: different board outline and a Mini-USB connector instead of the Uno's USB-B, but the same D0–D13/A0–A5 pin numbering and the same ATmega328P underneath:

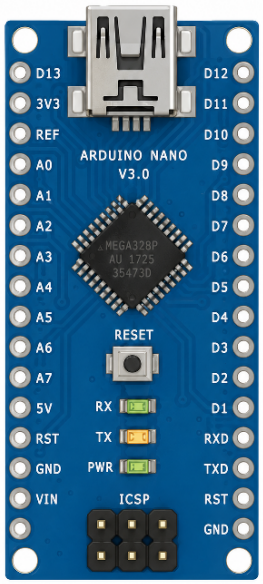


Figure 2.2 The Arduino Nano: the same pin numbering, plus the analog-only pins A6 and A7.

`PORTB` and `PORTD` are purely digital I/O. `PORTC` is the analog-capable port (used with the ADC, Chapter 4) but behaves as ordinary digital I/O when you're not doing analog reads.

For consistency, every breadboard circuit diagram in this section (and in §2.2) shows an Arduino Uno. If you're using a Nano, wire it the same way: same chip, same D0–D13/A0–A5 pin numbering, the only difference is the board outline and USB connector in the picture.

2.1.3 Bitwise operations (this is the whole game)

There's no `digitalWrite()` here; you write directly to registers, and a register is just a byte, so every pin operation is a bitwise operation on that byte.

Operation	C operator	Effect
OR	`|`	sets one or more bits, leaves the rest ("bitwise addition")
AND	`&`	reads a bit, or (with NOT) clears one ("bitwise multiplication")
NOT	`~`	inverts every bit
XOR	`^`	clears matching bits, sets differing bits
NOR	`~(a | b)`	inverse of OR: bit set only where both operands are 0
NAND	`~(a & b)`	inverse of AND: bit set unless both operands are 1
XNOR	`~(a ^ b)`	inverse of XOR: bit set where operands match

There's no single C operator for NOR/NAND/XNOR; you get them by combining ~ with `|`/`&`/`^` as shown above. Don't confuse NAND with the clear-bit idiom below: `reg &= ~mask` is AND with a *negated mask*, not a NAND of `reg` and `mask`; those are different operations that happen to both involve ~ and &:

- Set bit 3 without touching the rest: `reg |= 0b00001000;`
- Clear bit 3 without touching the rest: `reg &= ~0b00001000;`

Shifting

`1U << n` puts a single `1` in bit position `n`; this is how you address "which pin" without hand-writing the bit pattern:

```
#define LED1 1U         // D9 -> PORTB1.
PORTB |= (1U << LED1);  // Enable LED1.
PORTB &= ~(1U << LED1); // Disable LED1.
```

`x << n` and `x >> n` are also multiply/divide by `2^n`, if you ever need that framing.

2.1.4 Driving an output, reading an input

- `DDRx` (**D**ata **D**irection **R**egister) decides, per bit, whether a pin is an output (`1`) or input (`0`).
- On an output pin, `PORTx` sets the pin high/low.
- On an input pin, `PORTx` instead enables (`1`) or disables (`0`) the internal pull-up resistor. With it enabled, the pin reads a clean `1` when unconnected/floating and `0` when pulled to ground, instead of floating unpredictably.
- To read the actual input level, read `PINx` (not `PORTx`).

```
#define BTN1 5U                              // D13 -> PORTB5
#define BTN1_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↩
    ↪ otherwise.
```

The button circuit used in §2.1.5 and §2.1.6 doesn't actually rely on this internal pull-up: it wires an **external pull-down** resistor instead (one leg of the button to 5 V, the other leg to D13 and, through a 10 kΩ resistor, to GND). That means D13 reads `0` when idle and `1` when pressed, the opposite of the internal-pull-up pattern described above. The code still enables the internal pull-up anyway (`PORTB = (1U << BTN1);`), so you see `PORTx`'s input-mode behavior used directly, but it isn't what makes this circuit work. It's harmless in practice: the internal pull-up and the external 10 kΩ pull-down form a voltage divider, and with the internal resistor's typical value of around 35 kΩ the idle pin sits near 1.1 V, comfortably below the 1.5 V (0.3×5 V) under which a pin is guaranteed to read `0`. It isn't guaranteed, though: the datasheet only promises 20–50 kΩ, and at 20 kΩ the idle pin rises to

about 1.7 V, where the reading is undefined. In a design of your own, don't enable a pull-up against an external pull-down.

2.1.5 First program: LED follows a button

An LED is connected to D9, and a button is connected to D13. The button is continuously polled:

- The LED is enabled if the button is pressed.
- The LED is disabled if the button isn't pressed.

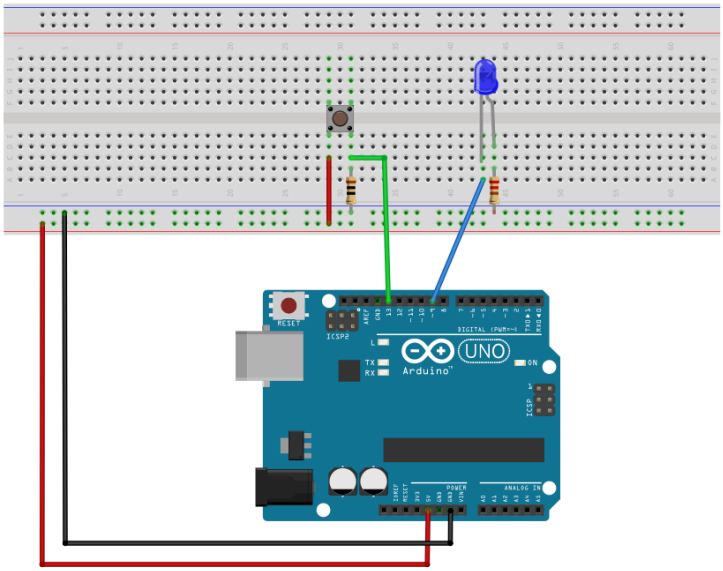


Figure 2.3 An LED on D9 and a button on D13, with its 10 kΩ pull-down resistor to GND.

```
#include <avr/io.h>

/** GPIO pins. */
#define LED1 1U // D9  -> PORTB1.
#define BTN1 5U // D13 -> PORTB5.

/** GPIO operations. */
#define LED1_ON PORTB |= (1U << LED1)         // Enable LED1.
#define LED1_OFF PORTB &= ~(1U << LED1)       // Disable LED1.
#define BTN1_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↪
    ↪ otherwise.
```

```c
/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Configure BTN1 as input with its pull-up enabled.
    PORTB = (1U << BTN1);
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        // Read BTN1, enable LED1 if pressed, disable if not.
        if (BTN1_PRESSED) { LED1_ON; }
        else { LED1_OFF; }
    }
    return 0;
}
```

`LED1_ON`, `LED1_OFF`, and `BTN1_PRESSED` are the object-like macros covered in §1.1.12: plain text substitution, with no parameters. `BTN1_PRESSED` is used as a value, inside a condition, so its whole body is parenthesized and it behaves correctly no matter what surrounds it at the call site. `LED1_ON` and `LED1_OFF` are complete statements, only ever used on their own, followed by a `;`.

You'll see a Doxygen comment above every non-trivial function throughout this codebase, written in the imperative mood ("Configure...", "Compute...", not "Configures..." or "This function configures...").

2.1.6 Blinking two LEDs, with a hand-rolled delay

LEDs on D9/D10 alternate every 200 ms while the button on D13 is held; both off otherwise.

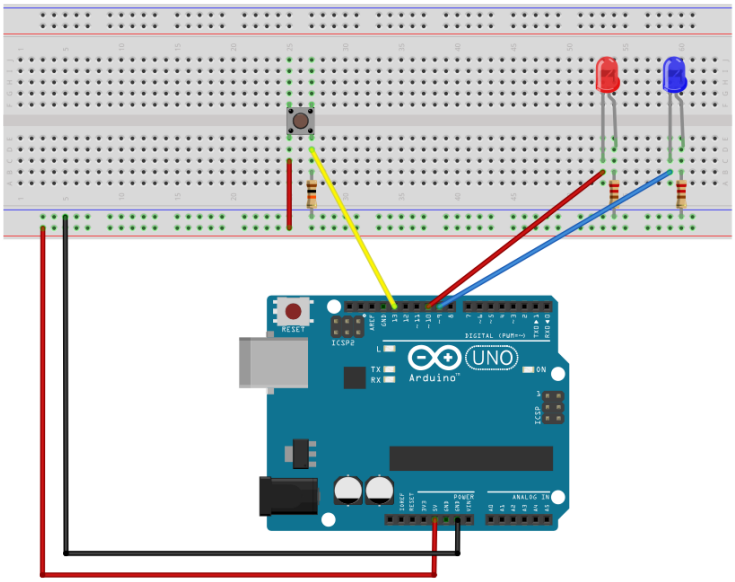


Figure 2.4 Two LEDs on D9 and D10, and the button on D13.

```
#define F_CPU 16000000UL // CPU frequency in Hz.

#include <stdint.h>

#include <avr/io.h>
#include <util/delay.h>

/** GPIO pins. */
#define LED1 1U // D9  -> PORTB1.
#define LED2 2U // D10 -> PORTB2.
#define BTN1 5U // D13 -> PORTB5.

/** GPIO operations. */
#define LED1_ON PORTB |= (1U << LED1)      // Enable LED1.
#define LED1_OFF PORTB &= ~(1U << LED1)    // Disable LED1.
#define LED2_ON PORTB |= (1U << LED2)      // Enable LED2.
#define LED2_OFF PORTB &= ~(1U << LED2)    // Disable LED2.
#define BTN1_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↩
    ↪ otherwise.

/** Time parameters. */
#define TICK_1MS 1U         // Generate 1 ms tick in delay_ms().
```

```c
#define BLINK_SPEED_MS 200U // Blink speed in ms.

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 and LED2 as outputs.
    DDRB = (1U << LED1) | (1U << LED2);

    // Configure BTN1 as input with its internal pull-up enabled.
    PORTB = (1U << BTN1);
}

/**
 * @brief Generate delay.
 *
 * @param[in] ms Delay duration in ms.
 */
static void delay_ms(const uint16_t ms)
{
    for (uint16_t i = 0U; i < ms; ++i)
    {
        _delay_ms(TICK_1MS);
    }
}

/**
 * @brief Blink the LEDs.
 *
 * @param[in] blink_speed_ms Blink speed in ms.
 */
static void led_blink(const uint16_t blink_speed_ms)
{
    // Enable LED1, disable LED2.
    LED1_ON;
    LED2_OFF;
    delay_ms(blink_speed_ms);

    // Disable LED1, enable LED2.
    LED1_OFF;
    LED2_ON;
    delay_ms(blink_speed_ms);
}
```

```c
/**
 * @brief Turn both LEDs off.
 */
static void leds_off(void)
{
    LED1_OFF;
    LED2_OFF;
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        // Read BTN1, blink the LEDs if pressed, disable both LEDs if not.
        if (BTN1_PRESSED) { led_blink(BLINK_SPEED_MS); }
        else { leds_off(); }
    }
    return 0;
}
```

A few things worth noting:

- `F_CPU` must be `#defined` *before* including `<util/delay.h>`, since the delay functions compute their loop counts from the clock frequency at compile time. `int` is only 16 bits on this target, far too small for 16,000,000; the `UL` suffix makes the constant an `unsigned long` explicitly, rather than leaving its type to C's rules for unsuffixed constants.
- `<stdint.h>` is included explicitly for `uint16_t`, even though `<avr/io.h>` happens to pull it in already; include what you directly use rather than relying on a header's own includes, which can change.
- `_delay_ms()` only accepts a compile-time constant or macro as its argument, not a variable, so it can't take `ms` directly. `delay_ms()` works around this by calling `_delay_ms(TICK_1MS)` in a loop, incrementing a local counter each pass until it reaches `ms`.
- `_delay_ms()` (and by extension `delay_ms()`) **blocks** the entire program while it waits, fine for this example, but this exact limitation is what the timers chapter, Chapter 5, exists to solve.

2.2 Arduino via Microchip Studio

2.2.1 Why this workflow

If you don't have a dedicated ISP programmer, you can still flash a Microchip Studio build onto the board: the Arduino bootloader already sits on the chip, and the Arduino IDE already knows the exact `avrdude` command line needed to talk to it over your board's COM port. The trick is simply to borrow that command line and point it at Microchip Studio's `.hex` output instead of the Arduino IDE's own. Everything below builds toward one reusable "External Tool" entry in Microchip Studio that does exactly that.

2.2.2 Wire the circuit

Same layout as the first example of §2.1: LED anode on D9 (through a 220–470 Ω resistor to ground, 330 Ω is a good middle value), button on D13 (one leg to 5 V, the other to ground through a 10 kΩ resistor, with D13 connected between the button and the resistor).

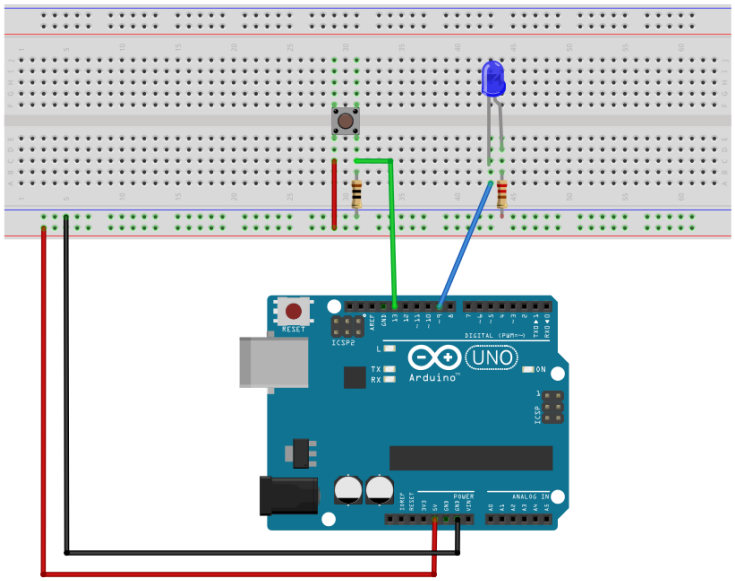


Figure 2.5 The circuit on the breadboard: the LED on D9 and the button on D13, with its pull-down resistor.

Like the diagrams in §2.1, this one shows an Arduino Uno. If you're using a Nano instead: same chip, same pin numbering, just a different board outline in the picture.

2.2.3 Build and test the sketch in Arduino IDE

This step uses the Arduino libraries (C++, an abstraction layer this book doesn't otherwise use) purely to get a known-working upload before touching Microchip Studio. Open the Arduino IDE, start a new sketch (`File → New`), and replace its contents with:

```
/** GPIO pins. */
#define LED1 9U  // D9.
#define BTN1 13U // D13.

/**
 * @brief Set up system.
 */
void setup()
{
    // Configure LED1 as output.
    pinMode(LED1, OUTPUT);

    // Configure BTN1 as input (the external 10 kOhm resistor pulls it down).
    pinMode(BTN1, INPUT);
}

void loop()
{
    // Read BTN1, enable LED1 if pressed, disable if not.
    if (digitalRead(BTN1)) { digitalWrite(LED1, HIGH); }
    else { digitalWrite(LED1, LOW); }
}
```

This lights the LED on D9 while the button on D13 is held. It'll be rebuilt as real C in §2.2.7.

`pinMode()`, `digitalRead()`, and `digitalWrite()` are Arduino's abstraction over the register-level I/O §2.1 covers directly: `pinMode(pin, mode)` sets a pin `INPUT` or `OUTPUT` (a `DDRx` write), `digitalRead(pin)` returns `HIGH`/`LOW` for a pin's current level (a `PINx` read), and `digitalWrite(pin, value)` sets an output pin `HIGH` or `LOW` (a `PORTx` write). Underneath, the Arduino core implements all three with exactly the register operations from §2.1, just hidden behind a function call and a pin-number lookup table.

Turn on verbose upload logging, since the next step depends on reading `avrdude`'s exact command line out of it: `File → Preferences`, then check the box next to "Show verbose output during: Upload".

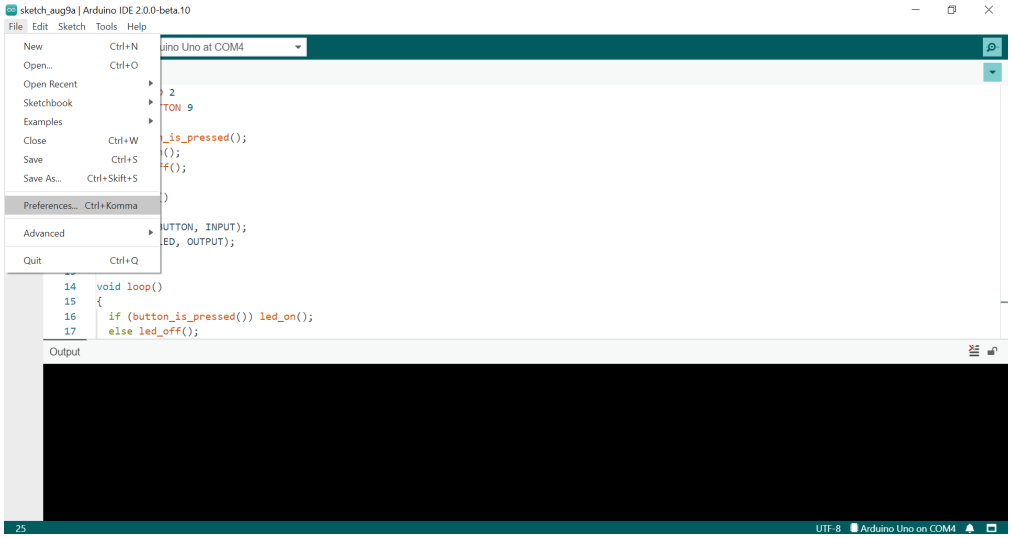


Figure 2.6 The Arduino IDE's File menu, with Preferences.

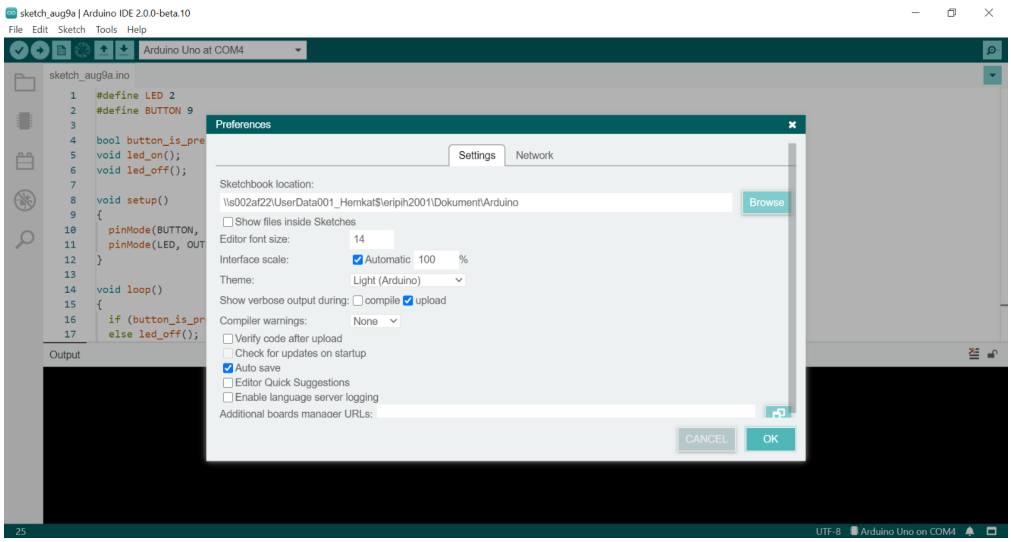


Figure 2.7 The Preferences dialog, with verbose output turned on for upload.

2.2.4 Install the board package, select the board and port

Install the AVR board package if it isn't already: `Tools → Board → Boards Manager`, then click Install on "Arduino AVR Boards by Arduino" (skip this if it's already installed, as in the screenshot below).

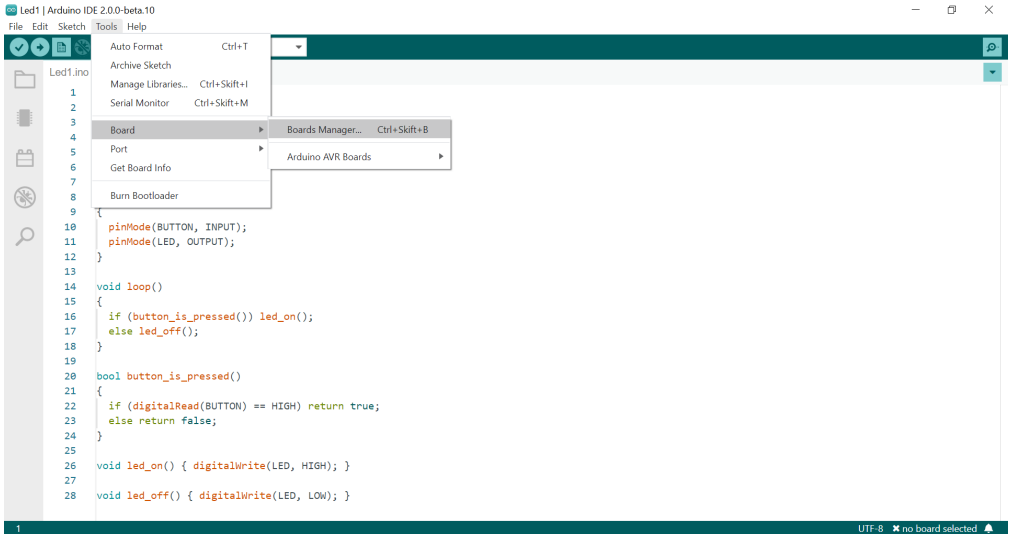


Figure 2.8 Tools, Board, Boards Manager.



Figure 2.9 The Boards Manager, with Arduino AVR Boards already installed.

With the board connected, select it: `Tools → Board → Arduino AVR Boards → Arduino Uno`. Then select its port: `Tools → Port`, and pick the COM port shown (e.g. COM4; the exact number doesn't matter and can vary by machine).

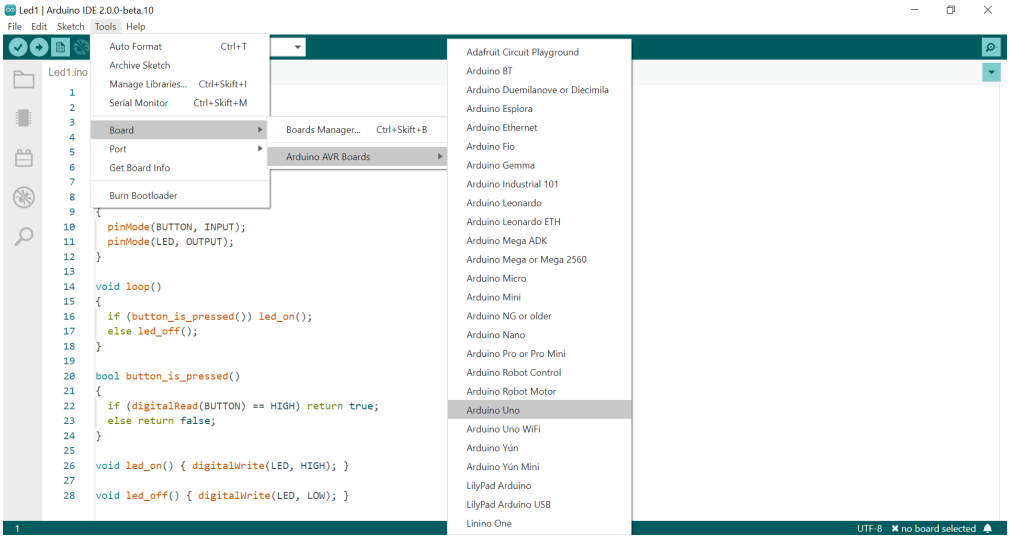


Figure 2.10 Selecting the Arduino Uno as the board.

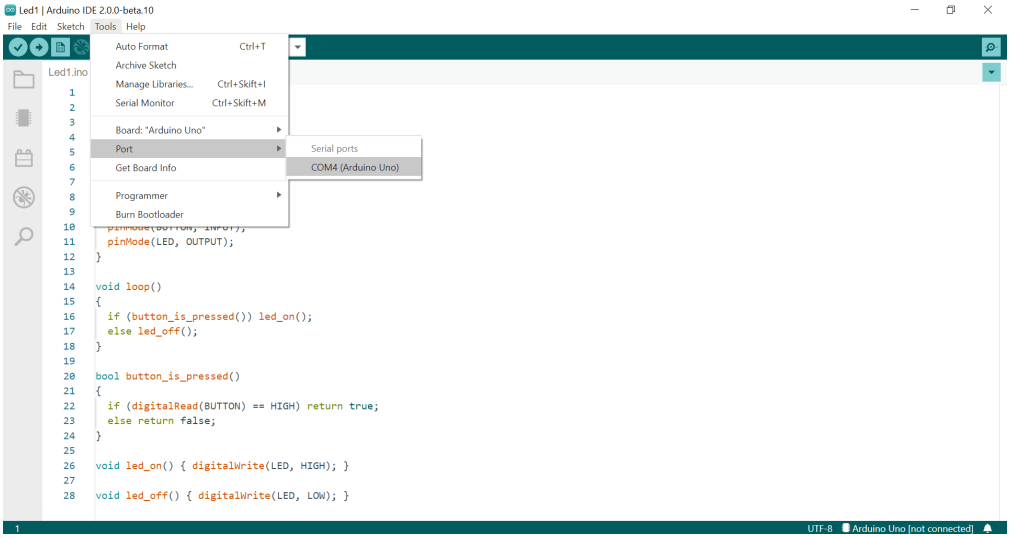


Figure 2.11 Selecting the board's port, here COM4.

2.2.5 Upload and read back avrdude's command line

Click Upload in the top-left corner.



Figure 2.12 The Upload button, the arrow in the top-left corner.

In the Output window (the black pane), scroll up to the line reading "Compilation complete." and copy the `avrdude` command on the line beneath it; it should look something like:

```
"C:\Users\eripih2001\AppData\Local\Arduino15\packages\arduino\tools\avrdude\6.
 ↪  3.0-arduino17/bin/avrdude"
"-CC:\Users\eripih2001\AppData\Local\Arduino15\packages\arduino\tools\avrdude\
 ↪  6.3.0-arduino17/etc/avrdude.conf"
-v -V -patmega328p -carduino "-PCOM4" -b115200 -D
"-Uflash:w:C:\Users\ERIPIH~1\AppData\Local\Temp\arduino-sketch4CE86550DB843096
 ↪  8B53E52972DC68D2/Led1.ino.hex:i"
```

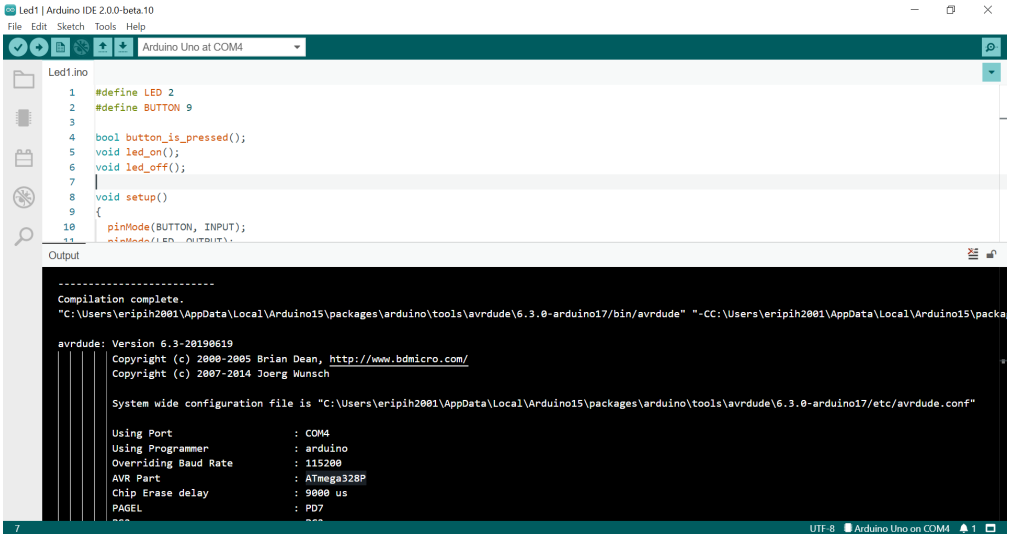


Figure 2.13 The avrdude command in the Output window, on the line after "Compilation complete."

2.2.6 Turn it into a reusable command

This is the fiddliest part, but it's a one-time setup: reshaping that copied line into a command/arguments pair Microchip Studio's "External Tools" feature can invoke directly.

1. Open Notepad, paste the copied path in, and strip out all the quotation marks. Save it as `Arduino via Microchip Studio.txt`.
2. After `bin/avrdude`, press Enter twice so the rest of the path lands two lines down.
3. Add the extension `.exe` after `avrdude` on the top line.



Figure 2.14 The command in Notepad, its quotation marks removed and the avrdude path on a line of its own.

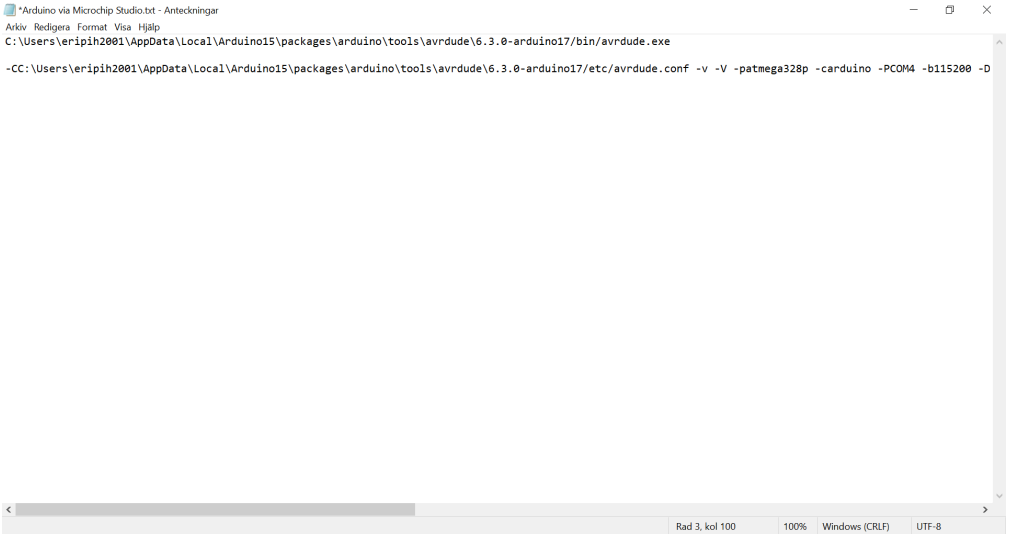


Figure 2.15 `.exe` added after `avrdude` on the top line.

4. Far to the right on the second line, just before `-Uflash`, press Enter so the rest lands on a new third line.

5. On the two bottom lines, add quotes around the two file paths (the ones starting with `C:\` and ending in `.conf` and `.hex` respectively). Afterward the two lines should look roughly like:

```
-C"C:\Users\eripih2001\AppData\Local\Arduino15\packages\arduino\tools\avrdude\
  6.3.0-arduino17/etc/avrdude.conf" -v -V -patmega328p -carduino -PCOM4
  -b115200 -D
-Uflash:w:"C:\Users\ERIPIH~1\AppData\Local\Temp\arduino-sketch4CE86550DB843096
  8B53E52972DC68D2/Led1.ino.hex":i
```

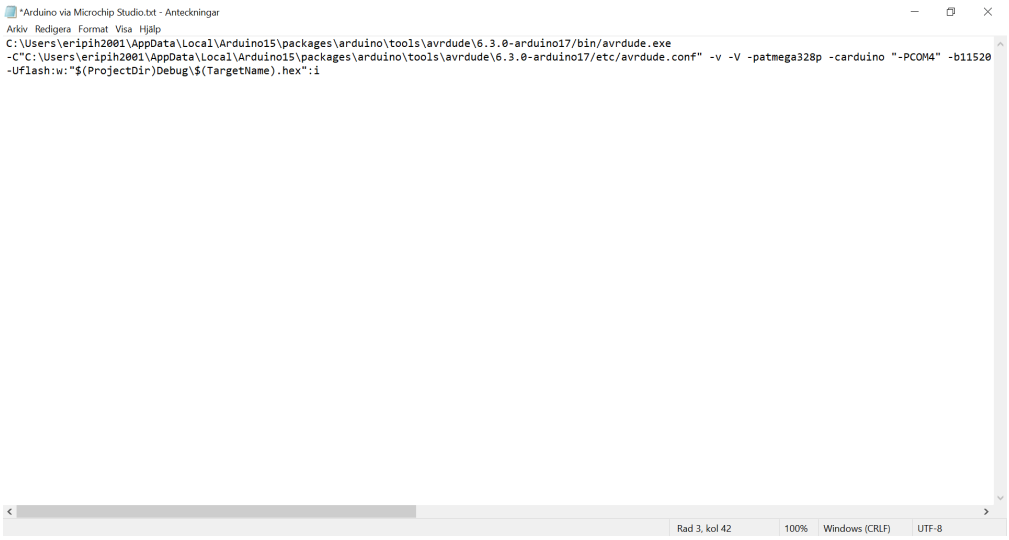


Figure 2.16 The arguments split before `-Uflash`.

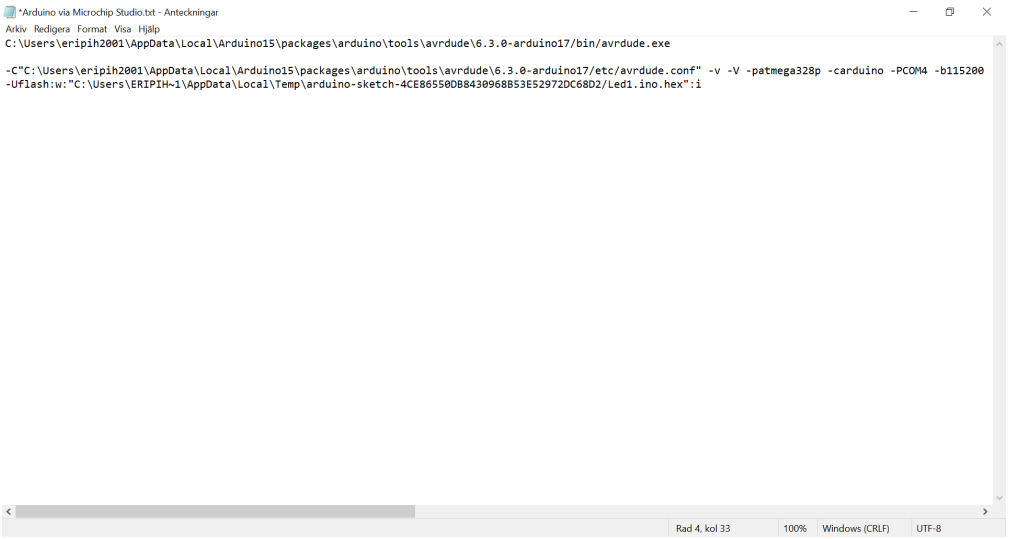


Figure 2.17 Quotes added around the `.conf` and `.hex` paths.

6. Cut the file path on the bottom line (the part after `-Uflash`, between the quotes ending in `.hex`) and paste in `$(ProjectDir)Debug\$(TargetName).hex` instead: Microchip Studio expands these build-variable placeholders to the actual output path at run time, so this line no longer needs to change per project.
7. Merge the two bottom lines back into one: click at the very start of the last line (right before `-Uflash`) and press Backspace. Only two lines should remain in total. Save the file.

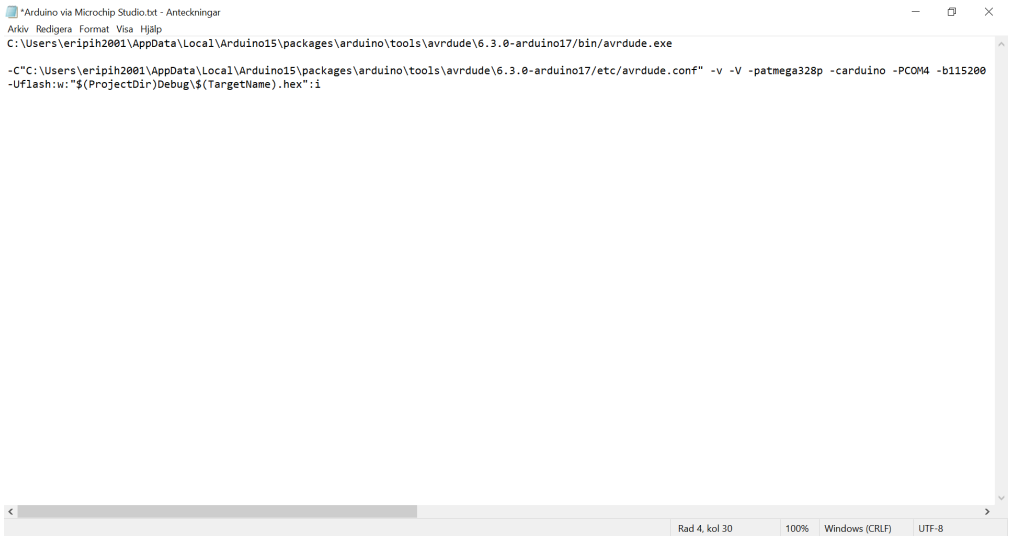


Figure 2.18 The `.hex` path replaced by Microchip Studio's build variables.

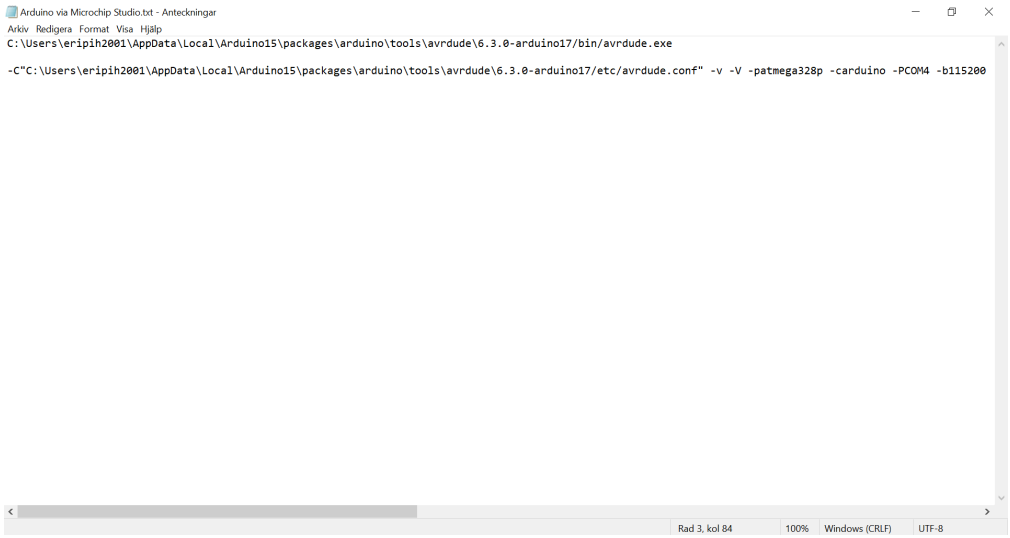


Figure 2.19 The finished file: the Command on the top line, the Arguments below it.

You now have two lines: the `avrdude.exe` path (the *Command*) and everything else (the *Arguments*): ready to paste into Microchip Studio's External Tools dialog in §2.2.8.

2.2.7 Recreate the program in Microchip Studio

Open Microchip Studio and create a new **GCC C Executable Project**, named `Led1`.

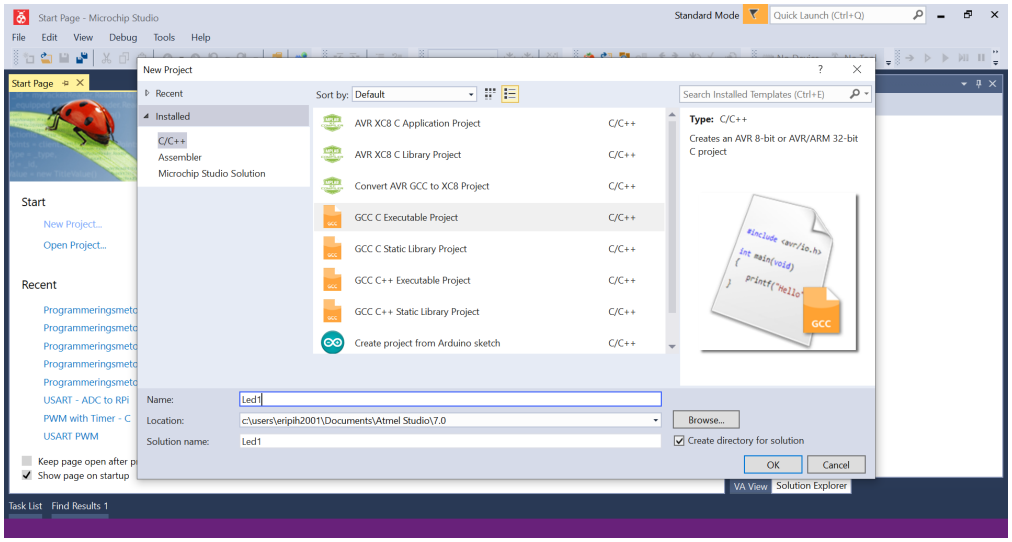


Figure 2.20 A new GCC C Executable Project, named Led1.

In the Device Selection dialog, choose **ATmega328P**.

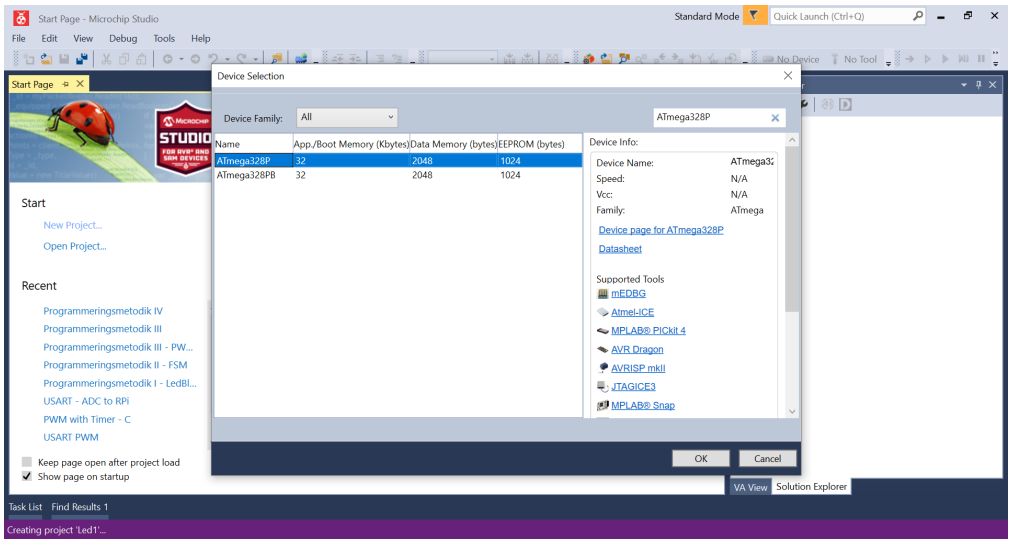


Figure 2.21 Choosing the ATmega328P in the Device Selection dialog.

Replace the contents of `main.c` with the real-C equivalent of the sketch from §2.2.3, written to this codebase's conventions:

```c
#include <stdint.h>

#include <avr/io.h>

/** GPIO pins. */
#define LED1 1U // D9  -> PORTB1.
#define BTN1 5U // D13 -> PORTB5.

/** GPIO operations. */
#define LED1_ON PORTB |= (1U << LED1)         // Enable LED1.
#define LED1_OFF PORTB &= ~(1U << LED1)       // Disable LED1.
#define BTN1_IS_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↩
    ↪ otherwise.

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Configure BTN1 as input with its pull-up enabled.
    PORTB = (1U << BTN1); // D13 as input, pull-up enabled
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        // Read BTN1, enable LED1 if pressed, disable if not.
        if (BTN1_IS_PRESSED) { LED1_ON; }
        else { LED1_OFF; }
    }
```

```
    return 0;
}
```

Then build it: `Build → Build Solution`.

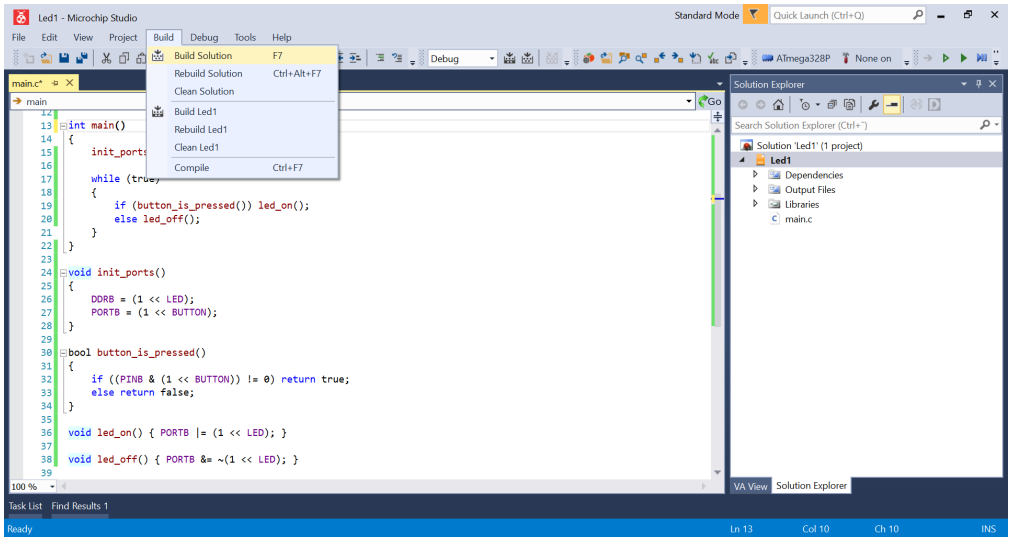


Figure 2.22 Build, Build Solution.

The screenshots for this walkthrough were taken against earlier drafts of the program: of the Arduino sketch in §2.2.3–2.2.5, with other pin numbers, and of `main.c`, with plain (non-`static`) functions and no Doxygen comments. The *code shown in the text* is what you should actually type in; the IDE mechanics (menus, dialogs, build steps) are otherwise identical.

2.2.8 Wire up the External Tool

Programming the chip externally requires the Advanced UI profile: `Tools → Select Profile`, choose **Advanced**, then **Apply**.

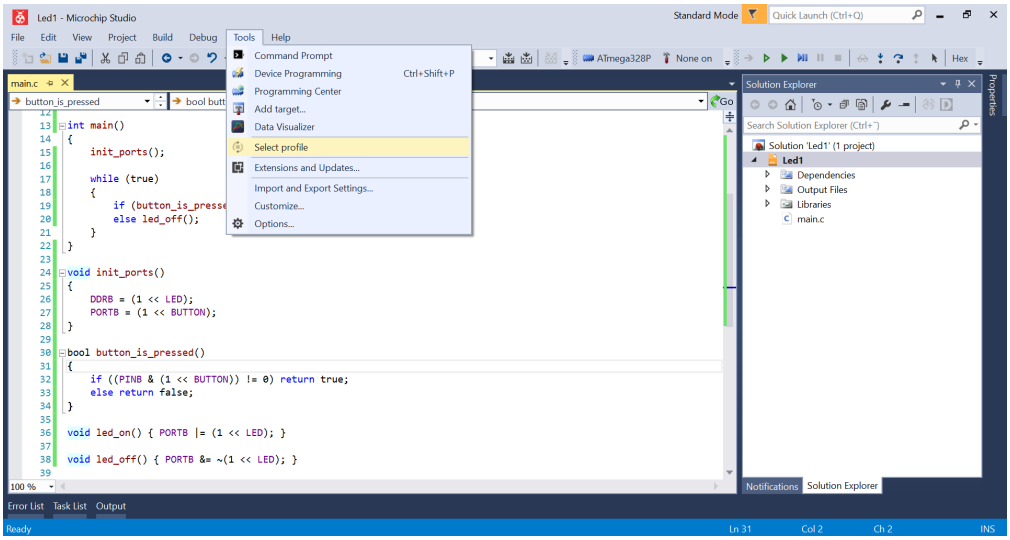


Figure 2.23 Tools, Select Profile.

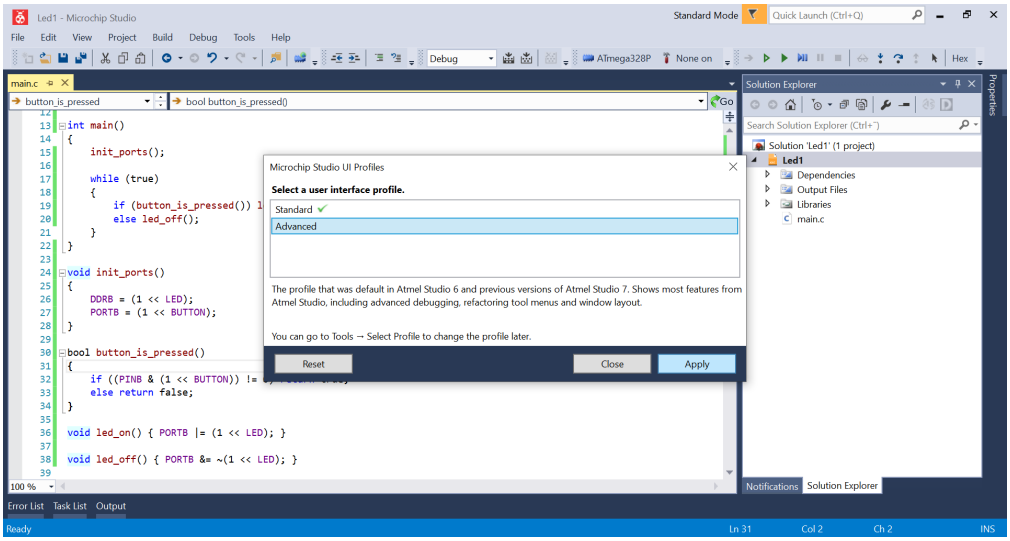


Figure 2.24 The Advanced profile.

Now create the External Tool entry: `Tools → External Tools`.

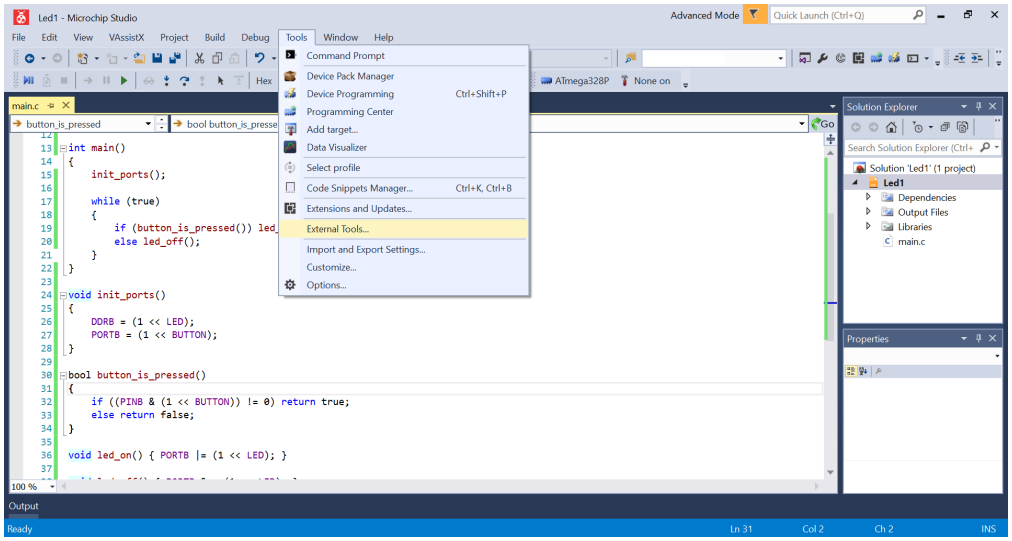


Figure 2.25 Tools, External Tools.

In the dialog that appears, click **Add**. Title it `Arduino` followed by the COM port used earlier in the Arduino IDE: e.g. `Arduino - COM4`. Paste the first line from `Arduino via Microchip Studio.txt` into **Command**, and the second line into **Arguments**. Check **Use Output Window**, then click **OK**.

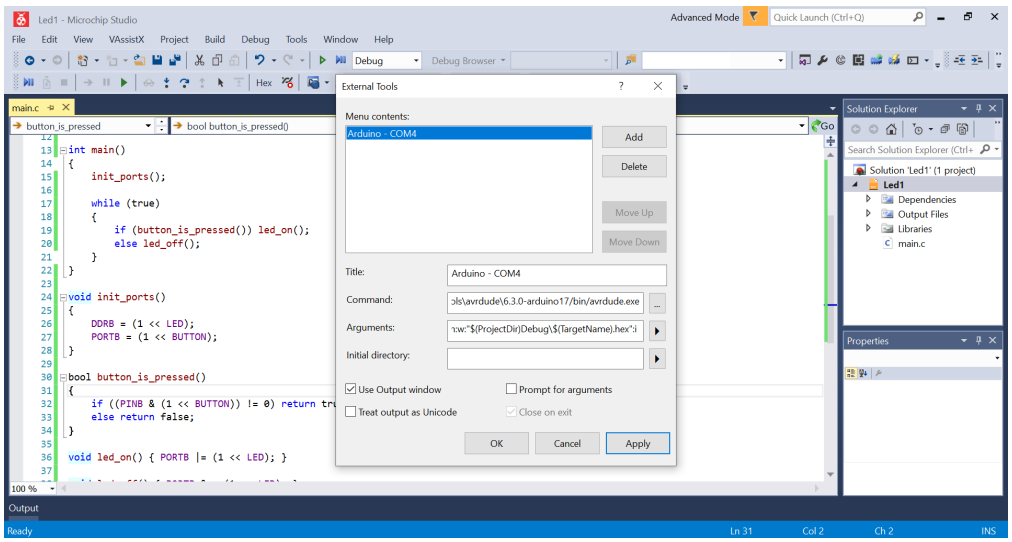


Figure 2.26 The External Tools dialog with the Arduino - COM4 entry: its Command and its Arguments.

Go back into `Tools` and select the tool you just created (e.g. `Arduino - COM4`).

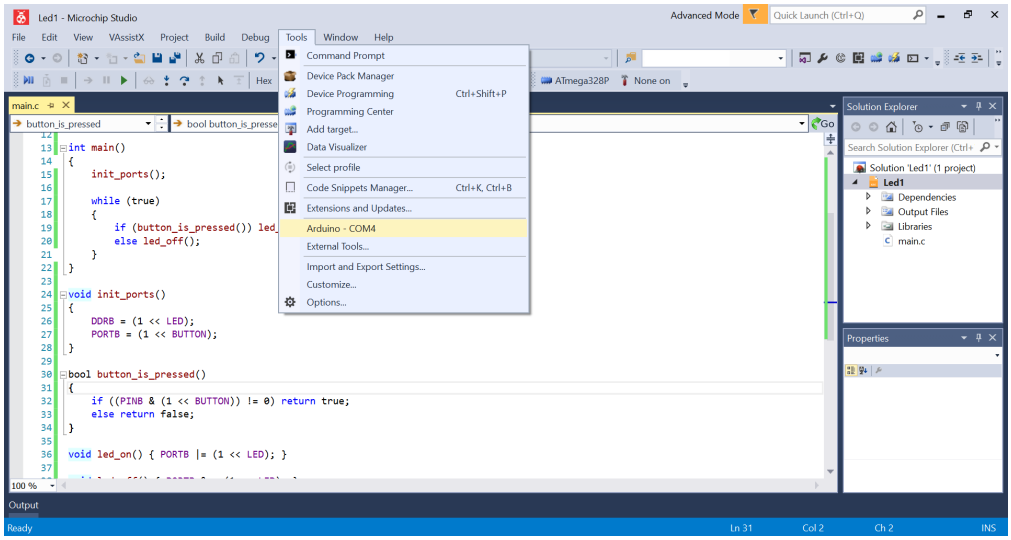


Figure 2.27 Running the new tool from the Tools menu.

2.2.9 Flash and verify

If everything worked, the Output window should end with the number of bytes written, e.g.:

```
Writing | ################################################## | 100% 0.05s

avrdude.exe: 194 bytes of flash written
avrdude.exe: safemode: lfuse reads as 0
avrdude.exe: safemode: hfuse reads as 0
avrdude.exe: safemode: efuse reads as 0
avrdude.exe: safemode: Fuses OK (E:00, H:00, L:00)

avrdude.exe done. Thank you.
```

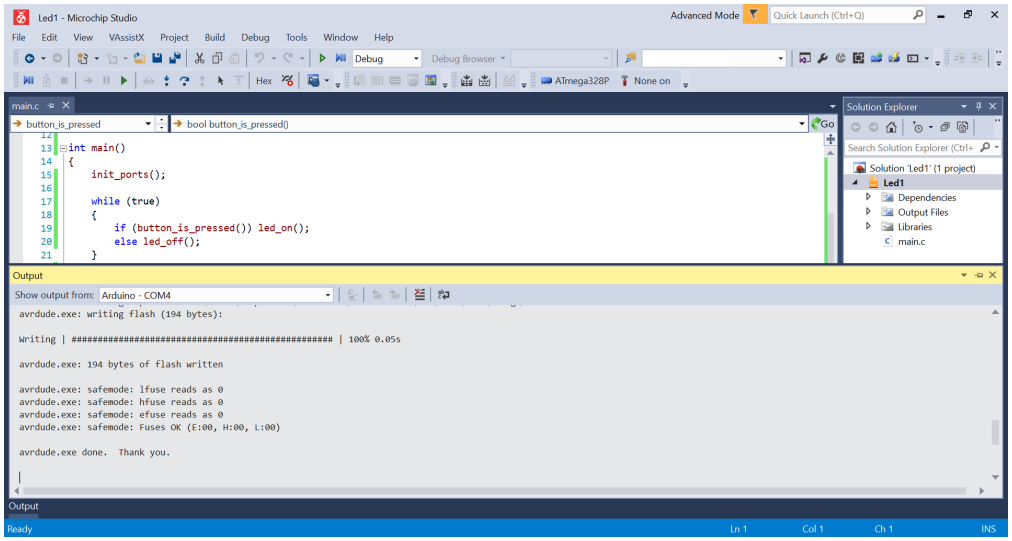


Figure 2.28 A successful flash, as the Output window reports it.

The LED on D9 should now light up while the button on D13 is held, and stay off otherwise. If the write fails instead, with something like:

```
avrdude.exe: ser_open(): can't open device "\\.\COM4": Det går inte att hitta
  ↪  filen.
```

(the last part is Swedish Windows for "The system cannot find the file specified"), check that the board is connected, then check the Arduino IDE for its current COM port; if it's changed, see below.

If the COM port changes

Reconnecting the board sometimes assigns a different COM port, which surfaces as the same `ser_open()` error above. The fix is a second External Tool entry, identical to the first except for the port: back in `Tools → External Tools`, add a new entry (e.g. `Arduino - COM3`), reusing the same Command, and an Arguments value with `PCOM4` swapped for the new port (e.g. `PCOM3`).

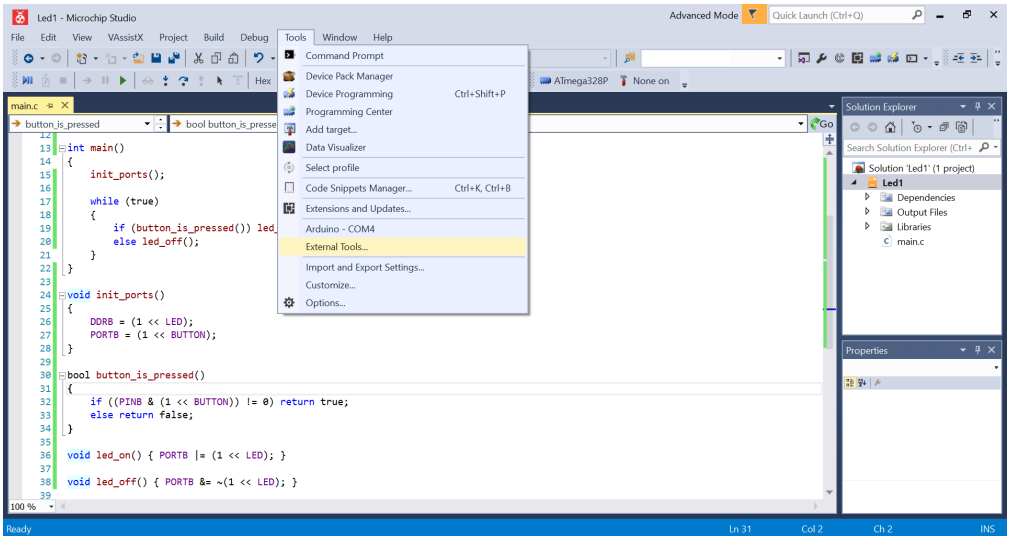


Figure 2.29 Back to Tools, External Tools, for a second entry.

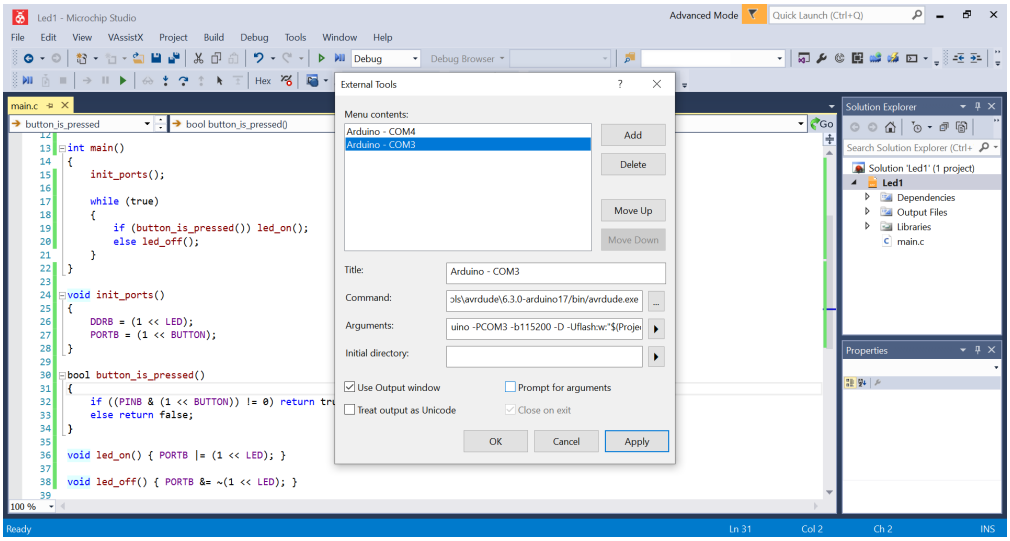


Figure 2.30 The Arduino - COM3 entry: the same Command, with `-PCOM3` in its Arguments.

Then select the new tool from `Tools` and run it the same way as before.

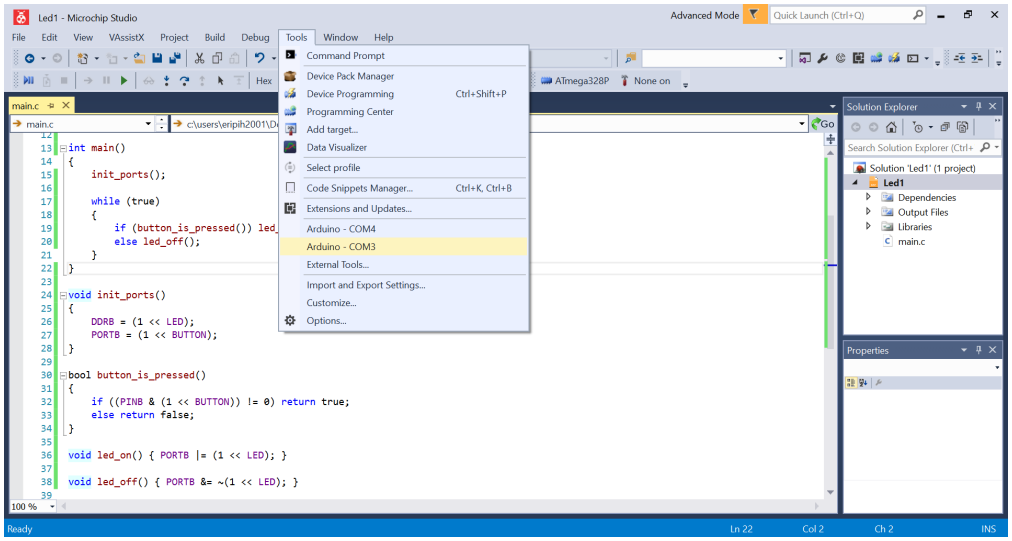


Figure 2.31 Running the COM3 tool from the Tools menu.

2.3 Building and Flashing on Linux

2.3.1 Why this section

§2.2 covers the Windows/Microchip Studio path. This is the native Linux equivalent: the same `avr-gcc/avrdude` toolchain Microchip Studio and the Arduino IDE both wrap under the hood, used directly from the command line. No IDE required.

This was written and tested from WSL2 (Ubuntu on Windows); the compiler/`avrdude` steps are identical on any Debian/Ubuntu-based Linux, but the USB step of §2.3.5 is WSL2-specific; skip it on a native Linux install, where the board just shows up as a device node once plugged in.

2.3.2 Install the toolchain

Three packages: the AVR cross-compiler, its C standard library, and the flashing tool.

```
sudo apt -y update
sudo apt -y install gcc-avr avr-libc avrdude
```

Verify:

```
avr-gcc --version
avrdude -v
```

2.3.3 Write the program

Same first program as §2.1.5: LED on D9, button on D13, LED lit while the button is held.

```c
#include <avr/io.h>

/** GPIO pins. */
#define LED1 1U // D9  -> PORTB1.
#define BTN1 5U // D13 -> PORTB5.

/** GPIO operations. */
#define LED1_ON PORTB |= (1U << LED1)         // Enable LED1.
#define LED1_OFF PORTB &= ~(1U << LED1)       // Disable LED1.
#define BTN1_IS_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↩
    ↪ otherwise.

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Configure BTN1 as input with its pull-up enabled.
    PORTB = (1U << BTN1);
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        // Read BTN1, enable LED1 if pressed, disable if not.
        if (BTN1_IS_PRESSED) { LED1_ON; }
        else { LED1_OFF; }
    }
    return 0;
```

```
}
```

Save it as `main.c`.

2.3.4 Compile

Two steps: compile and link to an ELF, then extract the flash-able Intel HEX from it.

```
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os -std=c11 -Wall -Wextra -o main.elf
    ↪ main.c
avr-objcopy -O ihex -R .eeprom main.elf main.hex
```

-

- `mmcu=atmega328p` targets the right instruction set and register definitions.

-

- `DF_CPU=16000000UL` matters the moment any code uses `<util/delay.h>`; harmless to always include.

-

- `Os` (optimize for size) is the AVR default of choice; flash is measured in kilobytes.
- `avr-objcopy` strips the ELF down to the raw `.hex` `avrdude` actually writes to flash.

`avr-size main.elf` shows how much flash/RAM the build uses, worth checking once code grows past a toy example.

2.3.5 WSL2 only: attach the board's USB device

WSL2 doesn't share Windows' USB devices by default. `usbipd-win` bridges the two; this is a one-time install, then a per-session attach.

On Windows (PowerShell, once): install the bridge:

```
winget install usbipd
```

On Windows (PowerShell as Administrator, every time the board is (re)plugged in):

```
usbipd list                              # Find the board's BUSID.
usbipd bind --busid <BUSID>              # One-time per device.
usbipd attach --wsl --busid <BUSID>      # Do this each time you reconnect.
```

Back in WSL2, confirm it showed up:

```
lsusb
ls /dev/ttyACM* /dev/ttyUSB* 2>/dev/null
```

Arduino Uno-style boards (ATmega16U2 as USB-serial bridge) usually show up as `/dev/ttyACM0`; boards using a CH340 or FTDI USB-serial chip (many Nano clones) show up as `/dev/ttyUSB0` instead.

2.3.6 Permissions

Without this, `avrdude` fails with a permissions error on the serial device. Add your user to the `dialout` group (one-time), then start a new shell session for it to take effect:

```
sudo usermod -aG dialout $USER
```

Log out and back in (or just open a new WSL terminal) for the group change to apply.

2.3.7 Flash

Same idea as §2.2: the board runs the Arduino bootloader, which `avrdude`'s `arduino` programmer type talks to over serial.

```
avrdude -c arduino -p atmega328p -P /dev/ttyACM0 -b 115200 -U flash:w:main.hex:i
```

-
- `c arduino` selects the STK500v1-over-serial protocol the Arduino bootloader speaks (this is the same programmer type the `avrdude` command of §2.2 uses via `-carduino`).

-
- `p atmega328p` must match the `-mmcu` of §2.3.4.

-
- `P` is whatever device §2.3.5 (or, on native Linux, just plugging the board in) produced.

-
- `b 115200` is the Optiboot-era default. Some older Nano boards ship a bootloader that only works at `-b 57600`; if 115200 fails with sync errors, that's the first thing to try instead.

A successful run ends with something like:

```
avrdude: 194 bytes of flash written
avrdude: safemode: Fuses OK
avrdude done. Thank you.
```

The LED on D9 should now light up while the button on D13 is held, exactly like the result of §2.2, just reached without Windows or an IDE.

2.3.8 Optional: a Makefile

Ties §2.3.4 and §2.3.7 together to replace the manual commands:

- Call this file `Makefile`.

- Run `make` to compile the code.
- Run `make flash` to flash the firmware.

Adjust `PORT` for your setup:

```
MCU   := atmega328p
F_CPU := 16000000UL
PORT  := /dev/ttyACM0
BAUD  := 115200

all: main.hex

main.elf: main.c
    avr-gcc -mmcu=$(MCU) -DF_CPU=$(F_CPU) -Os -std=c11 -Wall -Wextra -o $@ $<

main.hex: main.elf
    avr-objcopy -O ihex -R .eeprom $< $@

flash: main.hex
    avrdude -c arduino -p $(MCU) -P $(PORT) -b $(BAUD) -U flash:w:$<:i

clean:
    rm -f main.elf main.hex

.PHONY: all flash clean
```

2.4 Review

What you should be able to do now

After this chapter, you should:

- Understand why embedded systems are programmed close to the hardware, in C, without an OS underneath.
- Be able to read and write bitwise operations (`|`, `&`, `~`, `<<`, `>>`) to set, clear, and read individual bits in a register.
- Be able to configure a pin as input or output and drive/read it correctly.

Questions to test yourself

You should be able to explain:

1. What's the difference between `DDRx`, `PORTx`, and `PINx`, and when do you write to one versus read from another?

2. How do you set bit 3 of a register without disturbing the other bits? How do you clear it?
3. Why does an input pin need its pull-up resistor enabled, and what would you expect to happen if you left it disabled with nothing else connected to the pin?

Looking ahead

Chapter 3 is about interrupts:

- Interrupts: reacting to events instead of polling for them.
- External interrupts (INT0/INT1) and Pin Change Interrupts.

2.5 Exercises

> **How to work these.** The Reflection exercise is answered on paper, from the chapter. Exercise 2.1 needs no board: as its tip shows, a small program on your own computer checks the expressions. The other Code exercises are programs for the board, and this is the chapter that sets up the toolchain for them: build and flash them the way §2.2 (Microchip Studio) or §2.3 (Linux) describes, or run them in the simulator of Appendix A if you have no board yet.
> **How to check your work.** The Reflection exercise, and the register values the expressions of Exercise 2.1 should produce, are answered in Appendix B. The Code exercises have no printed solution: the board tells you whether a program does what the exercise describes. Try each exercise before you read its answer.

EXERCISE SET 1

Bit manipulation

Exercise 2.1 Set, clear and read a bit

CODE

Given a register currently holding `0b10110010`, write single-line C expressions (using `|`, `&`, `~`, `<<`) to:

a) Set bit 0 without affecting any other bit.

b) Clear bit 5 without affecting any other bit.

c) Read whether bit 4 is set, as a boolean expression.

Tip: You don't need real hardware to test this; write a small program on your own machine and check the results by invoking `printf()` from `<stdio.h>`:

```
#include <stdint.h>
#include <stdio.h>
```

```c
int main(void)
{
    const uint8_t reg = (1U << 3U);
    printf("%u\n", reg);
    return 0;
}
```

Exercise 2.2 LED macros

CODE

A macro is defined as `#define LED 6U`. Write the two macros `LED_ON` and `LED_OFF`, operating on a register called `PORTD`, following the OR/AND-NOT pattern used in the chapter.

EXERCISE SET 2

Registers

Exercise 2.3 Which register?

REFLECTION

For each of the following, name the correct register (`DDRx`, `PORTx`, or `PINx`) and explain in one sentence why:

a) Configuring D5 as an output.

b) Reading whether a button on D2 is currently pressed.

c) Enabling the internal pull-up resistor on D7 configured as an input.

EXERCISE SET 3

Program

Exercise 2.4 Two buttons, two LEDs

CODE

Wire up LEDs and buttons as described below:

- `LED1` on D6 and `LED2` on D7.
- `BTN1` on D2 and `BTN2` on D3.

Write a complete program, from scratch, that:

- Lights `LED1` if `BTN1` is pressed and turns it off otherwise.
- Lights `LED2` if `BTN2` is released and turns it off otherwise.

Don't copy the chapter's example directly; use these different pin numbers and re-derive the macros yourself.

Exercise 2.5 Cycling three LEDs

CODE

Wire up LEDs and buttons as described below:

- `LED1` on D5, `LED2` on D6, and `LED3` on D7.
- `BTN1` on D4.

Write a complete program, from scratch, that:

- While `BTN1` is held, cycles through the LEDs one at a time, in order (`LED1` → `LED2` → `LED3` → `LED1` …), lighting exactly one for 100 ms before moving to the next.
- Turns off all LEDs otherwise.

CHAPTER 3

Interrupts

Letting the hardware say when something has happened, instead of asking it over and over.

In this chapter

- Why polling doesn't scale, and what an interrupt actually is.
- External interrupts: INT0/INT1 (`SREG`, `EICRA`, `EIMSK`), edge selection, and the `ISR`/vector mechanism.
- Worked example: toggle an LED via INT0.
- Worked example: blink an LED via INT1.
- Pin Change Interrupts (PCI): any other pin, at the cost of per-port grouping and both-edges triggering.
- Worked example: toggle an LED via a PCI interrupt, and the general pattern for any port.
- When to reach for INT0/INT1 versus PCI.

Every input the book has read so far has been polled. This chapter is about the alternative: why interrupts exist, and what problem they solve compared to polling. You will configure an external interrupt, INT0 or INT1, for a specific edge and write its ISR; configure a Pin Change Interrupt on an arbitrary pin, including working out the correct `PCINTx` number; and see why PCI interrupts fire on both edges, and how to filter for the one you care about.

Read §3.1 first: it is the chapter's core material, with its worked examples. Then work the exercises in §3.3. The Reflection and Calculation exercises are answered in Appendix B; the Code exercises have no printed solution, because the board tells you whether a program does what the exercise asks.

3.1 Interrupts

3.1.1 Why interrupts

- Every input we've read so far has been polled: the `while` loop checks `BTN1_PRESSED` every iteration. Fine for one button, but it wastes cycles and doesn't scale; imagine polling a

dozen inputs, or needing sub-millisecond reaction time on one of them while the rest of the program does something else entirely.

- An interrupt lets a hardware event (button edge, timer overflow, ADC done, UART byte received…) suspend whatever the CPU is doing, run a dedicated handler (an **Interrupt Service Routine**, or **ISR**), and then resume exactly where it left off. It's the embedded equivalent of an event handler, except the "event loop" is the CPU itself.
- Analogy: an MP3 player's main loop plays the current track. Pressing "next" doesn't get checked between every audio sample; it interrupts playback, the ISR queues the next track, and playback resumes.
- Because you don't control *when* an ISR fires, keep them short: flip a flag or update a small piece of state, and let `main()`'s loop do the actual work. We'll follow that convention throughout.

3.1.2 External interrupts: INT0 / INT1

The ATmega328P has two pins wired straight to dedicated interrupt logic: **INT0** on D2 (`PORTD2`) and **INT1** on D3 (`PORTD3`).

Three things to configure:

1. **Global interrupt enable**, the `I` bit in the status register `SREG`. `<avr/interrupt.h>` provides `sei()/cli()` for this, thin wrappers around the `SEI/CLI` assembly instructions:

```
sei(); // Enable interrupts globally.
cli(); // Disable interrupts globally.
```

2. **Edge/level selection**, per interrupt, in `EICRA` (bits `ISCn1:ISCn0`):

`ISCn1:ISCn0`	Triggers on
`00`	pin is low
`01`	any logic change
`10`	falling edge (high → low)
`11`	rising edge (low → high)

```
EICRA = (1U << ISC00) | (1U << ISC01); // Enable INT0 on rising edge.
```

3. **Enable the specific interrupt** in `EIMSK`:

```
EIMSK = (1U << INT0);
```

The handler itself is a function named `ISR`, parameterized by a *vector* that ties it to a specific interrupt source, `INT0_vect` for INT0, `INT1_vect` for INT1. No prototype, no explicit call site; the vector is how the linker wires it up.

```
ISR(INT0_vect)
{
    // Runs on the configured edge, then execution resumes exactly where the main ↪
    ↪ program
    // was interrupted.
}
```

3.1.3 Example: toggle an LED via INT0

Button on D2 (INT0, rising edge), LED on D9. Every press flips the LED.

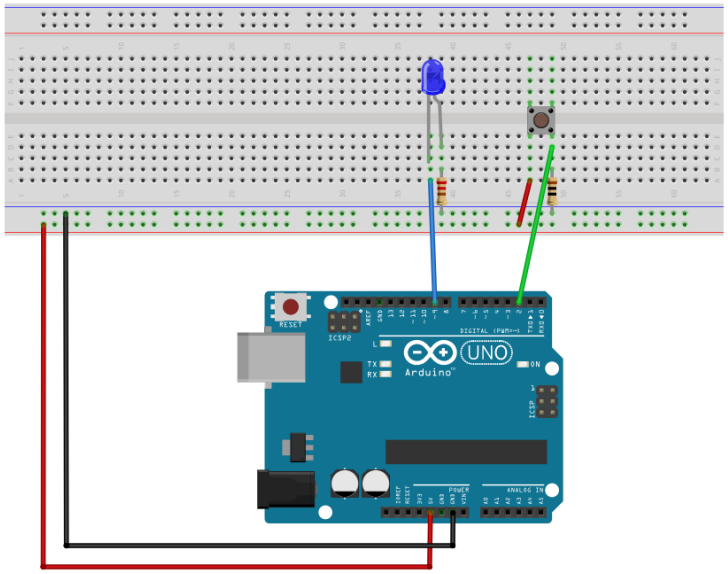


Figure 3.1 A button on D2 (INT0) and an LED on D9.

```
#include <avr/interrupt.h>
#include <avr/io.h>

/** GPIO pins. */
#define LED1 1U // D9  -> PORTB1.

/** GPIO operations. */
#define LED1_TOGGLE PINB = (1U << LED1) // Toggle LED1.

/**
```

```
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Enable rising edge interrupt on INT0 (D2) => button pressdown will cause an↩
    ↪  interrupt.
    EICRA = (1U << ISC00) | (1U << ISC01);
    EIMSK = (1U << INT0);

    // Enable interrupts globally.
    sei();
}

/**
 * @brief Toggle the LED on button event.
 */
ISR(INT0_vect) { LED1_TOGGLE; }

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();
    while (1) {}
    return 0;
}
```

Why `LED1_TOGGLE` writes to `PINB`, not `PORTB`. `PINx` registers are normally read-only: reading `PINB` returns the actual logic level currently on each of Port B's pins, regardless of whether they're configured as inputs or outputs. But writing a `1` to a `PINxn` bit is a documented exception: on this chip (and other modern AVR cores), it *toggles* the corresponding `PORTxn` bit instead of doing nothing, independent of that pin's `DDRxn` setting. Only the bits written as `1` are affected; any `0` bits in the write leave their `PORTx` counterpart untouched.

This isn't a hack, it's a deliberate hardware feature, and it exists because a toggle written the "obvious" way, `PORTB ^= (1U << LED1);`, is a read-modify-write: the CPU reads `PORTB`, flips the bit, then writes the result back. If an interrupt changes another bit in `PORTB` between that read and that write, the interrupt's change gets silently overwritten once the write

completes. Writing to `PINB` instead needs no read at all, so there's no window for that race; the toggle is atomic no matter what else might be touching `PORTB`.

INT1 on D3 works identically for configuration; swap `ISC00`/`ISC01` for `ISC10`/`ISC11`, `INT0` for `INT1` in `EIMSK`, and `INT0_vect` for `INT1_vect`. §3.1.4 walks through a full example.

3.1.4 Example: blink an LED via INT1

Button on D3 (INT1, falling edge, i.e. on release), LED on D9. Each press/release cycle toggles between the LED blinking every 100 ms and staying off.

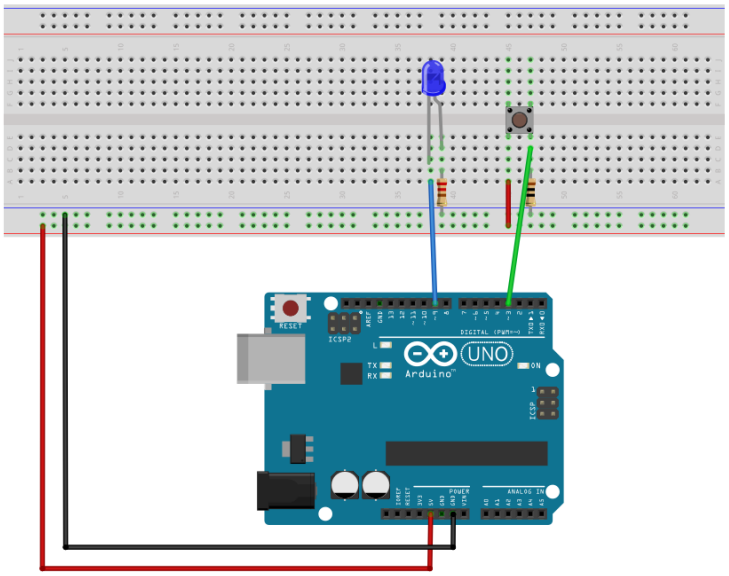


Figure 3.2 A button on D3 (INT1) and an LED on D9.

```c
#define F_CPU 16000000UL // CPU frequency in Hz.

#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>
#include <util/delay.h>

/** GPIO pins. */
#define LED1 1U // D9 -> PORTB1.
```

```c

/** GPIO operations. */
#define LED1_ON PORTB |= (1U << LED1)   // Enable LED1.
#define LED1_OFF PORTB &= ~(1U << LED1) // Disable LED1.

/** Time parameters. */
#define TICK_1MS 1U         // Generate a 1 ms tick in delay_ms().
#define BLINK_SPEED_MS 100U // Blink speed in ms.

/** Blink state (true = blinking, false = off); shared with the ISR, see below. */
static volatile bool blink_state = false;

/** Current blink speed in ms; also shared with the ISR. */
static volatile uint16_t blink_speed_ms = 0U;

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Enable falling edge interrupt on INT1 (D3).
    EICRA = (1U << ISC11);
    EIMSK = (1U << INT1);

    // Enable interrupts globally.
    sei();
}

/**
 * @brief Generate delay.
 *
 * @param[in] ms Pointer to the duration in ms.
 *
 * @note A pointer is used so that the duration can be set to 0 on blink state ↩
    ↪ change.
 */
static void delay_ms(const volatile uint16_t* ms)
{
    // Null check included to demonstrate the practice; ms is only ever passed as
    // &blink_speed_ms below, so this never actually triggers. Both this check and↩
    ↪  the
```

```
    // <stddef.h> include it needs (for NULL) can be omitted.
    if (NULL == ms) { return; }

    for (uint16_t i = 0U; i < *ms; ++i)
    {
        _delay_ms(TICK_1MS);
    }
}

/**
 * @brief Blink the LED.
 */
static void led_blink(void)
{
    LED1_ON;
    delay_ms(&blink_speed_ms);
    LED1_OFF;
    delay_ms(&blink_speed_ms);
}

/**
 * @brief Toggle the blink state on button event, update the blink speed ↩
    ↪ accordingly.
 */
ISR(INT1_vect)
{
    blink_state    = !blink_state;
    blink_speed_ms = blink_state ? BLINK_SPEED_MS : 0U;
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        if (blink_state) { led_blink(); }
        else { LED1_OFF; }
    }
```

```
    return 0;
}
```

Why `blink_state` and `blink_speed_ms` are `volatile`. Both are written inside `ISR(INT1_vect)` and read elsewhere: `blink_state` in `main()`'s loop, `blink_speed_ms` via the pointer `delay_ms()` dereferences. Without `volatile`, the compiler is free to assume a variable never changes on its own between two reads in the same function, and is allowed to cache it in a register once instead of re-reading memory every time; it has no way to know an interrupt can run between those reads and change the value. Skipping `volatile` here wouldn't necessarily fail right away, either: the loop might work fine at low optimization levels and only start ignoring button presses once a build enables optimizations aggressive enough to actually act on that caching assumption, which is exactly the kind of bug that's miserable to track down later. `volatile` tells the compiler to always re-read (or re-write) the variable from memory, never to assume its value between accesses. Chapters 5 and 6 both reuse this same pattern for their own ISR-shared variables.

Why `delay_ms()` takes a pointer, not a plain `uint16_t`. §1.1.7 introduced pointers as C's way to let a function reach into the caller's own object instead of working on a private copy, there, so a function could write an out-parameter (`*x = 3;`). This is the same mechanism, used the other way around: `delay_ms(&blink_speed_ms)` doesn't hand the function `blink_speed_ms`'s value, it hands over its *address*, so every `*ms` inside the loop re-reads whatever `blink_speed_ms` currently holds, not a value frozen at the moment `delay_ms()` was called.

That distinction matters here specifically because `blink_speed_ms` can change out from under `delay_ms()` mid-delay, the ISR overwrites it the instant the button is released. Had `delay_ms()` taken `const uint16_t ms` by value instead, it would have received a private copy at call time and stayed blind to the ISR's update for the rest of the delay. With a pointer, the very next `i < *ms` check after the ISR runs sees the new value, so a press/release is noticed within about one 1 ms tick instead of waiting for the whole delay to finish first.

This live-reading trick relies on the same `volatile` guarantee explained above: without it, the compiler could assume `*ms` never changes across loop iterations and cache it in a register instead of re-reading memory each time, silently defeating the exact behavior this pointer parameter exists to enable.

The blink itself still uses a blocking `_delay_ms()` loop, same as the hand-rolled delay of §2.1.6; this chapter only changes *how* the toggle is triggered, not how the blink timing works. Non-blocking timing is the subject of Chapter 5, on timers.

3.1.5 Pin Change Interrupts (PCI) for any other pin

INT0/INT1 only cover two physical pins. Every other pin can still trigger an interrupt via **Pin Change Interrupts**, at the cost of some extra bookkeeping:

- PCI interrupts are grouped **by port, not by pin**: all of `PORTB` shares one vector (`PCINT0_vect`), all of `PORTC` shares another (`PCINT1_vect`), all of `PORTD` a third (`PCINT2_vect`). Enable PCI on two pins in the same port and both fire the *same* ISR; the vector alone doesn't tell you which pin changed.
- PCI fires on **both** edges (press and release); there's no edge-select register like `EICRA` for it. If you only care about one edge, check the pin's actual level inside the ISR and ignore the call otherwise.

Configuration, per port:

Port	Enable bit (`PCICR`)	Mask register	Bit-name formula
B (D8–D13)	`PCIE0`	`PCMSK0`	`PCINTx = PORTBx`
C (A0–A5)	`PCIE1`	`PCMSK1`	`PCINTx = PORTCx + 8`
D (D0–D7)	`PCIE2`	`PCMSK2`	`PCINTx = PORTDx + 16`

Example: button on D13 (`PORTB5`) → `PCINT5`.

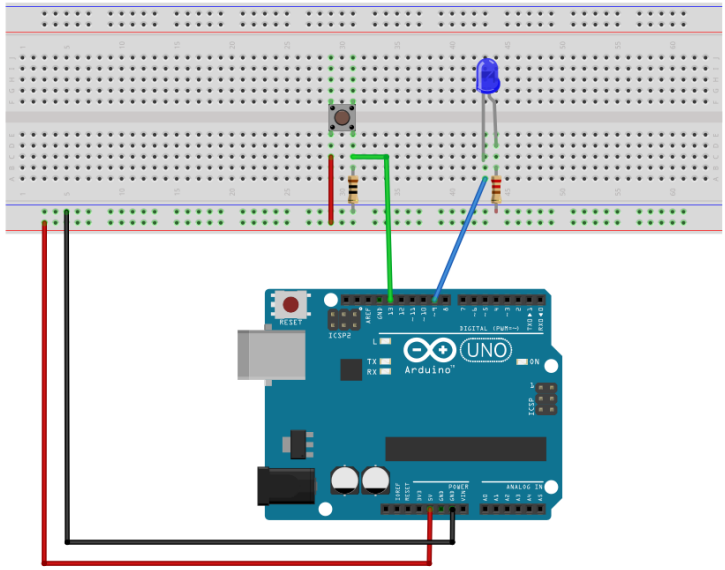


Figure 3.3 A button on D13 (`PCINT5`) and an LED on D9.

```
#include <avr/interrupt.h>
#include <avr/io.h>

/** GPIO pins. */
#define LED1 1U // D9  -> PORTB1.
```

```c
#define BTN1 5U // D13 -> PORTB5.

/** GPIO operations. */
#define LED1_TOGGLE PINB = (1U << LED1)    // Toggle LED1.
#define BTN1_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↩
    ↪ otherwise.

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Configure BTN1 as input with its internal pull-up enabled.
    PORTB = (1U << BTN1);

    // Enable pin change interrupt on D13 (for BTN1).
    PCICR  = (1U << PCIE0);
    PCMSK0 = (1U << BTN1);

    // Enable interrupts globally.
    sei();
}

/**
 * @brief Toggle the LED on rising button edge.
 */
ISR(PCINT0_vect)
{
    if (BTN1_PRESSED) { LED1_TOGGLE; }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();
    while (1) {}
    return 0;
```

```
}
```

3.1.6 PCI on any port: the general pattern

The port B example of §3.1.5 generalizes directly to ports C and D: only the enable bit, mask register, and vector change, following the same formula table from §3.1.5.

Port	Enable (`PCICR`)	Mask register	Vector
B	`PCICR = (1U << PCIE0);`	`PCMSK0 = (1U << PCINTx);`	`ISR(PCINT0_vect)`
C	`PCICR = (1U << PCIE1);`	`PCMSK1 = (1U << PCINTx);`	`ISR(PCINT1_vect)`
D	`PCICR = (1U << PCIE2);`	`PCMSK2 = (1U << PCINTx);`	`ISR(PCINT2_vect)`

Worked example: a button on D2 (`PORTD2`) needs port D's enable bit and mask register, and its ISR is tied to `PCINT2_vect`:

```
#define BTN2 2U // D2 -> PORTD2.

static void setup(void)
{
    // ...
    PCICR = (1U << PCIE2);  // Enable PCI on port D.
    PCMSK2 = (1U << BTN2);  // Specifically D2 / PORTD2.
    sei();                  // Enable interrupts globally.
}

ISR(PCINT2_vect) // Shared by every enabled PCI pin on port D.
{
    // ...
}
```

`PCMSKn`'s bit position always matches the pin's own bit position within its port (`DDRx`/`PORTx`/`PINx`); the `PCINTx = PORTDx + 16`-style formula of §3.1.5 only matters for computing the *global* `PCINTx` number (useful for looking a pin up in the datasheet), not for what bit to actually set in `PCMSKn`. So `PCMSK2 = (1U << BTN2)` above sets exactly the same bit as avr-libc's named `PCMSK2 = (1U << PCINT18)` would; defining your own pin-bit macro, the same convention §3.1.5 already used for `BTN1`, skips the global-number lookup entirely. Same idea for a button on D7 (`PORTD7`): `PCMSK2 = (1U << 7)` sets the same bit as `(1U << PCINT23)`, no formula needed.

The same button on A0 (`PORTC0`) instead needs `PCIE1` / `PCMSK1`, and an `ISR(PCINT1_vect)`.

3.1.7 Choosing between INT0/INT1 and PCI

- Need a specific edge (rising *or* falling, not both) and don't mind being limited to D2/D3? Use INT0/INT1; it's simpler, and the edge filtering happens in hardware.
- Need any other pin, or don't care which edge? Use PCI, and filter edges yourself by reading the pin level in the ISR, as above.
- Multiple PCI sources on the same port share an ISR, and if you have more than one, the handler needs to work out which pin actually changed (e.g. by keeping a "previous state" byte and XOR-ing it against the current `PINx` read to see which bit flipped); §3.1.8 shows how.

3.1.8 Example: three buttons on one PCI vector

§3.1.5's ISR gets away with checking the pin level, because it has only one button on the port. Add more buttons and that's no longer enough. Here, three buttons each toggle their own LED: BTN1 on D11 toggles LED1 on D8, BTN2 on D12 toggles LED2 on D9, and BTN3 on D13 toggles LED3 on D10. All six pins are on port B, so all three buttons fire the same `PCINT0_vect`.

The circuit is the one from Exercise 3.8; this example doesn't use the buttons on D2 and D3.

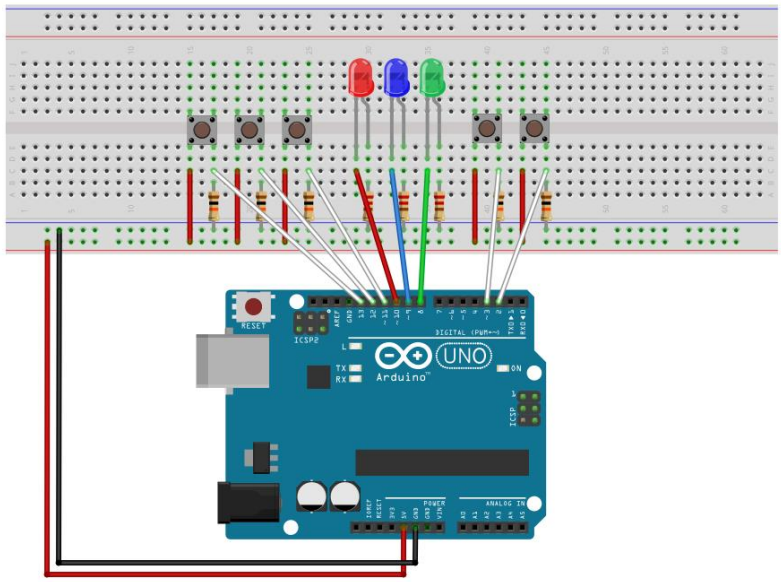


Figure 3.4 Three LEDs on D8–D10 and three buttons on D11–D13; the buttons on D2 and D3 are unused here.

With a level check like §3.1.5's, each button's test passes whenever that button is down, whichever pin caused the interrupt. Hold BTN2 down and press BTN1: the ISR runs once, for BTN1, but both pins read pressed, so both LED1 and LED2 toggle. Release BTN1 and the ISR runs again; BTN2 still reads pressed, so its LED toggles once more.

The fix is to remember what the buttons looked like the last time the ISR ran, and compare:

1. Keep the previous button states in a `static volatile` local variable inside the ISR, `prev`. A `static` local is created once and keeps its value from one call to the next, like a global, but only the ISR can see it, and the ISR is the only place it's needed. It starts at `0`, which matches all three buttons idling low. `volatile` follows this book's habit of marking every variable an ISR writes; since nothing outside the ISR touches `prev`, the caching problem explained in §3.1.4 can't actually happen to it, but the habit costs next to nothing.
2. Read `PINB` once into `now`. Read it only once: a button could change between two separate reads, and the tests would then disagree. `PINB` shows every pin of port B, the LED outputs included, so only the button bits of the result are tested; a changing LED bit is ignored.
3. XOR `prev` with `now`: each bit that's `1` in the result is a button that changed since the last interrupt. AND that with `now` to keep only the buttons that changed *and* are now high, the ones just pressed; releases drop out.
4. Store `now` in `prev` for the next interrupt, then toggle the LED of each button found pressed.

```
#include <avr/interrupt.h>
#include <avr/io.h>

#define LED1 0U // D8  -> PORTB0.
#define LED2 1U // D9  -> PORTB1.
#define LED3 2U // D10 -> PORTB2.

#define BTN1 3U // D11 -> PORTB3.
#define BTN2 4U // D12 -> PORTB4.
#define BTN3 5U // D13 -> PORTB5.

#define LED1_TOGGLE PINB = (1U << LED1) // Toggle LED1.
#define LED2_TOGGLE PINB = (1U << LED2) // Toggle LED2.
#define LED3_TOGGLE PINB = (1U << LED3) // Toggle LED3.

/**
 * @brief Interrupt service routine for I/O port B.
 *
 *        Toggle a given LED on rising edge of the corresponding button.
 */
ISR(PCINT0_vect)
{
```

```
    static volatile uint8_t prev = 0U;

    const uint8_t now    = PINB;
    const uint8_t event  = now ^ prev;
    const uint8_t rising = event & now;
    prev                 = now;

    if (rising & (1U << BTN1)) { LED1_TOGGLE; }
    if (rising & (1U << BTN2)) { LED2_TOGGLE; }
    if (rising & (1U << BTN3)) { LED3_TOGGLE; }
}

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LEDs as outputs.
    DDRB = (1U << LED1) | (1U << LED2) | (1U << LED3);

    // Enable pin change interrupts for BTN1-BTN3.
    PCICR  = (1U << PCIE0);
    PCMSK0 = (1U << BTN1) | (1U << BTN2) | (1U << BTN3);
    sei();
}

/**
 * @brief Application entry point.
 */
int main(void)
{
    setup();
    while (1) {}
}
```

Note:

- Nothing outside the ISR uses `prev`, so it doesn't need to be a global, and keeping it local means no other function can change it by mistake.
- A button held down at reset reads as just pressed the first time *any* button fires the ISR, because `prev` starts at `0`. If that matters, keep `prev` at file scope instead and initialize it from `PINB` in `setup()`, before `sei()`, at the cost of a global every function can see.
- The same steps work for any number of buttons on the port: enable each button's bit in `PCMSK0`, and add a test for it after the XOR and AND.

3.2 Review

What you should be able to do now

After this chapter, you should:

- Explain, in your own words, why interrupts exist and what problem they solve compared to polling.
- Be able to configure an external interrupt (INT0 or INT1) for a specific edge and write its ISR.
- Be able to configure a Pin Change Interrupt on an arbitrary pin, including working out the correct `PCINTx` number.
- Understand why PCI interrupts fire on both edges, and how to filter for the one you care about.

Questions to test yourself

You should be able to answer:

1. What does `EICRA` configure that `EIMSK` doesn't, and vice versa?
2. Why do multiple Pin Change Interrupts on the same port share a single ISR, and what does that mean for how you write it?
3. A PCI-based button handler toggles an LED on both press *and* release. What line is missing, and why?

Looking ahead

Chapter 4 is about analog-to-digital conversion:

- Analog-to-digital conversion (ADC): reading a continuous signal instead of a simple high/low.
- A worked example combining what you already know (PCI, bitwise register I/O) with the ADC.

3.3 Exercises

> **How to work these.** The Reflection and Calculation exercises are answered on paper, from the chapter. The Code exercises are programs for the board: build and flash them the way §2.2 (Microchip Studio) or §2.3 (Linux) describes, or run them in the simulator of Appendix A if you have no board yet.
> **How to check your work.** The Reflection and Calculation exercises are answered in Appendix B. The Code exercises have no printed solution: the board tells you whether a program does what the exercise describes. Try each exercise before you read its answer.

EXERCISE SET 1

Concepts

Exercise 3.1 Polling versus interrupts

REFLECTION

In your own words (2–3 sentences), explain why polling `BTN1_PRESSED` in a `while` loop scales poorly once you have several inputs to watch, and how interrupts solve that.

Exercise 3.2 `EICRA` and `EIMSK`

REFLECTION

What does `EICRA` configure that `EIMSK` doesn't? Could you enable INT0 without touching `EICRA` at all; what would happen if you did?

EXERCISE SET 2

INT0/INT1

Exercise 3.3 Moving the button to INT1 CODE

Rewire the INT0 example of §3.1.3 (button on D2) to use **INT1** on D3 instead. Write out the three configuration lines that change.

Exercise 3.4 The wrong edge REFLECTION

A colleague configures `EICRA` for a falling edge instead of rising, but leaves the rest of the INT0 example unchanged, and complains the LED toggles when the button is *released*, not pressed. Explain what happened.

EXERCISE SET 3

Pin Change Interrupts

Exercise 3.5 An interrupt on D4 CALCULATION

A project needs an interrupt on D4. Work out, from the formula table in §3.1.5, which `PCINT` number that is, which port group it belongs to, and which three registers/bits need to be touched to enable it.

Exercise 3.6 Reading `PINB` once REFLECTION

§3.1.8's ISR reads `PINB` once into `now` and uses that copy everywhere: in the XOR, in the AND, and when updating `prev`. Suppose it read `PINB` afresh in each of those three places instead. Describe a button press this version could miss entirely, and explain why the single read can't miss it.

EXERCISE SET 4

Program

Exercise 3.7 A PCI blink toggle CODE

Wire up an LED on D6 and a button on D4. Using a Pin Change Interrupt (not INT0/INT1) on the button, write a complete program, from scratch, that:

- Starts with the LED off and blinking disabled.
- Each button press toggles between two states: blinking (LED on/off every 200 ms) and disabled (LED held off).
- The ISR only flags that the button was pressed, filtering the release edge the same way as the PCI example of §3.1.5; the actual state toggle and the blink timing both happen in `main()`.

You'll need a blocking delay for the blink timing itself (`_delay_ms()`, as in Chapter 2); non-blocking delays are the subject of Chapter 5, on timers, not of this one. This chapter's own §3.1.4 (blink an LED via INT1) is the closest worked example to lean on.

Exercise 3.8 An interrupt-driven state machine CODE

Build a 5-state state machine, driven entirely by interrupts, using every kind covered in this chapter at once.

Wire up three LEDs, `LED1` on D8, `LED2` on D9, and `LED3` on D10, plus five buttons. Declare the five states as an `enum` (`STATE_OFF`, `STATE_SLOW`, `STATE_MEDIUM`, `STATE_FAST`, `STATE_ON`), and wire each button as follows, advancing/rewinding with wraparound at both ends of the range:

- `BTN1` on D2 (**INT0**, falling edge) is a reset: jump straight to `STATE_OFF`, ignoring the normal advance/rewind stepping.
- `BTN2` on D3 (**INT1**, rising edge) rewinds the state by two steps.
- `BTN3` on D11 (**PCI**) rewinds the state by one step.
- `BTN4` on D12 (**PCI**) advances the state by one step.
- `BTN5` on D13 (**PCI**) advances the state by two steps.

The three LEDs' behavior depends on the current state:

- `STATE_OFF`: all three LEDs held off.
- `STATE_SLOW` / `STATE_MEDIUM` / `STATE_FAST`: the three LEDs chase in a loop (`LED1` → `LED2` → `LED3` → `LED1` …), lighting exactly one at a time, holding each for 500 ms / 250 ms / 50 ms respectively.
- `STATE_ON`: all three LEDs held on.

As with every other example in this chapter, keep each ISR minimal: flag which button fired (and, for the three PCI buttons sharing one vector, work out *which* pin changed, as in §3.1.8), and let `main()` do the actual work of updating `state` and driving the LEDs.

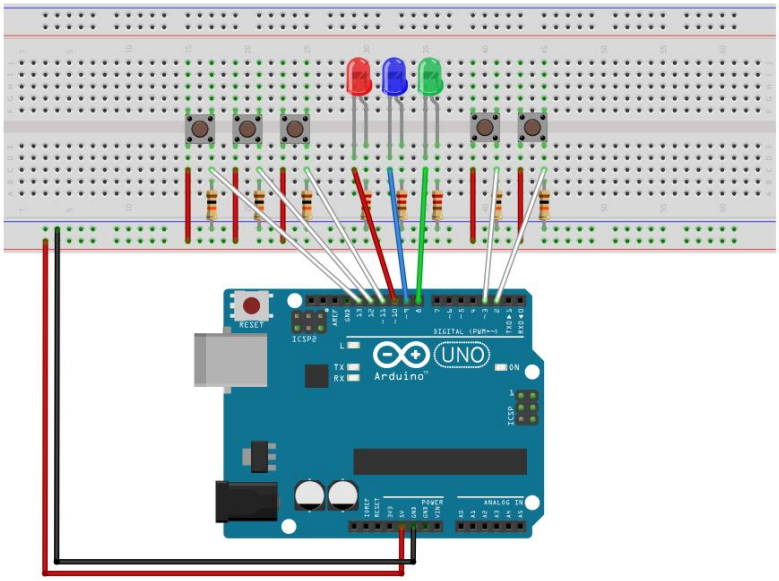


Figure 3.5 Three LEDs on D8–D10 and five buttons, on D2, D3 and D11–D13.

CHAPTER 4

Analog-to-Digital Conversion

Reading a voltage as a number, and turning a number back into brightness.

IN THIS CHAPTER

- Why ADC: reading continuous voltages instead of a simple high/low.
- The 10-bit ADC on the ATmega328P and the linear scaling formula.
- Registers: `ADMUX`, `ADCSRA`, and the combined `ADC` result register.
- A reusable `adc_read()` function.
- Worked example: potentiometer-controlled LED brightness via software PWM.

Until now every input has been a pin that is high or low. This chapter is about the analog-to-digital converter: what an ADC does, and why its result is a discrete number, not a continuous value. You will compute the voltage a given ADC reading corresponds to, and vice versa; configure and read from a chosen ADC channel; and see why the ADC needs its own clock prescaler, separate from the CPU clock.

Read §4.1 first: it is the chapter's core material, and it ends with a worked example that combines the ADC with the Pin Change Interrupts of Chapter 3. Then work the exercises in §4.3. The Reflection and Calculation exercises are answered in Appendix B; the Code exercises have no printed solution, because the board tells you whether a program does what the exercise asks.

4.1 Analog-to-Digital Conversion (ADC)

4.1.1 Why ADC

- Everything so far has been binary: a pin is high or low. Real-world signals, like a potentiometer, a temperature sensor, or a microphone, vary continuously. An ADC (Analog-to-Digital Converter) samples that continuous voltage and returns a discrete number representing it.
- The ATmega328P has a 10-bit ADC: the result is an integer between `0` and `1023` ($2^{10} - 1$), representing an input voltage between `0 V` and whatever reference voltage you configure

(normally the chip's own supply, `AVCC`, 5 V on the Uno/Nano; §4.1.2 shows how to select it).

The mapping is linear:

```
result = 1023 * (V_in / V_ref)
```

So `V_in = 0 V` → `0`, `V_in = V_ref` → `1023`, `V_in = V_ref / 2` → `~512`. More bits would mean finer resolution for the same voltage range; 10 bits is enough for most sensor work, which is why the chip stops there.

The datasheet writes the same mapping with `1024` instead of `1023`, counting full scale as `V_ref` itself, so that the top code, `1023`, means one step below `V_ref`. The two formulas never differ by more than one step (about 5 mV at a 5 V reference); this book uses `1023` throughout, so that `1023` converts back to exactly `V_ref`.

There's a flip side to this same problem worth previewing: this chip's outputs are just as binary as its inputs, a pin is either driving high or low, nothing in between. So how do you make an LED look dimmer instead of just on or off? By toggling it faster than the eye can follow and varying how much of each cycle it spends on versus off. That ratio, the fraction of one cycle spent "on," is called the **duty cycle**; the **on-time** and **off-time** are just that fraction (and its complement) converted into actual milliseconds for a chosen cycle length. §4.1.4 combines both ideas: an ADC reading drives the duty cycle, so turning a potentiometer changes how bright an LED appears.

4.1.2 Which pin, which registers

The chip exposes 8 analog input channels, `ADC0`–`ADC7` (plus a 9th, `ADC8`, wired to an internal temperature sensor). On the Uno/Nano, `ADC0`–`ADC5` map to `A0`–`A5`; you select a channel by writing its number into the low 4 bits of `ADMUX`.

Three registers matter:

1. **`ADMUX`** selects the input channel (bits 0–3) and the voltage reference (bits 6–7). Setting `REFS0` (and leaving `REFS1` clear) selects the supply voltage `AVCC` as the reference, the normal choice. Leave both clear, as they are after a reset, and the reference is whatever is connected to the `AREF` pin instead, which on an Uno/Nano is nothing at all:

   ```
   // Use AVCC as voltage reference, select ADC channel (0 for A0, 1 for A1 etc.)
       .
   ADMUX = (1U << REFS0) | channel;
   ```

2. **`ADCSRA`** enables the ADC, starts a conversion, and sets its clock prescaler. The ADC needs a clock well below the CPU's; under ~200 kHz for full accuracy. At 16 MHz, a ÷128 prescaler gets you to 125 kHz:

```
// Enable the ADC (ADEN), start a conversion (ADSC), set prescaler to 128 (↩
    ↪ ADPSx).
ADCSRA = (1U << ADEN) | (1U << ADSC) | (1U << ADPS0) | (1U << ADPS1) | (1U << ↩
    ↪ ADPS2);
```

A conversion isn't instantaneous; poll the `ADIF` flag (or use an interrupt) until it's set, then clear it by writing a `1` back to it:

```
while (0U == (ADCSRA & (1U << ADIF))) {} // Wait for the conversion.
ADCSRA = (1U << ADIF);                   // Clear the flag for next time.
```

That plain = writes every other bit of `ADCSRA` too, `ADEN` included, so it also switches the ADC off until the next read turns it back on. That's harmless in §4.1.3's `adc_read()`, which rewrites the whole register at the start of every read, but it makes every conversion a first conversion after enabling, 25 ADC clock cycles (200 µs at 125 kHz) instead of 13.

3. **`ADC`** holds the result. In C this macro already combines the two underlying 8-bit registers (`ADCH`/`ADCL`) into the full 10-bit value; you don't assemble it by hand. Without it, you'd have to do it yourself, and the read order matters:

```
#define BYTE_SHIFT 8U // Parameter to shift an entire byte.

const uint8_t low = ADCL;  // Read ADCL first: locks ADCH until it's read.
const uint8_t high = ADCH; // Same conversion's result; safe to read now.
const uint16_t result = ((uint16_t)(high) << BYTE_SHIFT) | (uint16_t)(low);
```

Note:

- Reading `ADCH` first risks tearing the result: if a new conversion finishes between the two reads, the two halves can end up belonging to different conversions.
- Reading `ADCL` first locks both registers against further updates until `ADCH` is read too, so the pair always comes from the same conversion; the `ADC` macro already does this for you.
- Both operands are cast to `uint16_t` before combining. The cast on `high` matters for correctness in general, not just for this specific value: `high` is a `uint8_t`, and shifting it left by 8 without first widening it relies on C's automatic promotion to `int`.
- On AVR, `int` is 16 bits, so shifting a *full-range* byte left by 8 (e.g. `(uint8_t)0xFF << 8U`) can produce a value that no longer fits in a 16-bit signed `int`, and left-shifting past what the type can hold is undefined behavior in C.
- `ADCH` never actually uses more than its bottom 2 bits, so this particular shift is safe either way, but casting to `uint16_t` first is the habit that keeps it safe unconditionally: the shift then happens in an unsigned 16-bit type, where the result always wraps predictably instead of risking undefined behavior.

- The cast on `low` isn't load-bearing the same way (`uint8_t` always fits safely in the promoted type), but keeps both operands of `|` explicitly `uint16_t` instead of leaning on C's implicit-conversion rules to sort it out.

4.1.3 A read function

```
/**
 * @brief Read and return a 10-bit ADC conversion result.
 *
 * @param[in] channel ADC channel to read.
 *
 * @return Conversion result (0-1023).
 */
static uint16_t adc_read(const uint8_t channel)
{
    // Use AVCC as voltage reference and select ADC channel to read.
    ADMUX = (1U << REFS0) | channel;

    // Perform conversion, wait for conversion to complete.
    ADCSRA = (1U << ADEN) | (1U << ADSC) | (1U << ADPS0) | (1U << ADPS1) | (1U << ↩
    ↪ ADPS2);
    while (0U == (ADCSRA & (1U << ADIF))) {}

    // Clear the ADC interrupt flag for next conversion, then return the result.
    ADCSRA = (1U << ADIF);
    return ADC;
}
```

4.1.4 Example: potentiometer-controlled LED brightness (software PWM)

There's no true analog output on this chip (it has no digital-to-analog converter), but as §4.1.1 previewed, you can fake brightness control in software instead: flick an LED on and off fast enough that the eye averages it into perceived brightness, driven by duty cycle.

Potentiometer on A0, LED on D9, toggle button on D13 (enables/disables reading; the LED stays off while disabled).

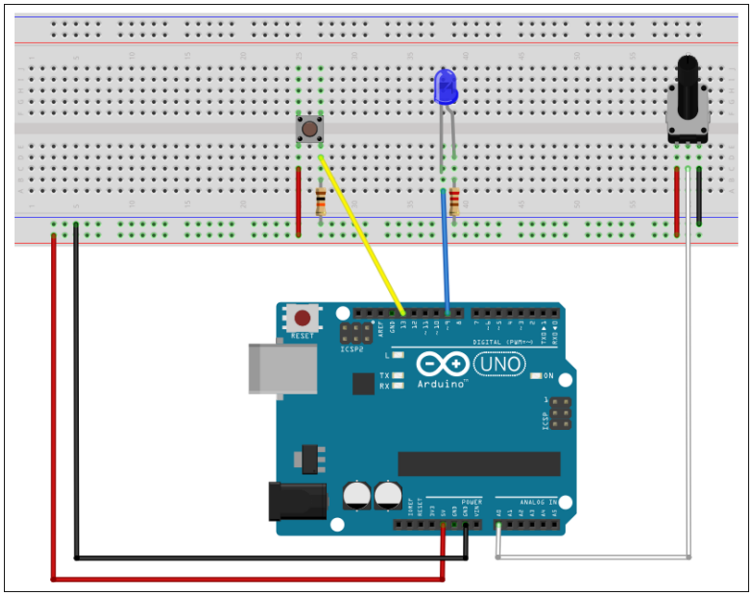


Figure 4.1 A potentiometer on A0, an LED on D9 and a button on D13.

```c
#define F_CPU 16000000UL // CPU frequency in Hz.

#include <stdbool.h>
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>
#include <util/delay.h>

/** GPIO pins. */
#define LED1 1U // D9  -> PORTB1.
#define BTN1 5U // D13 -> PORTB5.
#define POT1 0U // A0  -> ADC0.

/** GPIO operations. */
#define LED1_ON PORTB |= (1U << LED1)      // Enable LED1.
#define LED1_OFF PORTB &= ~(1U << LED1)    // Disable LED1.
#define BTN1_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↪
    ↪ otherwise.

/** Time parameters. */
#define PWM_PERIOD_MS 10U // PWM period in ms.
```

```
#define TICK_1MS 1U       // Generate a 1 ms tick in delay_ms().
#define ADC_MAX 1023U     // Maximum ADC conversion value.

/** ADC state (true = enabled, false = disabled). */
static volatile bool adc_state = false;

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Configure BTN1 as input with its internal pull-up enabled.
    PORTB = (1U << BTN1);

    // Enable pin change interrupts for BTN1.
    PCICR  = (1U << PCIE0);
    PCMSK0 = (1U << BTN1);
    sei();
}

/**
 * @brief Generate delay.
 *
 * @param[in] ms Delay duration in ms.
 */
static void delay_ms(const uint16_t ms)
{
    for (uint16_t i = 0U; i < ms; ++i)
    {
        _delay_ms(TICK_1MS);
    }
}

/**
 * @brief Read and return a 10-bit ADC conversion result.
 *
 * @param[in] channel ADC channel to read.
 *
 * @return Conversion result (0-1023).
 */
static uint16_t adc_read(const uint8_t channel)
```

```
{
    // Use AVCC as voltage reference and select ADC channel to read.
    ADMUX = (1U << REFS0) | channel;

    // Perform conversion, wait for conversion to complete.
    ADCSRA = (1U << ADEN) | (1U << ADSC) | (1U << ADPS0) | (1U << ADPS1) | (1U << ↩
    ↪ ADPS2);
    while (0U == (ADCSRA & (1U << ADIF))) {}

    // Clear the ADC interrupt flag for next conversion, then return the result.
    ADCSRA = (1U << ADIF);
    return ADC;
}

/**
 * @brief Run a single PWM cycle.
 */
static void run_pwm_cycle(void)
{
    // Check if the ADC is enabled, terminate the function if not.
    if (!adc_state)
    {
        LED1_OFF;
        return;
    }

    // Read the ADC and compute the duty cycle.
    const uint16_t adc_val = adc_read(POT1);
    const float duty_cycle = (float)(adc_val) / ADC_MAX;

    // Compute the PWM durations based on the duty cycle, round to the nearest ↩
    ↪ integer.
    const uint8_t on_ms  = (uint8_t)(duty_cycle * PWM_PERIOD_MS + 0.5F);
    const uint8_t off_ms = PWM_PERIOD_MS - on_ms;

    // Enable LED1 during the active duration.
    LED1_ON;
    delay_ms(on_ms);

    // Disable LED1 during the inactive duration.
    LED1_OFF;
    delay_ms(off_ms);
}
```

```
/**
 * @brief Toggle the ADC state on button press.
 */
ISR(PCINT0_vect)
{
    if (BTN1_PRESSED) { adc_state = !adc_state; }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        run_pwm_cycle();
    }
    return 0;
}
```

Turning the potentiometer changes the ADC reading, which changes the computed duty cycle, which changes how bright the LED appears.

Note `delay_ms()` is built on `_delay_ms()`, so it has the same blocking limitation flagged in §2.1.6; this software-PWM approach ties up the CPU for the whole 10 ms period. Chapter 5, on timers, shows the way out: a hardware timer that raises an interrupt when a period has passed, so the CPU is free in between instead of spinning inside `_delay_ms()`. (The timers can also generate PWM entirely in hardware, which this book doesn't cover.)

4.1.5 Integer arithmetic instead of `float`

§4.1.4 computes the duty cycle with a `float`, which reads naturally but costs a lot on this chip. The ATmega328P has no floating-point hardware: every `float` division, multiplication and conversion is carried out in software, by routines from the compiler's support library that the linker adds to your program, and each one takes hundreds of CPU cycles.

The arithmetic here never needs a fraction, though. Multiply before you divide, and the duty cycle comes out as a whole percentage, `0`–`100` instead of `0.0`–`1.0`; one more multiplication and division turns that percentage into milliseconds. The version below does exactly that. It drops §4.1.4's button and reads the potentiometer on A2:

```c
#define F_CPU 16000000UL // CPU frequency (16 MHz).

#include <avr/io.h>
#include <util/delay.h>

/** Pin definitions. */
#define POT1 2U // Pin A2 -> PORTC2.
#define LED1 1U // Pin D9 -> PORTB1.

#define LED1_ON PORTB |= (1U << LED1)   // Enable LED1.
#define LED1_OFF PORTB &= ~(1U << LED1) // Disable LED1.

#define PWM_PERIOD_MS 10U // PWM period in ms (on time + off time).
#define TICK_MS 1U        // Generate a 1 ms delay.
#define ADC_MAX_RAW 1023U // Maximum raw value from an ADC conversion.
#define PERCENT 100U      // Multiplier to get ratio in percent [0, 100] instead ↩
    ↪ of [0.0, 1.0].

/**
 * @brief Read the input of the given analog pin.
 *
 * @param[in] pin Analog pin to read.
 *
 * @return 10-bit raw ADC value in the range [0, 1023].
 */
static uint16_t adc_read(const uint8_t pin)
{
    // Select internal voltage supply and pin.
    ADMUX = (1U << REFS0) | pin;

    // Enable the ADC, start the conversion with the slowest possible conversion ↩
    ↪ time
    // for the best possible result.
    ADCSRA = (1U << ADEN) | (1U << ADSC) | (1U << ADPS2) | (1U << ADPS1) | (1U << ↩
    ↪ ADPS0);

    // Wait for the conversion to complete (the ADC interrupt flag will then be ↩
    ↪ set).
    while (0U == (ADCSRA & (1U << ADIF))) {}

    // Reset the interrupt flag for the next conversion (the hardware won't do it,↩
    ↪  since no
    // interrupt was invoked).
    ADCSRA = (1U << ADIF);
```

```
    // Return the 10-bit ADC result.
    return ADC;
}

/**
 * @brief Generate delay in ms.
 *
 * @param[in] ms Duration in ms.
 */
static void delay_ms(const uint16_t ms)
{
    for (uint16_t i = 0U; i < ms; ++i)
    {
        _delay_ms(TICK_MS);
    }
}

/**
 * @brief Initialize the system.
 */
static void setup(void)
{
    // Configure the LED as output.
    DDRB = (1U << LED1);
}

/**
 * @brief Run PWM on the LED based on the input from the potentiometer.
 */
static void run_pwm(void)
{
    // Get the raw ADC value [0, 1023].
    const uint32_t raw = (uint32_t)(adc_read(POT1));

    // Compute the ratio in the range [0, 100].
    const uint8_t ratio = (raw * PERCENT) / ADC_MAX_RAW;

    // Compute the on time and off time.
    const uint8_t on_time  = (uint8_t)((ratio * PWM_PERIOD_MS) / PERCENT);
    const uint8_t off_time = PWM_PERIOD_MS - on_time;

    LED1_ON;
    delay_ms(on_time);
```

```
    LED1_OFF;
    delay_ms(off_time);
}

/**
 * @brief Application entry point.
 */
int main(void)
{
    setup();

    while (1)
    {
        // Read the pot, control the LED brightness based on the input.
        run_pwm();
    }
}
```

Built with `avr-gcc -Os` for the ATmega328P, the same program computing the on-time with a `float`, as §4.1.4 does, takes 1326 bytes of flash, 4.0 % of the chip's 32 KB; this integer version takes 420 bytes, 1.3 %. Two-thirds of the `float` program is the software floating-point code. (`avr-size -C --mcu=atmega328p main.elf`, from §2.3.4, prints these figures.)

Note:

- `raw` is a `uint32_t` because the largest intermediate result, `1023 * 100 = 102300`, doesn't fit in 16 bits. Without the cast, `adc_read()`'s `uint16_t` result and `PERCENT` (`100U`) would be multiplied as a 16-bit `unsigned int`, AVR's `int` width, and the product would silently wrap past `65535`, giving nonsense duty cycles for every reading above `655`.
- The second line needs no such care: `ratio` is at most `100`, so `ratio * PWM_PERIOD_MS` is at most `1000`, well within 16 bits.
- The order matters. `(raw / ADC_MAX_RAW) * PERCENT` would divide first, and integer division throws away the remainder, so the result would be `0` for every reading except `1023`.
- Integer division truncates (rounds toward zero), where §4.1.4 rounds to the nearest millisecond with `+ 0.5F`. Here every step of the on-time comes half a step later in the potentiometer's travel than in §4.1.4, and the full 10 ms on-time only at a reading of exactly `1023`. To round instead, add half the divisor to the numerator before dividing.

4.2 Review

What you should be able to do now

After this chapter, you should:

- Explain what an ADC does and why its result is a discrete number, not a continuous value.
- Be able to compute the voltage a given ADC reading corresponds to, and vice versa.
- Be able to configure and read from a chosen ADC channel.
- Understand why the ADC needs its own clock prescaler, separate from the CPU clock.

Questions to test yourself

You should be able to answer:

1. At a 5 V reference, what analog voltage does an ADC reading of `0`, `511`, and `1023` each correspond to?
2. Why does the code wait on the `ADIF` flag instead of reading `ADC` immediately after starting a conversion?
3. What is "duty cycle," and how does the software-PWM example use it to control perceived LED brightness?

Looking ahead

Chapter 5 is about timers:

- Timers: replacing blocking `_delay_ms()` calls with non-blocking, interrupt-driven timing.
- Normal Mode vs. CTC Mode, and the prescaler math behind both.

4.3 Exercises

How to work these. The Reflection and Calculation exercises are answered on paper, from the chapter. The Code exercises are programs for the board: build and flash them the way §2.2 (Microchip Studio) or §2.3 (Linux) describes, or run them in the simulator of Appendix A if you have no board yet.
How to check your work. The Reflection and Calculation exercises are answered in Appendix B. The Code exercises have no printed solution: the board tells you whether a program does what the exercise describes. Try each exercise before you read its answer.

EXERCISE SET 1

Concepts

Exercise 4.1 From reading to voltage

CALCULATION

A potentiometer reads `768` from `adc_read()`. Given a 5 V reference, what voltage does that correspond to? Show the calculation.

Exercise 4.2 The ADC's own prescaler

REFLECTION

Why does the ADC need its own clock prescaler, separate from the CPU's? What would you expect to happen to the reading's accuracy if the ADC ran at the full 16 MHz CPU clock instead?

EXERCISE SET 2

Registers

Exercise 4.3 From A0 to A1

REFLECTION

Which single bit would you flip in `ADMUX` to read from `A1` instead of `A0`, assuming the reference voltage stays the same?

Exercise 4.4 Waiting for `ADIF`

REFLECTION

Explain, in your own words, why the code polls the `ADIF` flag in a `while` loop after starting a conversion, rather than reading `ADC` immediately after setting `ADSC`.

EXERCISE SET 3

Program

Exercise 4.5 A threshold switch

CODE

Extend the PWM-brightness example of §4.1.4 so that, instead of software PWM, it simply lights the LED solid whenever the potentiometer reading exceeds `512`, and turns it off otherwise, a basic threshold switch with no PWM. Rewrite `run_pwm_cycle()` (rename it too; it's no longer PWM).

Exercise 4.6 Removing the wait

REFLECTION

What would you expect to happen if the `while (0U == (ADCSRA & (1U << ADIF))) {}` wait were removed entirely from `adc_read()`? Would the program crash outright, or misbehave more subtly? Explain your reasoning.

Exercise 4.7 Five dimmable LEDs

CODE

Capstone. Extend the software-PWM approach of §4.1.4 to dim five LEDs from a single potentiometer, each individually switchable:

- LEDs on D6–D10.
- Buttons on D2, D3, D11–D13:
  - D2/D3 are `INT0`/`INT1` from Chapter 3.
  - D11–D13 share a Pin Change Interrupt, also from Chapter 3.

Each button shall toggle the PWM state of an LED:

- Button D2 toggles LED D6.
- D3 toggles D7.
- D11 toggles D8.
- D12 toggles D9.
- D13 toggles D10.

Only LEDs currently "on" should respond to the potentiometer's duty cycle; "off" LEDs stay dark no matter where the pot is turned.

Reuse `adc_read()` from §4.1.3 for the potentiometer reading and the duty-cycle math from §4.1.4.

- Track which of the five LEDs are enabled using a `uint8_t` bitmask with bitwise tests.
- Drive the on/off timing for all five within one PWM period instead of just one.
- The Pin Change Interrupt runs on any change of D11–D13, so its ISR must work out which button was just pressed; use the `prev`/`now` technique from §3.1.8.

CHAPTER 5

Timers

Letting the hardware keep time, so the CPU is free to do something else.

In this chapter

- Why `_delay_ms()` doesn't scale, and what a hardware timer gives you instead.
- The prescaler math: from CPU clock to tick period to interrupt count.
- Normal Mode vs. CTC Mode.
- Worked example: non-blocking LED blink using Timer 0 (Normal Mode).
- Timer 1 in CTC Mode: hitting an arbitrary period directly.

Every delay so far has been a busy-wait. This chapter explains why a busy-wait delay (`_delay_ms`) is unsuitable once a program needs to do more than one thing at a time, and replaces it with the chip's hardware timers. You will compute a timer's tick period from the CPU clock and a chosen prescaler, work out how many overflow interrupts an 8-bit timer needs for a given delay, and learn the practical difference between Normal Mode and CTC Mode, and when you'd reach for each. Along the way the chapter returns to `volatile`, and to why a variable shared between an ISR and `main()` needs it.

Read §5.1 first: it is the chapter's core material, with its worked examples. Then work the exercises in §5.3. The Reflection and Calculation exercises, and the questions some Code exercises ask, are answered in Appendix B; the Code exercises themselves have no printed solution, because the board tells you whether a program does what the exercise asks.

5.1 Timers

5.1.1 Why timers

In the previous examples, a function `delay_ms()` has been used to generate delays between LED blinks and PWM pulses:

```
/**
 * @brief Generate delay.
```

```
 *
 * @param[in] ms Delay duration in ms.
 */
static void delay_ms(const uint16_t ms)
{
    for (uint16_t i = 0U; i < ms; ++i)
    {
        _delay_ms(TICK_1MS);
    }
}
```

This function works as intended, but it has some flaws:

- `_delay_ms()` blocks: the CPU sits in a busy-loop doing nothing else for the duration. Fine for a two-line demo, not for anything that also needs to poll a button, read a sensor, or respond to interrupts while "waiting."
- To avoid CPU blocks, a timer circuit can be used that flags when the given duration has passed, i.e. when timeout occurs.
- A timer is hardware that counts up on its own, driven by the CPU clock (divided down by a **prescaler**), completely independent of your code. Configure it once and it can either be polled, or, more usefully, fire an interrupt when it reaches a target value, letting you build non-blocking delays and precise periodic events.
- The ATmega328P has three: **Timer 0** and **Timer 2** are 8-bit (count 0–255), **Timer 1** is 16-bit (count 0–65535). Their registers follow a consistent naming pattern; swap the digit (`0`/`1`/`2`) and the names carry over, although, as §5.1.2 and §5.1.5 show, not every bit pattern does (Timer 2's prescaler bits, and the bit that selects CTC mode on Timer 1).

5.1.2 The math: prescaler → tick time → interrupt count

Pick a **prescaler** (a clock divider) to get a manageable tick rate. At 16 MHz with a ÷1024 prescaler:

```
tick frequency = 16,000,000 / 1024 = 15,625 Hz
tick period    = 1 / 15,625 = 0.064 ms
```

For an 8-bit timer, one full count (0→255, i.e. 256 ticks) triggers one **overflow**. To get a specific delay, work out how many overflows you need:

```
ticks needed     = desired_delay_ms / tick_period_ms
overflows needed = ticks needed / 256          (8-bit timer)
```

E.g. a 100 ms delay at 0.064 ms/tick needs ~1562 ticks ≈ 6 overflows. You don't have to hit this exactly with a single hardware compare; instead count overflow interrupts in software and act once you've counted enough. That's the pattern in the example below.

Prescaler bit combinations for Timer 0 / Timer 1 are shown below:

Prescaler bits	Divisor	Tick @ 16 MHz
`001`	1	62.5 ns
`010`	8	0.5 µs
`011`	64	4 µs
`100`	256	16 µs
`101`	1024	64 µs

Timer 2's prescaler bit values differ somewhat:

Prescaler bits	Divisor	Tick @ 16 MHz
`001`	1	62.5 ns
`010`	8	0.5 µs
`011`	32	2 µs
`100`	64	4 µs
`101`	128	8 µs
`110`	256	16 µs
`111`	1024	64 µs

5.1.3 Normal Mode vs. CTC Mode

There are two modes that will be covered in this chapter:

- **Normal Mode**: the timer always counts to its full range (255 or 65535) before overflowing and firing `TIMERn_OVF_vect`.
- **CTC Mode** (Clear Timer on Compare): the timer counts up to a value *you* choose (written to `OCRnA`), fires `TIMERn_COMPA_vect`, and resets to 0, useful when the period you want doesn't cleanly divide into 256 or 65536 ticks.

Both need the same steps: set the prescaler (which also starts the timer), enable the timer's own interrupt in `TIMSKn`, and enable interrupts globally. CTC additionally needs the compare value and the mode bit.

5.1.4 Non-blocking blink using Timer 0 (Normal Mode)

LED on D9 toggles approximately every 500 ms (31 overflows of 16.384 ms each, 508 ms); no `_delay_ms()` anywhere, so the rest of `main()`'s loop stays free to do other work. The circuit is Chapter 2's first one, Figure 2.3; its button on D13 isn't used here.

```c
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>

/** GPIO pins. */
#define LED1 1U // D9 -> PORTB1

/** GPIO operations. */
#define LED1_TOGGLE PINB = (1U << LED1) // Toggle LED1.

/** Time parameters. */
#define TICK_PERIOD_MS 0.064F // Tick period, prescaler 1024.
#define TICK_MAX 256U         // Ticks per overflow when using an 8-bit timer.
#define TIMEOUT_MS 500U       // Timeout in ms.

/** Limit parameters; ticks and overflows needed for the desired timeout. */
#define OVF_PERIOD_MS (TICK_PERIOD_MS * TICK_MAX)            // Time between each ↩
    ↪ overflow in ms.
#define OVF_MAX (uint8_t)(TIMEOUT_MS / OVF_PERIOD_MS + 0.5F) // Timeout measured ↩
    ↪ in overflows.

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Enable overflow interrupt for timer 0, use prescaler 1024.
    TCCR0B = (1U << CS00) | (1U << CS02);
    TIMSK0 = (1U << TOIE0);
    sei();
}

/**
 * @brief Count overflows, toggle LED1 on timeout.
 */
ISR(TIMER0_OVF_vect)
{
    static volatile uint8_t ovf_count = 0U;
```

```
    // Increment the overflow counter, toggle LED1 and clear the counter on ↩
    ↪ timeout.
    if (OVF_MAX <= ++ovf_count)
    {
        LED1_TOGGLE;
        ovf_count = 0U;
    }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();
    while (1) {}
    return 0;
}
```

`OVF_PERIOD_MS` and `OVF_MAX` are the calculation macros covered in §1.1.12: `OVF_PERIOD_MS` parenthesizes its whole body the way `TICKS_NEEDED` does there, and `OVF_MAX` gets the same safety from its leading `(uint8_t)` cast instead: the cast applies to the parenthesized division that follows it, and a cast binds tighter than any arithmetic or comparison operator, so the expanded macro still acts as a single operand wherever it's used.

These macros use `float`, but they cost nothing at runtime: every operand is a constant, so the compiler computes the result while compiling and only the final integer ends up in the program. §4.1.5's advice to avoid `float` applies to math on values only known at runtime, like an ADC reading.

`ovf_count` here is a function-local `static` variable, scoped entirely inside the ISR: nothing outside `ISR(TIMER0_OVF_vect)` ever touches it. That means `volatile` isn't actually load-bearing on this specific variable: the compiler can already see every read and write to `ovf_count` within this one function body, and nothing else can change it between two of them. Leaving `volatile` on here is a harmless habit. The moment to actually rely on it is the moment code outside this ISR needs to read `ovf_count` too, above all a loop in `main()` that waits for it to change, the situation the `blink_state` of §3.1.4 was in.

5.1.5 Timer 1 in CTC Mode (16-bit, arbitrary period)

Same idea, but Timer 1's extra range plus CTC mode let you hit a period directly, without stacking overflow counts except for very long delays. Register names swap the `0` for a `1`; CTC needs one extra bit, `WGM12`, to select CTC instead of Normal Mode.

The counter, `TCNT1`, counts from `0` up to the value you write to `OCR1A`, so it takes `(OCR1A + 1)` ticks to reach it and fire a compare-match interrupt. To hit a specific timeout, work out how many ticks that timeout needs, then subtract one:

```
ticks needed = timeout_ms / TICK_PERIOD_MS
OCR1A        = ticks needed - 1
```

E.g. for a 100 ms timeout at a 0.064 ms tick (÷1024 prescaler): `100 / 0.064 = 1562.5` ticks needed, exactly halfway between 1562 ticks (99.968 ms) and 1563 (100.032 ms), which are equally close. Rounding up gives 1563, so `OCR1A = 1563 - 1 = 1562`. The macros below make the same calculation in floating point and land on `1561` instead, one tick short: `0.064F` can't be stored exactly and is a hair above `0.064`, so the division gives a hair under `1562.5`, and the rounding goes down. On a half tick either answer is as good as the other; a timeout that doesn't land on one, like the two seconds of Exercise 5.6, comes out the same either way:

```
#define TIMEOUT_MS 100U                           // Timeout in ms.
#define TICK_PERIOD_MS 0.064F                     // Tick period, prescaler 1024.
#define CMP_MAX (TIMEOUT_MS / TICK_PERIOD_MS - 1U) // Compare value.
#define TICK_MAX (uint16_t)(CMP_MAX + 0.5F)       // Rounded compare value.

TCCR1B = (1U << CS10) | (1U << CS12) | (1U << WGM12); // Set 1024 prescaler, use ↪
    ↪ CTC mode.
OCR1A  = TICK_MAX;                                    // Set timeout to 100 ms.
TIMSK1 = (1U << OCIE1A);                              // Enable compare-match ↪
    ↪ interrupt.
```

Timer 1's 16-bit range means `1561` fits directly; Timer 0's 8-bit range (max `255`) couldn't hold a value that large, which is exactly why §5.1.4 had to count multiple overflows in software instead.

Two LEDs on D9/D10 alternate once every 100 ms while enabled; a button on D13 toggles them on and off entirely, cutting the LEDs immediately rather than waiting for the next compare match. The circuit is the one from §2.1.6, Figure 2.4.

```
#include <stdbool.h>
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>

/** GPIO pins. */
#define LED1 1U // D9  -> PORTB1.
#define LED2 2U // D10 -> PORTB2.
#define BTN1 5U // D13 -> PORTB5.
```

```
/** GPIO operations. */
#define LED1_ON PORTB |= (1U << LED1)                    // Enable LED1.
#define LED1_OFF PORTB &= ~(1U << LED1)                  // Disable LED1.
#define LED2_ON PORTB |= (1U << LED2)                    // Enable LED2.
#define LED2_OFF PORTB &= ~(1U << LED2)                  // Disable LED2.
#define LEDS_OFF PORTB &= ~((1U << LED1) | (1U << LED2)) // Disable both LEDs.
#define BTN1_PRESSED (PINB & (1U << BTN1))               // High if BTN1 is ↩
    ↪ pressed, low otherwise.

/** Time parameters. */
#define TIMEOUT_MS 100U                             // Timeout in ms.
#define TICK_PERIOD_MS 0.064F                       // Tick period, prescaler 1024.
#define CMP_MAX (TIMEOUT_MS / TICK_PERIOD_MS - 1U) // Compare value.
#define TICK_MAX (uint16_t)(CMP_MAX + 0.5F)        // Rounded compare value.

/** LED state (true = enabled, false = disabled). */
static volatile bool led_state = false;

/** True if LED1 is lit, false otherwise. */
static volatile bool led1_lit = false;

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 and LED2 as outputs.
    DDRB = (1U << LED1) | (1U << LED2);

    // Configure BTN1 as input with its internal pull-up enabled.
    PORTB = (1U << BTN1);

    // Enable pin change interrupt for BTN1.
    PCICR  = (1U << PCIE0);
    PCMSK0 = (1U << BTN1);

    // Enable timer 1 in CTC mode, set timeout to 100 ms.
    TCCR1B = (1U << CS10) | (1U << CS12) | (1U << WGM12);
    OCR1A  = TICK_MAX;
    TIMSK1 = (1U << OCIE1A);

    // Enable interrupts globally.
    sei();
```

```
}

/**
 * @brief Toggle the LEDs on button press.
 */
ISR(PCINT0_vect)
{
    if (BTN1_PRESSED)
    {
        led_state = !led_state;
        if (!led_state) { LEDS_OFF; }
    }
}

/**
 * @brief Alternate which of the two LEDs is lit if enabled.
 */
ISR(TIMER1_COMPA_vect)
{
    // Check the LED state, terminate if disabled.
    if (!led_state) { return; }

    // Alternate which LED is lit.
    led1_lit = !led1_lit;

    if (led1_lit)
    {
        LED1_ON;
        LED2_OFF;
    }
    else
    {
        LED1_OFF;
        LED2_ON;
    }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
```

```
    setup();
    while (true) {}
    return 0;
}
```

`led_state` is `volatile` because it's shared: the button's ISR writes it and the timer's ISR reads it. That follows the book's rule of making anything shared with an ISR `volatile`, although between two ISRs the qualifier is, strictly, the same harmless habit as on the `ovf_count` of §5.1.4: on the AVR an ISR runs to completion with interrupts disabled, so nothing can change `led_state` in the middle of one, and each ISR reads it afresh every time it runs. It becomes load-bearing the moment `main()`'s loop reads it too. `led1_lit` is only ever touched by the timer's ISR, so it could just as well be a `static` local inside it, like `ovf_count`.

Timer 2 mirrors Timer 0 (8-bit, Normal/CTC), with one gotcha: its ÷1024 prescaler bit pattern is `111`, not `101` as on Timers 0/1.

5.2 Review

What you should be able to do now

After this chapter, you should:

- Explain why a busy-wait delay (`_delay_ms`) is unsuitable once a program needs to do more than one thing at a time.
- Be able to compute a timer's tick period from the CPU clock and a chosen prescaler.
- Be able to work out how many overflow interrupts are needed for a given delay, for an 8-bit timer.
- Understand the practical difference between Normal Mode and CTC Mode, and when you'd reach for each.
- Know why a variable shared between an ISR and `main()` needs to be declared `volatile`.

Questions to test yourself

You should be able to answer:

1. At 16 MHz with a ÷256 prescaler, what's the resulting tick period?
2. What's the difference between what triggers `TIMERn_OVF_vect` and what triggers `TIMERn_COMPA_vect`?
3. Why is `volatile` optional on `ovf_count` in the Timer 0 example, and when does a variable shared with an ISR genuinely need it?

Looking ahead

Chapter 6 is about serial communication:

- Serial communication (UART): transmitting data to a PC instead of only driving LEDs.
- Combining timers with UART to report readings periodically.

5.3 Exercises

> **How to work these.** The Reflection and Calculation exercises are answered on paper, from the chapter. The Code exercises are programs for the board: build and flash them the way §2.2 (Microchip Studio) or §2.3 (Linux) describes, or run them in the simulator of Appendix A if you have no board yet.
> **How to check your work.** The Reflection and Calculation exercises, and the questions some Code exercises ask about their design, are answered in Appendix B. The Code exercises have no printed solution: the board tells you whether a program does what the exercise describes. Try each exercise before you read its answer.

EXERCISE SET 1

Concepts

Exercise 5.1 Tick at ÷256 CALCULATION

At 16 MHz with a ÷256 prescaler, what's the resulting tick frequency and tick period? Show your work.

Exercise 5.2 Overflows for 250 ms CALCULATION

Using your answer to Exercise 5.1, how many overflow interrupts (8-bit timer) would be needed to time a 250 ms delay?

EXERCISE SET 2

Normal vs. CTC

Exercise 5.3 When to reach for CTC REFLECTION

In your own words, explain when you'd reach for CTC mode instead of Normal mode with software overflow-counting. Is there a period that's awkward to hit precisely in Normal mode but trivial in CTC?

EXERCISE SET 3

Program

Exercise 5.4 Blinking at 2 Hz CODE

Adapt the Timer 0 blink example of §5.1.4 to blink at 2 Hz (250 ms on, 250 ms off) instead of 1 Hz. What value changes, and what does the new `OVF_MAX` work out to?

Exercise 5.5 When `volatile` matters REFLECTION

The chapter's Timer 0 example declares `ovf_count` as `volatile`, and §5.1.4 says that removing the qualifier there would change nothing. Explain why. Then suppose `ovf_count` moved to file scope so that `main()` could wait for it, in a loop such as `while (10U > ovf_count) {}`. What specific failure could removing `volatile` now introduce, and why might it show up in one build and not in another?

Exercise 5.6 A 2-second period in CTC CODE

Configure Timer 1 in CTC mode for a period of exactly 2 seconds at 16 MHz with a ÷1024 prescaler. What value goes in `OCR1A`? (Timer 1 is 16-bit, so check whether your answer fits.)

Exercise 5.7 Three timers, one blink CODE

Capstone. Two LEDs on D9/D10 blink at a speed selected by whichever of three buttons (D11–D13) was most recently pressed, each mapped to a different timer:

- D11 selects Timer 0 (1000 ms per toggle), D12 selects Timer 1 (500 ms per toggle), D13 selects Timer 2 (100 ms per toggle).
- Pressing a button toggles *that* timer's blinking on or off. If it was off, turn it on and turn the other two off (only one timer drives the LEDs at a time). If it was already the active timer, turn it off entirely and the LEDs go dark.
- All three buttons are on port B and share `PCINT0_vect`; detect which one was just pressed with the `prev`/`now` edge detection from §3.1.8.
- Configure all three timers to tick at the same rate (÷1024 prescaler, ~0.064 ms per tick) so the overflow-counting logic in §5.1.4 works identically for each. Timer 1 needs CTC mode with `OCR1A` set so its compare-match period matches Timer 0/2's overflow period, even though its 16-bit range could hit the target directly, so all three interrupt handlers end up the same shape.

- Define a small struct to hold `required_interrupts` and `executed_interrupts` for one timer (see §5.1.2 and §5.1.4 for the counting pattern), and use three instances, one per timer, instead of three sets of loose globals.
- Track the currently active timer in a single variable, using either an enum (`TIMER_NONE`, `TIMER_0`, `TIMER_1`, `TIMER_2`) or macros with timer indexes.

CHAPTER 6

Serial Communication (UART)

A real channel to the PC, one bit at a time.

IN THIS CHAPTER

- Why serial: a real communication channel to a PC instead of blink-and-infer debugging.
- Configuring UART: `UCSR0B`, `UCSR0C`, and the `UBRR0` baud-rate formula.
- Sending data: `UDR0`, the `UDRE0` flag, and building a reusable text-transmit helper.
- Mixing text and numbers with `sprintf()`.
- Receiving data: `RXEN0`, the `RXC0` flag, and reading `UDR0` instead of writing it.
- Worked example: button toggles an LED and reports its new state over serial, using a driver split across its own header and source file.

This chapter gives the board a way to talk back. Until now every program has reported what it was doing by blinking an LED; with the UART it can send text to a PC. After the chapter you should be able to explain what baud rate is and compute `UBRR0` for a given clock and baud rate, write a byte and then a whole string over the UART, combine `sprintf()` with transmission to report a mix of text and numbers, explain why the code waits on `UDRE0` before writing a new byte, and receive a byte and explain how `RXC0` mirrors `UDRE0`.

Read §6.1, the substance of the chapter, in order: transmit first, then the chapter's demo, the book's first program split across several files, then receiving. Then work the exercises in §6.3. The Reflection and Calculation exercises are answered in Appendix B; the Code exercises have no printed solution.

6.1 Serial Communication (UART)

6.1.1 Why serial

- Every debugging technique so far has been "blink an LED and infer what happened." UART (Universal Asynchronous Receiver/Transmitter) gives you a real communication channel to a PC, `printf()`-style debugging, logging sensor data, or eventually two-way communication with a host application.

- It's called *serial* because it sends one bit at a time over a single wire, framed by a start bit and one or more stop bits per byte, cheap on wiring, at the cost of throughput. This chapter builds up transmit first (§6.1.2–6.1.6), then receiving (§6.1.7), which reuses almost the same registers.
- Speed is measured in **baud rate**, bits per second. 9600 baud is the traditional default and what we'll use here.

6.1.2 Configuring UART

Three registers:

1. **`UCSR0B`** enables transmit (and, later, receive):

   ```
   UCSR0B = (1U << TXEN0);   // Enable transmission.
   ```

2. **`UCSR0C`** sets the frame format. 8 data bits, no parity, 1 stop bit (the near-universal default) needs:

   ```
   UCSR0C = (1U << UCSZ00) | (1U << UCSZ01);
   ```

3. **`UBRR0`** holds the baud rate, computed from the CPU clock:

   ```
   UBRR0 = F_CPU / (16 * baud_rate) - 1
   ```

   At 16 MHz for 9600 baud, that's `103`:

   ```
   UBRR0 = 103U;
   ```

```
#define BAUD 103U // UBRR value for 9600 bps with a 16 MHz clock.

/**
 * @brief Initialize UART with a 9600 bps baud rate.
 */
static void serial_init(void)
{
    // Enable UART transmission.
    UCSR0B = (1U << TXEN0);

    // Set character size to eight bits.
    UCSR0C = (1U << UCSZ00) | (1U << UCSZ01);

    // Set baud rate to 9600 bps.
    UBRR0 = BAUD;
}
```

6.1.3 Sending data

Transmission happens one byte at a time through the `UDR0` data register. Before writing a new byte, wait for the previous one to move out of `UDR0` (into the transmit shift register, which sends it bit by bit); signalled by the `UDRE0` (Data Register Empty) flag in `UCSR0A`:

```
/**
 * @brief Transmit a single byte over UART, blocking until the data register is ↩
    ↪ free.
 *
 * @param[in] byte Byte to transmit.
 */
static void serial_write_byte(const uint8_t byte)
{
    while (0U == (UCSR0A & (1U << UDRE0))) {} // Wait for the register to free up.
    UDR0 = byte;
}
```

Sending a whole string is just calling that in a loop, plus a trailing `\r\n` (carriage return, then line feed) so each message starts on a fresh line in the terminal:

```
/**
 * @brief Transmit text over UART.
 *
 * @param[in] text Text to transmit.
 */
static void serial_print(const char* text)
{
    // Terminate if the text pointer is NULL.
    if (NULL == text) { return; }

    // Transmit each character one by one.
    for (uint16_t i = 0U; text[i]; ++i)
    {
        serial_write_byte((uint8_t)(text[i]));
    }
    serial_write_byte('\r');
    serial_write_byte('\n');
}
```

6.1.4 Simple TX example

A first fully working example: §6.1.2's `serial_init()` and §6.1.3's `serial_write_byte()`/`serial_print()` combined into a single flashable program. LED1 on D9 blinks every 500 ms, and each transition is reported over serial. `delay_ms()` is the same hand-rolled helper built

on `_delay_ms()` from §2.1.6, reused rather than redefined from scratch. Everything still lives in one file here; §6.1.6 splits the driver out into its own header and source file once there's enough of it to be worth the split.

```c
#define F_CPU 16000000UL // CPU frequency in Hz.

#include <stddef.h>
#include <stdint.h>

#include <avr/io.h>
#include <util/delay.h>

/** GPIO pins. */
#define LED1 1U // D9  -> PORTB1.

/** GPIO operations. */
#define LED1_TOGGLE PINB = (1U << LED1)    // Toggle LED1.
#define LED1_ENABLED (PORTB & (1U << LED1)) // High if LED1 is enabled, low ↩
    ↪ otherwise.

/** Time parameters. */
#define BLINK_SPEED_MS 500U // Blink speed in ms.
#define TICK_1MS 1U         // Generate a 1 ms tick in delay_ms().

/** Serial parameters. */
#define BAUD 103U            // UBRR value for 9600 bps with a 16 MHz clock.
#define NEW_LINE '\n'        // New line character.
#define CARRIAGE_RETURN '\r' // Carriage return.

/**
 * @brief Initialize UART with a 9600 bps baud rate.
 */
static void serial_init(void)
{
    // Enable UART transmission.
    UCSR0B = (1U << TXEN0);

    // Set character size to eight bits.
    UCSR0C = (1U << UCSZ00) | (1U << UCSZ01);

    // Set baud rate to 9600 bps.
    UBRR0 = BAUD;
}
```

```
/**
 * @brief Transmit a single byte over UART, blocking until the data register is ↪
    ↪ free.
 *
 * @param[in] byte Byte to transmit.
 */
static void serial_write_byte(const uint8_t byte)
{
    // Wait until the transmit buffer is ready for a new byte.
    while (0U == (UCSR0A & (1U << UDRE0))) {}

    // Place the new byte in the transmit data register.
    UDR0 = byte;
}

/**
 * @brief Transmit text over UART.
 *
 * @param[in] text Text to transmit.
 */
static void serial_print(const char* text)
{
    // Terminate if the text pointer is NULL.
    if (NULL == text) { return; }

    // Transmit each character one by one.
    for (uint16_t i = 0U; text[i]; ++i)
    {
        serial_write_byte((uint8_t)(text[i]));
    }
    serial_write_byte('\r');
    serial_write_byte('\n');
}

/**
 * @brief Generate delay.
 *
 * @param[in] ms Delay duration in ms.
 */
static void delay_ms(const uint16_t ms)
{
    for (uint16_t i = 0U; i < ms; ++i)
    {
        _delay_ms(TICK_1MS);
```

```
    }
}

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Initialize serial driver.
    serial_init();
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    // Toggle LED1 and print its new state every 500 ms.
    while (1)
    {
        LED1_TOGGLE;
        if (LED1_ENABLED) { serial_print("LED1 enabled!"); }
        else { serial_print("LED1 disabled!"); }
        delay_ms(BLINK_SPEED_MS);
    }
    return 0;
}
```

`LED1_ENABLED` reads `PORTB` here, not `PINB`. A pin's level reaches `PINB` through a synchronizer that runs one clock cycle behind, so reading back a level the program has just set needs an instruction in between (the datasheet uses a `nop`). In this loop the compiler puts the read directly after the toggle, and a `PINB` read would still see the old level, reporting every transition the wrong way round. `PORTB` holds what the program last wrote, and changes at once.

6.1.5 Mixing text and numbers

`sprintf()` (from `<stdio.h>`) formats into a buffer exactly like `printf()` formats to the terminal; build the string there, then hand it to `serial_print()`. A full program, extending §6.1.4 with a counter reported once every 500 ms instead of a fixed message:

```
#define F_CPU 16000000UL // CPU frequency in Hz.

#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

#include <avr/io.h>
#include <util/delay.h>

/** Time parameters. */
#define REPORT_INTERVAL_MS 500U // Report interval in ms.
#define TICK_1MS 1U             // Generate a 1 ms tick in delay_ms().

/** Serial parameters. */
#define BAUD 103U // UBRR value for 9600 bps with a 16 MHz clock.

/** Message parameters. */
#define MSG_LEN 50U // Max length of a formatted message, including the NUL ↩
    ↪ terminator.

/**
 * @brief Initialize UART with a 9600 bps baud rate.
 */
static void serial_init(void)
{
    // Enable UART transmission.
    UCSR0B = (1U << TXEN0);

    // Set character size to eight bits.
    UCSR0C = (1U << UCSZ00) | (1U << UCSZ01);

    // Set baud rate to 9600 bps.
    UBRR0 = BAUD;
}

/**
 * @brief Transmit a single byte over UART, blocking until the data register is ↩
    ↪ free.
```

```
 *
 * @param[in] byte Byte to transmit.
 */
static inline void serial_write_byte(const uint8_t byte)
{
    // Wait until the transmit buffer is ready for a new byte.
    while (0U == (UCSR0A & (1U << UDRE0))) {}

    // Place the new byte in the transmit data register.
    UDR0 = byte;
}

/**
 * @brief Transmit text over UART.
 *
 * @param[in] text Text to transmit.
 */
static void serial_print(const char* text)
{
    // Terminate if the text pointer is NULL.
    if (NULL == text) { return; }

    // Transmit each character one by one.
    for (uint16_t i = 0U; text[i]; ++i)
    {
        serial_write_byte((uint8_t)(text[i]));
    }
    serial_write_byte('\r');
    serial_write_byte('\n');
}

/**
 * @brief Generate delay.
 *
 * @param[in] ms Delay duration in ms.
 */
static void delay_ms(const uint16_t ms)
{
    for (uint16_t i = 0U; i < ms; ++i)
    {
        _delay_ms(TICK_1MS);
    }
}
```

```
/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Initialize serial driver.
    serial_init();
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    uint16_t counter = 0U;
    setup();

    // Report an incrementing counter every 500 ms.
    while (1)
    {
        // Parse the counter value into a string:
        char msg[MSG_LEN] = {'\0'};
        sprintf(msg, "Count: %u", counter++);

        // Transmit the string content over UART, then wait 500 ms.
        serial_print(msg);
        delay_ms(REPORT_INTERVAL_MS);
    }
    return 0;
}
```

`%d` for `int`, `%u` for unsigned, `%u` used here since `counter` is a `uint16_t`, which is an `unsigned int` on this target. Floating-point formatting in AVR's default `sprintf()` is often disabled or costly to link in; prefer integers (e.g. scale a reading to hundredths and print the two parts separately) unless you've confirmed float formatting is available in your toolchain.

6.1.6 Example: button toggles an LED, reports over serial

Button on D13, LED on D9. Each press flips the LED and reports its new state.

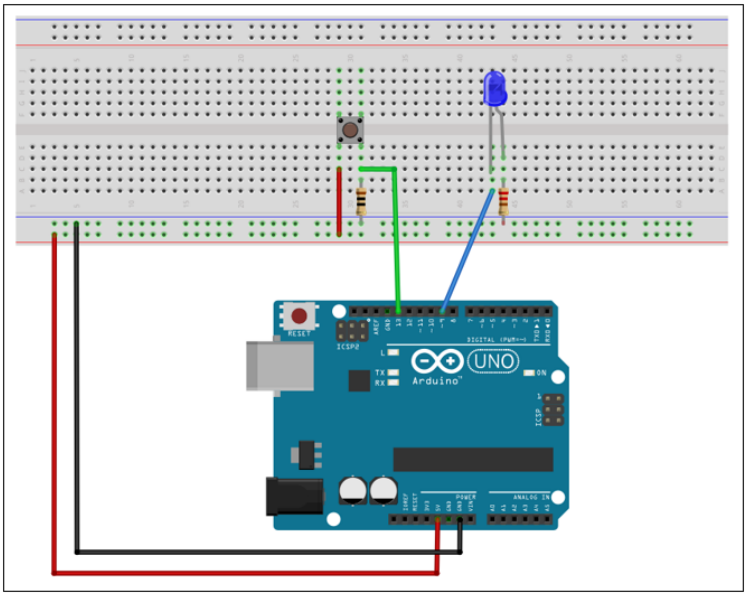


Figure 6.1 The serial demo's circuit: a button on D13 and an LED on D9, with the board's USB port as the serial link to the PC.

This is also the book's first multi-file example: rather than inlining every function here, the serial driver lives in its own header and source file, and `main.c` `#includes` it like any other driver:

```
serial_driver_demo/
├── Makefile
├── main.c
├── include/
│   └── driver/
│       └── serial.h
└── source/
    └── driver/
        └── serial.c
```

The driver (`driver/serial.h` and `driver/serial.c`, both printed in full below) adds one function beyond §6.1.3–6.1.4's `serial_write_byte()`/`serial_print()`: `serial_write()`, which takes a raw `uint8_t` buffer and a length, for arbitrary binary data rather than NUL-terminated text. `main.c` below still only needs `serial_print()`, the same convenience wrapper already familiar from §6.1.3–6.1.4, with one difference: the driver's version doesn't append a line ending of its own. It sends the text as written, turning each `\n` in it into `\r\n` on the way out, which is why the messages below end in `\n`. `serial_write_byte()` is a `static`

helper inside `serial.c`, no longer something callers can reach. `main.c` itself is shown in full after the driver.

The driver in full

lectures/L06/serial_driver_demo/include/driver/serial.h

```c
/**
 * @file Serial driver.
 */
#ifndef DRIVER_SERIAL_H_
#define DRIVER_SERIAL_H_

#include <stdint.h>

/**
 * @brief Initialize UART with a 9600 baud rate.
 */
void serial_init(void);

/**
 * @brief Transmit data over UART.
 *
 * @param[in] buf Buffer holding serial data to write.
 * @param[in] buflen Size of buf in bytes.
 *
 * @return Number of bytes transmitted, or -1 if buf is NULL or buflen is 0.
 */
int16_t serial_write(const uint8_t* buf, uint16_t buflen);

/**
 * @brief Transmit text over UART.
 *
 * @param[in] text Text to transmit.
 *
 * @return Number of bytes consumed from text, or -1 if text is NULL.
 *         Injected \r bytes are not counted.
 */
int16_t serial_print(const char* text);

#endif /** DRIVER_SERIAL_H_ */
```

lectures/L06/serial_driver_demo/source/driver/serial.c

```c
/**
 * @file Serial driver implementation details.
 */
#include <stddef.h>
#include <stdint.h>

#include <avr/io.h>

#include "driver/serial.h"

#define BAUD 103U            // UBRR value for 9600 bps with a 16 MHz clock.
#define NEW_LINE '\n'        // New line character.
#define CARRIAGE_RETURN '\r' // Carriage return.

// -----------------------------------------------------------------------------
static inline void serial_write_byte(const uint8_t byte)
{
    // Wait until the transmit buffer is ready for a new byte.
    while (0U == (UCSR0A & (1U << UDRE0))) {}

    // Place the new byte in the transmit data register.
    UDR0 = byte;
}

// -----------------------------------------------------------------------------
void serial_init(void)
{
    // Enable UART transmission.
    UCSR0B = (1U << TXEN0);

    // Set character size to eight bits.
    UCSR0C = (1U << UCSZ00) | (1U << UCSZ01);

    // Set baud rate to 9600 bps.
    UBRR0 = BAUD;
}

// -----------------------------------------------------------------------------
int16_t serial_write(const uint8_t* buf, const uint16_t buflen)
{
    // Check the input parameters, return -1 if invalid.
    if ((NULL == buf) || (0U == buflen)) { return -1; }
    uint16_t i = 0U;
```

```
    // Transmit each byte one by one.
    for (; i < buflen; ++i)
    {
        serial_write_byte(buf[i]);
    }
    // Return the number of transmitted bytes.
    return (int16_t)(i);
}

// ------------------------------------------------------------------------
int16_t serial_print(const char* text)
{
    // Check the text, return -1 if invalid.
    if (NULL == text) { return -1; }
    uint16_t i = 0U;

    // Transmit each character one by one.
    for (; text[i]; ++i)
    {
        const char byte = text[i];

        // Convert each newline character from LF to CRLF.
        if (NEW_LINE == byte) { serial_write_byte(CARRIAGE_RETURN); }
        serial_write_byte((uint8_t)(byte));
    }
    // Return the number of characters processed.
    return (int16_t)(i);
}
```

The program

lectures/L06/serial_driver_demo/main.c

```
/**
 * @file Serial driver demo.
 */
#include <stdbool.h>
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>

#include "driver/serial.h"
```

```
#define LED1 1U // D9  -> PORTB1.
#define BTN1 5U // D13 -> PORTB5.

/** GPIO operations. */
#define LED1_TOGGLE PINB = (1U << LED1)    // Toggle LED1.
#define LED1_ENABLED (PINB & (1U << LED1)) // High if LED1 is enabled, low ↩
    ↪ otherwise.
#define BTN1_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↩
    ↪ otherwise.

/** Timer operations. */
#define TIMER0_ENABLE TIMSK0 = (1U << TOIE0) // Enable timer 0 interrupt.
#define TIMER0_DISABLE TIMSK0 = 0U           // Disable timer 0 interrupt.

/** Interrupt operations. */
#define BTN1_INT_ENABLE PCMSK0 |= (1U << BTN1)   // Enable pin change interrupt ↩
    ↪ for BTN1.
#define BTN1_INT_DISABLE PCMSK0 &= ~(1U << BTN1) // Disable pin change interrupt ↩
    ↪ for BTN1.

/** Time parameters */
#define TICK_PERIOD_MS 0.064F    // Time between each tick, prescaler 1024.
#define TICK_MAX 256U            // Ticks per overflow for 8-bit timer.
#define DEBOUNCE_TIMEOUT_MS 300U // Debounce timeout in ms.

/** Limit parameters. */
#define OVF_TIME_MS (TICK_PERIOD_MS * TICK_MAX)     // Time between each overflow ↩
    ↪ in ms.
#define OVF_MAX (DEBOUNCE_TIMEOUT_MS / OVF_TIME_MS) // Overflows needed for ↩
    ↪ timeout.
#define OVF_TIMEOUT (uint8_t)(OVF_MAX + 0.5F)       // Overflows needed for ↩
    ↪ timeout, rounded.

/** Flags that a LED1 event has occurred. */
static volatile bool led1_event = false;

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);
```

```
    // Configure BTN1 as input with its internal pull-up enabled.
    PORTB = (1U << BTN1);

    // Enable pin change interrupt for BTN1.
    PCICR = (1U << PCIE0);
    BTN1_INT_ENABLE;

    // Set up 300 ms debounce timer.
    TCCR0B = (1U << CS00) | (1U << CS02);

    // Initialize serial driver.
    serial_init();

    // Enable interrupts globally.
    sei();
}

/**
 * @brief Toggle LED1 on button press, and flag the event for main() to report.
 *
 *        Disable button interrupts for 300 ms to suppress contact bounce.
 */
ISR(PCINT0_vect)
{
    // Disable button interrupts for 300 ms.
    BTN1_INT_DISABLE;
    TIMER0_ENABLE;

    // Toggle LED1 if BTN1 is pressed, flag that a LED1 event has occurred.
    if (BTN1_PRESSED)
    {
        LED1_TOGGLE;
        led1_event = true;
    }
}

/**
 * @brief Re-enable button interrupts after 300 ms.
 */
ISR(TIMER0_OVF_vect)
{
    static volatile uint8_t ovf_counter = 0U;
```

```
    // Wait for 300 ms, then re-enable button interrupts and disable timer 0.
    if (OVF_TIMEOUT <= ++ovf_counter)
    {
        BTN1_INT_ENABLE;
        TIMER0_DISABLE;
        ovf_counter = 0U;
    }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        // Check if a LED1 event has occurred, report the new state over UART if ↪
    ↪ true.
        if (led1_event)
        {
            if (LED1_ENABLED) { serial_print("LED1 enabled!\n"); }
            else { serial_print("LED1 disabled!\n"); }
            led1_event = false;
        }
    }
    return 0;
}
```

`ISR(PCINT0_vect)` only toggles LED1 and sets `led1_event`; the UART report happens in `main()`'s loop instead, once per event, never inside the ISR itself. `serial_print()` blocks byte-by-byte on `UDRE0` until each byte is sent, and an ISR that spends that time transmitting holds up every other interrupt for the whole transmission, including the debounce timer this very feature depends on. Keep ISRs to the minimum work needed (flip a pin, set a flag) and let `main()`'s loop handle anything that can block; §7.1.4 reuses this exact pattern for `eeprom_write()`, another slow, blocking call that doesn't belong in an ISR either.

Building and running

On Linux, `cd` into `lectures/L06/serial_driver_demo` and run `make` to build, then `make flash` to program the board, the same `Makefile` pattern as §2.3.8:

lectures/L06/serial_driver_demo/Makefile

```
# Target MCU.
MCU := atmega328p

# CPU frequency in Hz.
F_CPU := 16000000UL

# Serial port for flashing.
PORT := /dev/ttyACM0

# Baud rate for flashing (in bps).
BAUD := 115200

# C compiler.
C_COMPILER := avr-gcc

# C compiler flags.
C_FLAGS := -Wall -Wextra -Werror -Iinclude -std=c11

# Source files.
SRC_FILES := main.c \
             source/driver/serial.c \

# Build the firmware image by default.
all: main.hex

# Compile and link the source files into an ELF binary.
main.elf: $(SRC_FILES)
    @$(C_COMPILER) -mmcu=$(MCU) -DF_CPU=$(F_CPU) -Os -o $@ $^ $(C_FLAGS)

# Convert the ELF binary into an Intel HEX image, ready to flash.
main.hex: main.elf
    @avr-objcopy -O ihex -R .eeprom $< $@

# Flash the firmware image onto the board.
flash: main.hex
    @avrdude -c arduino -p $(MCU) -P $(PORT) -b $(BAUD) -U flash:w:$<:i

# Remove build artifacts.
clean:
    @rm -f main.elf main.hex

# Targets that don't produce a file of the same name.
.PHONY: all flash clean
```

On Windows via Microchip Studio, there's no Makefile support: recreate this same folder structure inside the project, then add `include` as an include directory under the project's toolchain settings (Project Properties → Toolchain → AVR/GNU C Compiler → Directories), the IDE's equivalent of the Makefile's `-Iinclude`.

Open a serial terminal (the Arduino IDE's Serial Monitor, or `screen`/`minicom` from the command line) at 9600 baud on the Uno/Nano's port to see the output.

6.1.7 Receiving data

Receiving is transmit's mirror image: one more enable bit in `UCSR0B`, one more status flag in `UCSR0A` to poll, and the same `UDR0` data register, just read instead of written.

1. **`RXEN0`** (Receiver Enable) sits right next to `TXEN0` in `UCSR0B`, the register §6.1.2 introduced. Set both together to enable transmit and receive at once:

```
UCSR0B = (1U << TXEN0) | (1U << RXEN0);   // Enable transmission and reception↩
    ↪ .
```

2. **`RXC0`** (RX Complete), in `UCSR0A`, is `UDRE0`'s counterpart: it's clear while nothing new has arrived, and sets itself the moment a fully-received byte is sitting in `UDR0`, waiting to be read. Reading `UDR0` clears it automatically, exactly as reading `UDR0` also readies it for the next incoming byte.
3. **`UDR0`** is the very same register `serial_write_byte()` writes to in §6.1.3; reading it instead of writing it returns whatever byte just arrived.

Put together, the receive-side equivalent of `serial_write_byte()` is:

```
/**
 * @brief Receive a single byte over UART, blocking until one arrives.
 *
 * @return Byte received.
 */
static char serial_read_byte(void)
{
    while (0U == (UCSR0A & (1U << RXC0))) {} // Wait for a byte to arrive.
    return UDR0;
}
```

This blocks indefinitely if nothing ever arrives, fine for a first pass, but worth keeping in mind once a program also needs to do other work while waiting. Reading more than one byte into a buffer, with an optional timeout so a silent sender can't hang the program forever, builds directly on `serial_read_byte()` the same way §6.1.6's driver `serial_write()` builds on `serial_write_byte()`.

`read()` below does exactly that: it fills a caller-supplied buffer up to `buflen` bytes, and takes an optional `timeout_ms` so a sender that never shows up can't hang the program forever.

```
/**
 * @brief Receive up to buflen bytes over UART into buf.
 *
 * @param[out] buf Buffer to receive into.
 * @param[in] buflen Size of buf in bytes.
 * @param[in] timeout_ms Milliseconds to wait for buflen bytes before giving up.
 *            0 blocks indefinitely.
 *
 * @return Number of bytes received, or -1 if buf is NULL or buflen is 0.
 */
int16_t read(uint8_t* buf, const uint16_t buflen, const uint16_t timeout_ms)
{
    // Check the input parameters, return -1 if invalid.
    if ((NULL == buf) || (0U == buflen)) { return -1; }
    uint16_t rx_count = 0U;

    if (0U == timeout_ms)
    {
        // Read indefinitely until the buffer is full if no timeout has been ↩
    ↪ specified.
        while (buflen > rx_count)
        {
            // Wait until a byte has been received.
            while (0U == (UCSR0A & (1U << RXC0))) {}
            buf[rx_count++] = UDR0;
        }
    }
    else
    {
        for (uint16_t i = 0U; i < timeout_ms; ++i)
        {
            while ((buflen > rx_count) && (UCSR0A & (1U << RXC0)))
            {
                buf[rx_count++] = UDR0;
            }
            // Stop reading if the read buffer is full.
            if (buflen <= rx_count) { break; }

            // Wait a millisecond before reading again.
            delay_ms(1U);
        }
```

```
    }
    // Return the number of bytes read.
    return (int16_t)(rx_count);
}
```

`read()` extends `serial_read_byte()` from a single byte to a whole buffer, with `timeout_ms` choosing between two modes:

- **`timeout_ms == 0`**: block indefinitely, the same as `serial_read_byte()`, until `buflen` bytes have arrived.
- **`timeout_ms` nonzero**: give up after roughly that many milliseconds even if the buffer isn't full yet, pacing itself with the `delay_ms()` helper first defined in §2.1.6, one millisecond at a time, rather than redefining it here.

`buf` is checked against `NULL` before use, the same reasoning as `vector_resize()` back in §1.1.11: this is written as a reusable, exported function with call sites outside this file, not a private helper like the `delay_ms()` of §3.1.4, so the check is doing real work rather than just demonstrating the habit.

The return value distinguishes three outcomes: `-1` for a bad call (`NULL` buffer or `buflen == 0`), the number of bytes actually received if the timeout elapsed first, or `buflen` itself once the buffer filled completely.

6.2 Review

What you should be able to do now

After this chapter, you should be able to:

- Explain what baud rate is and compute `UBRR0` for a given clock and baud rate.
- Write a byte, and then a whole string, over UART.
- Combine `sprintf()` with UART transmission to report a mix of text and numeric data.
- Understand why the code waits on `UDRE0` before writing a new byte.
- Receive a byte over UART and explain how `RXC0` mirrors `UDRE0`.

Questions to test yourself

1. What does the `UDRE0` flag represent, and what would happen if `serial_write_byte()` didn't wait for it?
2. Compute `UBRR0` for 19200 baud at a 16 MHz clock.
3. Why is `sprintf()` used to build a message before transmitting it, rather than trying to transmit an `int` directly?
4. What does `RXC0` represent, and what would `serial_read_byte()` do if it didn't wait for it?

Looking ahead

This chapter closes the introductory arc:

- C fundamentals (Chapter 1).
- Digital I/O and bit manipulation (Chapter 2).
- Reacting to events instead of polling for them (Chapter 3).
- Reading the analog world (Chapter 4).
- Timing without blocking (Chapter 5).
- Reporting back to a PC (this chapter).

Chapter 7 rounds out the peripheral survey with EEPROM, and Chapter 8 covers the watchdog timer. A follow-on arc on driver design starts after that: bundling a peripheral's state into a struct instead of one-off macros and globals, building directly on the LEDs, buttons, and timers from these six chapters.

6.3 Exercises

> **How to work these.** The Reflection and Calculation exercises are answered on paper, from the chapter. The Code exercises are programs for the board, except Exercise 6.3, which is a plain PC program: build and flash the board programs the way §2.2 (Microchip Studio) or §2.3 (Linux) describes, or run them in the simulator of Appendix A if you have no board yet.
> **How to check your work.** The Reflection and Calculation exercises are answered in Appendix B. The Code exercises have no printed solution: the board, or for Exercise 6.3 the terminal, tells you whether a program does what the exercise describes. Try each exercise before you read its answer.

EXERCISE SET 1

Concepts

Exercise 6.1 `UBRR0` for 19200 baud

CALCULATION

Calculate the value of `UBRR0` for a baud rate of `19200` at a 16 MHz clock. Round to the nearest integer.

Exercise 6.2 What `UDRE0` means

REFLECTION

Explain what the `UDRE0` flag represents and why `serial_write_byte()` waits for it before writing to `UDR0`.

EXERCISE SET 2

Program

Exercise 6.3 Decimal digits by hand

CODE

Write a `serial_write_uint(const uint8_t byte)` helper that prints `byte` as decimal text, one digit at a time, e.g. `serial_write_uint(243)` prints `243`. Extract each digit yourself via division/modulo; don't just hand the whole number to `printf("%u", byte)`/`sprintf()` and let it do the conversion for you, that's the same idea you'll need on any future toolchain where `sprintf()`/float formatting isn't available.

This is a console exercise: a plain PC program is enough, no board or UART required. `printf()` (or `putchar()`) stands in for `serial_write_byte()`, one call per digit.

Hint: dividing/modulo-ing a number by `10` repeatedly gives you its digits least-significant first; printing them in that order comes out backwards. Instead, find the largest power of ten that still fits inside `byte` first (`uint8_t` maxes out at `255`, so at most 3 digits), then walk that divisor down by a factor of `10` each step, digits come out in the right order with no need to reverse anything afterward. Watch out for `0`: make sure the search for the starting divisor doesn't shrink it all the way down to nothing before the loop that actually emits digits ever runs.

Exercise 6.4 Receiving a command

CODE

Extend `serial_driver_demo`'s driver with receive support, following §6.1.7's `RXEN0`/`RXC0` pattern: enable the receiver in `serial_init()`, then add a `serial_read()` to `serial.h`/`serial.c` that blocks until a byte arrives.

Use it in `main()` so that receiving `'t'` over serial toggles LED1 the same way pressing the button already does, reporting the new state the same way `main()` already reports a button press.

Exercise 6.5 A temperature reporter CODE

Capstone, tying together ADC (Chapter 4), timers (Chapter 5), and UART (this chapter). Read ambient temperature from a TMP36 sensor on A0 and report it over UART, either once a minute (Timer-driven) or immediately on a button press on D13, debounced with a timer so contact bounce doesn't spam the report:

- Reuse `adc_read()` from §4.1.3 to sample A0.
- TMP36 outputs `750 mV` at `25 °C` and rises `10 mV` per `°C`; converting the ADC's `0-1023` reading to a voltage and then to a temperature gives:

```
T (°C) = 500 * adc_reading / 1023 - 50
```

- The reported message should look like `"Temperature: 23 C"` (build it with `sprintf()`, as in §6.1.5).

For a harder version: keep the five most recent readings in a small array and report the running average alongside the instantaneous reading; and instead of a fixed one-minute report interval, track the time between the last five button presses and auto-adjust the report interval to match how often the button is actually being pressed.

CHAPTER 7

EEPROM

Memory that survives a reset, and costs milliseconds and wear to write.

In this chapter

- EEPROM: non-volatile storage that survives a power cycle or reset, and why it's slow and wear-limited compared to RAM.
- EEPROM registers: `EEAR`, `EEDR`, `EECR`, and the timed write sequence (`EEMPE` then `EEPE`).
- `eeprom_write_byte()` / `eeprom_read_byte()`, and why interrupts are disabled around a write.
- Why a plain function is the right tool here: there's exactly one EEPROM on the chip.
- Worked example: persisting LED1's state across resets, and why the EEPROM write and UART report both happen in `main()`, never inside an ISR.

This chapter is about the one memory on the chip that keeps its contents when the power goes. After it you should be able to explain what EEPROM is for and why it's unsuitable for data that changes every loop iteration, read from and write to EEPROM at a given address, explain why the EEPROM's write sequence disables interrupts and what could go wrong if it didn't, and explain why slow, blocking calls like `eeprom_write()` and `serial_print()` don't belong inside an ISR.

Read §7.1, the substance of the chapter; its worked example extends the serial demo of §6.1.6. Then work the exercises in §7.3. The Reflection exercises, and the question the Code exercise asks, are answered in Appendix B; the Code exercise has no printed solution.

7.1 EEPROM

7.1.1 EEPROM: what it is and why

- RAM forgets everything the instant power is removed. EEPROM (Electrically Erasable Programmable ROM) doesn't: it's a separate, much smaller memory that keeps its contents across a reset or power loss, useful for settings, calibration values, or a counter that should survive being unplugged.

- The ATmega328P has 1024 bytes of EEPROM, addressed `0–1023`, entirely separate from flash (where your program lives) and RAM (where your variables live).
- Two costs come with that durability: a single byte write takes a few milliseconds (eternity, compared to a RAM write), and each EEPROM cell only tolerates roughly 100,000 write cycles before it wears out. Don't write to EEPROM in a tight loop or once per iteration of `main()`; write only when a value actually needs to persist.

7.1.2 EEPROM registers

Three registers, all specific to the EEPROM peripheral:

1. **`EEAR`** (EEPROM Address Register) holds the address, `0–1023`, to read from or write to.
2. **`EEDR`** (EEPROM Data Register) holds the byte being written, or receives the byte just read.
3. **`EECR`** (EEPROM Control Register) controls and reports on the read/write process:
   - `EERE` (EEPROM Read Enable) starts a read.
   - `EEMPE` (EEPROM Master Write Enable) must be set immediately before `EEPE`, it's a safety interlock, not a data bit.
   - `EEPE` (EEPROM Write Enable) starts a write, and stays set until the write completes; a new write must wait for it to clear first.

The datasheet requires `EEPE` to be set within four CPU cycles of `EEMPE` being set, or the write silently doesn't happen at all. An interrupt firing in that window is a real risk (a few cycles is nothing), so a write disables interrupts for the handful of instructions between the two:

```
while (EECR & (1U << EEPE)) {} // Wait for any earlier write to finish.

EEAR = addr; // Set destination address (0-1023).
EEDR = byte; // Set data to write (one byte).

cli();                 // Disable interrupts during the write sequence.
EECR |= (1U << EEMPE); // EEMPE must be followed by EEPE within 4 cycles.
EECR |= (1U << EEPE);
sei();                 // Re-enable interrupts once the write sequence is complete
    ↪ .
```

Reading doesn't have this timing constraint, no write is happening, so there's no equivalent interlock, just the same "wait for the peripheral to be free" pattern already familiar from `ADIF` (Chapter 4) and `UDRE0` (Chapter 6):

```
while (EECR & (1U << EEPE)) {} // Wait for any earlier write to finish.
EEAR = addr;                   // Set the address to read from.
EECR |= (1U << EERE);          // Start read operation.
```

```
return EEDR;                    // Return the byte at the specified address.
```

7.1.3 eeprom_write_byte() / eeprom_read_byte()

```
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>

/**
 * @brief Write a byte to EEPROM at the given address.
 *
 * @param[in] addr EEPROM address to write to (0-1023).
 * @param[in] byte Byte to write.
 */
static void eeprom_write_byte(const uint16_t addr, const uint8_t byte)
{
    // Wait for any earlier write to finish.
    while (EECR & (1U << EEPE)) {}

    // Set destination address and data.
    EEAR = addr;
    EEDR = byte;

    // Perform write operation.
    cli();
    EECR |= (1U << EEMPE);
    EECR |= (1U << EEPE);
    sei();
}

/**
 * @brief Read a byte from EEPROM at the given address.
 *
 * @param[in] addr EEPROM address to read from (0-1023).
 *
 * @return Byte stored at that address.
 */
static uint8_t eeprom_read_byte(const uint16_t addr)
{
    // Wait for any earlier write to finish.
    while (EECR & (1U << EEPE)) {}
```

```
    // Set address to read from.
    EEAR = addr;

    // Start read operation.
    EECR |= (1U << EERE);

    // Return the byte stored at the given address.
    return EEDR;
}
```

7.1.4 Worked example: persisting LED1's state across resets

Extends §6.1.6's button-toggle-report demo: LED1's state now survives a reset or power cycle, read back from EEPROM in `setup()` and written back out every time it changes. The circuit is the one of Figure 6.1, unchanged.

```
eeprom_driver_demo/
├── Makefile
├── main.c
├── include/
│   └── driver/
│       ├── eeprom.h
│       └── serial.h
└── source/
    └── driver/
        ├── eeprom.c
        └── serial.c
```

The driver files (`driver/eeprom.h` and `driver/eeprom.c`, both printed in full below) wrap §7.1.3's `eeprom_write_byte()`/`eeprom_read_byte()` behind a buffer-and-length interface, `eeprom_write()`/`eeprom_read()`, the same shape as Chapter 6's `serial_write()`. `main.c` in full follows them.

The driver in full

lectures/L07/eeprom_driver_demo/include/driver/eeprom.h

```
/**
 * @file EEPROM driver.
 */
#ifndef DRIVER_EEPROM_H_
#define DRIVER_EEPROM_H_

#include <stdint.h>
```

```
/**
 * @brief Write data to EEPROM.
 *
 * @param[in] buf Buffer containing the data to write.
 * @param[in] buflen Number of bytes to write.
 * @param[in] addr Address of the first EEPROM byte to write.
 *
 * @return Number of bytes written, or -1 on failure.
 */
int16_t eeprom_write(const uint8_t* buf, uint16_t buflen, uint16_t addr);

/**
 * @brief Read data from EEPROM.
 *
 * @param[out] buf Buffer in which to store the data read.
 * @param[in] buflen Number of bytes to read.
 * @param[in] addr Address of the first EEPROM byte to read.
 *
 * @return Number of bytes read, or -1 on failure.
 */
int16_t eeprom_read(uint8_t* buf, uint16_t buflen, uint16_t addr);

#endif /** DRIVER_EEPROM_H_ */
```

lectures/L07/eeprom_driver_demo/source/driver/eeprom.c

```
/**
 * @file EEPROM driver implementation details.
 */
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>

#include "driver/eeprom.h"

#define EEPROM_ADDR_MAX 1023U // Highest address in EEPROM.

// ----------------------------------------------------------------------------
```

```
static inline bool addr_valid(const uint16_t addr) { return EEPROM_ADDR_MAX >= ↩
    ↪ addr; }

// ----------------------------------------------------------------------------
static bool eeprom_write_byte(const uint16_t addr, const uint8_t byte)
{
    // Return false if the address is invalid.
    if (!addr_valid(addr)) { return false; }

    // Wait for any previous write to finish.
    while (EECR & (1U << EEPE)) {}

    // Set destination address and data.
    EEAR = addr;
    EEDR = byte;

    // Perform write operation, then return true to indicate success.
    cli();
    EECR |= (1U << EEMPE);
    EECR |= (1U << EEPE);
    sei();
    return true;
}

// ----------------------------------------------------------------------------
static bool eeprom_read_byte(const uint16_t addr, uint8_t* byte)
{
    // Return false if the address is invalid or if byte is NULL.
    if (!addr_valid(addr) || (NULL == byte)) { return false; }

    // Wait for any previous write to finish.
    while (EECR & (1U << EEPE)) {}

    // Set the address to read from.
    EEAR = addr;

    // Perform read operation.
    EECR |= (1U << EERE);

    // Store the retrieved byte, then return true to indicate success.
    *byte = EEDR;
    return true;
}
```

```
// ----------------------------------------------------------------------------
int16_t eeprom_write(const uint8_t* buf, const uint16_t buflen, const uint16_t
    ↪ addr)
{
    // Check the input arguments, return -1 if invalid.
    if ((NULL == buf) || (0U == buflen) || !addr_valid(addr)) { return -1; }
    uint16_t i = 0U;

    // Write each byte one by one.
    for (; i < buflen; ++i)
    {
        // Terminate operation on write failure (indicates out of range).
        if (!eeprom_write_byte(addr + i, buf[i])) { break; }
    }
    // Return the number of bytes written.
    return (int16_t)(i);
}

// ----------------------------------------------------------------------------
int16_t eeprom_read(uint8_t* buf, const uint16_t buflen, const uint16_t addr)
{
    // Check the input arguments, return -1 if invalid.
    if ((NULL == buf) || (0U == buflen) || !addr_valid(addr)) { return -1; }
    uint16_t i = 0U;

    // Receive bytes one by one.
    for (; i < buflen; ++i)
    {
        // Terminate operation on read failure (indicates out of range).
        if (!eeprom_read_byte(addr + i, buf + i)) { break; }
    }
    // Return the number of bytes read.
    return (int16_t)(i);
}
```

The program

lectures/L07/eeprom_driver_demo/main.c

```
/**
 * @file EEPROM driver demo.
 */
#include <stdbool.h>
#include <stdint.h>
```

```
#include <avr/interrupt.h>
#include <avr/io.h>

#include "driver/eeprom.h"
#include "driver/serial.h"

#define LED1 1U // D9  -> PORTB1.
#define BTN1 5U // D13 -> PORTB5.

/** GPIO operations. */
#define LED1_TOGGLE PINB = (1U << LED1)    // Toggle LED1.
#define LED1_ENABLED (PINB & (1U << LED1)) // High if LED1 is enabled, low ↩
    ↪ otherwise.
#define BTN1_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↩
    ↪ otherwise.

/** Timer operations. */
#define TIMER0_ENABLE TIMSK0 = (1U << TOIE0) // Enable timer 0 interrupt.
#define TIMER0_DISABLE TIMSK0 = 0U           // Disable timer 0 interrupt.

/** Interrupt operations. */
#define BTN1_INT_ENABLE PCMSK0 |= (1U << BTN1)   // Enable pin change interrupt ↩
    ↪ for BTN1.
#define BTN1_INT_DISABLE PCMSK0 &= ~(1U << BTN1) // Disable pin change interrupt ↩
    ↪ for BTN1.

/** Time parameters */
#define TICK_PERIOD_MS 0.064F    // Time between each tick, prescaler 1024.
#define TICK_MAX 256U            // Ticks per overflow for 8-bit timer.
#define DEBOUNCE_TIMEOUT_MS 300U // Debounce timeout in ms.

/** Limit parameters. */
#define OVF_TIME_MS (TICK_PERIOD_MS * TICK_MAX)     // Time between each overflow ↩
    ↪ in ms.
#define OVF_MAX (DEBOUNCE_TIMEOUT_MS / OVF_TIME_MS) // Overflows needed for ↩
    ↪ timeout.
#define OVF_TIMEOUT (uint8_t)(OVF_MAX + 0.5F)       // Overflows needed for ↩
    ↪ timeout, rounded.

/** EEPROM parameters. */
#define EEPROM_LED1_ADDR 1000U // EEPROM address containing the LED1 state.
#define EEPROM_LED1_ON 1U      // Stored value indicating that LED1 is on.
#define EEPROM_LED1_OFF 0U     // Stored value indicating that LED1 is off.
```

```
/** Flags that a LED1 event has occurred. */
static volatile bool led1_event = false;

/**
 * @brief Print the LED1 state over UART.
 */
static void print_led1_state(void)
{
    if (LED1_ENABLED) { serial_print("LED1 enabled!\n"); }
    else { serial_print("LED1 disabled!\n"); }
}

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Configure BTN1 as input with its internal pull-up enabled.
    PORTB = (1U << BTN1);

    // Enable pin change interrupt for BTN1.
    PCICR = (1U << PCIE0);
    BTN1_INT_ENABLE;

    // Set up 300 ms debounce timer.
    TCCR0B = (1U << CS00) | (1U << CS02);

    // Initialize serial driver.
    serial_init();

    // Restore the previous LED state from EEPROM.
    uint8_t led_state = 0U;
    if (sizeof(led_state) == eeprom_read(&led_state, sizeof(led_state), ↩
    ↪ EEPROM_LED1_ADDR))
    {
        if (EEPROM_LED1_ON == led_state) { LED1_TOGGLE; }
    }
    print_led1_state();

    // Enable interrupts globally.
```

```
    sei();
}

/**
 * @brief Toggle LED1 on button press, and flag the event for main() to report.
 *
 *        Disable button interrupts for 300 ms to suppress contact bounce.
 */
ISR(PCINT0_vect)
{
    // Disable button interrupts for 300 ms.
    BTN1_INT_DISABLE;
    TIMER0_ENABLE;

    // Toggle LED1 and flag the event if BTN1 is pressed.
    if (BTN1_PRESSED)
    {
        LED1_TOGGLE;
        led1_event = true;
    }
}

/**
 * @brief Re-enable button interrupts after 300 ms.
 */
ISR(TIMER0_OVF_vect)
{
    static volatile uint8_t ovf_counter = 0U;

    // Wait for 300 ms, then re-enable button interrupts and disable timer 0.
    if (OVF_TIMEOUT <= ++ovf_counter)
    {
        BTN1_INT_ENABLE;
        TIMER0_DISABLE;
        ovf_counter = 0U;
    }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
```

```
{
    setup();

    while (1)
    {
        // Check if a LED1 event has occurred, store the new state in EEPROM if ↩
    ↪ true.
        if (led1_event)
        {
            const uint8_t led_state = LED1_ENABLED ? EEPROM_LED1_ON : ↩
    ↪ EEPROM_LED1_OFF;
            eeprom_write(&led_state, sizeof(led_state), EEPROM_LED1_ADDR);
            led1_event = false;

            // Report the new LED1 state over UART.
            print_led1_state();
        }
    }
    return 0;
}
```

`ISR(PCINT0_vect)` only toggles LED1 and sets `led1_event`; the EEPROM write and the UART report both happen in `main()`'s loop instead, once per event, never inside the ISR itself. That's deliberate, not incidental, and it's good practice generally: both `eeprom_write()` (which can block for several milliseconds waiting on `EECR`'s `EEPE` bit, §7.1.2) and `serial_print()` (which blocks byte-by-byte on `UDRE0`) are slow, blocking calls, and an ISR that spends milliseconds inside either one holds up every other interrupt for that whole time, including the debounce timer this very feature depends on. Keep ISRs to the minimum work needed (flip a pin, set a flag) and let `main()`'s loop handle anything that can block, the same reasoning §6.1.6 gave for never calling `serial_print()` from an ISR.

Building and running

Build and flash the same way as §6.1.6: on Linux, run `make` in the demo's directory, `lectures/L07/eeprom_driver_demo` (`make flash` to program the board); on Windows via Microchip Studio, recreate this folder structure inside the project and add `include` as an include directory. The demo's `Makefile` is the one printed in §6.1.6, with the EEPROM driver added to its source files:

```
SRC_FILES := main.c \
             source/driver/eeprom.c \
             source/driver/serial.c \
```

7.2 Review

What you should be able to do now

After this chapter, you should be able to:

- Explain what EEPROM is for and why it's unsuitable for data that changes every loop iteration.
- Read from and write to EEPROM at a given address.
- Explain why EEPROM's write sequence disables interrupts, and what could go wrong if it didn't.
- Explain why slow, blocking calls like `eeprom_write()` and `serial_print()` don't belong inside an ISR.

Questions to test yourself

1. Why does `eeprom_write_byte()` wait on the `EEPE` flag before starting a new write?
2. What real AVR datasheet requirement forces `EEMPE` to be followed by `EEPE` within 4 clock cycles, and why does the example disable interrupts for that window?
3. In §7.1.4's worked example, why does `ISR(PCINT0_vect)` only toggle LED1 and set a flag, leaving the actual `eeprom_write()` and UART report to `main()`?

Looking ahead

Chapter 8 is about the watchdog timer, another peripheral the chip has exactly one of, and a hardware safety net against a program that hangs.

7.3 Exercises

How to work these. The Reflection exercises are answered on paper, from the chapter. The Code exercise is a program for the board: build and flash it the way §2.2 (Microchip Studio) or §2.3 (Linux) describes, or run it in the simulator of Appendix A if you have no board yet: its simulation harness (§A.6) saves the EEPROM to a file, so what the program stores survives a restart there too.

How to check your work. The Reflection exercises, and the question the Code exercise asks about its design, are answered in Appendix B. The Code exercise has no printed solution: the board tells you whether the program does what the exercise describes. Try each exercise before you read its answer.

EXERCISE SET 1

Concepts

Exercise 7.1 Waiting before, not after

REFLECTION

Why does `eeprom_write_byte()` wait on the `EEPE` flag before it starts, but `eeprom_read_byte()` doesn't need to wait for anything after its read completes?

Exercise 7.2 The four-cycle window

REFLECTION

`EEMPE` must be followed by `EEPE` within four CPU cycles or the write silently fails. Why does that specifically mean interrupts need to be disabled for that window, and what real failure would you expect if an ISR fired in between and took longer than four cycles to return?

EXERCISE SET 2

Program

Exercise 7.3 Fewer EEPROM writes

CODE

§7.1.4's `eeprom_driver_demo` writes to EEPROM on every single button press. Given §7.1.1's write-cycle warning, is that actually a good idea for a device expected to run for years? Extend the demo to reduce wear: keep a small, non-persisted counter of presses since the last EEPROM write, and only call `eeprom_write()` once it reaches `5`, instead of on every press. `main()`'s event handling already separates "an event happened" from "write it out", build on that rather than restructuring it.

CHAPTER 8

The Watchdog Timer

A second timer whose only job is to restart a program that has stopped.

IN THIS CHAPTER

- The watchdog timer: a hardware safety net that resets the chip if the program stops "petting" it.
- Watchdog registers: `WDTCSR`, its own timed write sequence (`WDCE` then `WDE`), and the `WDR` instruction.
- Why a plain function is the right tool here: like EEPROM (Chapter 7), the watchdog is a singleton, there's exactly one on the chip.
- `WDRF`, the flag that can lock `WDE` on after a watchdog reset, and System Reset Mode vs. Interrupt Mode (`WDE` vs. `WDIE`).

This chapter is about the last peripheral of the survey, and the only one whose job is to catch the program itself failing. After it you should be able to explain what the watchdog timer protects against, and to enable it, reset it, and observe what happens when it isn't reset in time.

Read §8.1, the substance of the chapter; its worked example adds the watchdog to the EEPROM demo of §7.1.4. Then work the exercises in §8.3. The Reflection exercise is answered in Appendix B; the Code exercises have no printed solution.

The watchdog is the one topic in this book that's simpler to observe in simulation than on hardware: starve `watchdog_reset()` and the reboot loop shows up immediately, with no reflashing and no serial monitor. That is worth doing even with a board to hand, and the only option without one; the simulator guide of Appendix A walks through it in §A.7.1.

8.1 The Watchdog Timer

8.1.1 The watchdog timer: what it is and why

- A watchdog is a second, independent timer with one job: if it isn't reset ("petted") within its configured timeout, it resets the entire chip.
- The use cases are:

  - A program that might hang.
  - An infinite loop from a bug.
  - A sensor that never responds.
  - A peripheral wait-loop (like §7.1.2's `EEPE` wait, or Chapter 4's `ADIF` wait) that never sees its flag set.
- Without a watchdog, a hang like that freezes the device until someone power-cycles it by hand. With one, the chip recovers on its own within one timeout period.
- The watchdog timer follows this pattern:
  - Enable the watchdog with a timeout comfortably longer than one iteration of `main()`'s loop normally takes.
  - Reset the watchdog once per iteration of `main()`'s loop.
  - As long as the loop keeps running, the watchdog never fires. The moment it doesn't (because something hung), the watchdog's timeout expires and the chip restarts.

8.1.2 Watchdog registers

One register, **`WDTCSR`** (Watchdog Timer Control and Status Register), controls both enablement and timeout via bits `WDE` (Watchdog Enable) and `WDP0`–`WDP3` (Watchdog Prescaler, selecting the timeout duration).

The four prescaler bits, `WDP[3:0]`, set the timeout anywhere from 16 ms up to 8192 ms:

Timeout	`WDP[3:0]`	Value
16 ms	`0000`	0
32 ms	`0001`	1
64 ms	`0010`	2
128 ms	`0011`	3
256 ms	`0100`	4
512 ms	`0101`	5
1024 ms	`0110`	6
2048 ms	`0111`	7
4096 ms	`1000`	32
8192 ms	`1001`	33

The jump from 7 to 32 isn't a typo: `WDP2`–`WDP0` sit at contiguous bits 2–0 of `WDTCSR`, so the first eight timeouts (16 ms–2048 ms) map straight onto the integers 0–7. `WDP3`, needed for 4096 ms and 8192 ms, lives at bit 5 instead, so those two timeouts land on `1 << 5` (32) and `(1 << 5) | (1 << 0)` (33).

This maps cleanly onto a plain `enum`: the first eight members can take their default values (0–7, counting up in declaration order, same as §1.1.3's `sensor_state_t`), and only the last two need an explicit value:

```
typedef enum
{
    WATCHDOG_TIMEOUT_16MS,        ///< 16 ms timeout.
    WATCHDOG_TIMEOUT_32MS,        ///< 32 ms timeout.
    WATCHDOG_TIMEOUT_64MS,        ///< 64 ms timeout.
    WATCHDOG_TIMEOUT_128MS,       ///< 128 ms timeout.
    WATCHDOG_TIMEOUT_256MS,       ///< 256 ms timeout.
    WATCHDOG_TIMEOUT_512MS,       ///< 512 ms timeout.
    WATCHDOG_TIMEOUT_1024MS,      ///< 1024 ms timeout.
    WATCHDOG_TIMEOUT_2048MS,      ///< 2048 ms timeout.
    WATCHDOG_TIMEOUT_4096MS = 32, ///< 4096 ms timeout.
    WATCHDOG_TIMEOUT_8192MS = 33, ///< 8192 ms timeout.
} watchdog_timeout_t;
```

Cast a `watchdog_timeout_t` directly to `uint8_t` and write it to the `WDTCSR` prescaler bits. Its encoded value places the prescaler bits correctly for all ten timeout settings.

Clearing `WDE`, or changing the prescaler bits, needs the same kind of timed sequence as EEPROM's `EEMPE`/`EEPE` from Chapter 7, and for the same reason (setting `WDE` alone doesn't, but this book's code always writes `WDE` and the timeout together, so it always uses the sequence):

- It's a safety interlock, this time specifically so runaway code can't accidentally disable the one thing meant to catch it. `WDCE` (Watchdog Change Enable) must be set together with `WDE`, and the real configuration written within 4 cycles:

```
cli();
WDTCSR |= (1U << WDCE) | (1U << WDE);                    // Unlock the timed ↩
    ↪ sequence.
WDTCSR = (1U << WDE) | (uint8_t)(WATCHDOG_TIMEOUT_1024MS); // Enable watchdog, ↩
    ↪ 1024 ms timeout.
sei();
```

Resetting the countdown doesn't touch `WDTCSR` at all, it's a single dedicated CPU instruction, `WDR` (Watchdog Reset). Nothing about `WDR` lives behind a register bit, so there's no C statement (no `WDTCSR |= ...`) that produces it; the only way to emit a specific machine instruction from C is **inline assembly**, `asm(...)`, which drops a literal instruction straight into the compiled output, bypassing C entirely for that one line:

```
asm("WDR");
```

This is the simplest form GCC supports, "basic" inline assembly: just a literal instruction as a string, no inputs, no outputs, nothing declared as clobbered. (GCC also has "extended" inline assembly, with input/output operands and clobber lists, for wiring C variables into and

out of an `asm` block; `WDR` doesn't read or write anything C can see, so none of that machinery is needed here.)

One gotcha: `asm` is a GNU extension, not standard C, recognized only under a GNU dialect (`-std=gnu11`) or GCC's default mode. This book's demos compile with `-std=c11`, strict ISO C, where the compiler doesn't know the word `asm` at all and this line fails to build. `__asm__`, the double-underscore spelling, is always recognized regardless of `-std=`, so that's the form used from here on:

```
__asm__("wdr");
```

This book writes straight against the registers instead of reaching for a library, the same as every peripheral so far (UART, ADC, timers, EEPROM): no header between the code and the hardware, so nothing about what the chip is actually doing is hidden behind a macro.

```
#include <avr/interrupt.h>
#include <avr/io.h>

/**
 * @brief Enable the watchdog timer with a 1024 ms timeout.
 */
static void watchdog_enable(void)
{
    cli();
    WDTCSR |= (1U << WDCE) | (1U << WDE);
    WDTCSR = (1U << WDE) | (uint8_t)(WATCHDOG_TIMEOUT_1024MS);
    sei();
}

/**
 * @brief Reset the watchdog timer, restarting its countdown.
 */
static void watchdog_reset(void)
{
    __asm__("wdr");
}
```

8.1.3 `WDRF`: don't get locked out of disabling the watchdog

A watchdog-caused reset sets `WDRF` (Watchdog Reset Flag) in **`MCUSR`** (MCU Status Register). That flag isn't just informational: the datasheet has `WDE` read back as set for as long as `WDRF` is set, no matter what gets written to it. Try to clear `WDE` (or disable the watchdog some other way) while `WDRF` is still set, and it silently does nothing, the watchdog stays armed and fires again next timeout.

This is deliberate, not a bug: it stops a chip that just failed and reset from immediately turning its own safety net back off, which is exactly the failure mode a watchdog exists to catch. The practical consequence is that `WDRF` must be cleared explicitly, and early, before anything else touches `WDTCSR`:

```
MCUSR &= ~(1U << WDRF); // Clear the watchdog reset flag; WDE can't be cleared ↩
    ↪ while it's set.
```

Put this as the first line of `setup()`, unconditionally, before `watchdog_enable()` or any other watchdog call. Skip it, and a single watchdog-triggered reset can leave the chip unable to ever turn the watchdog off again, whatever the running code tries to do about it.

Clearing `WDRF` doesn't stop the watchdog, though, and after a watchdog reset it is still running. The reset cleared the prescaler bits, so it now runs at its shortest timeout, 16 ms. Configure the watchdog straight after clearing the flag, with the timeout you actually want (or turn it off), before anything slow: a `setup()` that first spends 16 ms on something else, and printing a single line at 9600 baud already takes that long, is reset again before it gets there, and again, forever. That's why §8.1.5's `setup()` calls `watchdog_init()`, which clears `WDRF` and then arms the watchdog, before anything else.

On an Uno you may never see this happen, because its bootloader clears `MCUSR` and switches the watchdog off before it starts your program. Don't rely on that: a program flashed without a bootloader, or run in a simulator, gets no such help. Nor do Nano boards with the older bootloader (the ones that need `-b 57600`, §2.3.7), which after a watchdog reset are known to get stuck resetting inside the bootloader itself, before your program ever runs.

8.1.4 System Reset Mode vs. Interrupt Mode

`WDE` and `WDIE` (Watchdog Interrupt Enable), both in `WDTCSR`, pick what a timeout actually does, independently of each other:

- `WDE` alone: **System Reset Mode**. Timeout resets the chip, same as §8.1.2's `watchdog_enable()`.
- `WDIE` alone: **Interrupt Mode**. Timeout jumps to `ISR(WDT_vect)` instead of resetting anything.
- Both set: timeout fires `ISR(WDT_vect)` *and* clears `WDIE` in hardware on the way in, leaving `WDE` armed. Pet the watchdog inside that ISR (or soon after) and the reset never happens; miss the next timeout too and the chip resets. That's one warning before the safety net actually trips.

Setting the prescaler bits still needs §8.1.2's `WDCE` timed sequence, same as `watchdog_enable()`, just with `WDE` left at `0` in the value that gets committed. `WDIE` itself isn't behind that sequence, so it's set separately, straight after:

```
/**
 * @brief Enable the watchdog in Interrupt Mode: ISR(WDT_vect) runs on timeout ↪
    ↪ instead of a
 *        system reset.
 *
 * @param[in] timeout Timeout to arm the watchdog with (see A.2's table).
 */
static void watchdog_enable_interrupt(const watchdog_timeout_t timeout)
{
    watchdog_reset();
    cli();
    WDTCSR |= (1U << WDCE) | (1U << WDE); // Unlock the timed sequence.
    WDTCSR = (uint8_t)(timeout);          // Commit prescaler bits, WDE left at 0.
    sei();
    WDTCSR |= (1U << WDIE);               // Not part of the timed sequence.
}
```

8.1.5 Worked example: petting the watchdog in Chapter 7's demo

Extends §7.1.4's button-toggles-LED1-and-persists-to-EEPROM demo with a 1024 ms watchdog (System Reset Mode, §8.1.2) on top. `setup()` calls `watchdog_init(WATCHDOG_TIMEOUT_1024MS)` once, before anything else, for the reason §8.1.3 gives; `main()`'s loop calls `watchdog_reset()` first thing, every iteration, whether or not a button event happened that iteration, same as §8.1.1's pattern. The circuit is the one of Figure 6.1, unchanged.

```
watchdog_driver_demo/
├── Makefile
├── main.c
├── include/
│   └── driver/
│       ├── eeprom.h
│       ├── serial.h
│       └── watchdog.h
└── source/
    └── driver/
        ├── eeprom.c
        ├── serial.c
        └── watchdog.c
```

The serial and EEPROM drivers were introduced in Chapters 6 and 7 respectively. `driver/watchdog.h` and `driver/watchdog.c`, printed in full below, are §8.1.2's `watchdog_timeout_t` plus `watchdog_init()`/`watchdog_reset()`, nothing more.

The driver in full

lectures/L08/watchdog_driver_demo/include/driver/watchdog.h

```
/**
 * @file Watchdog timer driver.
 */
#ifndef DRIVER_WATCHDOG_H_
#define DRIVER_WATCHDOG_H_

/**
 * @brief Enumeration of watchdog timeouts.
 */
typedef enum
{
    WATCHDOG_TIMEOUT_16MS,        ///< 16 ms timeout.
    WATCHDOG_TIMEOUT_32MS,        ///< 32 ms timeout.
    WATCHDOG_TIMEOUT_64MS,        ///< 64 ms timeout.
    WATCHDOG_TIMEOUT_128MS,       ///< 128 ms timeout.
    WATCHDOG_TIMEOUT_256MS,       ///< 256 ms timeout.
    WATCHDOG_TIMEOUT_512MS,       ///< 512 ms timeout.
    WATCHDOG_TIMEOUT_1024MS,      ///< 1024 ms timeout.
    WATCHDOG_TIMEOUT_2048MS,      ///< 2048 ms timeout.
    WATCHDOG_TIMEOUT_4096MS = 32, ///< 4096 ms timeout.
    WATCHDOG_TIMEOUT_8192MS = 33, ///< 8192 ms timeout.
} watchdog_timeout_t;

/**
 * @brief Initialize the watchdog timer.
 *
 * @param[in] timeout Watchdog timeout.
 */
void watchdog_init(watchdog_timeout_t timeout);

/**
 * @brief Reset the watchdog timer.
 */
void watchdog_reset(void);

#endif /** DRIVER_WATCHDOG_H_ */
```

lectures/L08/watchdog_driver_demo/source/driver/watchdog.c

```c
/**
 * @file Watchdog timer driver implementation details.
 */
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>

#include "driver/watchdog.h"

// ----------------------------------------------------------------------------
void watchdog_init(const watchdog_timeout_t timeout)
{
    // Clear the reset flag before anything else touches the watchdog.
    MCUSR &= ~(1U << WDRF);

    // Update the watchdog timeout.
    cli();
    WDTCSR |= (1U << WDCE) | (1U << WDE);
    WDTCSR = (1U << WDE) | (uint8_t)(timeout);
    sei();
}

// ----------------------------------------------------------------------------
void watchdog_reset(void) { __asm__("wdr"); }
```

The program

`main.c` in full:

lectures/L08/watchdog_driver_demo/main.c

```c
/**
 * @file Watchdog driver demo.
 */
#include <stdbool.h>
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>

#include "driver/eeprom.h"
#include "driver/serial.h"
#include "driver/watchdog.h"
```

```
#define LED1 1U // D9  -> PORTB1.
#define BTN1 5U // D13 -> PORTB5.

/** GPIO operations. */
#define LED1_TOGGLE PINB = (1U << LED1)    // Toggle LED1.
#define LED1_ENABLED (PINB & (1U << LED1)) // High if LED1 is enabled, low ↩
    ↪ otherwise.
#define BTN1_PRESSED (PINB & (1U << BTN1)) // High if BTN1 is pressed, low ↩
    ↪ otherwise.

/** Timer operations. */
#define TIMER0_ENABLE TIMSK0 = (1U << TOIE0) // Enable timer 0 interrupt.
#define TIMER0_DISABLE TIMSK0 = 0U           // Disable timer 0 interrupt.

/** Interrupt operations. */
#define BTN1_INT_ENABLE PCMSK0 |= (1U << BTN1)   // Enable pin change interrupt ↩
    ↪ for BTN1.
#define BTN1_INT_DISABLE PCMSK0 &= ~(1U << BTN1) // Disable pin change interrupt ↩
    ↪ for BTN1.

/** Time parameters */
#define TICK_PERIOD_MS 0.064F    // Time between each tick, prescaler 1024.
#define TICK_MAX 256U            // Ticks per overflow for 8-bit timer.
#define DEBOUNCE_TIMEOUT_MS 300U // Debounce timeout in ms.

/** Limit parameters. */
#define OVF_TIME_MS (TICK_PERIOD_MS * TICK_MAX)     // Time between each overflow ↩
    ↪ in ms.
#define OVF_MAX (DEBOUNCE_TIMEOUT_MS / OVF_TIME_MS) // Overflows needed for ↩
    ↪ timeout.
#define OVF_TIMEOUT (uint8_t)(OVF_MAX + 0.5F)       // Overflows needed for ↩
    ↪ timeout, rounded.

/** EEPROM parameters. */
#define EEPROM_LED1_ADDR 1000U // EEPROM address containing the LED1 state.
#define EEPROM_LED1_ON 1U      // Stored value indicating that LED1 is on.
#define EEPROM_LED1_OFF 0U     // Stored value indicating that LED1 is off.

/** Flags that a LED1 event has occurred. */
static volatile bool led1_event = false;

/**
 * @brief Print the LED1 state over UART.
```

```
 */
static void print_led1_state(void)
{
    if (LED1_ENABLED) { serial_print("LED1 enabled!\n"); }
    else { serial_print("LED1 disabled!\n"); }
}

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Initialize the watchdog first: after a watchdog reset it is still running, ↩
    ↪ with a 16 ms
    // timeout, and the rest of setup() takes longer than that.
    watchdog_init(WATCHDOG_TIMEOUT_1024MS);

    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Configure BTN1 as input with its internal pull-up enabled.
    PORTB = (1U << BTN1);

    // Enable pin change interrupt for BTN1.
    PCICR = (1U << PCIE0);
    BTN1_INT_ENABLE;

    // Set up 300 ms debounce timer.
    TCCR0B = (1U << CS00) | (1U << CS02);

    // Initialize serial driver.
    serial_init();

    // Restore the previous LED state from EEPROM.
    uint8_t led_state = 0U;
    if (sizeof(led_state) == eeprom_read(&led_state, sizeof(led_state), ↩
    ↪ EEPROM_LED1_ADDR))
    {
        if (EEPROM_LED1_ON == led_state) { LED1_TOGGLE; }
    }
    print_led1_state();

    // Enable interrupts globally.
    sei();
```

```
}

/**
 * @brief Toggle LED1 on button press, and flag the event for main() to report.
 *
 *        Disable button interrupts for 300 ms to suppress contact bounce.
 */
ISR(PCINT0_vect)
{
    // Disable button interrupts for 300 ms.
    BTN1_INT_DISABLE;
    TIMER0_ENABLE;

    // Toggle LED1 and flag the event if BTN1 is pressed.
    if (BTN1_PRESSED)
    {
        LED1_TOGGLE;
        led1_event = true;
    }
}

/**
 * @brief Re-enable button interrupts after 300 ms.
 */
ISR(TIMER0_OVF_vect)
{
    static volatile uint8_t ovf_counter = 0U;

    // Wait for 300 ms, then re-enable button interrupts and disable timer 0.
    if (OVF_TIMEOUT <= ++ovf_counter)
    {
        BTN1_INT_ENABLE;
        TIMER0_DISABLE;
        ovf_counter = 0U;
    }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
```

```
    setup();

    while (1)
    {
        // Reset the watchdog once every iteration of the loop.
        watchdog_reset();

        // Check if a LED1 event has occurred, store the new state in EEPROM if ↩
    ↪ true.
        if (led1_event)
        {
            const uint8_t led_state = LED1_ENABLED ? EEPROM_LED1_ON : ↩
    ↪ EEPROM_LED1_OFF;
            eeprom_write(&led_state, sizeof(led_state), EEPROM_LED1_ADDR);
            led1_event = false;

            // Report the new LED1 state over UART.
            print_led1_state();
        }
    }
    return 0;
}
```

Try commenting out just the `watchdog_reset();` call and reflashing: nothing about the rest of the code looks wrong, `setup()` still runs, the button still toggles LED1 and reports over UART, but roughly every 1024 ms the chip reboots on its own, visible as `setup()`'s LED1-state report firing over and over in the serial monitor even with nobody touching the button. That's §8.1.1's watchdog doing exactly what it's for: the timeout wasn't pet in time, so it fired.

This is the one experiment in the book that's simpler to run in the simulator than on the board, and it's worth doing even if you have hardware: no reflashing between the two versions and no serial monitor to wire up, so the before-and-after is seconds apart instead of minutes. Petting the watchdog gives one state report and then silence; starving it gives the same report over and over. See Appendix A, which walks through this demo specifically in §A.7.1.

Building and running

Build and flash the same way as §6.1.6: on Linux, run `make` in the demo's directory, `lectures/L08/watchdog_driver_demo` (`make flash` to program the board); on Windows via Microchip Studio, recreate this folder structure inside the project and add `include` as an include directory. The demo's `Makefile` is the one printed in §6.1.6, with the EEPROM and watchdog drivers added to its source files:

```
SRC_FILES := main.c \
             source/driver/eeprom.c \
             source/driver/serial.c \
             source/driver/watchdog.c \
```

8.1.6 Why no structs here

UART (Chapter 6), EEPROM (Chapter 7) and the watchdog all get away with plain functions and no struct, unlike where this book is headed next. The ATmega328P has exactly one UART, exactly one EEPROM and exactly one watchdog; there's no second instance to distinguish between, no `self` parameter needed, nothing to bundle. A struct would just be ceremony around state that only ever exists once.

LEDs, buttons, and timers aren't so lucky, a program regularly wants several of each, and Chapter 4's five-LED exercise (Exercise 4.7) and Chapter 5's three-timer exercise (Exercise 5.7) already ran into what that means without a better tool. That's exactly the gap Chapter 9 closes.

8.2 Review

What you should be able to do now

After this chapter, you should be able to:

- Explain what the watchdog timer protects against, and enable it, reset it, and observe what happens when it isn't reset in time.

Questions to test yourself

1. What is the watchdog timer for, and what's the practical difference between a program that resets it every loop iteration and one that never resets it at all?

Looking ahead

Now that EEPROM and the watchdog have shown what plain functions look like for a peripheral with exactly one instance, Chapter 9 covers what changes once a program needs several instances of the same kind of peripheral: structs as drivers.

8.3 Exercises

> **How to work these.** The Reflection exercise is answered on paper, from the chapter. The Code exercises are programs for the board: build and flash them the way §2.2 (Microchip Studio) or §2.3 (Linux) describes, or run them in the simulator of Appendix A if you have no board yet. The simulator is the quicker way to see a watchdog reset (§A.7.1), though its reboot loop runs faster than the board's.
> **How to check your work.** The Reflection exercise is answered in Appendix B. The Code exercises have no printed solution: the board tells you whether a program does what the exercise describes. Try each exercise before you read its answer.

EXERCISE SET 1

Concepts

Exercise 8.1 Why the watchdog is locked REFLECTION

The watchdog's `WDE`/prescaler bits need the same `WDCE`-then-write-within-four-cycles pattern as EEPROM's `EEMPE`/`EEPE` (Chapter 7). As with EEPROM, this isn't just a hardware quirk, it's a deliberate safety feature. What is it protecting against?

EXERCISE SET 2

Program

Exercise 8.2 A loop that is too slow CODE

Using §8.1.5's demo, add a `_delay_ms(1500)` (`<util/delay.h>`) inside `main()`'s loop, longer than the watchdog's configured 1024 ms timeout. Confirm the board still resets on its own, even though `watchdog_reset()` is still called every iteration, just not often enough. Calling it somewhere in the code isn't enough; the loop has to actually get back around to it in time.

Exercise 8.3 Every tenth press CODE

Capstone, harder. Combine EEPROM (Chapter 7) and the watchdog: reduce EEPROM wear by only writing every 10th button press instead of every press. Keep a small `uint8_t` counter (not itself persisted) that tracks presses since the last EEPROM write, and only call `eeprom_write()` once it reaches `10`.

Exercise 8.4 A warning before the reset CODE

Capstone, harder. §8.1.5's driver only supports System Reset Mode. Extend it yourself with Interrupt Mode (§8.1.4):

- Add a way to arm the watchdog with `WDIE` set instead of `WDE`, plus an `ISR(WDT_vect)`.
- Then combine both modes (`WDE` and `WDIE` together) so the first missed timeout logs a warning over UART instead of resetting, and only a second consecutive missed timeout actually resets the chip.

CHAPTER 9

Structs as Drivers

One struct per instance, and one set of functions for all of them.

IN THIS CHAPTER

- Why Chapter 4's ADC and Chapters 6–8's UART/EEPROM/watchdog functions got away with no struct (they're singletons), and why per-instance macros and globals stop scaling for everything else: revisiting Chapter 4's multi-LED exercise.
- Bundling a peripheral's state into a struct, and writing "methods" as functions that take a pointer to that struct as their first argument.
- Building a GPIO (`gpio_t`) driver, split into a header and a source file.
- Worked example: rebuilding Chapter 8's watchdog demo on `gpio_t`, adding pin-change-interrupt support to the GPIO driver along the way.
- Exercise: a complete `timer_t` driver, with its own header/source split and a callback, built the same way as `gpio_t`.

Every driver so far has served exactly one of something: one ADC, one UART, one EEPROM, one watchdog, and LEDs and buttons hardcoded into macros one pin at a time. This chapter is about what to do when a program needs several of the same thing. After it you should be able to explain why hardcoded per-instance functions or macros break down once a program needs more than one of something, design a struct that bundles a peripheral's configuration and state, write "methods" as plain functions taking a pointer to that struct as their first parameter (the `self` convention), and refactor a program built on duplicated globals into one built on an array of struct instances.

Read §9.1, the substance of the chapter, in order: it builds a GPIO driver one step at a time and ends with Chapter 8's demo rebuilt on it. Then work the exercises in §9.3. The Reflection exercises, and the design questions the Code exercises ask, are answered in Appendix B; the Code exercises have no printed solution.

9.1 Structs as Drivers

9.1.1 Why bundle state

- The drivers covered in Chapters 4 and 6–8 — the ADC, UART, EEPROM, and the watchdog — did not need any structs:
  - The chip has exactly one ADC (`adc_read()` already takes a channel number as a parameter, so it was never hardcoded to a single pin the way the examples below are).
  - It has exactly one UART.
  - It has exactly one EEPROM.
  - It has exactly one watchdog.
  - Because there is only one instance of each peripheral, plain functions operating on implicit global state were the simplest correct solution.
- The same single-instance pattern shows up again with peripherals the chip actually has many of, where it isn't the right solution:
  - `LED_ON` assumes one specific LED, even though the chip has many GPIO pins.
  - The configuration is effectively hardcoded into macros, functions, and global state.
- This pattern works well as long as the program needs only one instance of each thing.
- The pattern starts to break down when the program needs several instances:
  - §4.3 contains an exercise with five LEDs, Exercise 4.7.
  - Without a better abstraction, adding another LED means copying an entire family of macros or functions and renaming them.
  - That quickly becomes repetitive, difficult to maintain, and easy to get wrong.
- C already provides the tool needed to solve this problem: the `struct`:
  - Store one peripheral instance's configuration and state in a `struct`.
  - Create one struct value for each LED (or other multi-instance peripheral) instance.
  - Write ordinary functions that receive a pointer to the relevant struct.
  - The same functions can then operate on several different instances.
- By convention, the pointer parameter is called `self`:
  - `self` points to the particular peripheral instance that the function should operate on.
  - It plays a role similar to `this` in languages with built-in object support, such as C++, C#, and Java.
  - Unlike `this`, however, `self` is just an ordinary function parameter written explicitly in the code.
- However, C has no built-in support for object-oriented programming:
  - There is no hidden behaviour.
  - There are no constructors.

  - There is no inheritance.
  - There is no automatic method dispatch.
  - There is only a `struct` containing data and a set of ordinary functions that accept a pointer to that data.
- This chapter keeps the design deliberately simple:
  - The struct's fields remain directly accessible.
  - Functions are called normally.
  - No function pointers are required.
- The next chapter, Chapter 10, explores how much further this approach can be taken:
  - Hiding a struct's internal fields.
  - Calling operations through function pointers.
  - Creating a more object-like interface while still using C.

9.1.2 The `self` convention: a `GPIO` driver

A minimal driver is a struct holding whatever a function needs to know to act on one specific instance: here, pointers to the GPIO's data direction (`DDRx`), output (`PORTx`), and input (`PINx`) registers, plus the bit position of its pin within them.

See §1.1.3 for a refresher on structs and `typedef`; `gpio_t` follows the same anonymous-struct-plus-`_t`-suffix convention introduced there.

```
/**
 * @brief GPIO driver structure.
 */
typedef struct
{
    /** Pointer to the data direction register associated with the GPIO. */
    volatile uint8_t* ddrx;

    /** Pointer to the output register associated with the GPIO. */
    volatile uint8_t* portx;

    /** Pointer to the input register associated with the GPIO. */
    volatile uint8_t* pinx;

    /** Bit position of the GPIO pin within the associated port. */
    const uint8_t pin;
} gpio_t;

/**
 * @brief Enumeration of GPIO modes.
```

```
 */
typedef enum
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/**
 * @brief Initialize GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] mode GPIO mode.
 */
void gpio_init(gpio_t* self, const gpio_mode_t mode)
{
    if (NULL == self) { return; }

    switch (mode)
    {
        case GPIO_MODE_INPUT_PULLUP:
            *self->ddrx &= ~(1U << self->pin);
            *self->portx |= (1U << self->pin);
            break;
        case GPIO_MODE_OUTPUT:
            *self->ddrx |= (1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
        default:
            *self->ddrx &= ~(1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
    }
}

/**
 * @brief Set state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] state GPIO state (true = high, false = low).
 *
 * @note This operation is only supported for GPIOs configured as outputs.
 */
void gpio_write(gpio_t* self, const bool state)
```

```
{
    if (NULL == self) { return; }
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }

    if (state) { *self->portx |= (1U << self->pin); }
    else { *self->portx &= ~(1U << self->pin); }
}

/**
 * @brief Read state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 *
 * @return GPIO state (true = high, false = low).
 */
bool gpio_read(const gpio_t* self)
{
    return (NULL != self) ? (*self->pinx & (1U << self->pin)) : false;
}

/**
 * @brief Toggle state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 */
void gpio_toggle(gpio_t* self)
{
    if (NULL == self) { return; }
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }
    *self->pinx = (1U << self->pin);
}
```

A GPIO output instance can then be initialized as shown below:

```
#define LED1_PIN 1U // D9 -> PORTB1.

/** Initialize LED on D9 (PORTB1). */
gpio_t led1 = {&DDRB, &PORTB, &PINB, LED1_PIN};

gpio_init(&led1, GPIO_MODE_OUTPUT);
```

Most of these functions take `gpio_t*` rather than `gpio_t`: passing a pointer avoids copying the struct on every call, and they need to write through `self` to the hardware registers it points at. `gpio_read()` takes `const gpio_t*` instead, since it only ever reads `self`, never writes through it.

9.1.3 Splitting the driver into `gpio.h` and `gpio.c`

So far §9.1.2's `gpio_t`, `gpio_mode_t`, and every `gpio_*()` function have lived together in one code block. In a real project a driver is split across a header and a source file, the same split §1.1.6 introduced for `sensor.h`/`sensor.c`, include guard and all:

- The header (`gpio.h`) is the public interface: the `gpio_t` and `gpio_mode_t` type definitions, plus a documented prototype for every function callers are allowed to use.
- The source file (`gpio.c`) `#includes` its own header, after the standard and AVR headers as everywhere in this book, then defines those same functions. Callers only ever see the header; the definitions in `gpio.c` are what actually gets compiled.

One thing is new compared to `sensor.h`: that header only ever declared a function, callers never touched anything but the value it returned. `gpio.h` also declares `gpio_t` itself, because a caller needs to create and initialize their own `gpio_t` instances directly (as in §9.1.2's `led1` example), not just receive values back from an opaque handle. That's also why `gpio_t`'s fields stay public here instead of hidden behind a pointer to an incomplete type; Chapter 10 revisits that trade-off.

Here's `driver/gpio.h`:

```
/**
 * @file GPIO driver.
 */
#ifndef DRIVER_GPIO_H_
#define DRIVER_GPIO_H_

#include <stdbool.h>
#include <stdint.h>

/**
 * @brief GPIO driver structure.
 */
typedef struct
{
    /** Pointer to the data direction register associated with the GPIO. */
    volatile uint8_t* ddrx;

    /** Pointer to the output register associated with the GPIO. */
    volatile uint8_t* portx;
```

```
    /** Pointer to the input register associated with the GPIO. */
    volatile uint8_t* pinx;

    /** Bit position of the GPIO pin within the associated port. */
    const uint8_t pin;
} gpio_t;

/**
 * @brief Enumeration of GPIO modes.
 */
typedef enum
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/**
 * @brief Initialize GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] mode GPIO mode.
 */
void gpio_init(gpio_t* self, gpio_mode_t mode);

/**
 * @brief Set state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] state GPIO state (true = high, false = low).
 *
 * @note This operation is only supported for GPIOs configured as outputs.
 */
void gpio_write(gpio_t* self, bool state);

/**
 * @brief Read state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 *
 * @return GPIO state (true = high, false = low).
 */
bool gpio_read(const gpio_t* self);
```

```
/**
 * @brief Toggle state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 */
void gpio_toggle(gpio_t* self);

#endif /** DRIVER_GPIO_H_ */
```

Here's `driver/gpio.c`:

```
/**
 * @file GPIO driver implementation details.
 */
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include <avr/io.h>

#include "driver/gpio.h"

// ----------------------------------------------------------------------------
void gpio_init(gpio_t* self, const gpio_mode_t mode)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Configure GPIO mode as specified.
    switch (mode)
    {
        case GPIO_MODE_INPUT_PULLUP:
            // Configure GPIO as input with its internal pull-up enabled.
            *self->ddrx &= ~(1U << self->pin);
            *self->portx |= (1U << self->pin);
            break;
        case GPIO_MODE_OUTPUT:
            // Configure GPIO as output.
            *self->ddrx |= (1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
        default:
            // Configure GPIO as standard input.
```

```
            *self->ddrx &= ~(1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
    }
}

// ----------------------------------------------------------------------------
void gpio_write(gpio_t* self, const bool state)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Check GPIO direction, terminate if not output.
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }

    // Set GPIO state as specified.
    if (state) { *self->portx |= (1U << self->pin); }
    else { *self->portx &= ~(1U << self->pin); }
}

// ----------------------------------------------------------------------------
bool gpio_read(const gpio_t* self)
{
    // Return GPIO state if GPIO instance is valid, else false.
    return (NULL != self) ? (*self->pinx & (1U << self->pin)) : false;
}

// ----------------------------------------------------------------------------
void gpio_toggle(gpio_t* self)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Check GPIO direction, terminate if not output.
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }

    // Toggle GPIO state.
    *self->pinx = (1U << self->pin);
}
```

Notice `gpio_init()`'s `mode` parameter (and `gpio_write()`'s `state`) drop the `const` in the header but keep it in the source file. A top-level `const` on a parameter only constrains what

the function body can do with its own local copy; it isn't part of the function's type and can't affect how callers call it, so by convention it's omitted from the prototype and kept in the definition, where it actually matters.

9.1.4 Worked example: an array of LEDs

The whole point of a driver struct is that `gpio_write()/gpio_read()` don't care which GPIO they're called on. An array of `gpio_t` instances plus a loop replaces a whole family of per-LED macros:

```
#define LED_COUNT 5U

// Create five LED instances.
static gpio_t leds[LED_COUNT] =
{
    { &DDRD, &PORTD, &PIND, 6U}, // D6  -> PORTD6.
    { &DDRD, &PORTD, &PIND, 7U}, // D7  -> PORTD7.
    { &DDRB, &PORTB, &PINB, 0U}, // D8  -> PORTB0.
    { &DDRB, &PORTB, &PINB, 1U}, // D9  -> PORTB1.
    { &DDRB, &PORTB, &PINB, 2U}, // D10 -> PORTB2.
};

/**
 * @brief Set up system.
 */
static void setup(void)
{
    for (uint8_t i = 0U; i < LED_COUNT; i++)
    {
        gpio_init(&leds[i], GPIO_MODE_OUTPUT);
    }
}
```

Compare this to §4.3's five-LED exercise, Exercise 4.7, which needed a hand-written `LED1_ON` through `LED5_ON` (and `_OFF`), one pair of macros per pin. Here, adding a sixth LED on the same port means adding one line to the `leds[]` array, no new functions.

9.1.5 Worked example: `gpio_t` wired into Chapters 6–8's demo

Combines Chapter 6's serial driver, Chapter 7's EEPROM driver, and Chapter 8's watchdog driver with this section's `gpio_t`: LED1 (D9) persists its state across resets via EEPROM, BTN1 (D13) toggles LED1 through a debounced pin-change interrupt exactly like §8.1.5, and the watchdog resets the chip if the main loop ever stalls. Every register access that went through Chapter 8's `LED1_TOGGLE/LED1_ENABLED/BTN1_PRESSED` macros now goes through `gpio_toggle()/gpio_read()` instead.

The circuit is Chapter 6's, in Figure 6.1.

```
gpio_driver_demo/
├── Makefile
├── main.c
├── include/
│   └── driver/
│       ├── eeprom.h
│       ├── gpio.h
│       ├── serial.h
│       └── watchdog.h
└── source/
    └── driver/
        ├── eeprom.c
        ├── gpio.c
        ├── serial.c
        └── watchdog.c
```

- The serial, EEPROM, and watchdog drivers were introduced in Chapters 6–8.
- `driver/gpio.h` and `driver/gpio.c`, both printed in full below, extend the `gpio_t` type introduced in §9.1.2 and §9.1.3.
- The extended driver adds one capability not covered in those sections:
  - `gpio_enable_pci()` enables or disables the pin-change interrupt for a GPIO.
  - It does this by writing the same `PCICR` and `PCMSKx` bits that were configured manually in §8.1.5.
- Supporting `gpio_enable_pci()` requires two additional fields in `gpio_t`, beyond `ddrx`, `portx`, `pinx`, and `pin`:
  - A pointer to the port's `PCMSKx` register.
  - The bit position of the port's `PCIEx` bit in `PCICR`.
- Every `gpio_t` instance contains these two additional fields:
  - This is true even when that particular GPIO never uses `gpio_enable_pci()`.
  - For example, LED1 still carries the pin-change-interrupt configuration fields.

The driver in full

lectures/L09/gpio_driver_demo/include/driver/gpio.h

```
/**
 * @file GPIO driver.
 */
#ifndef DRIVER_GPIO_H_
#define DRIVER_GPIO_H_
```

```c
#include <stdbool.h>
#include <stdint.h>

/**
 * @brief GPIO driver structure.
 */
typedef struct
{
    /** Pointer to the data direction register associated with the GPIO. */
    volatile uint8_t* ddrx;

    /** Pointer to the output register associated with the GPIO. */
    volatile uint8_t* portx;

    /** Pointer to the input register associated with the GPIO. */
    volatile uint8_t* pinx;

    /** Pointer to the pin change mask register associated with the GPIO. */
    volatile uint8_t* pcmskx;

    /** Pin change interrupt control register bit (PCIEx) associated with the GPIO's port. */
    const uint8_t pcicrx;

    /** Bit position of the GPIO pin within the associated port. */
    const uint8_t pin;
} gpio_t;

/**
 * @brief Enumeration of GPIO modes.
 */
typedef enum
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/**
 * @brief Initialize GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] mode GPIO mode.
```

```
 */
void gpio_init(gpio_t* self, gpio_mode_t mode);

/**
 * @brief Set state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] state GPIO state (true = high, false = low).
 *
 * @note This operation is only supported for GPIOs configured as outputs.
 */
void gpio_write(gpio_t* self, bool state);

/**
 * @brief Read state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 *
 * @return GPIO state (true = high, false = low).
 */
bool gpio_read(const gpio_t* self);

/**
 * @brief Toggle state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 */
void gpio_toggle(gpio_t* self);

/**
 * @brief Enable/disable pin change interrupts for the given GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] enable True to enable pin change interrupts, false otherwise.
 */
void gpio_enable_pci(gpio_t* self, bool enable);

#endif /** DRIVER_GPIO_H_ */
```

lectures/L09/gpio_driver_demo/source/driver/gpio.c

```c
/**
 * @file GPIO driver implementation details.
 */
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include <avr/io.h>

#include "driver/gpio.h"

// ----------------------------------------------------------------------------
void gpio_init(gpio_t* self, const gpio_mode_t mode)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Configure GPIO mode as specified.
    switch (mode)
    {
        case GPIO_MODE_INPUT_PULLUP:
            // Configure GPIO as input with its internal pull-up enabled.
            *self->ddrx &= ~(1U << self->pin);
            *self->portx |= (1U << self->pin);
            break;
        case GPIO_MODE_OUTPUT:
            // Configure GPIO as output.
            *self->ddrx |= (1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
        default:
            // Configure GPIO as standard input.
            *self->ddrx &= ~(1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
    }
}

// ----------------------------------------------------------------------------
void gpio_write(gpio_t* self, const bool state)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }
```

```
    // Check GPIO direction, terminate if not output.
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }

    // Set GPIO state as specified.
    if (state) { *self->portx |= (1U << self->pin); }
    else { *self->portx &= ~(1U << self->pin); }
}

// ----------------------------------------------------------------------------
bool gpio_read(const gpio_t* self)
{
    // Return GPIO state if GPIO instance is valid, else false.
    return (NULL != self) ? (*self->pinx & (1U << self->pin)) : false;
}

// ----------------------------------------------------------------------------
void gpio_toggle(gpio_t* self)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Check GPIO direction, terminate if not output.
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }

    // Toggle GPIO state.
    *self->pinx = (1U << self->pin);
}

// ----------------------------------------------------------------------------
void gpio_enable_pci(gpio_t* self, const bool enable)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Enable/disable interrupts for the given pin as specified.
    if (enable)
    {
        PCICR |= (1U << self->pcicrx);
        *self->pcmskx |= (1U << self->pin);
    }
    else { *self->pcmskx &= ~(1U << self->pin); }
```

```
}
```

`main.c` in full

lectures/L09/gpio_driver_demo/main.c

```
/**
 * @file GPIO driver demo.
 */
#include <stdbool.h>
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>

#include "driver/eeprom.h"
#include "driver/gpio.h"
#include "driver/serial.h"
#include "driver/watchdog.h"

#define LED1_PIN 1U // D9  -> PORTB1.
#define BTN1_PIN 5U // D13 -> PORTB5.

/** Timer operations. */
#define TIMER0_ENABLE TIMSK0 = (1U << TOIE0) // Enable timer 0 interrupt.
#define TIMER0_DISABLE TIMSK0 = 0U           // Disable timer 0 interrupt.

/** Time parameters */
#define TICK_PERIOD_MS 0.064F    // Time between each tick, prescaler 1024.
#define TICK_MAX 256U            // Ticks per overflow for 8-bit timer.
#define DEBOUNCE_TIMEOUT_MS 300U // Debounce timeout in ms.

/** Limit parameters. */
#define OVF_TIME_MS (TICK_PERIOD_MS * TICK_MAX)     // Time between each overflow ↩
    ↪ in ms.
#define OVF_MAX (DEBOUNCE_TIMEOUT_MS / OVF_TIME_MS) // Overflows needed for ↩
    ↪ timeout.
#define OVF_TIMEOUT (uint8_t)(OVF_MAX + 0.5F)       // Overflows needed for ↩
    ↪ timeout, rounded.

/** EEPROM parameters. */
#define EEPROM_LED1_ADDR 1000U // EEPROM address containing the LED1 state.
#define EEPROM_LED1_ON 1U      // Stored value indicating that LED1 is on.
#define EEPROM_LED1_OFF 0U     // Stored value indicating that LED1 is off.
```

```
/** Flags that a LED1 event has occurred. */
static volatile bool led1_event = false;

/** GPIO instances. */
static gpio_t led1 = {&DDRB, &PORTB, &PINB, &PCMSK0, PCIE0, LED1_PIN};
static gpio_t btn1 = {&DDRB, &PORTB, &PINB, &PCMSK0, PCIE0, BTN1_PIN};

/**
 * @brief Print the LED1 state over UART.
 */
static void print_led1_state(void)
{
    if (gpio_read(&led1)) { serial_print("LED1 enabled!\n"); }
    else { serial_print("LED1 disabled!\n"); }
}

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Initialize the watchdog first: after a watchdog reset it is still running, ↩
    ↪ with a 16 ms
    // timeout, and the rest of setup() takes longer than that.
    watchdog_init(WATCHDOG_TIMEOUT_1024MS);

    // Initialize GPIOs.
    gpio_init(&led1, GPIO_MODE_OUTPUT);
    gpio_init(&btn1, GPIO_MODE_INPUT_PULLUP);
    gpio_enable_pci(&btn1, true);

    // Set up 300 ms debounce timer.
    TCCR0B = (1U << CS00) | (1U << CS02);

    // Initialize serial driver.
    serial_init();

    // Restore the previous LED state from EEPROM.
    uint8_t led_state = 0U;
    if (sizeof(led_state) == eeprom_read(&led_state, sizeof(led_state), ↩
    ↪ EEPROM_LED1_ADDR))
    {
        if (EEPROM_LED1_ON == led_state) { gpio_toggle(&led1); }
```

```
    }
    print_led1_state();

    // Enable interrupts globally.
    sei();
}

/**
 * @brief Toggle LED1 on button press, and flag the event for main() to report.
 *
 *        Disable button interrupts for 300 ms to suppress contact bounce.
 */
ISR(PCINT0_vect)
{
    // Disable button interrupts for 300 ms.
    gpio_enable_pci(&btn1, false);
    TIMER0_ENABLE;

    // Toggle LED1 and flag the event if BTN1 is pressed.
    if (gpio_read(&btn1))
    {
        gpio_toggle(&led1);
        led1_event = true;
    }
}

/**
 * @brief Re-enable button interrupts after 300 ms.
 */
ISR(TIMER0_OVF_vect)
{
    static volatile uint8_t ovf_counter = 0U;

    // Wait for 300 ms, then re-enable button interrupts and disable timer 0.
    if (OVF_TIMEOUT <= ++ovf_counter)
    {
        gpio_enable_pci(&btn1, true);
        TIMER0_DISABLE;
        ovf_counter = 0U;
    }
}

/**
 * @brief Application entry point.
```

```
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        // Reset the watchdog once every iteration of the loop.
        watchdog_reset();

        // Check if a LED1 event has occurred, store the new state in EEPROM if ↩
    ↪ true.
        if (led1_event)
        {
            const uint8_t led_state = gpio_read(&led1) ? EEPROM_LED1_ON : ↩
    ↪ EEPROM_LED1_OFF;
            eeprom_write(&led_state, sizeof(led_state), EEPROM_LED1_ADDR);
            led1_event = false;

            // Report the new LED1 state over UART.
            print_led1_state();
        }
    }
    return 0;
}
```

`led1_event`, the debounce timer, and the EEPROM-persistence logic are unchanged from §8.1.5; only the GPIO handling changed, from bespoke macros tied to `PORTB` specifically to a driver that works for however many `gpio_t` instances a program declares, on whichever pins and ports they happen to be wired to.

Building and running

Build and flash the same way as §8.1.5: on Linux, run `make` in the demo's directory, `lectures/L09/gpio_driver_demo` (`make flash` to program the board); on Windows via Microchip Studio, recreate this folder structure inside the project and add `include` as an include directory.

The demo's `Makefile` is the one printed in §6.1.6, with `source/driver/eeprom.c`, `source/driver/gpio.c` and `source/driver/watchdog.c` added to its `SRC_FILES`. Its serial, EEPROM and watchdog drivers are byte for byte the ones printed in §6.1.6, §7.1.4 and §8.1.5.

9.2 Review

What you should be able to do now

After this chapter, you should:

- Explain why hardcoded per-instance functions or macros break down once a program needs more than one of something.
- Be able to design a struct that bundles a peripheral's configuration and state.
- Be able to write "methods" as plain functions taking a pointer to that struct as their first parameter (the `self` convention).
- Refactor a program built on duplicated globals into one built on an array of struct instances.

Questions to test yourself

1. What problem does bundling a pin number into a struct solve that a `#defined` macro can't?
2. Why does `gpio_read()` take a pointer to a `gpio_t` rather than a `gpio_t` by value?
3. How would you rewrite five separate `LED1_ON`/`LED2_ON`/... macros as a single `gpio_write()` function plus an array of `gpio_t` instances?

Looking ahead

Chapter 10: hiding a struct's implementation details (opaque structs) and manual polymorphism via function pointers and vtables.

9.3 Exercises

> **How to work these.** The Reflection exercises are answered on paper, from the chapter. The Code exercises are programs for the board: build and flash them the way §2.2 (Microchip Studio) or §2.3 (Linux) describes, or run them in the simulator of Appendix A if you have no board yet.
> **How to check your work.** The Reflection exercises, and the questions Code Exercises 9.3 to 9.6 ask about their design, are answered in Appendix B. The Code exercises have no printed solution: the board tells you whether a program does what the exercise describes. Try each exercise before you read its answer.

EXERCISE SET 1
Concepts

Exercise 9.1 A struct or a macro

REFLECTION

What problem does bundling a pin number into a `gpio_t` struct solve that a `#defined LED_ON` macro can't? What has to change to support a second LED under each approach?

Exercise 9.2 By pointer or by value

REFLECTION

`gpio_read()` takes `const gpio_t* self` rather than `gpio_t self` (the whole struct, by value). What would change, functionally or in cost, if it took `gpio_t self` instead? Would `gpio_init()` still work correctly if it took `self` by value?

EXERCISE SET 2
Program

Exercise 9.3 Freeing a GPIO

CODE

Add `gpio_free(gpio_t* self)` to §9.1.2's driver:

- Reset the pin's `DDRx`/`PORTx` bits back to their power-on state (input, pull-up disabled).
- Decide whether `self->ddrx`, `self->portx`, and `self->pinx` should be set to `NULL` afterwards.
- Justify the choice:
  - What does calling `gpio_write()` or `gpio_toggle()` on a freed instance do in each case?
  - Which is preferable?

Exercise 9.4 Initializing from a pin number

CODE

Rewrite `gpio_init()` so callers pass a single pin number, 0–13, instead of `DDRx`/`PORTx`/`PINx` pointers:

- Pin mapping:
  - 0–7 map to `PORTD0`–`PORTD7`.
  - 8–13 map to `PORTB0`–`PORTB5`.
- `gpio_init()` should compute and store `ddrx`/`portx`/`pinx` (and the bit position) itself.
- Return `bool`:
  - `false` for any pin number outside 0–13, without touching any register.
  - `true` otherwise, after initialization.

- `pin` is `const` in §9.1.2's `gpio_t` because it's only ever set at struct-literal time; now that `gpio_init()` derives it from the pin number instead, does it still make sense for `pin` to stay `const`?

Exercise 9.5 A pin-reservation table CODE

Add a pin-reservation table:

- Declare a `static uint16_t pin_reg` in `gpio.c`.
- Use one bit for each pin number introduced in Exercise 9.4:
  - Bit n represents pin n.
  - `1` means that the pin is reserved.
  - `0` means that the pin is free.
- Update `gpio_init()`:
  - Check the bit corresponding to the requested pin.
  - If the bit is already set, return `false`.
  - In that case, do not modify any hardware register.
  - If the bit is clear, set it to reserve the pin.
  - Then initialize the GPIO as in Exercise 9.4.
  - Return `true` when the initialization succeeds.
- Extend the `gpio_free()` function from Exercise 9.3:
  - Clear the pin's bit in `pin_reg`.
  - This allows a later call to `gpio_init()` to reserve the same pin again.
- Answer the following questions:
  - Why must `pin_reg` be declared `static`?
  - What would break if it were not `static`?

Exercise 9.6 A timer driver CODE

Capstone — harder: Implement a timer driver named `timer_t`:

- Follow the header/source separation used in §9.1.3:
  - `driver/timer.h`
  - `driver/timer.c`
- The timer must run a callback function whenever its timeout expires.
- `timer_t` should follow the same ownership model as `gpio_t`:
  - The caller declares and owns each `timer_t` instance.
  - The caller may declare an array of timer instances, just as §9.1.4 declares an array of `gpio_t` instances.
  - `timer_init()` initializes an existing instance in place.

- The ATmega328P has only three hardware timer/counters:
  - At most three `timer_t` instances may therefore be active at the same time.
  - Use a reservation scheme similar to Exercise 9.5's `pin_reg`.
  - `timer_init()` must not create more active timer instances than there are hardware timer/counters available.
- `timer_t` is a complete timer driver instance:
  - Each instance has:
    - Its own callback.
    - Its own elapsed-time count.
    - Its own start/stop state.
    - One reserved hardware timer/counter.
  - It is driven directly by the hardware timer/counter and its compare-match interrupt.

Recommended `driver/timer.h`

`timer_callback_t` is a function pointer, see §1.1.8 if that syntax is unfamiliar. Document every declaration with Doxygen comments, the same way §9.1.2/§9.1.3's `gpio.h` does, and add the header guard yourself (suggested guard: `DRIVER_TIMER_H_`).

```
#include <stdbool.h>
#include <stdint.h>

/** Alias for timer callbacks. */
typedef void (*timer_callback_t)(void);

typedef struct
{
    // Left for you to design.
} timer_t;

bool timer_init(timer_t* self, uint32_t timeout_ms, timer_callback_t callback);
void timer_clear(timer_t* self);
bool timer_running(const timer_t* self);
void timer_start(timer_t* self);
void timer_stop(timer_t* self);
void timer_toggle(timer_t* self);
void timer_reset(timer_t* self);
```

Initialization and reservation

- `timer_init()` must return `false` when:
  - `timeout_ms == 0`.
  - `callback == NULL`.

  - No hardware timer/counter is available.
- Otherwise, `timer_init()` must:
  - Reserve one free hardware timer/counter.
  - Associate it with `self`.
  - Store the timeout and callback.
  - Initialize the timer instance.
  - Return `true`.
- `timer_clear()` must:
  - Stop the timer.
  - Release its reserved hardware timer/counter.
  - Allow that hardware timer/counter to be claimed by a later `timer_init()` call.
  - Work whether the later call uses the same `timer_t` instance or another one.
- Consider the same design question raised for `gpio_free()` in Exercise 9.3:
  - Should `timer_clear()` also reset the fields of `self`?
  - If so, which values would leave the instance in a safe, inert state?
- `timeout_ms` is only known at runtime, so converting it into a number of interrupts happens on the chip:
  - Avoid `float` for this conversion, see §4.1.5.
  - Hint: a 1 ms compare-match interrupt (prescaler 64 and a compare value of 249, since 16 MHz / 64 / 250 = 1 kHz) makes the number of interrupts equal to `timeout_ms`, with no division at all. The compare value fits all three timer/counters, but check each one's prescaler bits: Timer 2's differ from Timer 0's and Timer 1's.

Timer control

- `timer_start()` must start or enable the timer.
  - It must not change the elapsed-time count.
- `timer_stop()` must stop or disable the timer.
  - It must not change the elapsed-time count.
- `timer_toggle()` must switch between the running and stopped states.
  - It must not change the elapsed-time count.
- `timer_reset()` must:
  - Set the elapsed-time count to zero.
  - Preserve the timer's current running or stopped state.

Driver independence

- The timer driver must not know that `gpio_t` exists.
- `timer.c` must not directly control an LED or read a button.
- Connecting a timer callback to an LED is the responsibility of `main.c`.

Hardware demonstration

Wire the following circuit:

- Connect a button to D13.
- Connect an LED to D8.
- Configure a timer with a timeout of 100 ms.

The program must behave as follows:

- Pressing the button toggles the timer between running and stopped.
- While the timer is running:
  - Its callback runs every 100 ms.
  - The callback toggles the LED.
- While the timer is stopped:
  - The LED remains off.
  - Calling the callback manually must have no effect.
- To enforce this behavior, the callback itself must check `timer_running()` before toggling the LED.

Testing

Confirm `timer_init()` returns `false` for each of the three invalid cases in "Initialization and reservation": a `0` timeout, a `NULL` callback, and a fourth `timer_t` once three hardware timer/counters are already reserved. Then confirm `timer_clear()` actually frees a hardware timer/counter for reuse: after reserving all three, clear one and check that a new `timer_init()` call succeeds again.

CHAPTER 10

Encapsulation and Polymorphism in C

Hiding what a driver is made of, and letting one call reach different code.

In this chapter

- Hiding functions with `static`, and storing function pointers directly on an otherwise still public `gpio_t`, before tackling the struct's fields.
- Why exposing every struct field lets callers break invariants, and how an opaque struct (forward-declared in a header, defined only in a `.c` file) prevents that.
- Function pointers: syntax, storing one in a struct, calling through it.
- Worked example: an interrupt-driven `gpio_t` that dispatches a stored callback from `main()`'s loop, an ISR only flags that an event occurred.
- Vtables: a struct of function pointers as a hand-written interface, and how it relates to what C++'s `virtual` does under the hood.
- Worked example: a `gpio_interface_t` interface that real ATmega328P hardware sits behind, alongside a swappable in-memory stub for testing without a chip attached.

Chapter 9's drivers were structs with every field in plain view and one set of functions per driver. This chapter takes the same GPIO driver further, towards what an object-oriented language would give you for free. After it you should be able to explain how `static` restricts a function's visibility to its own file, and why that's a different kind of encapsulation than hiding struct fields; explain what an opaque struct is and why hiding a struct's definition in a `.c` file protects its invariants; declare, assign, and call through a function pointer, including one stored in a struct field; design a small vtable-style interface and provide more than one implementation of it; and understand, at a high level, how this manual pattern relates to virtual dispatch in higher-level languages.

The chapter assumes Chapter 9's `gpio_t` driver and `timer_t` struct (Exercise 9.6), plus the function pointer syntax of §1.1.8. Read §10.1, the substance of the chapter, in order: each of its three demos builds on what came before it. Then work the exercises in §10.3.

The Reflection exercises are answered in Appendix B; the Code exercises have no printed solution.

10.1 Encapsulation and Polymorphism in C

10.1.1 Why hide a struct's fields

- Every driver in Chapter 9 is fully exposed: `gpio_t` and `timer_t`'s fields are visible to any file that includes their definition.
  - For something as simple as `gpio_t`, that's mostly harmless.
  - For `timer_t`, it's less so: nothing stops calling code from reaching in and setting its elapsed-time field directly, skipping `timer_init()` and quietly breaking the invariant `timer_running()`/`timer_reset()` depend on.
- An **opaque struct** fixes this by splitting a driver across two files:
  - A header that only forward-declares the struct's name and lists function prototypes.
  - A `.c` file that defines the struct's actual fields.
  - Callers only ever see a pointer to an incomplete type: they can pass it around and hand it to the driver's functions, but they can't see or touch its fields, so they can't bypass the functions meant to maintain its invariants.
- Splitting a driver across a header and a `.c` file is what makes opacity possible:
  - §9.1.3 already did this split, for a different reason.
  - It only works when the struct's real definition lives somewhere the caller doesn't (and can't) include.
  - The next section shows a lighter version of the same idea, hiding *functions* rather than fields, before §10.1.3 hides fields too.

10.1.2 Hiding functions with `static`

- Chapter 9's `gpio_t` (§9.1.2) stays exactly as public here: every field still visible.
- What changes is how its behavior is reached:
  - `gpio_write()`, `gpio_read()`, `gpio_toggle()`, and `gpio_enable_pci()` (new in §9.1.5's demo) become `static`.
  - At file scope, `static` means internal linkage: invisible outside `gpio.c`, a different meaning than the storage-duration one it has on a local variable (see §1.1.4).
  - `gpio_t` gains four function pointers so the now-hidden functions are still reachable, just through the struct instead of by name.

`driver/gpio.h`, `gpio_t`'s fields all still public:

```c
/**
 * @file GPIO driver.
 */
#ifndef DRIVER_GPIO_H_
#define DRIVER_GPIO_H_

#include <stdbool.h>
#include <stdint.h>

/**
 * @brief Enumeration of GPIO modes.
 */
typedef enum
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/**
 * @brief GPIO driver structure.
 */
typedef struct gpio
{
    /** Pointer to the data direction register associated with the GPIO. */
    volatile uint8_t* ddrx;

    /** Pointer to the output register associated with the GPIO. */
    volatile uint8_t* portx;

    /** Pointer to the input register associated with the GPIO. */
    volatile uint8_t* pinx;

    /** Pointer to the pin change mask register associated with the GPIO. */
    volatile uint8_t* pcmskx;

    /** Pin change interrupt control register bit (PCIEx) associated with the GPIO
    ↪ 's port. */
    uint8_t pcicrx;

    /** Bit position of the GPIO pin within the associated port. */
    uint8_t pin;

    /** Set state of the given GPIO. Only supported for outputs. */
```

```
    void (*write)(struct gpio* self, bool state);

    /** Read state of the given GPIO. */
    bool (*read)(const struct gpio* self);

    /** Toggle state of the given GPIO. Only supported for outputs. */
    void (*toggle)(struct gpio* self);

    /** Enable/disable pin change interrupts for the given GPIO. */
    void (*enable_pci)(struct gpio* self, bool enable);
} gpio_t;

/**
 * @brief Initialize GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] pin  Digital pin to use. Must be in range [0, 13].
 * @param[in] mode GPIO mode.
 *
 * @return True on success, false on failure.
 */
bool gpio_init(gpio_t* self, uint8_t pin, gpio_mode_t mode);

#endif /* DRIVER_GPIO_H_ */
```

- Each function pointer's `self` parameter is typed `struct gpio*`, not `gpio_t*`:
  - `typedef struct gpio { ... } gpio_t;` only introduces the name `gpio_t` once the whole declaration finishes, at the final `;` after the closing brace.
  - Inside the braces, while the struct is still being defined, `gpio_t` doesn't exist yet: writing `gpio_t* self` there is a compile error (`unknown type name 'gpio_t'`).
  - The tag `struct gpio`, on the other hand, is in scope from the moment `struct gpio {` appears, so a member can refer to a pointer to its own struct type by tag, the same trick a linked list's `struct node* next` relies on.
  - Everywhere *outside* the struct, `gpio_t` is the normal name to use; only these member declarations, written before their own type has a name, need `struct gpio` instead.

Unlike Chapter 9's `gpio_init(self, mode)`, which expected `ddrx/portx/pinx/pin` already set through a struct literal, this `gpio_init(self, pin, mode)` takes an Arduino pin number and works the registers out itself, following the pin-number rewrite from Exercise 9.4: pins 0–7 map to `PORTD`, pins 8–13 map to `PORTB`, and an out-of-range pin returns `false` without touching any register. `driver/gpio.c`:

lectures/L10/function_pointer_demo/source/driver/gpio.c

```c
/**
 * @file GPIO driver implementation details.
 */
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>

#include <avr/io.h>

#include "driver/gpio.h"

#define PIN_OFFSET_B 8U  // Pin offset for I/O port B.
#define GPIO_PIN_MAX 13U // Maximum GPIO pin number.

// ----------------------------------------------------------------------------
static void gpio_write(gpio_t* self, const bool state)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Check GPIO direction, terminate if not output.
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }

    // Set GPIO state as specified.
    if (state) { *self->portx |= (1U << self->pin); }
    else { *self->portx &= ~(1U << self->pin); }
}

// ----------------------------------------------------------------------------
static bool gpio_read(const gpio_t* self)
{
    // Return GPIO state if GPIO instance is valid, else false.
    return (NULL != self) ? (*self->pinx & (1U << self->pin)) : false;
}

// ----------------------------------------------------------------------------
static void gpio_toggle(gpio_t* self)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Check GPIO direction, terminate if not output.
```

```
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }

    // Toggle GPIO state.
    *self->pinx = (1U << self->pin);
}

// ----------------------------------------------------------------------------
static void gpio_enable_pci(gpio_t* self, const bool enable)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Enable/disable interrupts for the given pin as specified.
    if (enable)
    {
        PCICR |= (1U << self->pcicrx);
        *self->pcmskx |= (1U << self->pin);
    }
    else { *self->pcmskx &= ~(1U << self->pin); }
}

// ----------------------------------------------------------------------------
bool gpio_init(gpio_t* self, const uint8_t pin, const gpio_mode_t mode)
{
    // Check the GPIO instance, return false if invalid.
    if (NULL == self) { return false; }

    // Check the pin number, return false if invalid.
    if (GPIO_PIN_MAX < pin) { return false; }

    // PIN 0 - 7 => I/O port D, pin = ID.
    if (PIN_OFFSET_B > pin)
    {
        self->ddrx   = &DDRD;
        self->portx  = &PORTD;
        self->pinx   = &PIND;
        self->pin    = pin;
        self->pcmskx = &PCMSK2;
        self->pcicrx = PCIE2;
    }
    // PIN 8 - 13 => I/O port B, pin = ID - 8.
    else
    {
```

```
        self->ddrx   = &DDRB;
        self->portx  = &PORTB;
        self->pinx   = &PINB;
        self->pin    = pin - PIN_OFFSET_B;
        self->pcmskx = &PCMSK0;
        self->pcicrx = PCIE0;
    }

    // Configure GPIO mode as specified.
    switch (mode)
    {
        case GPIO_MODE_INPUT_PULLUP:
            // Configure GPIO as input with its internal pull-up enabled.
            *self->ddrx &= ~(1U << self->pin);
            *self->portx |= (1U << self->pin);
            break;
        case GPIO_MODE_OUTPUT:
            // Configure GPIO as output.
            *self->ddrx |= (1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
        default:
            // Configure GPIO as standard input.
            *self->ddrx &= ~(1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
    }

    // Set function pointers.
    self->write      = gpio_write;
    self->read       = gpio_read;
    self->toggle     = gpio_toggle;
    self->enable_pci = gpio_enable_pci;
    return true;
}
```

- Two fields that used to be `const`, `pin` and `pcicrx`, aren't anymore:
  - Both used to be known at the call site and handed in through a struct literal before `gpio_init()` ever ran.
  - Now `gpio_init()` derives them itself from the pin number argument and assigns them with `self->pin = .../self->pcicrx = ....`
  - A `const` member can only be set once, by an initializer at the point a struct is declared, never by assignment afterward, so once the value is only known and set inside

`gpio_init()`, the field can no longer stay `const`. Exercise 9.4 asks this exact question for the non-function-pointer version of this same rewrite.

```
function_pointer_demo/
├── Makefile
├── main.c
├── include/
│   └── driver/
│       ├── eeprom.h
│       ├── gpio.h
│       ├── serial.h
│       └── watchdog.h
└── source/
    └── driver/
        ├── eeprom.c
        ├── gpio.c
        ├── serial.c
        └── watchdog.c
```

- `driver/gpio.h` and `driver/gpio.c` are the full driver shown above; the header is printed with its full comments at the end of this section.
- `driver/eeprom.h/.c`, `driver/serial.h/.c`, and `driver/watchdog.h/.c` are the same EEPROM, UART, and watchdog drivers from Chapters 6–8, wired in only so `main.c` has something to persist state to and report over, unrelated to this chapter's material.

`main.c` toggles LED1 (`D9`) whenever BTN1 (`D13`) is pressed, debounces the button with a 300 ms timer, persists the new LED state to EEPROM, reports it over serial, and kicks the watchdog every loop iteration:

lectures/L10/function_pointer_demo/main.c

```c
/**
 * @file GPIO driver demo.
 */
#include <stdbool.h>
#include <stdint.h>

#include <avr/interrupt.h>
#include <avr/io.h>

#include "driver/eeprom.h"
#include "driver/gpio.h"
#include "driver/serial.h"
#include "driver/watchdog.h"
```

```
#define LED1_PIN 9U  // D9  -> PORTB1.
#define BTN1_PIN 13U // D13 -> PORTB5.

/** Timer operations. */
#define TIMER0_ENABLE TIMSK0 = (1U << TOIE0) // Enable timer 0 interrupt.
#define TIMER0_DISABLE TIMSK0 = 0U           // Disable timer 0 interrupt.

/** Time parameters */
#define TICK_PERIOD_MS 0.064F    // Time between each tick, prescaler 1024.
#define TICK_MAX 256U            // Ticks per overflow for 8-bit timer.
#define DEBOUNCE_TIMEOUT_MS 300U // Debounce timeout in ms.

/** Limit parameters. */
#define OVF_TIME_MS (TICK_PERIOD_MS * TICK_MAX)     // Time between each overflow ↩
    ↪ in ms.
#define OVF_MAX (DEBOUNCE_TIMEOUT_MS / OVF_TIME_MS) // Overflows needed for ↩
    ↪ timeout.
#define OVF_TIMEOUT (uint8_t)(OVF_MAX + 0.5F)       // Overflows needed for ↩
    ↪ timeout, rounded.

/** EEPROM parameters. */
#define EEPROM_LED1_ADDR 1000U // EEPROM address containing the LED1 state.
#define EEPROM_LED1_ON 1U      // Stored value indicating that LED1 is on.
#define EEPROM_LED1_OFF 0U     // Stored value indicating that LED1 is off.

/** Flags that a LED1 event has occurred. */
static volatile bool led1_event = false;

/** GPIO instances. */
static gpio_t led1, btn1;

/**
 * @brief Print the LED1 state over UART.
 */
static void print_led1_state(void)
{
    if (led1.read(&led1)) { serial_print("LED1 enabled!\n"); }
    else { serial_print("LED1 disabled!\n"); }
}

/**
 * @brief Set up system.
 */
static void setup(void)
```

```
{
    // Initialize the watchdog first: after a watchdog reset it is still running, ↩
    ↪ with a 16 ms
    // timeout, and the rest of setup() takes longer than that.
    watchdog_init(WATCHDOG_TIMEOUT_1024MS);

    // Initialize GPIOs.
    gpio_init(&led1, LED1_PIN, GPIO_MODE_OUTPUT);
    gpio_init(&btn1, BTN1_PIN, GPIO_MODE_INPUT_PULLUP);
    btn1.enable_pci(&btn1, true);

    // Set up 300 ms debounce timer.
    TCCR0B = (1U << CS00) | (1U << CS02);

    // Initialize serial driver.
    serial_init();

    // Restore the previous LED state from EEPROM.
    uint8_t led_state = 0U;
    if (sizeof(led_state) == eeprom_read(&led_state, sizeof(led_state), ↩
    ↪ EEPROM_LED1_ADDR))
    {
        if (EEPROM_LED1_ON == led_state) { led1.toggle(&led1); }
    }
    print_led1_state();

    // Enable interrupts globally.
    sei();
}

/**
 * @brief Toggle LED1 on button press, and flag the event for main() to report.
 *
 *        Disable button interrupts for 300 ms to suppress contact bounce.
 */
ISR(PCINT0_vect)
{
    // Disable button interrupts for 300 ms.
    btn1.enable_pci(&btn1, false);
    TIMER0_ENABLE;

    // Toggle LED1 and flag the event if BTN1 is pressed.
    if (btn1.read(&btn1))
    {
```

```
        led1.toggle(&led1);
        led1_event = true;
    }
}

/**
 * @brief Re-enable button interrupts after 300 ms.
 */
ISR(TIMER0_OVF_vect)
{
    static volatile uint8_t ovf_counter = 0U;

    // Wait for 300 ms, then re-enable button interrupts and disable timer 0.
    if (OVF_TIMEOUT <= ++ovf_counter)
    {
        btn1.enable_pci(&btn1, true);
        TIMER0_DISABLE;
        ovf_counter = 0U;
    }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        // Reset the watchdog once every iteration of the loop.
        watchdog_reset();

        // Check if a LED1 event has occurred, store the new state in EEPROM if ↩
    ↪ true.
        if (led1_event)
        {
            const uint8_t led_state = led1.read(&led1) ? EEPROM_LED1_ON : ↩
    ↪ EEPROM_LED1_OFF;
            eeprom_write(&led_state, sizeof(led_state), EEPROM_LED1_ADDR);
            led1_event = false;
```

```
            // Report the new LED1 state over UART.
            print_led1_state();
        }
    }
    return 0;
}
```

- `led1.toggle(&led1)` and `btn1.enable_pci(&btn1, true)` instead of `gpio_toggle(&led1)` and `gpio_enable_pci(&btn1, true)`: the same call shape §10.1.6's vtables use later, just with the pointers sitting directly on `gpio_t` instead of behind a separate interface struct.
- `PCINT0_vect` only toggles the LED, a single register write, and sets `led1_event`; the EEPROM write and serial report both happen from `main()`'s loop once `led1_event` is seen, following the same ISR-minimality discipline as §6.1.6: keep the ISR to the minimum, let `main()`'s loop handle anything that can block.
- A second ISR, `TIMER0_OVF_vect`, re-enables the button's interrupt after the 300 ms debounce window, using the same `enable_pci` function pointer `main()` used to arm it.
- Nothing here hides `led1`'s or `btn1`'s *fields* yet: `led1.ddrx`, `led1.write`, all of it, are exactly as reachable as before. `static` only hid the *functions*; §10.1.3 hides the fields too.

Build and flash the same way as §9.1.5: on Linux, run `make` in the demo's directory, `lectures/L10/function_pointer_demo` (`make flash` to program the board); on Windows via Microchip Studio, recreate this folder structure inside the project and add `include` as an include directory.

The function-pointer demo in full

The demo in `lectures/L10/function_pointer_demo` is about function pointers on a still-public GPIO driver: a `gpio_t` driver where the fields stay fully public, but `write`/`read`/`toggle`/`enable_pci` are reached only through function pointers stored on the struct itself.

This example demonstrates encapsulation through `static` internal linkage rather than through an opaque struct. The fields of `gpio_t`, including its function pointers, remain visible and assignable from any source file that includes `driver/gpio.h`. Only the concrete functions behind those pointers are hidden, because their definitions have internal linkage inside `gpio.c`.

`gpio_init()` accepts an Arduino digital pin number and derives the relevant ATmega328P registers, bit positions, and pin-change interrupt configuration. It also configures the pin and assigns all function pointers in a single initialization call.

The implementation is written for compilation on an ATmega328P in Microchip Studio.

Files.

- `driver/gpio.h`:

  - Contains the GPIO driver's public interface.
  - Defines `struct gpio` and its `gpio_t` typedef directly in the header.
  - Exposes every field of the GPIO instance to application code.
  - Stores the GPIO hardware information required by the internal operations.
  - Stores the pin's bit number within its hardware port.
  - Stores the pin-change interrupt group bit used with `PCICR`.
  - Includes four function pointers as struct members:
    - `write()`
    - `read()`
    - `toggle()`
    - `enable_pci()`
  - Defines the function-pointer signatures used by the driver operations.
  - Defines the `gpio_mode_t` enum used to select the pin's operating mode.
  - Declares `gpio_init()`.
  - Exposes `gpio_init()` as the driver's only ordinary function with external linkage.
  - Allows callers to declare individual `gpio_t` objects or arrays of GPIO objects.
  - Requires callers to initialize each object before calling any stored function pointer.
  - Demonstrates that a public struct provides convention-based rather than enforced encapsulation.
- `driver/gpio.c`:
  - Contains the GPIO driver's hardware-specific implementation.
  - Defines `static` implementations of:
    - `gpio_write()`
    - `gpio_read()`
    - `gpio_toggle()`
    - `gpio_enable_pci()`
  - Gives these concrete functions internal linkage.
  - Prevents other translation units from calling the implementations directly by name.
  - Allows the functions to remain reachable indirectly through the pointers stored in a successfully initialized `gpio_t`.
  - Implements `gpio_init()`.
  - Validates the supplied Arduino pin number.
  - Returns `false` when the pin number is outside the supported range.
  - Performs validation before changing any hardware register.
  - Maps pins 0–7 to the `PORTD` register group.
  - Maps pins 8–13 to the `PORTB` register group.
  - Calculates the pin's hardware bit position.

  - Assigns the correct `DDRx`, `PORTx`, and `PINx` register pointers.
  - Assigns the appropriate pin-change mask register.
  - Assigns the correct pin-change interrupt group bit.
  - Configures the pin according to the requested `gpio_mode_t`.
  - Establishes the pin's initial output or pull-up state where required.
  - Assigns all four internal operation functions to the instance's function-pointer fields.
  - Leaves the caller-owned `gpio_t` ready for use after successful initialization.
  - Centralizes Arduino-pin-to-ATmega-register mapping in one implementation file.
- `driver/eeprom.h/driver/eeprom.c`:
  - Contains the EEPROM driver introduced in Chapter 7.
  - Stores the current LED state in non-volatile memory.
  - Allows the application to restore the LED state after a reset or power cycle.
  - Is called only from normal program context.
  - Remains independent of the GPIO driver.
  - Does not need access to `gpio_t` or its function pointers.
- `driver/serial.h/driver/serial.c`:
  - Contains the UART driver introduced in Chapter 6.
  - Initializes serial communication with the configured baud rate.
  - Reports whether the LED is currently on or off.
  - Performs output from the main loop rather than from an ISR.
  - Remains independent of the GPIO implementation.
  - Demonstrates composition of separate drivers at application level.
- `driver/watchdog.h/driver/watchdog.c`:
  - Contains the watchdog driver introduced in Chapter 8.
  - Configures the watchdog during application startup.
  - Provides the operation used to reset the watchdog counter.
  - Is serviced once during every main-loop iteration.
  - Helps recover the system if the main loop stops executing normally.
  - Operates independently of the GPIO, EEPROM, and UART drivers.
- `main.c`:
  - Declares one caller-owned `gpio_t` instance for the LED on D9.
  - Declares one caller-owned `gpio_t` instance for the button on D13.
  - Initializes the watchdog first, then the GPIOs, the debounce timer and the UART.
  - Calls `gpio_init()` with two pin numbers known to be valid, so it doesn't check the results; code that takes its pin numbers from elsewhere must.
  - Restores the previously saved LED state from EEPROM.

  - Applies the restored state to the LED.
  - Arms the button's pin-change interrupt through its `enable_pci()` function pointer.
  - Enables global interrupts only after initialization is complete.
  - Defines the shared event state used by the ISRs and main loop.
  - Implements `ISR(PCINT0_vect)`:
    - Disables the button's pin-change interrupt.
    - Starts or resets the debounce timing state.
    - Toggles the LED through the GPIO driver's function pointer.
    - Sets an event flag.
    - Performs no EEPROM write.
    - Performs no UART output.
  - Implements `ISR(TIMER0_OVF_vect)`:
    - Tracks the elapsed debounce interval.
    - Re-enables the button interrupt after approximately 300 ms.
    - Stops or resets the debounce timing state when re-arming is complete.
  - Uses the timer interrupt only for debounce timing.
  - Resets the watchdog once per main-loop iteration.
  - Checks the event flag from normal program context.
  - Clears the event flag once the new state has been written to EEPROM.
  - Reads the LED's new state.
  - Writes the new state to EEPROM.
  - Reports the new state through UART.
  - Keeps potentially slow operations outside the interrupt service routines.
  - Coordinates several independent drivers without coupling their implementations together.

`driver/gpio.c` and `main.c` are the files printed above. The header, with its full comments:

lectures/L10/function_pointer_demo/include/driver/gpio.h

```
/**
 * @file GPIO driver.
 */
#ifndef DRIVER_GPIO_H_
#define DRIVER_GPIO_H_

#include <stdbool.h>
#include <stdint.h>

/**
```

```
 * @brief Enumeration of GPIO modes.
 */
typedef enum
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/**
 * @brief GPIO driver structure.
 */
typedef struct gpio
{
    /** Pointer to the data direction register associated with the GPIO. */
    volatile uint8_t* ddrx;

    /** Pointer to the output register associated with the GPIO. */
    volatile uint8_t* portx;

    /** Pointer to the input register associated with the GPIO. */
    volatile uint8_t* pinx;

    /** Pointer to the pin change mask register associated with the GPIO. */
    volatile uint8_t* pcmskx;

    /** Pin change interrupt control register bit (PCIEx) associated with the GPIO
    ↪ 's port. */
    uint8_t pcicrx;

    /** Bit position of the GPIO pin within the associated port. */
    uint8_t pin;

    /**
     * @brief Set state of the given GPIO.
     *
     * @param[in] self GPIO instance.
     * @param[in] state GPIO state (true = high, false = low).
     *
     * @note This operation is only supported for GPIOs configured as outputs.
     */
    void (*write)(struct gpio* self, bool state);

    /**
```

```
     * @brief Read state of the given GPIO.
     *
     * @param[in] self GPIO instance.
     *
     * @return GPIO state (true = high, false = low).
     */
    bool (*read)(const struct gpio* self);

    /**
     * @brief Toggle state of the given GPIO.
     *
     * @param[in] self GPIO instance.
     */
    void (*toggle)(struct gpio* self);

    /**
     * @brief Enable/disable pin change interrupts for the given GPIO.
     *
     * @param[in] self GPIO instance.
     * @param[in] enable True to enable pin change interrupts, false otherwise.
     */
    void (*enable_pci)(struct gpio* self, bool enable);
} gpio_t;

/**
 * @brief Initialize GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] pin Digital pin to use. Must be in range [0, 13].
 * @param[in] mode GPIO mode.
 *
 * @return True on success, false on failure.
 */
bool gpio_init(gpio_t* self, uint8_t pin, gpio_mode_t mode);

#endif /** DRIVER_GPIO_H_ */
```

The serial, EEPROM and watchdog drivers are byte for byte the ones printed in §6.1.6, §7.1.4 and §8.1.5. The demo's `Makefile` is the one printed in §6.1.6, with the same three extra source files in its `SRC_FILES` as Chapter 9's demo (§9.1.5).

10.1.3 Opaque structs

- Starting over from §9.1.2/§9.1.3's `gpio_t`, not continuing from §10.1.2's version above:

  - Hiding fields and hiding functions are independent techniques, not two steps of one pipeline, so `gpio_write()`/`gpio_read()`/`gpio_toggle()` go back to being ordinary global functions instead of struct fields.
  - Same fields, same `gpio_write()`/`gpio_read()`/`gpio_toggle()` signatures as Chapter 9; §10.1.2's `enable_pci`/`pcmskx`/`pcicrx` additions don't carry over either. This section is about opacity, not interrupts.
- What's new relative to Chapter 9:
  - The struct itself moves out of `gpio.h` and into `gpio.c`.
  - `gpio_init()` is replaced by a `malloc()`-based `gpio_create()`, since callers can no longer declare their own `gpio_t` and pass its address once the struct's size and layout aren't visible to them.

`gpio.h`, the only file calling code includes:

```c
/**
 * @brief GPIO driver for ATmega328p.
 */
#ifndef DRIVER_GPIO_H_
#define DRIVER_GPIO_H_

#include <stdbool.h>
#include <stdint.h>

/**
 * @brief Enumeration of GPIO modes.
 */
typedef enum
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/**
 * @brief GPIO driver structure.
 */
typedef struct gpio gpio_t;   // incomplete type: fields are not visible here

/**
 * @brief Allocate and configure a new GPIO.
 *
 * @param[in] ddrx  Pointer to the data direction register associated with the ↩
    ↪ GPIO.
```

```
 * @param[in] portx Pointer to the output register associated with the GPIO.
 * @param[in] pinx  Pointer to the input register associated with the GPIO.
 * @param[in] pin   Bit position of the GPIO pin within the associated port.
 * @param[in] mode  GPIO mode.
 *
 * @return Pointer to the new GPIO, or NULL if allocation failed.
 */
gpio_t* gpio_create(volatile uint8_t* ddrx, volatile uint8_t* portx, volatile
    ↪ uint8_t* pinx,
                    uint8_t pin, gpio_mode_t mode);

/**
 * @brief Set state of the given GPIO.
 *
 * @param[in] self  GPIO instance.
 * @param[in] state GPIO state (true = high, false = low).
 *
 * @note This operation is only supported for GPIOs configured as outputs.
 */
void gpio_write(gpio_t* self, bool state);

/**
 * @brief Read state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 *
 * @return GPIO state (true = high, false = low).
 */
bool gpio_read(const gpio_t* self);

/**
 * @brief Toggle state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 */
void gpio_toggle(gpio_t* self);

#endif /* DRIVER_GPIO_H_ */
```

`gpio.c`, the only file that ever sees `gpio_t`'s actual fields:

```
#include <stddef.h>
#include <stdlib.h>
```

```c
#include "driver/gpio.h"

struct gpio
{
    volatile uint8_t* ddrx;
    volatile uint8_t* portx;
    volatile uint8_t* pinx;
    uint8_t pin;
};

gpio_t* gpio_create(volatile uint8_t* ddrx, volatile uint8_t* portx, volatile
    ↪ uint8_t* pinx,
                    const uint8_t pin, const gpio_mode_t mode)
{
    gpio_t* self = (gpio_t*)(malloc(sizeof(gpio_t)));
    if (NULL == self) { return NULL; } // Allocation failed.

    self->ddrx = ddrx;
    self->portx = portx;
    self->pinx = pinx;
    self->pin = pin;

    switch (mode)
    {
        case GPIO_MODE_INPUT_PULLUP:
            *self->ddrx &= ~(1U << self->pin);
            *self->portx |= (1U << self->pin);
            break;
        case GPIO_MODE_OUTPUT:
            *self->ddrx |= (1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
        default:
            *self->ddrx &= ~(1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
    }
    return self;
}

void gpio_write(gpio_t* self, const bool state)
{
    if (NULL == self) { return; }
```

```
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }

    if (state) { *self->portx |= (1U << self->pin); }
    else { *self->portx &= ~(1U << self->pin); }
}

bool gpio_read(const gpio_t* self)
{
    return (NULL != self) ? (*self->pinx & (1U << self->pin)) : false;
}

void gpio_toggle(gpio_t* self)
{
    if (NULL == self) { return; }
    const bool output = (*self->ddrx & (1U << self->pin));
    if (!output) { return; }
    *self->pinx = (1U << self->pin);
}
```

- `gpio_create()` calls `malloc()` once per instance rather than reserving a fixed-size static array:
  - A pool sized for the worst case, up to 14 possible Arduino pins, reserves that much RAM on every board, even ones that only ever create two or three GPIOs; `malloc()` only spends what a program actually asks for.
  - This isn't the "avoid dynamic memory" advice AVR programs usually follow, that's really about avoiding *repeated* allocation and `free()` churn at runtime. `gpio_create()` is called once per driver instance, at startup, and the instance lives for the rest of the program; there's no resizing or fragmentation risk to worry about here.
  - `gpio_create()` returning `NULL` on allocation failure is the caller's only signal that the heap ran out, same as any other `malloc()` call.

10.1.4 Function pointers

- §1.1.8 covered the syntax: a variable that holds a function's address, callable through the variable exactly like calling the function by name.
- What's new here is storing one in a struct, which lets a driver call back into code it knows nothing about.

Pairing one of §10.1.3's `gpio_t` inputs with a callback:

```c
/**
 * @brief Button structure.
 */
typedef struct
{
    /** GPIO associated with the button. */
    gpio_t* pin;

    /**
     * @brief Handle press events.
     */
    void (*on_press)(void); // Called from main(), not the ISR; see L06 Appendix A.6.
} button_t;

/**
 * @brief Check for button press event, invoke its callback if one occurred.
 *
 * @param[in] self Button instance.
 * @param[in] edge True if a press was just detected (e.g. from a flag an ISR set).
 */
static void button_check(const button_t* self, const bool edge)
{
    const bool valid = (NULL != self) && (NULL != self->on_press);
    if (valid && edge) { self->on_press(); }
}
```

`button_t`'s code never mentions what `on_press` actually does, toggle an LED, send a UART message, anything with a matching signature can be plugged in without touching `button_check()`.

10.1.5 Worked example: an interrupt-driven `gpio_t` with event flags

- §10.1.4's `button_t` took an edge flag as a parameter, wherever it came from was left unspecified. This demo wires the same idea into a real pin-change interrupt.
- §10.1.3's `gpio_t` gains:
  - A `gpio_callback_t cb` field.
  - `pcmskx/pcicrx` (the port's pin-change mask register and its enable bit in `PCICR`).
  - Three new functions that arm, flag, and dispatch the interrupt.

`gpio_enable_pci()` arms the interrupt for one GPIO:

```
void gpio_enable_pci(gpio_t* self)
{
    if (NULL == self) { return; }
    SET(PCICR, self->pcicrx);      // enable interrupts on the associated I/O port
    SET(*self->pcmskx, self->pin); // enable interrupts on the pin
    sei();                         // enable interrupts globally
}
```

- Following §6.1.6's discipline, none of the three ISRs (one per I/O port) do anything beyond flagging that *something* happened.
- `event_flags` is `volatile` because it's written from an ISR and read from `main()`'s loop, the exact situation `volatile` exists for:

```
/** Event flags for each I/O port (set from an ISR, cleared once handled). */
static volatile uint8_t event_flags = 0U;

static inline void flag_event(const uint8_t port)
{
    if (PORT_COUNT > port) { SET(event_flags, port); }
}

ISR(PCINT0_vect) { flag_event(PORT_OFFSET_B); } // port B: pins 8-13
ISR(PCINT1_vect) { flag_event(PORT_OFFSET_C); } // port C: pins 14-19
ISR(PCINT2_vect) { flag_event(PORT_OFFSET_D); } // port D: pins 0-7
```

Deciding *what* to do about a flagged event, and actually calling the callback, happens entirely outside the ISR, in code `main()`'s loop drives:

```
bool gpio_event_occurred(const gpio_t* self)
{
    return NULL != self ? READ(event_flags, self->pcicrx) : false;
}

bool gpio_handle_event(gpio_t* self)
{
    bool handled = false;
    if (gpio_event_occurred(self))
    {
        if (NULL != self->cb)
        {
            cli();
            CLEAR(event_flags, self->pcicrx);
            sei();
```

```
            self->cb();
            handled = true;
        }
    }
    return handled;
}
```

`CLEAR()` is a read-modify-write of `event_flags`, the same shape as the `PORTB ^=` toggle §3.1.3 warned about: read the byte, clear one bit, write the byte back. An ISR that flags another port between that read and that write would have its flag silently overwritten, so `gpio_handle_event()` clears the flag with interrupts disabled. And it clears it *before* calling the callback, not after: an edge that arrives while the callback runs then sets the flag again and is handled on the next pass, instead of being wiped out along with the one just handled.

`gpio_event_occurred()` reads `event_flags` by `pcicrx` (`PCIE0`/`PCIE1`/`PCIE2`), while the ISRs above flag it by `PORT_OFFSET_B/_C/_D` (`0`/`1`/`2`); these only line up because the ATmega328p happens to number `PCIE0`–`PCIE2` the same way. Worth noticing when reading unfamiliar register code: two differently-named constants lining up by coincidence of the datasheet, not by a shared definition.

```
encapsulation_demo/
├── Makefile
├── main.c
├── include/
│   └── driver/
│       ├── gpio.h
│       └── utils.h
└── source/
    └── driver/
        └── gpio.c
```

- `driver/gpio.h` and `driver/gpio.c`, printed at the end of this section, are the full driver, including `gpio_new()`/`gpio_del()`, extended with the callback and interrupt-handling pieces shown above.
- `driver/utils.h`, printed there too, has the `SET`/`CLEAR`/`TOGGLE`/`READ` register macros used throughout.

`main.c` configures pin 9 as an LED output and pin 13 as a button input with its pull-up enabled, registers `button_event()` as the button's callback, and lets pin-change interrupts drive it:

lectures/L10/encapsulation_demo/main.c

```c
/**
 * @brief Demonstrate GPIO usage with interrupt-driven callback handling.
 */
#include <stddef.h>

#include "driver/gpio.h"

/** Use GPIO pin 9 for the LED. */
#define LED_PIN 9U

/** Use GPIO pin 13 for the button. */
#define BUTTON_PIN 13U

/** GPIO devices (file-global so they can be accessed from the interrupt callback)
    . */
static gpio_t *led1, *btn1;

/**
 * @brief Handle button event.
 *
 *        Toggle the LED when the button is pressed.
 */
static void button_event(void)
{
    if (gpio_read(btn1)) { gpio_toggle(led1); }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur), or -1 on
 * failure.
 */
int main(void)
{
    // Initialize LED GPIO (output, no callback), return -1 on failure.
    led1 = gpio_new(LED_PIN, GPIO_MODE_OUTPUT, NULL);
    if (NULL == led1) { return -1; }

    // Initialize button GPIO (input with pull-up, use callback), return -1 on
    failure.
    btn1 = gpio_new(BUTTON_PIN, GPIO_MODE_INPUT_PULLUP, button_event);
    if (NULL == btn1) { return -1; }
```

```
    // Enable pin change interrupt for the button.
    gpio_enable_pci(btn1);

    // Keep program running.
    while (1)
    {
        // Handle event if occurred.
        gpio_handle_event(btn1);
    }

    // Unreachable in this example (infinite loop). Shown for completeness.
    gpio_del(&led1);
    gpio_del(&btn1);
    return 0;
}
```

`button_event()` itself never runs inside an ISR, it only ever runs from `gpio_handle_event()`, called from `main()`'s loop; the ISRs above never touch `led1`, `btn1`, or the callback at all.

Build and flash the same way as §9.1.5: on Linux, run `make` in the demo's directory, `lectures/L10/encapsulation_demo` (`make flash` to program the board); on Windows via Microchip Studio, recreate this folder structure inside the project and add `include` as an include directory.

The encapsulation demo in full

The demo in `lectures/L10/encapsulation_demo` is interrupt-driven GPIO with event flags: an opaque `gpio_t` driver extended with a stored callback and pin-change interrupt support.

This example demonstrates encapsulation (`gpio_t`'s fields are hidden behind `driver/gpio.h`) combined with a function pointer stored in a struct, dispatched safely from `main()`'s loop. The interrupt service routine only records that an event occurred; `gpio_handle_event()` is what actually invokes the callback.

The implementation is written for compilation on an ATmega328P in Microchip Studio.

Files.

- `driver/gpio.h`:
  - Contains the GPIO driver's public interface.
  - Forward-declares `gpio_t` without exposing the definition of `struct gpio`.
  - Keeps register pointers, pin numbers, callback pointers, and other internal state private.
  - Defines the `gpio_mode_t` enum used to select the pin's operating mode.
  - Declares the lifetime-management functions:
    - `gpio_new()`

    - `gpio_del()`
  - Declares the basic GPIO operations:
    - `gpio_write()`
    - `gpio_read()`
    - `gpio_toggle()`
  - Declares the pin-change interrupt control functions:
    - `gpio_enable_pci()`
    - `gpio_disable_pci()`
  - Declares the event-processing functions:
    - `gpio_event_occurred()`
    - `gpio_handle_event()`
  - Provides the only interface that application code needs in order to use the driver.
  - Prevents application code from depending on the driver's internal representation.
- `driver/gpio.c`:
  - Defines the private `struct gpio`.
  - Stores the hardware information needed to operate each GPIO pin.
  - Stores the callback function associated with an interrupt-driven GPIO instance.
  - Implements allocation and initialization performed by `gpio_new()`.
  - Implements cleanup performed by `gpio_del()`.
  - Implements GPIO register access for reading, writing, and toggling pins.
  - Maps each GPIO pin to the appropriate ATmega328P I/O registers.
  - Configures the corresponding pin-change interrupt mask bit.
  - Enables or disables the appropriate pin-change interrupt group.
  - Maintains the shared event flags used to communicate with the main loop.
  - Implements `gpio_event_occurred()` for checking whether an event is pending.
  - Implements `gpio_handle_event()` for clearing the pending event and invoking the stored callback.
  - Contains the pin-change interrupt handlers:
    - `ISR(PCINT0_vect)`
    - `ISR(PCINT1_vect)`
    - `ISR(PCINT2_vect)`
  - Keeps the interrupt handlers intentionally small.
  - Records events in interrupt context without directly calling application code.
  - Centralizes all ATmega328P-specific GPIO and interrupt details inside the driver implementation.
- `driver/utils.h`:

  - Contains function-like macros for common bit operations.
  - Provides macros for setting bits in a hardware register.
  - Provides macros for clearing bits in a hardware register.
  - Provides macros for toggling bits in a hardware register.
  - Provides macros for reading individual bits from a hardware register.
  - Makes low-level register operations more readable.
  - Reduces repetition in the GPIO driver implementation.
- `main.c`:
  - Demonstrates how application code uses the GPIO driver without accessing its internal fields.
  - Creates one GPIO instance for an LED.
  - Creates another GPIO instance for a push button.
  - Configures the LED pin as an output.
  - Configures the button pin as an input.
  - Registers a callback for the button GPIO.
  - Enables the button's pin-change interrupt.
  - Enables global interrupts through `gpio_enable_pci()`, once both GPIOs are initialized.
  - Polls for pending GPIO events in the main loop.
  - Calls `gpio_handle_event()` from normal program context.
  - Toggles the LED from the registered callback when the button event is handled.
  - Demonstrates deferred interrupt processing rather than executing the complete response inside the ISR.
  - Releases the GPIO objects through the driver's public interface when cleanup is required.

`main.c` is the file printed above. The driver:

lectures/L10/encapsulation_demo/include/driver/gpio.h

```
/**
 * @brief GPIO driver for ATmega328p.
 */
#ifndef GPIO_H_
#define GPIO_H_

#include <stdbool.h>
#include <stdint.h>

/**
 * @brief Enumeration of GPIO modes.
 */
```

```c
typedef enum gpio_mode
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/**
 * @brief GPIO driver structure.
 */
typedef struct gpio gpio_t;

/** GPIO callback. */
typedef void (*gpio_callback_t)(void);

/**
 * @brief Create a new GPIO.
 *
 * @param[in] pin Arduino pin number.
 * @param[in] mode GPIO mode.
 * @param[in] cb Optional callback. Pass NULL if none.
 *
 * @return The initialized GPIO on success, NULL otherwise.
 */
gpio_t* gpio_new(uint8_t pin, gpio_mode_t mode, gpio_callback_t cb);

/**
 * @brief Delete GPIO instance.
 *
 *        Release resources allocated for the GPIO and set the associated
 * pointer to null.
 *
 * @param[in, out] self Reference to the GPIO instance.
 */
void gpio_del(gpio_t** self);

/**
 * @brief Set state of the GPIO.
 *
 * @note This operation is only supported for outputs.
 *
 * @param[in, out] self GPIO instance.
 * @param[in] state GPIO state (true = enabled, false = disabled).
 */
```

```
void gpio_write(gpio_t* self, bool state);

/**
 * @brief Read the state of the GPIO.
 *
 * @param[in] self GPIO instance.
 *
 * @return True if the GPIO is enabled, false otherwise.
 */
bool gpio_read(const gpio_t* self);

/**
 * @brief Toggle state of the GPIO.
 *
 * @note This operation is only supported for outputs.
 *
 * @param[in, out] self GPIO instance.
 */
void gpio_toggle(gpio_t* self);

/**
 * @brief Enable pin change interrupt for the given GPIO.
 *
 * @param[in, out] self GPIO instance.
 */
void gpio_enable_pci(gpio_t* self);

/**
 * @brief Disable pin change interrupts for the given GPIO.
 *
 * @param[in, out] self GPIO instance.
 * @param[in] disable_port True to disable interrupts on the entire I/O port.
 */
void gpio_disable_pci(gpio_t* self, bool disable_port);

/**
 * @brief Check if a GPIO event has occurred on the associated port.
 *
 * @return True if a GPIO event on the associated port has occurred, false
 * otherwise.
 */
bool gpio_event_occurred(const gpio_t* self);

/**
```

```
 * @brief Handle GPIO event on the associated port.
 *
 * @param[in, out] self GPIO instance.
 *
 * @return True if a GPIO event on the associated port was handled, false
 * otherwise.
 *
 * @note A callback is needed to handle occurring events.
 */
bool gpio_handle_event(gpio_t* self);

#endif /* GPIO_H_ */
```

lectures/L10/encapsulation_demo/include/driver/utils.h

```
/**
 * @brief GPIO utilities.
 */
#ifndef UTILS_H_
#define UTILS_H_

#include <stdbool.h>
#include <stdint.h>

/**
 * @brief Set bit in the given register.
 *
 * @param[in, out] reg Destination register.
 * @param[in] bit The bit to set.
 */
#define SET(reg, bit) ((reg) |= (1UL << (bit)))

/**
 * @brief Clear bit in the given register.
 *
 * @param[in, out] reg Destination register.
 * @param[in] bit The bit to clear.
 */
#define CLEAR(reg, bit) ((reg) &= ~(1UL << (bit)))

/**
 * @brief Toggle bit in the given register.
 *
```

```
 * @param[in, out] reg Destination register.
 * @param[in] bit The bit to toggle.
 */
#define TOGGLE(reg, bit) ((reg) ^= (1UL << (bit)))

/**
 * @brief Read bit in the given register.
 *
 * @param[in] reg Register to read from.
 * @param[in] bit The bit to read.
 *
 * @return True if the bit is set, false otherwise.
 */
#define READ(reg, bit) (bool)((reg) & (1UL << (bit)))

#endif /* UTILS_H_ */
```

lectures/L10/encapsulation_demo/source/driver/gpio.c

```
/**
 * @brief Implementation details for ATmega328p GPIO driver.
 */
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>

#include <avr/interrupt.h>
#include <avr/io.h>

#include "driver/gpio.h"
#include "driver/utils.h"

/** Number of available pins. */
#define PIN_COUNT 20U

/** Pin offset for I/O port B. */
#define PIN_OFFSET_B 8U

/** Pin offset for I/O port C. */
#define PIN_OFFSET_C 14U

/** Pin offset for I/O port D. */
```

```
#define PIN_OFFSET_D 0U

/** Number of available ports. */
#define PORT_COUNT 3U

/** Port offset for I/O port B. */
#define PORT_OFFSET_B 0U

/** Port offset for I/O port C. */
#define PORT_OFFSET_C 1U

/** Port offset for I/O port D. */
#define PORT_OFFSET_D 2U

/**
 * @brief GPIO driver structure.
 */
typedef struct gpio
{
    /** Pointer to the data mode register associated with the GPIO. */
    volatile uint8_t* ddrx;

    /** Pointer to the output register associated with the GPIO. */
    volatile uint8_t* portx;

    /** Pointer to the input register associated with the GPIO. */
    volatile uint8_t* pinx;

    /** Pointer to the pin change mask register associated with the GPIO. */
    volatile uint8_t* pcmskx;

    /** Pointer to the callback associated with the GPIO. */
    gpio_callback_t cb;

    /** Pin change interrupt control register bit (PCIEx) associated with the GPIO↩
    ↪ 's port. */
    uint8_t pcicrx;

    /** Bit position of the GPIO pin within the associated port. */
    uint8_t pin;

    /** Pin ID (for reserving the pin). */
    uint8_t id;
} gpio_t;
```

```
/** Pin registry (1 = pin is reserved, 0 = pin is free). */
static uint32_t pin_reg = 0U;

/** Event flags for each I/O port. */
static volatile uint8_t event_flags = 0U;

// ----------------------------------------------------------------------------
static inline bool is_pin_free(const uint8_t pin)
{
    // Return true if the pin number is valid and the pin is free.
    return PIN_COUNT > pin ? !READ(pin_reg, pin) : false;
}

// ----------------------------------------------------------------------------
static inline bool is_mode_valid(const gpio_mode_t mode)
{
    // Cast to uint8_t to avoid negative integers.
    return (uint8_t)(GPIO_MODE_OUTPUT) >= (uint8_t)(mode);
}

// ----------------------------------------------------------------------------
static inline void flag_event(const uint8_t port)
{
    // Flag event on the given port.
    if (PORT_COUNT > port) { SET(event_flags, port); }
}

// ----------------------------------------------------------------------------
static void gpio_init_attributes(gpio_t* self, const uint8_t pin)
{
    // Store pin ID (for pin registration).
    self->id = pin;

    // PIN 0 - 7 => I/O port D, pin = ID.
    if (PIN_OFFSET_B > pin)
    {
        self->ddrx   = &DDRD;
        self->portx  = &PORTD;
        self->pinx   = &PIND;
        self->pin    = pin - PIN_OFFSET_D;
        self->pcmskx = &PCMSK2;
        self->pcicrx = PCIE2;
    }
```

```
    // PIN 8 - 13 => I/O port B, pin = ID - 8.
    else if (PIN_OFFSET_C > pin)
    {
        self->ddrx   = &DDRB;
        self->portx  = &PORTB;
        self->pinx   = &PINB;
        self->pin    = pin - PIN_OFFSET_B;
        self->pcmskx = &PCMSK0;
        self->pcicrx = PCIE0;
    }
    // PIN 14 - 19 => I/O port C, pin = ID - 14.
    else if (PIN_COUNT > pin)
    {
        self->ddrx   = &DDRC;
        self->portx  = &PORTC;
        self->pinx   = &PINC;
        self->pin    = pin - PIN_OFFSET_C;
        self->pcmskx = &PCMSK1;
        self->pcicrx = PCIE1;
    }
}

// ----------------------------------------------------------------------------
static void gpio_set_mode(gpio_t* self, const gpio_mode_t mode)
{
    // Set GPIO mode.
    switch (mode)
    {
        case GPIO_MODE_INPUT_PULLUP:
        {
            // Configure as input by clearing the corresponding bit in DDRx.
            // Enable pull-up resistor by setting the corresponding bit in PORTx.
            CLEAR(*self->ddrx, self->pin);
            SET(*self->portx, self->pin);
            break;
        }
        case GPIO_MODE_OUTPUT:
        {
            // Configure as output by setting the corresponding bit in DDRx.
            SET(*self->ddrx, self->pin);
            CLEAR(*self->portx, self->pin);
            break;
        }
        default:
```

```
        {
            // Configure as input by clearing the corresponding bit in DDRx.
            CLEAR(*self->portx, self->pin);
            CLEAR(*self->ddrx, self->pin);
            break;
        }
    }
}

// ----------------------------------------------------------------------------
gpio_t* gpio_new(const uint8_t pin, const gpio_mode_t mode, gpio_callback_t cb)
{
    // Check the pin, return NULL if reserved or invalid.
    if (!is_pin_free(pin)) { return NULL; }

    // Check GPIO mode, return NULL if invalid.
    if (!is_mode_valid(mode)) { return NULL; }

    // Allocate memory for a new GPIO, return NULL on failure.
    gpio_t* self = (gpio_t*)(malloc(sizeof(gpio_t)));
    if (NULL == self) { return NULL; }

    // Initialize GPIO.
    gpio_init_attributes(self, pin);
    gpio_set_mode(self, mode);
    self->cb = cb;

    // Register pin, then return a pointer to the GPIO.
    SET(pin_reg, self->id);
    return self;
}

// ----------------------------------------------------------------------------
void gpio_del(gpio_t** self)
{
    // Check the GPIO instance, terminate the function if invalid.
    if (NULL == self) { return; }

    // Retrieve the GPIO implementation, terminate the function if invalid.
    gpio_t* gpio = *self;
    if (NULL == gpio) { return; }

    // Reset/clear used hardware (DDRx and PORTx).
    CLEAR(*gpio->ddrx, gpio->pin);
```

```
    CLEAR(*gpio->portx, gpio->pin);

    // Release the pin from the pin registry.
    CLEAR(pin_reg, gpio->id);

    // Free allocated memory and set the associated pointer to NULL.
    free(gpio);
    *self = NULL;
}

// ----------------------------------------------------------------------------
void gpio_write(gpio_t* self, const bool state)
{
    // Terminate the function if the GPIO is invalid.
    if (NULL == self) { return; }

    // Check data mode, terminate the function if not configured as output.
    if (!READ(*(self->ddrx), self->pin)) { return; }

    // Check the desired state, set/clear the bit in PORTx.
    if (state) { SET(*self->portx, self->pin); }
    else { CLEAR(*self->portx, self->pin); }
}

// ----------------------------------------------------------------------------
bool gpio_read(const gpio_t* self)
{
    // Read PINx if the GPIO is valid, otherwise return false.
    return NULL != self ? READ(*self->pinx, self->pin) : false;
}

// ----------------------------------------------------------------------------
void gpio_toggle(gpio_t* self)
{
    // Terminate the function if the GPIO is invalid.
    if (NULL == self) { return; }

    // Check data mode, terminate the function if not configured as output.
    if (!READ(*self->ddrx, self->pin)) { return; }

    // Toggle the output by writing a 1 to its bit in PINx (the hardware toggles ↩
    ↪ PORTx). A plain
    // write, not SET(): a read-modify-write of PINx would toggle every pin that ↩
    ↪ reads high.
```

```
    *self->pinx = (1U << self->pin);
}

// ----------------------------------------------------------------------------
void gpio_enable_pci(gpio_t* self)
{
    // Terminate the function if the GPIO is invalid.
    if (NULL == self) { return; }

    // Enable interrupts on the associated I/O-port.
    SET(PCICR, self->pcicrx);

    // Enable interrupts on the pin.
    SET(*self->pcmskx, self->pin);

    // Enable interrupts globally.
    sei();
}

// ----------------------------------------------------------------------------
void gpio_disable_pci(gpio_t* self, bool disable_port)
{
    // Terminate the function if the GPIO is invalid.
    if (NULL == self) { return; }

    // Disable interrupts on the pin.
    CLEAR(*self->pcmskx, self->pin);

    // Disable interrupts on the entire I/O-port if specified.
    if (disable_port) { CLEAR(PCICR, self->pcicrx); }
}

// ----------------------------------------------------------------------------
bool gpio_event_occurred(const gpio_t* self)
{
    return NULL != self ? READ(event_flags, self->pcicrx) : false;
}

// ----------------------------------------------------------------------------
bool gpio_handle_event(gpio_t* self)
{
    bool handled = false;

    // Check event flag, invoke callback if an event has occurred.
```

```
    if (gpio_event_occurred(self))
    {
        // Clear the event flag, then invoke the callback and mark the event as ↩
    ↪ handled. Clearing
        // first means an event that arrives while the callback runs is flagged ↩
    ↪ again, not lost.
        if (NULL != self->cb)
        {
            // Clear the flag with interrupts disabled, since an ISR flagging ↩
    ↪ another port between
            // the read and the write of this read-modify-write would otherwise be↩
    ↪  lost.
            cli();
            CLEAR(event_flags, self->pcicrx);
            sei();

            self->cb();
            handled = true;
        }
    }
    // Return true if the event was handled, false otherwise.
    return handled;
}

// ----------------------------------------------------------------------------
ISR(PCINT0_vect)
{
    // Flag event on I/O port B.
    flag_event(PORT_OFFSET_B);
}

// ----------------------------------------------------------------------------
ISR(PCINT1_vect)
{
    // Flag event on I/O port C.
    flag_event(PORT_OFFSET_C);
}

// ----------------------------------------------------------------------------
ISR(PCINT2_vect)
{
    // Flag event on I/O port D.
    flag_event(PORT_OFFSET_D);
}
```

The demo's `Makefile` is the one printed in §6.1.6, with `source/driver/gpio.c` in place of `source/driver/serial.c` in its `SRC_FILES`: this demo has no serial driver.

10.1.6 Vtables: a `gpio_interface_t` interface

- §10.1.3's `gpio_t` hardcodes real ATmega328P registers into every instance:
  - There's no way to run code that uses it without a chip attached.
  - There's no way to substitute a fake GPIO for host-side testing.
- The fix is a struct of function pointers, an **interface**, with each concrete implementation, real hardware, or an in-memory stub, providing its own vtable:

```
/** GPIO interface. */
typedef struct gpio_interface gpio_interface_t;

/**
 * @brief GPIO virtual table.
 */
typedef struct gpio_vtable
{
    void (*del)(gpio_interface_t** self);
    bool (*read)(const gpio_interface_t* self);
    void (*write)(gpio_interface_t* self, bool value);
    void (*toggle)(gpio_interface_t* self);
} gpio_vtable_t;

/**
 * @brief GPIO interface.
 */
typedef struct gpio_interface
{
    const gpio_vtable_t* vptr;   // pointer to the concrete implementation's ↩
    ↪ vtable
} gpio_interface_t;
```

Unlike §10.1.3/§10.1.4's `gpio_t`, `gpio_interface_t` has no register pointers, no pin, nothing hardware-specific at all, just a pointer to a table of functions. Every operation goes through it: `self->vptr->toggle(self)` instead of `gpio_toggle(self)`.

A concrete type implements the interface by embedding `gpio_interface_t` as its *first* field:

```
typedef struct gpio_impl
{
```

```
    gpio_interface_t itf;   // must be first: makes gpio_impl_t* safely ↩
    ↪ reinterpretable as
                            // gpio_interface_t*
    volatile uint8_t* ddrx;
    volatile uint8_t* portx;
    volatile uint8_t* pinx;
    uint8_t pin;
    uint8_t id;
} gpio_impl_t;
```

- Because `itf` is the struct's first member, C guarantees a `gpio_impl_t*` and a pointer to its first field share the same address, so a `gpio_impl_t*` can be cast to `gpio_interface_t*` and back without adjusting the pointer.
- That's the entire trick a C++ compiler automates for single inheritance: an object's address *is* its base class's address, and `self->vptr` is exactly the hidden vtable pointer every polymorphic C++ object carries.

Each vtable function casts back to the concrete type, then operates on its own fields exactly like §10.1.3's `gpio_write()`/`gpio_toggle()` did:

```
static inline gpio_impl_t* get_impl(gpio_interface_t* self) { return (gpio_impl_t↩
    ↪ *)(self); }

static void gpio_toggle(gpio_interface_t* self)
{
    gpio_impl_t* impl = get_impl(self);
    if ((NULL == impl) || !gpio_impl_is_output(impl)) { return; }
    *(impl->pinx) = (1U << impl->pin); // Writing a 1 to PINx toggles PORTx.
}

static const gpio_vtable_t* gpio_vptr_get_instance(void)
{
    static const gpio_vtable_t vtable = {
        .del = gpio_del, .read = gpio_read, .write = gpio_write, .toggle = ↩
    ↪ gpio_toggle,
    };
    return &vtable;   // one shared vtable; every instance's itf.vptr points at ↩
    ↪ this same address
}

gpio_interface_t* gpio_atmega328p_new(const uint8_t pin, const gpio_mode_t mode)
{
    gpio_impl_t* impl = (gpio_impl_t*)(malloc(sizeof(gpio_impl_t)));
```

```
    if (NULL == impl) { return NULL; }
    // ... configure impl->ddrx/portx/pinx/pin from pin, same mapping A.2's ↩
    ↪ gpio_init() did ...
    impl->itf.vptr = gpio_vptr_get_instance();
    return (gpio_interface_t*)(impl);
}
```

- Like §10.1.3's `gpio_create()`, `gpio_atmega328p_new()` calls `malloc()`, one instance per call, never resized, never repeatedly allocated:
  - What's actually new here isn't the allocation strategy, it's what can sit behind the pointer: `gpio_interface_t*` can point at a `gpio_impl_t` or a `gpio_stub_t`, two differently-sized concrete structs. §10.1.3 only ever had one struct shape (`struct gpio`) to allocate.
- `del()` takes `gpio_interface_t**` rather than `gpio_interface_t*`:
  - It can `free()` the instance and null out the caller's own pointer in one call, guarding against a second `del()` or a use-after-free through that same variable.
  - That's a different defense than Exercise 9.3 considered for `gpio_free()` (whether the freed instance's *own* fields should be reset), but the same underlying worry: a pointer to a GPIO that no longer safely means what it used to.

A second implementation, `gpio_stub_t`, satisfies the exact same interface without touching any hardware, a `bool state` field stands in for the registers:

```
typedef struct gpio_stub
{
    gpio_interface_t itf;   // still first
    bool state;
} gpio_stub_t;

static bool gpio_read(const gpio_interface_t* self)
{
    const gpio_stub_t* impl = (const gpio_stub_t*)(self);
    return NULL != impl ? impl->state : false;
}
```

Calling code never knows the difference: anything holding a `gpio_interface_t*` can call `self->vptr->read(self)`, whether `self` came from `gpio_atmega328p_new()` or `gpio_stub_new()`.

This is the same mechanism higher-level languages build `virtual` functions on top of: an object with virtual methods carries a hidden pointer to a table of function addresses (its vtable), and calling a virtual method is, under the hood, exactly the `self->vptr->toggle(...)` pattern above, just generated by the compiler instead of written out by hand.

```
interface_demo/
├── Makefile
├── main.c
├── include/
│   └── driver/
│       └── gpio/
│           ├── atmega328p.h
│           ├── interface.h
│           └── stub.h
└── source/
    └── driver/
        └── gpio/
            ├── atmega328p.c
            └── stub.c
```

- `driver/gpio/interface.h` is the full interface shown above.
- `driver/gpio/atmega328p.h/.c` and `driver/gpio/stub.h/.c` are the two implementations in full, all printed at the end of this section.
  - Including the pin-registry reservation `gpio_atmega328p_new()` uses: the same idea as Exercise 9.5's `pin_reg`, just widened from a `uint16_t` covering pins 0–13 to a `uint32_t` covering pins 0–19, since this demo's pin mapping also reaches `PORTC`.

`main.c` toggles a real LED on D8 whenever a *simulated* button, a `gpio_stub_t` whose state flips every 60,000 main-loop iterations instead of being wired to real hardware, reports a rising edge. Both constructors can fail, the ATmega328P one also when its pin is already taken, so `main()` checks each result before using it:

lectures/L10/interface_demo/main.c

```c
/**
 * @brief GPIO driver example with interfaces in C.
 */
#include <stddef.h>

#include "driver/gpio/atmega328p.h"
#include "driver/gpio/stub.h"

#define LED1_PIN 8U // D8.

/** Max value of the loop counter used to simulate button events. */
#define LOOP_CNT_MAX 60000UL

/**
 * @brief Simulate button events.
```

```
 *
 * @param[out] btn Pointer to the simulated button.
 */
void simulate_btn_event(gpio_interface_t* btn)
{
    static uint16_t loop_cnt = 0U;

    // Simulate a button toggle every LOOP_CNT_MAX calls.
    if (LOOP_CNT_MAX <= ++loop_cnt)
    {
        btn->vptr->toggle(btn);
        loop_cnt = 0U;
    }
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur), or -1 on
 * failure.
 */
int main(void)
{
    // Create and initialize GPIO instances, return -1 on failure.
    gpio_interface_t* led1 = gpio_atmega328p_new(LED1_PIN, GPIO_MODE_OUTPUT);
    if (NULL == led1) { return -1; }

    gpio_interface_t* btn1 = gpio_stub_new();
    if (NULL == btn1) { return -1; }
    bool btn1_prev = false;

    while (1)
    {
        // Simulate button events every LOOP_CNT_MAX main-loop iterations.
        simulate_btn_event(btn1);

        // Detect pressdown (rising edge).
        const bool btn1_current = btn1->vptr->read(btn1);
        const bool btn1_pressed = btn1_current && !btn1_prev;

        // Toggle the LED on button pressdown (rising edge).
        if (btn1_pressed) { led1->vptr->toggle(led1); }
        btn1_prev = btn1_current;
    }
```

```
    // Unreachable in this example (infinite loop). Shown for completeness.
    led1->vptr->del(&led1);
    btn1->vptr->del(&btn1);
    return 0;
}
```

Neither `simulate_btn_event()` nor `main()`'s edge-detection logic cares that `led1` is real hardware and `btn1` isn't: both are just `gpio_interface_t*`. Swap `gpio_stub_new()` for a second `gpio_atmega328p_new()` call and a real button, and nothing else in this file changes.

Build and flash the same way as §9.1.5: on Linux, run `make` in the demo's directory, `lectures/L10/interface_demo` (`make flash` to program the board); on Windows via Microchip Studio, recreate this folder structure inside the project and add `include` as an include directory.

The interface demo in full

The demo in `lectures/L10/interface_demo` is about interfaces and vtables in C: a hand-written interface (`gpio_interface_t`) backed by two independent concrete GPIO implementations — real ATmega328P hardware and an in-memory stub — that satisfy the same contract and can be used interchangeably.

This example demonstrates, in simplified form, how C++ commonly implements `virtual` functions and single inheritance under the hood. The interface object contains only a pointer to a table of function pointers. Each concrete type embeds that interface object as its first field, allowing a pointer to the concrete object to be used as a pointer to the interface without changing its address.

Calling code holds only a `gpio_interface_t*` and dispatches operations through `self->vptr->...`. It does not know whether the object controls a physical ATmega328P pin or stores its state entirely in memory.

The implementation is written for compilation on an ATmega328P in Microchip Studio.

Files.

- `driver/gpio/interface.h`:
  - Contains the implementation-independent GPIO contract.
  - Defines `gpio_mode_t`.
  - Defines `gpio_vtable_t`.
  - Declares the complete set of operations that a concrete GPIO implementation must provide:
    - `del()`
    - `read()`
    - `write()`
    - `toggle()`

  - Defines the signatures of all GPIO operation pointers.
  - Uses `gpio_interface_t*` as the common `self` parameter type.
  - Defines `gpio_interface_t`.
  - Stores only `const gpio_vtable_t* vptr` in the interface object.
  - Contains no ATmega328P register pointers.
  - Contains no pin number or hardware-specific state.
  - Contains no stub-specific state.
  - Allows calling code to dispatch operations without knowing the concrete object layout.
  - Is the only GPIO header required by code that receives an already-created interface object.
  - Serves a role similar to an abstract base class in C++.
  - Describes behavior without selecting an implementation.
  - Allows additional GPIO implementations to be added without changing the interface-using logic.
- `driver/gpio/atmega328p.h/driver/gpio/atmega328p.c`:
  - Declare and define `gpio_atmega328p_new(uint8_t pin, gpio_mode_t mode)`.
  - Provide the concrete implementation backed by real ATmega328P GPIO registers.
  - Return a `gpio_interface_t*` rather than exposing the private concrete type.
  - Keep `gpio_impl_t` private to `atmega328p.c`.
  - Define `gpio_impl_t` with `gpio_interface_t` as its first field.
  - Extend the embedded interface with hardware-specific state:
    - `ddrx`
    - `portx`
    - `pinx`
    - `pin`
    - `id`
  - Store register pointers for the selected hardware port.
  - Store the bit position used within those registers.
  - Store an identifier used to release the pin from the registry.
  - Define a `static const gpio_vtable_t` for the ATmega328P implementation.
  - Assign the ATmega328P vtable to the embedded interface during construction.
  - Implement private concrete operations for:
    - Reading a hardware pin.
    - Writing a hardware pin.
    - Toggling a hardware pin.
    - Deleting the hardware-backed object.

  - Convert the incoming interface pointer back to `gpio_impl_t*` inside each concrete operation.
  - Validate the supplied Arduino pin number.
  - Map the pin number to `PORTB`, `PORTC`, or `PORTD`.
  - Select the corresponding `DDRx` and `PINx` registers.
  - Calculate the bit position within the selected port.
  - Configure the pin according to the requested `gpio_mode_t`.
  - Allocate one private `gpio_impl_t` object with `malloc()`.
  - Return `NULL` if allocation fails.
  - Maintain a pin registry representing currently owned physical pins.
  - Check the registry before claiming a pin.
  - Return `NULL` if another live object already owns the requested pin.
  - Mark the pin as reserved after successful validation and allocation.
  - Prevent conflicting GPIO instances from silently modifying the same hardware.
  - Implement concrete deletion through the interface:
    - Validate the supplied pointer.
    - Recover the concrete `gpio_impl_t`.
    - Reset the pin's data-direction and output state.
    - Release the pin from `pin_registry`.
    - Free the concrete allocation.
    - Set the caller's interface pointer to `NULL`.
  - Keep register mapping, reservation, and cleanup details hidden from interface-using code.
- `driver/gpio/stub.h/driver/gpio/stub.c`:
  - Declare and define `gpio_stub_new(void)`.
  - Provide a concrete implementation backed only by memory.
  - Return the same public type, `gpio_interface_t*`, as the hardware constructor.
  - Keep `gpio_stub_t` private to `stub.c`.
  - Define `gpio_stub_t` with `gpio_interface_t` as its first field.
  - Extend the embedded interface with a `bool state`.
  - Define a separate `static const gpio_vtable_t` for the stub implementation.
  - Assign the stub vtable during construction.
  - Allocate one `gpio_stub_t` object with `malloc()`.
  - Initialize its logical state.
  - Return `NULL` if allocation fails.
  - Implement `write()` by assigning the in-memory state.
  - Implement `read()` by returning the in-memory state.

  - Implement `toggle()` by inverting the state.
  - Implement `del()` by freeing the stub allocation and nulling the caller's pointer.
  - Perform no hardware register access.
  - Require no ATmega328P GPIO pin.
  - Require no external wiring.
  - Satisfy the same behavioral contract as the real GPIO implementation.
  - Allow higher-level logic to be exercised in a deterministic software-only environment.
- `main.c`:
  - Uses `gpio_interface_t*` for both GPIO objects.
  - Creates one real LED with `gpio_atmega328p_new()`.
  - Assigns the LED to Arduino digital pin D8.
  - Configures the LED pin as an output.
  - Creates one simulated button with `gpio_stub_new()`.
  - Checks both constructor results before using the returned objects.
  - Does not access the private fields of either concrete implementation.
  - Does not need to know the size of either concrete struct.
  - Calls every GPIO operation through `self->vptr`.
  - Implements `simulate_btn_event()`.
  - Toggles the stub button after approximately every 60,000 main-loop iterations.
  - Uses the simulated change in state as a substitute for a physical button press.
  - Stores the previous button state.
  - Reads the current button state through the interface.
  - Detects a rising edge when:
    - The previous state was low.
    - The current state is high.
  - Toggles the physical LED when a rising edge is detected.
  - Performs both the button read and LED toggle through the same interface contract.
  - Demonstrates one application simultaneously using two different concrete implementations.
  - Contains no special branch for distinguishing the real object from the stub object.
  - Demonstrates that dynamic dispatch is determined by each object's vtable.
  - Deletes both objects through their interface-provided `del()` operations.
  - Leaves implementation-specific cleanup to the concrete types.
  - Demonstrates hardware-independent application logic built on top of a C interface.

`main.c` is the file printed above. The interface and its two implementations:

lectures/L10/interface_demo/include/driver/gpio/interface.h

```c
/**
 * @brief GPIO interface.
 */
#ifndef DRIVER_GPIO_INTERFACE_H_
#define DRIVER_GPIO_INTERFACE_H_

#include <stdbool.h>
#include <stdint.h>

/**
 * @brief Enumeration of GPIO modes.
 */
typedef enum
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/** GPIO interface. */
typedef struct gpio_interface gpio_interface_t;

/**
 * @brief GPIO virtual table.
 */
typedef struct gpio_vtable
{
    /**
     * @brief Delete given GPIO.
     *
     *        Release allocated resources and set the corresponding pointer to
     * null.
     *
     * @param[in] self Double pointer to the GPIO to delete.
     */
    void (*del)(gpio_interface_t** self);

    /**
     * @brief Read input of given GPIO.
     *
     * @param[in] self The GPIO to read.
     *
     * @return True if the GPIO is enabled, false otherwise.
```

```
     */
    bool (*read)(const gpio_interface_t* self);

    /**
     * @brief Write output of given GPIO.
     *
     *        This operation is only supported for outputs.
     *
     * @param[in] self The GPIO to write.
     * @param[in] value The value to write.
     */
    void (*write)(gpio_interface_t* self, bool value);

    /**
     * @brief Toggle output of given GPIO.
     *
     *        This operation is only supported for outputs.
     *
     * @param[in] self The GPIO to toggle.
     */
    void (*toggle)(gpio_interface_t* self);
} gpio_vtable_t;

/**
 * @brief GPIO interface.
 */
typedef struct gpio_interface
{
    /** Pointer to the GPIO virtual table. */
    const gpio_vtable_t* vptr;
} gpio_interface_t;

#endif /* DRIVER_GPIO_INTERFACE_H_ */
```

lectures/L10/interface_demo/include/driver/gpio/atmega328p.h

```
/**
 * @brief GPIO driver for ATmega328p.
 */
#ifndef DRIVER_GPIO_ATMEGA328P_H_
#define DRIVER_GPIO_ATMEGA328P_H_

#include <stdbool.h>
```

```
#include <stdint.h>

#include "driver/gpio/interface.h"

/**
 * @brief Create new GPIO.
 *
 * @param[in] pin Arduino pin number.
 * @param[in] mode GPIO mode.
 *
 * @return The new GPIO instance, or NULL on failure.
 */
gpio_interface_t* gpio_atmega328p_new(uint8_t pin, gpio_mode_t mode);

#endif /* DRIVER_GPIO_ATMEGA328P_H_ */
```

lectures/L10/interface_demo/source/driver/gpio/atmega328p.c

```
/**
 * @brief GPIO driver implementation details for ATmega328p.
 */
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <stdlib.h>

#include <avr/io.h>

#include "driver/gpio/atmega328p.h"

#define PIN_COUNT 20U // The number of available pins.

/** I/O port offsets (when mapping against Arduino pins). */
#define PIN_OFFSET_B 8U  // Pin offset for I/O port B (pin 9 = PORTB1).
#define PIN_OFFSET_C 14U // Pin offset for I/O port C (pin 14 = PORTC0).
#define PIN_OFFSET_D 0U  // Pin offset for I/O port D (pin 5 = PORTD5).

/**
 * @brief GPIO structure.
 */
typedef struct gpio_impl
{
    /** GPIO interface. */
```

```
    gpio_interface_t itf;

    /** Pointer to the data direction register. */
    volatile uint8_t* ddrx;

    /** Pointer to the port register. */
    volatile uint8_t* portx;

    /** Pointer to the pin register. */
    volatile uint8_t* pinx;

    /** Pin on the associated I/O port. */
    uint8_t pin;

    /** Pin ID (equal to the Arduino Uno pin). */
    uint8_t id;
} gpio_impl_t;

/** Pin registry holding the state of each pin (1 = reserved, 0 = free). */
static uint32_t pin_registry = 0U;

/** Get a pointer to the GPIO virtual table. */
static const gpio_vtable_t* gpio_vptr_get_instance(void);

// ----------------------------------------------------------------------------
static inline gpio_impl_t* get_impl(gpio_interface_t* self) { return (gpio_impl_t↩
    ↪ *)(self); }

// ----------------------------------------------------------------------------
static inline const gpio_impl_t* get_const_impl(const gpio_interface_t* self)
{
    return (const gpio_impl_t*)(self);
}

// ----------------------------------------------------------------------------
static bool is_pin_free(const uint8_t id)
{
    // Return true if the corresponding bit in the pin registry is 0, false ↩
    ↪ otherwise.
    return PIN_COUNT > id ? (0U == (pin_registry & (1UL << id))) : false;
}

// ----------------------------------------------------------------------------
static void reserve_pin(const uint8_t id)
```

```
{
    // Set the corresponding bit in the pin registry.
    pin_registry |= (1UL << id);
}

// ----------------------------------------------------------------------------
static void release_pin(const uint8_t id)
{
    // Clear the corresponding bit in the pin registry.
    pin_registry &= ~(1UL << id);
}

// ----------------------------------------------------------------------------
static bool gpio_impl_is_output(const gpio_impl_t* impl)
{
    // Retrieve implementation details, return false on failure.
    return NULL != impl ? (bool)(*(impl->ddrx) & (1U << impl->pin)) : false;
}

// ----------------------------------------------------------------------------
static void gpio_impl_init(gpio_impl_t* impl, const uint8_t pin, const gpio_mode_t
    ↪ mode)
{
    // Save the pin ID for reservation and cleanup.
    impl->id = pin;

    // Initialize the vtable pointer.
    impl->itf.vptr = gpio_vptr_get_instance();

    // If 0 <= pin <= 7 => the GPIO is connected to I/O port D.
    if (PIN_OFFSET_B > pin)
    {
        impl->ddrx  = &DDRD;
        impl->portx = &PORTD;
        impl->pinx  = &PIND;
        impl->pin   = pin - PIN_OFFSET_D;
    }
    // Else if 8 <= pin <= 13 => the GPIO is connected to I/O port B.
    else if (PIN_OFFSET_C > pin)
    {
        impl->ddrx  = &DDRB;
        impl->portx = &PORTB;
        impl->pinx  = &PINB;
        impl->pin   = pin - PIN_OFFSET_B;
```

```
    }
    // Else if 14 <= pin <= 19 => the GPIO is connected to I/O port C.
    else
    {
        impl->ddrx  = &DDRC;
        impl->portx = &PORTC;
        impl->pinx  = &PINC;
        impl->pin   = pin - PIN_OFFSET_C;
    }

    // Set GPIO mode.
    switch (mode)
    {
        case GPIO_MODE_INPUT_PULLUP:
        {
            // Clear the associated bit in DDRx (configure as input).
            // Set the associated bit in PORTx (enable pull-up).
            *(impl->ddrx) &= ~(1U << impl->pin);
            *(impl->portx) |= (1U << impl->pin);
            break;
        }
        case GPIO_MODE_OUTPUT:
        {
            // Set the associated bit in DDRx (configure as output).
            // Clear the associated bit in PORTx (set the initial output value to ↩
    ↪ 0).
            *(impl->ddrx) |= (1U << impl->pin);
            *(impl->portx) &= ~(1U << impl->pin);
            break;
        }
        default:
        {
            // Clear the associated bits in DDRx and PORTx as default.
            *(impl->ddrx) &= ~(1U << impl->pin);
            *(impl->portx) &= ~(1U << impl->pin);
            break;
        }
    }
    // Reserve the pin before terminating the function.
    reserve_pin(pin);
}

// ----------------------------------------------------------------------------
static void gpio_impl_cleanup(gpio_impl_t* impl)
```

```
{
    // Clear the associated bits in the GPIO registers and the pin registry.
    *(impl->ddrx) &= ~(1U << impl->pin);
    *(impl->portx) &= ~(1U << impl->pin);
    release_pin(impl->id);
}

// ----------------------------------------------------------------------------
static void gpio_del(gpio_interface_t** self)
{
    // Terminate the function if self is NULL.
    if (NULL == self) { return; }

    // Retrieve implementation details, terminate the function on failure.
    gpio_impl_t* impl = get_impl(*self);
    if (NULL == impl) { return; }

    // Clean up GPIO registers and the pin registry.
    gpio_impl_cleanup(impl);

    // Free allocated memory.
    free(impl);

    // Set the associated pointer to null.
    *self = NULL;
}

// ----------------------------------------------------------------------------
static bool gpio_read(const gpio_interface_t* self)
{
    // Retrieve implementation details.
    const gpio_impl_t* impl = get_const_impl(self);

    // Return true if the input is high, false otherwise.
    return NULL != impl ? (bool)(*(impl->pinx) & (1U << impl->pin)) : false;
}

// ----------------------------------------------------------------------------
static void gpio_write(gpio_interface_t* self, const bool value)
{
    // Retrieve implementation details.
    // Terminate the function on failure or if the GPIO is configured as input.
    gpio_impl_t* impl = get_impl(self);
    if ((NULL == impl) || !gpio_impl_is_output(impl)) { return; }
```

```
    // Set/clear the output in accordance with the given value.
    if (value) { *(impl->portx) |= (1U << impl->pin); }
    else { *(impl->portx) &= ~(1U << impl->pin); }
}

// ----------------------------------------------------------------------------
static void gpio_toggle(gpio_interface_t* self)
{
    // Retrieve implementation details.
    // Terminate the function on failure or if the GPIO is configured as input.
    gpio_impl_t* impl = get_impl(self);
    if ((NULL == impl) || !gpio_impl_is_output(impl)) { return; }

    // Toggle the output by writing a 1 to its bit in PINx (the hardware toggles ↩
    ↪ PORTx). A plain
    // write, not |=: a read-modify-write of PINx would toggle every pin that ↩
    ↪ reads high.
    *(impl->pinx) = (1U << impl->pin);
}

// ----------------------------------------------------------------------------
static const gpio_vtable_t* gpio_vptr_get_instance(void)
{
    // Create and initialize vtable holding function pointers (done once at ↩
    ↪ startup).
    static const gpio_vtable_t vtable = {
        .del    = gpio_del,
        .read   = gpio_read,
        .write  = gpio_write,
        .toggle = gpio_toggle,
    };
    // Return a pointer to the vtable.
    return &vtable;
}

// ----------------------------------------------------------------------------
gpio_interface_t* gpio_atmega328p_new(const uint8_t pin, const gpio_mode_t mode)
{
    // Return NULL if the pin is reserved.
    if (!is_pin_free(pin)) { return NULL; }

    // Create new GPIO instance, return NULL on failure.
    gpio_impl_t* impl = (gpio_impl_t*)(malloc(sizeof(gpio_impl_t)));
```

```
    if (NULL == impl) { return NULL; }

    // Initialize the GPIO instance.
    gpio_impl_init(impl, pin, mode);
    impl->itf.vptr = gpio_vptr_get_instance();

    // Return the initialized GPIO instance, cast to the corresponding interface.
    return (gpio_interface_t*)(impl);
}
```

lectures/L10/interface_demo/include/driver/gpio/stub.h

```
/**
 * @brief GPIO driver stub.
 */
#ifndef DRIVER_GPIO_STUB_H_
#define DRIVER_GPIO_STUB_H_

#include <stdbool.h>
#include <stdint.h>

#include "driver/gpio/interface.h"

/**
 * @brief Create new GPIO stub.
 *
 * @return The new GPIO instance, or NULL on failure.
 */
gpio_interface_t* gpio_stub_new(void);

#endif /* DRIVER_GPIO_STUB_H_ */
```

lectures/L10/interface_demo/source/driver/gpio/stub.c

```
/**
 * @brief GPIO stub driver implementation.
 */
#include <stdbool.h>
#include <stdint.h>
#include <stdlib.h>

#include "driver/gpio/stub.h"
```

```c
/**
 * @brief GPIO stub structure.
 */
typedef struct gpio_stub
{
    /** GPIO interface. */
    gpio_interface_t itf;

    /** GPIO state (true = high, false = low). */
    bool state;
} gpio_stub_t;

// ----------------------------------------------------------------------------
static inline gpio_stub_t* get_impl(gpio_interface_t* self) { return (gpio_stub_t*)(self); }

// ----------------------------------------------------------------------------
static inline const gpio_stub_t* get_const_impl(const gpio_interface_t* self)
{
    return (const gpio_stub_t*)(self);
}

// ----------------------------------------------------------------------------
static void gpio_del(gpio_interface_t** self)
{
    // Terminate the function if self is NULL.
    if (NULL == self) { return; }

    // Retrieve implementation details, terminate the function on failure.
    gpio_stub_t* impl = get_impl(*self);
    if (NULL == impl) { return; }

    // Free allocated memory.
    free(impl);

    // Set the associated pointer to null.
    *self = NULL;
}

// ----------------------------------------------------------------------------
static bool gpio_read(const gpio_interface_t* self)
{
    // Retrieve implementation details.
    const gpio_stub_t* impl = get_const_impl(self);
```

```
    // Return true if the input is high, false otherwise.
    return NULL != impl ? impl->state : false;
}

// ----------------------------------------------------------------------------
static void gpio_write(gpio_interface_t* self, const bool value)
{
    // Retrieve implementation details, terminate the function on failure.
    gpio_stub_t* impl = get_impl(self);
    if (NULL == impl) { return; }

    // Set/clear the output in accordance with the given value.
    impl->state = value;
}

// ----------------------------------------------------------------------------
static void gpio_toggle(gpio_interface_t* self)
{
    // Retrieve implementation details, terminate the function on failure.
    gpio_stub_t* impl = get_impl(self);
    if (NULL == impl) { return; }

    // Toggle the output.
    impl->state = !impl->state;
}

// ----------------------------------------------------------------------------
static const gpio_vtable_t* gpio_vptr_get_instance(void)
{
    // Create and initialize vtable holding function pointers (done once at ↩
    ↪ startup).
    static const gpio_vtable_t vtable = {
        .del    = gpio_del,
        .read   = gpio_read,
        .write  = gpio_write,
        .toggle = gpio_toggle,
    };
    // Return a pointer to the vtable.
    return &vtable;
}

// ----------------------------------------------------------------------------
gpio_interface_t* gpio_stub_new(void)
```

```
{
    // Create new GPIO instance, return NULL on failure.
    gpio_stub_t* self = (gpio_stub_t*)(malloc(sizeof(gpio_stub_t)));
    if (NULL == self) { return NULL; }

    // Initialize the instance.
    self->itf.vptr = gpio_vptr_get_instance();
    self->state    = false;

    // Return the initialized GPIO instance, cast to the corresponding interface.
    return (gpio_interface_t*)(self);
}
```

The demo's `Makefile` is the one printed in §6.1.6, with `source/driver/gpio/atmega328p.c` and `source/driver/gpio/stub.c` in place of `source/driver/serial.c` in its `SRC_FILES`: this demo has no serial driver either.

10.2 Review

What you should be able to do now

After this chapter, you should:

- Explain how `static` restricts a function's visibility to its own file, and why that's a different kind of encapsulation than hiding struct fields.
- Explain what an opaque struct is and why hiding a struct's definition in a `.c` file protects its invariants.
- Be able to declare, assign, and call through a function pointer, including one stored in a struct field.
- Be able to design a small vtable-style interface and provide more than one implementation of it.
- Understand, at a high level, how this manual pattern relates to virtual dispatch in higher-level languages.

Questions to test yourself

1. What stops calling code from calling the `static` `gpio_write()` in §10.1.2's `gpio.c` directly by name, and what does it *not* stop it from doing to `led1.write` or `led1.pin`?
2. Why can't code outside `timer.c` read or write a `timer_t`'s fields directly, once `timer_t` is made opaque?
3. What does `void (*on_press)(void);` declare, and how would you call it?
4. Why does the pin-change ISR only set a flag instead of calling the callback directly?

5. What problem does a vtable-style interface solve that hardcoding a single GPIO implementation doesn't?

Looking ahead

- This closes the driver-design arc (Chapters 9 and 10), built on the peripherals covered in Chapters 1–8.
- Everything here, opaque structs, function pointers, vtables, was C standing in for object orientation. The next step is Modern Embedded C++, where these patterns become language features (classes, access specifiers, virtual functions) instead of conventions enforced by hand.
- Covered in a separate course, *Modern Embedded C++*: `https://github.com/qrtech-academy/modern-embedded-cpp`.

10.3 Exercises

> **How to work these.** The Reflection exercises are answered on paper, from the chapter. The Code exercises are programs for the board: build and flash them the way §2.2 (Microchip Studio) or §2.3 (Linux) describes, or run them in the simulator of Appendix A if you have no board yet.
> **How to check your work.** The Reflection exercises are answered in Appendix B. The Code exercises have no printed solution: the board tells you whether a program does what the exercise describes. Try each exercise before you read its answer.

EXERCISE SET 1

Concepts

Exercise 10.1 Only a forward declaration

REFLECTION

Why does `gpio.h` only forward-declare `gpio_t` (`typedef struct gpio gpio_t;`) instead of including the full field list? What would go wrong, from an encapsulation standpoint, if `struct gpio { ... };` were written directly in the header instead?

Exercise 10.2 When `malloc()` fails

REFLECTION

`gpio_create()` returns `NULL` when `malloc()` fails instead of, say, overwriting the first GPIO it ever created. Why does that matter, and what should calling code do with the return value?

EXERCISE SET 2

Program

Exercise 10.3 Active-low GPIOs CODE

Add a second behavior to §10.1.2's driver: `static` functions `gpio_write_inverted()`, `gpio_read_inverted()`, and `gpio_toggle_inverted()` that treat the pin as active-low (a `true` write clears the bit rather than setting it), plus a second `gpio_init_inverted(gpio_t* self, uint8_t pin, gpio_mode_t mode)` that behaves like `gpio_init()` but wires a `gpio_t`'s pointers to these instead of the originals. Confirm the same `led1.write(&led1, true)` call site now does the opposite, with no change to any calling code, only which functions `gpio_init_inverted()` wired up.

Exercise 10.4 An opaque timer CODE

Make Chapter 9's `timer_t` opaque: move `struct timer { ... };` into a `timer.c` file, expose only `typedef struct timer timer_t;` plus prototypes for `timer_create()`, `timer_clear()`, `timer_running()`, `timer_start()`, `timer_stop()`, `timer_toggle()`, and `timer_reset()`, from `timer.h`. `timer_create(uint32_t timeout_ms, timer_callback_t callback)` should follow `gpio_create()`'s allocation pattern from §10.1.3, since an opaque `timer_t` can no longer be declared and passed by address the way Exercise 9.6's `timer_init(timer_t* self, uint32_t timeout_ms, timer_callback_t callback)` expected.

Exercise 10.5 A button callback for real CODE

Wire up §10.1.4's `button_t` for real: create one `gpio_t` configured as a button input and another as an LED output, then set `on_press` to a function that calls `gpio_toggle()` on the LED. `button_check()`'s own code should never mention the LED's `gpio_t` by name. See §10.1.5 for a fully wired, interrupt-driven version of this same idea.

Exercise 10.6 Testing through the stub CODE

Capstone, harder. `gpio_stub_new()` (§10.1.6) exists specifically so GPIO-dependent logic can be exercised without a chip attached. Write a small test that creates a `gpio_interface_t*` via `gpio_stub_new()` alone, no `gpio_atmega328p_new()`, no hardware, and confirms `write()`/`read()`/`toggle()` behave correctly purely through the interface (write `true`, read back `true`, `toggle()` flips it to `false`). Then write a third implementation, one that logs every `write()`/`toggle()` call before updating its own `state`, and confirm `main.c`'s edge-detection logic (§10.1.6) works against it completely unmodified, just by changing which `gpio_interface_t*` gets created.

APPENDIX A

Running the Demos Without Hardware

This book targets real ATmega328P hardware, and the worked demos are written to be flashed onto an Arduino Uno/Nano. If you don't have a board yet, you can still run every demo in the companion repository using simavr (`https://github.com/buserror/simavr`), a cycle-accurate AVR simulator.

Everything described here has been verified against the demos in `lectures/`: the LED toggles, the button interrupt fires, the debounce timer works, UART output appears, EEPROM survives a restart, and starving the watchdog resets the MCU. Two things behave differently from the board, and both are explained where they come up: the simulated button is wired the other way round (see §A.8), and the watchdog's reboot loop runs faster (see §A.7.1).

> **This is a fallback, not a replacement.** Simulation gets you unblocked, but it will not teach you what a floating input, a missing pull-up, or a badly seated jumper wire feels like. Get a board when you can.

A.1 What You Get, and What You Don't

There are two ways to use simavr, and it's worth being clear about the difference up front.

	Stock `simavr` CLI	With the harness below
Run the firmware	Yes	Yes
Debug with `avr-gdb`	Yes	Yes (`simavr -g`)
See UART output	**No**	Yes
Press a button	**No**	Yes
See the LED change	**No**	Yes
Persist EEPROM across restarts	**No**	Yes

The stock `simavr` command-line tool has no way to display UART traffic and no way to drive an input pin. It runs your firmware in complete silence. That is fine for a GDB session, but useless for the LED-and-button demos.

To actually interact with the demos you need a small host-side program that links against `libsimavr` and connects a simulated UART and a simulated button to the chip. That program is provided in full below: a couple of hundred lines, over a third of it comments, and you only build it once.

A.2 Install

On Debian/Ubuntu:

```
sudo apt install -y \
    gcc-avr avr-libc avrdude \
    simavr libsimavr-dev libsimavrparts1 libelf-dev \
    gdb-avr build-essential
```

- **macOS:** `brew install simavr` and `brew install osx-cross/avr/avr-gcc`. The harness builds the same way; adjust include/library paths to your Homebrew prefix.
- **Windows:** use WSL and follow the Debian/Ubuntu instructions. The rest of this book assumes a Linux-like shell anyway.

A.3 Build the Firmware

Nothing changes. The existing project makefiles already produce exactly what simavr wants:

```
make -C lectures/L09/gpio_driver_demo
```

This gives you `main.elf` and `main.hex`. **Prefer the `.elf`**; it carries symbols, so simavr can report meaningful addresses and GDB can show you source.

Throughout this appendix, shell commands run from the **repository root** unless a `cd` says otherwise. Watch for this when you open a second terminal: a new shell starts in your home directory, not where the previous one was.

A.4 Quick Path: Just Run It

```
simavr -m atmega328p -f 16000000 lectures/L09/gpio_driver_demo/main.elf
```

`-m` and `-f` are required. The demo firmwares don't embed a `.mmcu` ELF section (that would mean including simavr's `avr_mcu_section.h` in the firmware source), so simavr can't infer the target or the clock and you must tell it.

Be aware: this prints nothing. The firmware is running, but no UART is attached and no button can be pressed. Press `Ctrl+C` to stop. To see anything happen, use the harness.

A.5 Debugging With avr-gdb

This works with the stock CLI and is worth learning; it is by far the fastest way to understand what a driver is doing.

The project makefiles compile with `-Os` only, so first rebuild with debug symbols:

```
# Terminal 1, from the repository root.
cd lectures/L09/gpio_driver_demo
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os -g \
    -o main_dbg.elf main.c source/driver/*.c \
    -Wall -Iinclude -std=c11
```

Start simavr as a GDB server (it waits for a connection on port 1234):

```
simavr -g -m atmega328p -f 16000000 main_dbg.elf
```

In another terminal. Note the `cd`: the second terminal starts in your home directory, so without it `avr-gdb` reports `main_dbg.elf: No such file or directory`.

```
# Terminal 2, from the repository root.
cd lectures/L09/gpio_driver_demo
avr-gdb main_dbg.elf
```

```
(gdb) target remote :1234
(gdb) break main
(gdb) continue
(gdb) next
(gdb) info locals
(gdb) print led1
(gdb) backtrace
```

You get real source-level stepping, breakpoints, locals, and struct inspection, including things like `print led1` showing `{ddrx = ..., portx = ..., pcicrx = 0, pin = 1}`.

Two pieces of harmless noise you may see on Ubuntu, so you don't waste time chasing them:

- `Python Exception <class 'ModuleNotFoundError'>: No module named 'gdb':` Ubuntu's `avr-gdb` package ships without GDB's Python support files. Purely cosmetic.

- `warning: pc 0x800024 in address map, but not in symtab` when printing a `gpio_t`. Those pointers point into the AVR's I/O register space, which has no symbols. Also cosmetic; the values printed are correct.

A.6 Full Path: The Simulation Harness

Save this as `simulator.c` somewhere outside the repo (a scratch directory is fine; it is host code, not firmware, and does not belong in `lectures/`).

```
/**
 * @file Host-side simavr harness for the Embedded C course demos.
 *
 *       Connects a simulated UART and a simulated button to an ATmega328P ↩
    ↪ firmware image, reports
 *       LED state changes, and persists EEPROM contents across runs.
 *
 * Build:
 *   gcc -O2 -o simulator simulator.c -I/usr/include/simavr \
 *       -lsimavr -lsimavrparts -lelf -lpthread -lm
 *
 * Usage:
 *   ./simulator [--led B1] [--button B5] [--eeprom eeprom.bin] <firmware.elf>
 */
#include <pthread.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "avr_eeprom.h"
#include "avr_ioport.h"
#include "parts/button.h"
#include "parts/uart_pty.h"
#include "sim_avr.h"
#include "sim_elf.h"

/** Size of the ATmega328P EEPROM in bytes. */
#define EEPROM_SIZE 1024U

/** Button hold time in microseconds, longer than the 300 ms firmware debounce. */
#define BUTTON_HOLD_US 500000U
```

```c
/** Simulated MCU instance. */
static avr_t* avr = NULL;

/** Simulated UART, bridged to a host pseudo-terminal. */
static uart_pty_t uart = {0};

/** Simulated button. */
static button_t button = {0};

/** File used to persist EEPROM contents between runs. */
static const char* eeprom_path = "eeprom.bin";

/** Port and pin the LED is connected to, defaults to PB1 (Arduino D9). */
static char led_port = 'B';
static int led_pin   = 1;

/** Port and pin the button is connected to, defaults to PB5 (Arduino D13). */
static char button_port = 'B';
static int button_pin   = 5;

/** Set from the signal handler to break out of the run loop. */
static volatile sig_atomic_t stop = 0;

/**
 * @brief Write the simulated EEPROM contents to disk.
 */
static void eeprom_save(void)
{
    uint8_t buf[EEPROM_SIZE];
    avr_eeprom_desc_t desc = {.ee = buf, .offset = 0U, .size = EEPROM_SIZE};
    avr_ioctl(avr, AVR_IOCTL_EEPROM_GET, &desc);

    FILE* file = fopen(eeprom_path, "wb");
    if (NULL == file) { return; }
    fwrite(buf, 1U, EEPROM_SIZE, file);
    fclose(file);
    printf("[sim] EEPROM saved to %s\n", eeprom_path);
}

/**
 * @brief Restore the simulated EEPROM contents from disk, if a saved image exists
    ↪ .
 */
static void eeprom_restore(void)
```

```
{
    uint8_t buf[EEPROM_SIZE];
    FILE* file = fopen(eeprom_path, "rb");
    if (NULL == file) { return; }

    if (EEPROM_SIZE == fread(buf, 1U, EEPROM_SIZE, file))
    {
        avr_eeprom_desc_t desc = {.ee = buf, .offset = 0U, .size = EEPROM_SIZE};
        avr_ioctl(avr, AVR_IOCTL_EEPROM_SET, &desc);
        printf("[sim] EEPROM restored from %s\n", eeprom_path);
    }
    fclose(file);
}

/**
 * @brief Request termination on Ctrl+C.
 *
 *        Only sets a flag, so that the EEPROM is saved from the main loop rather ↩
    ↪ than from this
 *        handler, where file I/O would not be async-signal-safe.
 *
 * @param[in] signum The received signal, unused.
 */
static void on_terminate(int signum)
{
    (void)(signum);
    stop = 1;
}

/**
 * @brief Report a change of the LED pin.
 *
 * @param[in] irq The IRQ that fired, unused.
 * @param[in] value The new pin state.
 * @param[in] param User data, unused.
 */
static void on_led_changed(struct avr_irq_t* irq, uint32_t value, void* param)
{
    (void)(irq);
    (void)(param);
    printf("[LED] %s\n", value ? "ON" : "OFF");
    fflush(stdout);
}
```

```
/**
 * @brief Press the simulated button whenever the user hits Enter.
 *
 * @param[in] arg Thread argument, unused.
 *
 * @return NULL on termination of stdin.
 */
static void* button_thread(void* arg)
{
    (void)(arg);
    char line[64] = {0};

    while (NULL != fgets(line, sizeof(line), stdin))
    {
        printf("[BTN] press\n");
        fflush(stdout);
        button_press(&button, BUTTON_HOLD_US);
    }
    return NULL;
}

/**
 * @brief Application entry point.
 *
 * @param[in] argc Number of command line arguments.
 * @param[in] argv Command line arguments.
 *
 * @return 0 on success, 1 on failure.
 */
int main(int argc, char** argv)
{
    const char* firmware = NULL;

    // Parse command line arguments.
    for (int i = 1; i < argc; ++i)
    {
        if ((0 == strcmp(argv[i], "--led")) && (i + 1 < argc))
        {
            led_port = argv[++i][0];
            led_pin  = atoi(argv[i] + 1);
        }
        else if ((0 == strcmp(argv[i], "--button")) && (i + 1 < argc))
        {
            button_port = argv[++i][0];
```

```
        button_pin  = atoi(argv[i] + 1);
    }
    else if ((0 == strcmp(argv[i], "--eeprom")) && (i + 1 < argc)) { ↩
↪ eeprom_path = argv[++i]; }
    else { firmware = argv[i]; }
}
if (NULL == firmware)
{
    fprintf(stderr, "usage: %s [--led B1] [--button B5] [--eeprom f] <firmware↩
↪ .elf>\n",
            argv[0]);
    return 1;
}

// Read the firmware image, defaulting to an ATmega328P at 16 MHz.
elf_firmware_t f = {{0}};
if (0 > elf_read_firmware(firmware, &f))
{
    fprintf(stderr, "error: cannot read %s\n", firmware);
    return 1;
}
if ('\0' == f.mmcu[0]) { strcpy(f.mmcu, "atmega328p"); }
if (0U == f.frequency) { f.frequency = 16000000U; }

// Create and initialize the simulated MCU.
avr = avr_make_mcu_by_name(f.mmcu);
if (NULL == avr)
{
    fprintf(stderr, "error: unknown MCU %s\n", f.mmcu);
    return 1;
}
avr_init(avr);
avr_load_firmware(avr, &f);

// Restore EEPROM contents, and save them again on exit.
eeprom_restore();
signal(SIGINT, on_terminate);
signal(SIGTERM, on_terminate);

// Bridge UART0 to a host pseudo-terminal.
uart_pty_init(avr, &uart);
uart_pty_connect(&uart, '0');

// Connect the simulated button to the input pin.
```

```
    button_init(avr, &button, "button");
    avr_connect_irq(button.irq + IRQ_BUTTON_OUT,
                    avr_io_getirq(avr, AVR_IOCTL_IOPORT_GETIRQ(button_port), ↩
    ↪ button_pin));

    // Report every change of the LED pin.
    avr_irq_register_notify(avr_io_getirq(avr, AVR_IOCTL_IOPORT_GETIRQ(led_port), ↩
    ↪ led_pin),
                            on_led_changed, NULL);

    // Press the button whenever the user hits Enter.
    pthread_t thread;
    pthread_create(&thread, NULL, button_thread, NULL);

    printf("[sim] running %s (%s @ %u Hz), press Enter to push the button\n", ↩
    ↪ firmware, f.mmcu,
           f.frequency);
    fflush(stdout);

    // Run until the firmware stops or crashes, or the user interrupts us.
    while (0 == stop)
    {
        const int state = avr_run(avr);
        if ((cpu_Done == state) || (cpu_Crashed == state))
        {
            printf("[sim] CPU stopped, state %d\n", state);
            break;
        }
    }
    eeprom_save();
    return 0;
}
```

Build it once, then remember where it landed:

```
gcc -O2 -o simulator simulator.c -I/usr/include/simavr \
    -lsimavr -lsimavrparts -lelf -lpthread -lm
export SIM="$PWD/simulator"
```

`$SIM` is used for the rest of this appendix. Because it holds an absolute path, it works from any directory, which matters once you start `cd`-ing into demo directories. Add that `export` line to your `~/.bashrc` so new terminals inherit it.

Run a demo from its own directory, so that `main.elf` resolves and the EEPROM image is kept alongside the firmware it belongs to. That image is written as `eeprom.bin` in the demo directory; `.gitignore` already excludes it, so it will not show up as an untracked file:

```
# From the repository root.
cd lectures/L09/gpio_driver_demo
$SIM main.elf
```

You'll see something like:

```
[sim] EEPROM restored from eeprom.bin
uart_pty_init bridge on port *** /dev/pts/14 ***
uart_pty_connect: /tmp/simavr-uart0 now points to /dev/pts/14
[sim] running .../main.elf (atmega328p @ 16000000 Hz), press Enter to push the
 ↪  button
[LED] OFF
```

Open a **second terminal** to read the UART:

```
cat /tmp/simavr-uart0
# or, for a proper terminal:
picocom /tmp/simavr-uart0
```

Now press **Enter** in the simulator terminal. The button is pushed, and you should see:

```
[BTN] press
[LED] ON
```

and on the UART terminal:

```
LED1 enabled!
```

Press `Ctrl+C` to stop. The EEPROM image is written to `eeprom.bin`; start the simulator again and the demo picks up where it left off — which is exactly the point of the EEPROM lesson of Chapters 7, 8 and 9.

A.7 Demo Reference

Every demo uses the same wiring: **LED on `PB1` (Arduino D9)** and **button on `PB5` (Arduino D13)**; with one exception, noted below.

`cd` into the demo directory under `lectures/`, then run the command below:

Demo	Command	UART?	Notes
`L06/serial_driver_demo`	`$SIM main.elf`	Yes	Press Enter, watch the state report
`L07/eeprom_driver_demo`	`$SIM main.elf`	Yes	State survives restart
`L08/watchdog_driver_demo`	`$SIM main.elf`	Yes	See the watchdog experiment below
`L09/gpio_driver_demo`	`$SIM main.elf`	Yes	Full struct-as-driver demo
`L10/encapsulation_demo`	`$SIM main.elf`	No	No serial driver in this one
`L10/function_pointer_demo`	`$SIM main.elf`	Yes	
`L10/interface_demo`	`$SIM --led B0 main.elf`	No	LED is on `PB0` (D8); its button is a software stub, so it toggles on its own with no input

A.7.1 Watching the Watchdog Bite

This is the one experiment in the book that is genuinely easier to run in simulation than on the board: there is no reflashing between the two versions and no serial monitor to wire up, so the before-and-after is seconds apart instead of minutes. It is referenced from §8.1.5.

Take a scratch copy of the demo outside the repo, comment out the `watchdog_reset()` call in the main loop, and rebuild:

```
cp -r lectures/L08/watchdog_driver_demo /tmp/wdt && cd /tmp/wdt
rm -f eeprom.bin   # cp brings along any saved EEPROM image; start from a blank ↩
    ↪ chip
sed -i 's|watchdog_reset();|// watchdog_reset();|' main.c
make clean && make
$SIM main.elf
```

Use `make clean && make`, not bare `make`. `cp -r` copies the prebuilt `main.elf` along with the sources, and the copied timestamps can leave `make` thinking the binary is current. You then run the old firmware and see no difference, which looks exactly like "the watchdog does nothing in the simulator". Check `md5sum main.elf` before and after if you are unsure.

Watch the **UART**. In a second terminal, `cat /tmp/simavr-uart0`:

```
LED1 disabled!LED1 disabled!LED1 disabled!LED1 disabled!LED1 disabled!...
```

That is `setup()`'s state report running again on every boot: the starved watchdog is resetting the MCU over and over. Restore the `watchdog_reset()` call, rebuild, and you get exactly one report and then silence.

The reports run together because each one is cut off before its line ending, and that is a simulator quirk worth knowing about. Measured in simulated time, the first report comes at boot and the first reset 1024 ms later, as on the board; after that, the MCU resets every **16 ms**. A watchdog reset leaves the watchdog running at its shortest timeout (§8.1.3), and

`watchdog_init()` changes that to 1024 ms first thing in `setup()`. The chip applies the change; simavr 1.6 only picks up a new timeout when the watchdog is switched on, not when it is changed while running. So every boot runs at 16 ms, which is just long enough to print `LED1 disabled!` but not its `\r\n`. On the board, the loop repeats every 1024 ms or so.

Commenting out `watchdog_init()` as well is a useful third data point, and gives 1 report again: the reboot loop really is the watchdog firing, not the demo restarting for some other reason.

(Which message repeats depends on the stored state, so it reads `LED1 enabled!` if the EEPROM image says the LED was on. The repetition is the signal, not the wording.)

The LED is a weaker indicator than the UART here. On each reboot the firmware restores LED1 from EEPROM, so you only see it flicker if `eeprom.bin` holds the "on" state from a previous session.

A.8 Gotchas

A short button press does nothing. This one comes from the simulated button, and it is worth understanding. simavr's button part pulls the pin *low* while pressed and lets it go *high* when released, the opposite of the book's circuit, where an external pull-down holds the pin low until the button connects it to 5 V. The pin change ISR checks `gpio_read(&btn1)` and only toggles the LED when the pin is high: on the board that is the press, but in the simulator it is the **release**. And the press edge also disables pin change interrupts for 300 ms of debounce. So if you release the button inside that 300 ms window, the release edge is never seen and nothing happens. The harness holds the button for 500 ms for exactly this reason.

Mashing Enter looks like a hang, but isn't. Press quickly and you will see `[BTN] press` lines with no `[LED]` change, as though the simulator has wedged. Two things compound. The 300 ms debounce above hides any press inside that window. And each `button_press()` schedules its release 500 ms later, so pressing again beforehand simply re-arms the hold: a burst of presses becomes one long press, not several. A measured burst of 20 presses 30 ms apart produced exactly one LED toggle. Stop pressing for a second and it responds normally again; nothing needs restarting.

Simulated time is not wall-clock time. simavr runs as fast as your host allows and is not throttled to real time. A 300 ms debounce is 300 ms of *simulated* time. Delays will feel wrong.

EEPROM starts blank unless you have an image. Delete `eeprom.bin` (or pass a different `--eeprom` path) to get a factory-fresh chip.

Use the `.elf`, not the `.hex`. The `.hex` has no symbols, so GDB is nearly useless with it and simavr can't report symbolic addresses.

A.9 Limits

The harness above covers digital I/O, interrupts, timers, UART, EEPROM and the watchdog — which is everything the committed demos need. It does not cover:

- **ADC input** (Chapter 4's potentiometer). simavr simulates the ADC, but feeding it a value requires raising the ADC IRQ from the harness. Adding that is a good exercise if you need it.
- **Timing realism.** Nothing here tells you whether your code is fast enough.
- **Anything physical.** Contact bounce, brown-outs, noise, and wiring mistakes are precisely the problems this book exists to teach you about, and a simulator has none of them.

Use simavr to keep moving. Get a board when you can.

APPENDIX B

Answers to the Exercises

This appendix answers the exercises that have one right answer: every CALCULATION and REFLECTION exercise, and the questions that some CODE exercises ask about the design they make you write. Calculations are shown with their working, so that an answer worked on paper can be checked step by step, not just at the end. Where an exercise asks you to explain something in your own words, the answer is one way of saying it, and yours can differ in wording as long as it says the same things.

The Code exercises have no printed solution. Each describes what a program must do on the board, and the board, or the simulator of Appendix A, is how you check it.

Chapter 1 C Fundamentals

Every exercise in this chapter is a Code exercise; what follows answers only the questions three of them ask.

Exercise 1.3 An enum of states

It depends on the compiler and its flags. GCC's `-Wswitch` is part of `-Wall`, so with `-Wall` a `switch` on a `state_t` that leaves out one of the named constants gives a warning along the lines of "enumeration value `STATE_FAST` not handled in switch"; a plain `gcc main.c` without it stays silent. Clang enables `-Wswitch` by default. The warning only appears while the `switch` has no `default` label: a `default` counts as handling every remaining value, so it silences the check (`-Wswitch-enum` warns even then). This is the help from the compiler that §1.1.3 gives as a reason to prefer an `enum` over a block of `#defined` constants, whose `switch` no compiler can check.

Exercise 1.5 A private helper

Its forward declaration goes in `temp_reading.c`, once, right after the `#includes`, not in `temp_reading.h`: the helper is `static`, an implementation detail rather than part of the module's public API. Its definition goes at the bottom of the file, after the public functions, which are defined in the order the header declares them. A reader then meets the public API first and reaches the private detail only by scrolling on. Following §1.1.2, the declaration leaves a by-value parameter unqualified and the definition makes it `const`.

Exercise 1.7 Reset and delete a reading

Because C passes every argument by value. A `temp_reading_t*` parameter is a copy of the caller's pointer: the function can change the struct it points to, which is all `temp_reading_reset()` needs, but setting the parameter itself to `NULL` changes only the local copy, and the caller's pointer still holds the old address. To change the caller's pointer, the function needs that pointer's address, a `temp_reading_t**`, and assigns `NULL` through it, the way `set_to_null()` does in §1.1.7.

Chapter 2 Hardware-Near Programming in C

Exercise 2.1 Set, clear and read a bit

The exercise asks for the expressions, and those are yours to write; what follows is what they must produce, for checking the output of the tip's program. The register holds `0b10110010` = `0xB2` = 178, with bits 7, 5, 4 and 1 set.

a) Setting bit 0 changes only the lowest bit: `0b10110011` = `0xB3` = 179.

b) Clearing bit 5 takes away its 32: `0b10010010` = `0x92` = 146.

c) Bit 4 is set, so the expression is true. The masked value itself is `0b00010000` = 16, which `printf("%u\n", ...)` prints as 16; compared against 0, or turned into a `bool`, it is 1.

Exercise 2.3 Which register?

a) `DDRD`, bit 5 (`DDD5`) set to 1: D5 is `PORTD5`, and the data direction register is what decides, bit by bit, whether a pin is an output (1) or an input (0).

b) `PIND`, bit 2 (`PIND2`): D2 is `PORTD2`, and `PINx` is the register that holds the level actually present on the pins; `PORTD` would only give back what was last written to it.

c) `PORTD`, bit 7 (`PORTD7`) set to 1, with `DDD7` left at 0: on a pin configured as an input, `PORTx` no longer drives the pin but switches its internal pull-up resistor on (1) or off (0).

Chapter 3 Interrupts

Exercise 3.1 Polling versus interrupts

Polling makes the main loop ask every input, on every pass, whether anything has happened. Each input added makes every pass longer, the CPU spends its time checking inputs that have not changed, and the reaction time to any one input is the time of a whole pass: while the loop is busy elsewhere, in a blocking delay say, a press is noticed late, and a pulse shorter than a pass can be missed altogether. An interrupt moves the watching into hardware: the pin logic detects the event and runs its ISR within a few clock cycles whatever `main()` is doing, so an input that is idle costs nothing, and the ISR only has to flag the event for `main()` to handle.

Exercise 3.2 `EICRA` and `EIMSK`

`EICRA` configures *what* triggers each external interrupt: its sense bits `ISCn1:ISCn0` choose low level, any change, falling edge or rising edge, for INT0 and INT1 separately. `EIMSK` configures *whether* each one is enabled at all: its `INT0` and `INT1` bits switch the two interrupts on. Neither register does the other's job, and neither replaces the global `I` bit in `SREG` that `sei()` sets.

INT0 can be enabled without touching `EICRA`, but it will not do what you want. `EICRA` is 0 after a reset, so `ISC01:ISC00` is `00`: a low-level interrupt, requested for as long as the pin is held low. With the book's button wiring, D2 idles low (the external pull-down) and goes high only while the button is held, so INT0 fires continuously while the button is *not* pressed: it is requested again the moment the ISR returns, and the CPU executes just one instruction of `main()` between two runs of it. In the toggle program of §3.1.3 the LED toggles hundreds of thousands of times a second and looks half-lit, and a press stops it in whichever state it happens to be in.

Exercise 3.4 The wrong edge

Nothing is broken: the edge is the wrong one for this circuit. The button is wired with an external pull-down (§2.1.4), so D2 reads low while the button is up and is pulled high while it is held down. A press is therefore a rising edge, low to high, and a release a falling edge, high to low. `ISC01:ISC00 = 10` selects the falling edge, so INT0 fires on the release, exactly as the colleague sees. Rising edge (`11`) is right for this wiring; a falling edge would mean the press only for a button wired the other way round, to ground against a pull-up. (Contact bounce can add stray edges in both directions, but the clean falling edge is the release.)

Exercise 3.5 An interrupt on D4

D4 is `PORTD4`: port D, bit 4. Port D's formula is `PCINTx = PORTDx + 16`:

$$x = 4 + 16 = 20,$$

so D4 is `PCINT20`. It belongs to port D's group, `PCIE2/PCMSK2`, whose interrupts all share `PCINT2_vect`. The three registers and bits to touch:

- `PCICR`: set `PCIE2`, to enable pin change interrupts for port D's group.
- `PCMSK2`: set bit 4, `PCINT20`, to select D4 within the group; `(1U << PCINT20)` and `(1U << 4)` are the same bit.
- `SREG`: set the global `I` bit, with `sei()`.

Exercise 3.6 Reading `PINB` once

The pins are live: a button can change while the ISR is running, between two reads of `PINB`. Say BTN2 is pressed just after the XOR has read `PINB`. The XOR saw BTN2 still released, so its bit is 0 in the result, and the AND drops it whatever the second read shows; BTN2's LED doesn't toggle. The third read, for `prev`, then sees BTN2 pressed and stores it. The press does set the pin change flag again, so the ISR runs a second time, but by then `prev` already says pressed, the XOR finds no change, and the press is lost for good. A press between the AND and the update of `prev` is lost the same way.

With a single read, `prev` always holds exactly the value the tests worked on. A press that lands after the read isn't in `now`, so it isn't in `prev` either; the flag it sets runs the ISR again, and that run sees it as a change and toggles the LED. Every change is handled once: either by this run, or by the next.

Chapter 4 Analog-to-Digital Conversion

Exercise 4.1 From reading to voltage

Invert the chapter's mapping, `result = 1023 * (V_in / V_ref)`:

$$V_{\text{in}} = \frac{\text{reading} \times V_{\text{ref}}}{1023} = \frac{768 \times 5\,\text{V}}{1023} = \frac{3840}{1023}\,\text{V} \approx 3.754\,\text{V}.$$

Check: $1023 \times 3.754/5 = 768.1$, which rounds back to 768. The datasheet's form, with 1024, gives $768 \times 5/1024 = 3.750\,\text{V}$, less than one step (about 5 mV) away.

Exercise 4.2 The ADC's own prescaler

The ADC is a successive-approximation converter. It samples the input onto a small capacitor, then decides the result one bit per ADC clock cycle, comparing the held voltage against a trial voltage generated inside the converter, and its analog circuitry needs time to settle at each step. The datasheet therefore asks for an ADC clock between 50 and 200 kHz for full 10-bit resolution. The CPU runs at 16 MHz, 80 times the upper limit, and slowing the whole CPU down to suit one peripheral is not an option; so the ADC has a prescaler of its own, the `ADPS2:0` bits of `ADCSRA`, which divides the system clock for the ADC alone. At 16 MHz, ÷128 (125 kHz) is the only setting that lands in range; ÷64 already gives 250 kHz.

At the full 16 MHz a conversion would finish sooner, but accuracy would collapse: the sampling capacitor and the comparator would get far too little time to settle, so the lower bits would turn into noise, readings would jump around and drift from the true voltage, and at 80 times the limit little of the 10 bits would be left. (The prescaler cannot in fact divide by less than 2, so 8 MHz is the fastest the ADC can be clocked, still 40 times too fast.)

Exercise 4.3 From A0 to A1

`MUX0`, bit 0 of `ADMUX`. The channel number is written into `MUX3:0`, the low four bits: `A0` is channel 0 (`0000`) and `A1` is channel 1 (`0001`), so moving from `A0` to `A1` sets `MUX0` and leaves `REFS1:REFS0`, the `AVCC` reference, as they are. With `adc_read()` that is simply a call with `channel` set to 1.

Exercise 4.4 Waiting for `ADIF`

Setting `ADSC` only *starts* a conversion. The conversion takes 13 ADC clock cycles, or 25 for the first one after the ADC is enabled; at 125 kHz that is 104 µs or 200 µs, 1,664 or 3,200 CPU cycles, while the CPU reaches the next statement after one or two. Until the conversion ends, `ADC` still holds the previous conversion's result, or 0 if there has been none, so reading it straight away returns an old value, not the one just asked for. The hardware sets `ADIF` at the moment the conversion completes and its result is in `ADCL/ADCH`, so waiting for `ADIF` is waiting for the result that belongs to this conversion. (`ADSC` reads as 1 while a conversion is in progress, so waiting for it to clear works too, and the ADC interrupt does the same without polling.)

Exercise 4.6 Removing the wait

It would not crash; it would quietly return 0 from every read. The reason is the statement that follows the wait: `ADCSRA = (1U << ADIF);` writes the whole register, not just the flag, and every bit written as 0 is cleared, `ADEN` included. Writing `ADEN` to zero switches the ADC off, and the datasheet is explicit that switching it off while a conversion is in progress terminates that conversion. Without the wait, this statement runs a few CPU cycles after the conversion was started, about 3,200 cycles before it could have finished (a conversion started together with `ADEN` takes 25 ADC clock cycles). So every conversion is aborted: none ever completes, sets `ADIF`, or writes `ADCL/ADCH`. `return ADC;` then reads a data register that no conversion has ever written, and returns its reset value, 0, on every call, whatever the potentiometer says.

Nothing illegal happens and the program runs exactly as before, only on wrong data. In the program of §4.1.4, every duty cycle is 0, `on_ms` is 0 and `off_ms` is 10, so the LED stays dark whenever reading is enabled, which looks more like a wiring fault than a software one. (Had the flag been cleared without switching the ADC off, the wait's absence would show up differently: each read would return the result of the conversion before it, one reading behind, which is stale data that is even harder to spot.)

Chapter 5 Timers

Exercise 5.1 Tick at ÷256

The prescaler divides the CPU clock, and the tick period is the inverse of the tick frequency:

$$f_{\text{tick}} = \frac{16\,000\,000\,\text{Hz}}{256} = 62\,500\,\text{Hz}, \qquad T_{\text{tick}} = \frac{1}{62\,500\,\text{Hz}} = 16\,\mu\text{s} = 0.016\,\text{ms},$$

the 16 µs of the ÷256 row in the table of §5.1.2.

Exercise 5.2 Overflows for 250 ms

An 8-bit timer overflows every 256 ticks:

$$T_{\text{ovf}} = 256 \times 16\,\mu\text{s} = 4.096\,\text{ms}, \qquad N = \frac{250\,\text{ms}}{4.096\,\text{ms}} = 61.04.$$

Rounded to the nearest whole number the way the chapter's `OVF_MAX` does it, $\lfloor 61.04 + 0.5 \rfloor = 61$ overflows. Check: 61×4.096 ms = 249.856 ms, 0.144 ms (0.06 %) short of 250 ms. (Counted in ticks it is the same: 250/0.016 = 15 625 ticks, and 15 625/256 = 61.04.)

Exercise 5.3 When to reach for CTC

Normal mode with overflow counting can only make whole multiples of the overflow period, 256 ticks for an 8-bit timer (16.384 ms at ÷1024), and it takes an interrupt for every overflow even though only the last one does anything. Any other period is rounded to the nearest multiple, an error of up to half an overflow period. CTC chooses the period in single ticks: the counter is cleared when it reaches `OCRnA`, so the period is `OCRnA` + 1 ticks, with one interrupt per period and no counting in software. Reach for CTC when the period has to be accurate, is not a multiple of the overflow period, or is short enough that the interrupt overhead matters; overflow counting is fine when roughly right is right enough.

Yes: 100 ms is one. At ÷1024, Timer 0 overflows every 16.384 ms, and 100/16.384 = 6.10, so the best Normal mode can do is 6 overflows, 98.304 ms, 1.7 % short. Timer 1 in CTC with `OCR1A = 1562` gives 1563 × 64 µs = 100.032 ms, 0.03 % off. A period that is a whole number of ticks is hit exactly: the 2 s of Exercise 5.6 is 31,250 ticks.

Exercise 5.4 Blinking at 2 Hz

Only `TIMEOUT_MS` changes, from `500U` to `250U`: 2 Hz is one on/off cycle every 500 ms, so the LED must toggle every 250 ms. The prescaler, `TICK_PERIOD_MS`, `TICK_MAX`, `OVF_PERIOD_MS` and the ISR all stay as they are, and `OVF_MAX` follows from the macro:

$$\texttt{OVF_MAX} = \left\lfloor \frac{250}{0.064 \times 256} + 0.5 \right\rfloor = \lfloor 15.26 + 0.5 \rfloor = 15.$$

Check: 15 × 16.384 ms = 245.76 ms per toggle, a 491.52 ms period, so the LED blinks at 2.03 Hz. (16 overflows would give 262.144 ms, further from 250.)

Exercise 5.5 When `volatile` matters

Inside the ISR. `ovf_count` is a `static` local of the ISR, so no other code can even name it, and on the AVR the ISR runs to completion with interrupts disabled, so nothing can change it between two of the ISR's own accesses. The compiler sees every access in one function body. It still has to load the value when the ISR starts and store it before the ISR returns, because a `static` outlives the call, and between those two points whatever it keeps in a register is correct. `volatile` only forbids optimizations that are already safe here, so removing it changes nothing about what the program does.

Waited on from `main()`. Now the loop `while (10U > ovf_count) {}` reads the variable, and its body neither writes it nor calls anything. Without `volatile`, the compiler may assume nothing else writes it either: it has no idea that an ISR exists. It is allowed to read `ovf_count` once, before the loop, and test that register copy forever; if the first read is below 10, `main()` hangs in an endless loop while the ISR keeps incrementing the variable in memory. Whether that happens depends on the build, not on the code: at `-O0` every access is compiled as a load from memory and the loop works, while with optimization, as at the `-Os` the book builds with, avr-gcc hoists the load out of the loop and it hangs. A different compiler version, a change in inlining, or a function call added to the loop body can each make the bug appear or disappear again. `volatile` makes every test of `ovf_count` in the loop a fresh read from memory.

Exercise 5.6 A 2-second period in CTC

The number of ticks in 2 s, less one, because the counter takes `OCR1A` + 1 ticks to reach the compare value:

$$\text{ticks} = \frac{2000\,\text{ms}}{0.064\,\text{ms}} = 31\,250, \qquad \texttt{OCR1A} = 31\,250 - 1 = 31\,249.$$

It fits: Timer 1 is 16-bit, so `OCR1A` holds up to 65,535, while an 8-bit timer's 255 could not. Check: $(31\,249 + 1) \times 64\,\mu\text{s} = 2.000\,\text{s}$ exactly, since 16 MHz divided by 1024 is exactly 15,625 ticks per second. The chapter's macros agree, and show why they round: in `float` arithmetic `2000U / 0.064F - 1U` comes to 31248.998, which a plain cast would truncate to 31248; adding 0.5 first gives 31249.

Chapter 6 Serial Communication (UART)

Exercise 6.1 `UBRR0` for 19200 baud

With the formula of §6.1.2:

$$\texttt{UBRR0} = \frac{f_{\text{CPU}}}{16 \cdot \mathit{baud}} - 1 = \frac{16\,000\,000}{16 \cdot 19\,200} - 1 = 52.08 - 1 = 51.08,$$

which rounds to `UBRR0 = 51`. Check the baud rate that value actually gives:

$$\mathit{baud} = \frac{f_{\text{CPU}}}{16\,(\texttt{UBRR0} + 1)} = \frac{16\,000\,000}{16 \cdot 52} = 19\,231 \text{ baud},$$

an error of $(19\,231 - 19\,200)/19\,200 = +0.16\,\%$, well inside what a UART receiver tolerates (the datasheet's own baud-rate table lists 51 and 0.2 % for this case).

Exercise 6.2 What `UDRE0` means

`UDRE0`, USART Data Register Empty, is a flag in `UCSR0A` that is set while the transmit buffer, `UDR0`, can take a new byte. Writing `UDR0` clears it; it sets again as soon as the hardware has moved that byte on into the transmit shift register, which then sends it bit by bit while `UDR0` is free for the next one. It does not mean that the byte has finished sending (that is `TXC0`); it means there is room for another.

`serial_write_byte()` waits for it because the CPU is far faster than the line. One 10-bit frame at 9600 baud takes about $10/9600 \approx 1.04$ ms, some 16 700 CPU cycles at 16 MHz, and a byte written to `UDR0` while `UDRE0` is clear is simply ignored by the transmitter. Without the wait, `serial_print()` would run through the whole string in a few microseconds and every character after the first two (one in the shift register, one waiting in `UDR0`) would be lost, so the terminal would show two characters instead of the message.

Chapter 7 EEPROM

Exercise 7.1 Waiting before, not after

Because a write goes on after the function has returned, and a read does not. Setting `EEPE` only starts the write: the EEPROM then spends a few milliseconds (about 3.3 ms) programming the cell in the background while the CPU carries on, and `EEPE` stays set until it is done. While it is set, no new write can start, no read can be made, and `EEAR` cannot be changed, so the next access, whichever it is, has to wait for it first. That is why `eeprom_read_byte()` also waits on `EEPE` before it starts: an earlier write may still be running.

A read has no such tail. Setting `EERE` makes the hardware read the cell straight away: the CPU is halted for four cycles and the byte is in `EEDR` by the time the next instruction runs. When `eeprom_read_byte()` returns `EEDR`, the read is already over and there is nothing left to wait for.

Exercise 7.2 The four-cycle window

The window is two consecutive instructions, and an interrupt can be taken between any two instructions. Taking one is never quick: the CPU needs at least four cycles just to respond, then jumps through the vector table, runs the ISR's prologue, body and epilogue, and returns with `reti`. So *any* interrupt that lands between setting `EEMPE` and setting `EEPE`, not only a slow one, pushes the second write out of the four-cycle window. Disabling interrupts is the only way to guarantee that nothing runs between the two.

The failure is a lost write that nobody notices. The hardware clears `EEMPE` again four cycles after it was set, so when the code finally sets `EEPE` it has no effect: the byte is never written and the cell keeps its old value. Nothing reports it: `eeprom_write()` still returns the full count, and the program runs on as if the write had happened. The damage shows only later, typically at the next reset, when `setup()` restores a stale state (in §7.1.4, the LED comes back in the state it had before the last press). And it happens only when a button press or a timer overflow happens to land in that two-instruction window, so it is rare, intermittent and very hard to reproduce.

Exercise 7.3 Fewer EEPROM writes

Not for a device that is used a lot. Every press rewrites the same cell, address 1000, and a cell is good for roughly 100,000 write cycles. At 50 presses a day that is about 2000 days, some five and a half years; a button pressed once a minute, around the clock, wears the cell out in 100 000 min ≈ 69 days. After that the stored state can no longer be trusted.

Writing only every fifth press cuts the wear to a fifth, so the same use lasts five times as long. The price is durability: up to four toggles since the last write are only in RAM, so a reset or a power loss at the wrong moment brings the LED back in an older state. That trade-off, how much recent state you can afford to lose against how long the cell lasts, is the design decision any EEPROM-backed setting makes. (Spreading the writes over several addresses, so that no single cell takes all of them, is the other common remedy.)

Chapter 8 The Watchdog Timer

Exercise 8.1 Why the watchdog is locked

Against the program that has gone wrong switching off the thing that is there to catch it going wrong. A watchdog earns its keep exactly when the code is misbehaving: stuck in a loop, running off through a corrupted pointer or return address, writing to registers it never meant to touch. If a single stray write to `WDTCSR` could clear `WDE` or stretch the timeout, the same fault the watchdog should recover from could disarm it first, and the chip would hang anyway.

With the interlock, one accidental write is not enough: turning the watchdog off or changing its timeout takes `WDCE` and `WDE` written together, and then the new configuration written with `WDCE` clear within four cycles. Runaway code is very unlikely to produce that

exact sequence by chance. (The datasheet is precise about which direction is protected: setting `WDE` is allowed at any time, because a stray write that arms the watchdog does no harm; it is clearing `WDE` and changing the prescaler that need the sequence.) EEPROM's `EEMPE` guards stored data against an accidental write; the watchdog's `WDCE` guards the safety net itself. `WDRF` (§8.1.3) is the same idea taken one step further: after a watchdog reset it holds `WDE` set until the flag is cleared.

Chapter 9 Structs as Drivers

Exercise 9.1 A struct or a macro

A `#defined LED_ON` bakes one specific pin, and its port, into the macro: it can only ever mean that one LED, so which LED it acts on is decided when the macro is written, not when it is used. A `gpio_t` turns the pin, and the registers that go with it, into data: one set of functions, `gpio_write()`, `gpio_toggle()` and the rest, works on whichever instance it is handed, so which LED becomes an argument.

To support a second LED:

- **Macros:** a whole new family, `LED2_ON`, `LED2_OFF`, a toggle, and its own setup lines, copied from the first and renamed by hand, with every copy a chance to leave a `1` where a `2` belongs.
- **`gpio_t`:** one more instance, `gpio_t led2 = {&DDRB, &PORTB, &PINB, 2U};`, and one more `gpio_init()` call (or one more line in §9.1.4's `leds[]` array). No new functions.

Exercise 9.2 By pointer or by value

`gpio_read()` by value would still return the right answer: the copy's `pinx` holds the address of the same `PINx` register, so reading through it reads the same pin. What changes is the cost and the safety net. Every call copies the whole struct instead of one 2-byte pointer: 7 bytes for §9.1.2's `gpio_t` (three 2-byte pointers and a byte), 10 for the demo's in §9.1.5, on a chip with 2 KB of RAM. And the `NULL` check has to go, because a struct passed by value cannot be `NULL`.

`gpio_init()` by value: §9.1.2's version would still configure the pin correctly, but only by accident. It never writes to the struct's own fields, only through its pointers to the hardware registers, and the copy's pointers point at the same registers. The moment an init function stores something in the instance, as Exercise 9.4's does when it computes `ddrx`, `portx`, `pinx` and the bit position, the stores land in the copy, which vanishes when the function returns, and the caller's instance is left uninitialized with no diagnostic.

Exercise 9.3 Freeing a GPIO

Pointers kept. `gpio_free()` has cleared the pin's `DDRx` bit, so the direction check in `gpio_write()` and `gpio_toggle()` fails and both return without touching anything: a harmless no-op, as long as the pin stays free. The danger comes later. Once something else claims the same pin and makes it an output again, the stale instance passes the direction check and silently drives a pin it no longer owns.

Pointers set to `NULL`. §9.1.2's functions check `self` but not its fields, so `*self->ddrx` dereferences a null pointer, which is undefined behaviour. The ATmega328P has no memory protection to catch it: address 0 of its data space is the CPU register `r0`, so instead of a crash you get a register silently read, and possibly written.

Preferable: set them to `NULL` *and* add the matching check (`NULL == self->ddrx`) to every function, so that a freed instance is recognisably dead and can never reach the pin again, whoever owns it by then. Nulling the pointers without adding the checks is worse than leaving them alone.

Exercise 9.4 Initializing from a pin number

No. A `const` member can only be given its value by an initializer where the object is defined; it can never be assigned afterwards. In §9.1.2 that is exactly how `pin` got its value, from the struct literal, so `const` cost nothing. Now `gpio_init()` computes the bit position and assigns it (for port B, the pin number minus 8) to an object that already exists, which a `const` member forbids: the compiler rejects the assignment ("assignment of read-only member"), and casting the `const` away to force it is undefined behaviour. So `pin` drops its `const`, like `ddrx`, `portx` and `pinx`, which never had one. §10.1.2 reaches the same conclusion for `pin` and `pcicrx`.

Exercise 9.5 A pin-reservation table

Why `static`. At file scope `static` is about linkage, not lifetime: a file-scope variable lives for the whole program either way. `static` gives `pin_reg` internal linkage, so the name is visible only inside `gpio.c`, and only `gpio_init()` and `gpio_free()` can change it. That is the whole point of the table: it enforces an invariant, one owner per pin, and it can only do that if nothing else can edit it.

Without `static`, `pin_reg` has external linkage, and two things break:

- Any other file can write `extern uint16_t pin_reg;` and set or clear bits behind the driver's back, handing out the same pin twice or leaking a reservation.
- The name is exported to the linker, where it collides with any other global `pin_reg` in the program, say a timer driver's reservation table of the same name: either a multiple-definition error at link time or, where uninitialized globals are merged as common symbols (`-fcommon`, GCC's default before version 10), one variable silently shared by two unrelated drivers.

Exercise 9.6 A timer driver

Yes. After `timer_clear()` the instance no longer owns a hardware timer/counter, and another `timer_t` may already have claimed the one it had; any field left pointing at it would let the old instance reach hardware it no longer owns, the same danger as a freed `gpio_t` in Exercise 9.3. The safe, inert values are the ones every function can recognise as "owns nothing":

- The reserved timer/counter set to a value no timer/counter has (e.g. `0xFF`), with every function checking it first, so `timer_start()` or `timer_toggle()` on a cleared instance does nothing instead of restarting hardware someone else now owns.
- Running `false` and the elapsed-time count `0`, so `timer_running()` reports it as stopped.
- The callback `NULL`, so nothing can call through it, and the timeout `0`, the one value `timer_init()` refuses, so the instance reads as never initialized.

The hardware side goes first: stop the timer/counter's clock, disable its compare-match interrupt, and clear the driver's own record of which instance owns it, before the reservation bit is released, so that its interrupt can never fire for an instance that no longer owns it.

Chapter 10 Encapsulation and Polymorphism in C

Exercise 10.1 Only a forward declaration

Because a forward declaration makes `gpio_t` an incomplete type everywhere outside `gpio.c`. Calling code can hold a `gpio_t*`, pass it around and hand it to the driver's functions, but it cannot name a field, take `sizeof(gpio_t)`, or declare a `gpio_t` of its own. The only way to get an instance is `gpio_create()`, and the only way to change one is through the driver's functions, so the invariants those functions maintain (register pointers that belong together, a bit position in range, the direction the pin was configured with) cannot be broken from outside.

With `struct gpio { ... };` in the header, every file that includes it sees the fields, and that protection is gone:

- Calling code can read and write `pin`, `ddrx` and the rest directly, for example point an instance at another port's registers, bypassing every check the driver makes.
- It can declare its own `gpio_t`, as a local or a global, that never went through `gpio_create()`: an instance whose register pointers are garbage or null, which the driver's functions then dereference.
- It can come to depend on the layout, so changing a field means recompiling, and possibly rewriting, every file that uses the driver.

Encapsulation is then back to being a convention, as it is for Chapter 9's `gpio_t` and §10.1.2's.

Exercise 10.2 When `malloc()` fails

Returning `NULL` makes the failure visible and unambiguous: there is no new GPIO, and the caller is told so. Handing back the first GPIO instead would hide the failure. The caller would believe it had a fresh instance for its own pin while holding a second pointer to an object that another part of the program still uses, and whatever it then did to "its" GPIO (configure it as an input, write to it, free it) would silently change the first one under its owner, a bug that shows up far from its cause. As §10.1.3 says, `NULL` is the caller's only signal that the heap ran out.

Calling code must compare the result with `NULL` before using it, and deal with the failure there and then rather than store the pointer and carry on: report it and give up on that part of the setup, typically by returning an error, as the chapter's demos do when `main()` returns `-1`. An unchecked `NULL` does not even crash: §10.1.3's functions check `NULL == self` and quietly do nothing, so the program runs but that pin never works.

APPENDIX C

The Self-Assessment Papers

This appendix and the four after it hold two complete three-hour papers for *Embedded C*, with worked solutions. Appendices D and E are the papers, questions only; Appendices F and G are their model answers, with marks.

Each paper is self-contained. It states its own rubric, and it closes with a register reference covering every ATmega328P register and bit it refers to, so nobody sitting one needs a datasheet, a chapter of this book, or this appendix.

What these are for

The book has **no exam and nothing is marked**. What you get instead is the exercise set after every chapter, and the questions in every chapter's review.

These papers do not replace that. **They are here for you to test your own skills and knowledge, nothing more.** They gate nothing, they are not a qualification, and no part of the book requires them. Nothing in the companion repository depends on them, and neither `make build` nor `make format-check` knows they exist.

What they check is something the exercises cannot. Every exercise is done at a keyboard, with the chapter open beside you, the datasheet beside that, and a board on the desk to tell you whether it worked. These papers give you a page of register names and take the rest away, then ask what you can reconstruct — which of the rules you have absorbed and which you have merely been looking up.

Take one after the last chapter. Both papers draw on all ten chapters, so sitting one partway through examines material you have not read yet, and the result says more about how far you have read than about what you have understood.

The two papers

Both cover the whole book, Chapters 1 to 10, at the same weighting. They share no question. Either can be used alone; use both as a main sitting and a resit, or in alternate years.

Question	Topic	Chapters	Marks
1	The C language underneath: pointers, macros, types	1	12
2	Registers and bitwise I/O	2	12
3	Interrupts, edges and `volatile`	3	12
4	The ADC, and timers that count	4, 5	13
5	Serial communication	6	13
6	EEPROM and the watchdog	7, 8	13
7	Structs as drivers	9	12
8	Encapsulation and polymorphism in C	10	13
			100

Both papers mix theory with code, because the subject does. Roughly half the marks are for prose — what `volatile` guarantees, why an ISR must not transmit, what an opaque struct protects — and roughly half are for reading or writing C on paper.

The two papers differ in which half they lean on:

- **Paper A leans towards code that is put in front of you.** All eight of its questions hand you a snippet that compiles and is wrong: a macro whose missing parentheses silently turn 50 into 32, a `free()` that leaves the caller holding the freed address, a `setup()` that writes board pin numbers where port bit positions belong and therefore writes zero to every register, an interrupt armed at the peripheral but never enabled globally, an ADC read with no reference and no wait, a UART driver with the baud *rate* written into `UBRR0`, an EEPROM write with `EEPE` set before `EEMPE`, a watchdog "pet" that sets a bit instead of executing an instruction, a `gpio_toggle()` that uses `^=` on `PINx`, and a vtable whose interface field is not the struct's first member. Naming the defect is half the marks; saying what it does to the running program is the other half.
- **Paper B leans towards code you write and design decisions you justify.** It asks for a complete three-LED chase program written from nothing, an enumeration and its validity check, a `realloc()`-based resize that survives a failed allocation, an INT1 handler with an atomic toggle, `adc_read()` in full, `driver/serial.h` in full with its Doxygen, the EEPROM read/write pair, the watchdog timeout enumeration and its two odd values, a GPIO driver that maps an Arduino pin number to a port, a pin reservation table, and a `gpio_stub_t` that satisfies the vtable interface with no hardware behind it. Several parts are pure judgement: which of four values belongs in EEPROM and why, where a report belongs relative to the interrupt that triggered it, and what opacity takes away from a caller.

Neither paper asks anything of a compiler that a careful reader could not answer from the chapters. Where a question turns on a rule that is easy to state and easy to get backwards — which register reports a pin's actual level, which of `EEMPE` and `EEPE` comes first, why `PINx = (1U << n)` is atomic and `PORTx ^= (1U << n)` is not — the solution says so at length, because that is where the marks and the understanding both are.

Conventions the papers assume

Both papers state these in their own rubric, so a candidate never has to have read this appendix.

- **C11 on an ATmega328P at 16 MHz**, built with `avr-gcc`. Standard and AVR headers may be assumed included unless a question asks for them.
- **Register and bit names are not memory-tested.** Each paper closes with a register reference of its own, so nobody sitting one needs a datasheet in the room. Misspelling a bit costs nothing; reaching for the wrong *register*, or the right register at the wrong *time*, costs everything. What the reference deliberately withholds is the ordering rules, the timing rules and the formulas — those are the parts being examined.
- **Code is marked on semantics, not syntax.** A missing semicolon, a forgotten `#include` or a brace in the wrong column costs nothing. A missing `volatile` costs everything.
- **The book's house style is expected** where a question asks for it: `snake_case` with `UPPER_CASE` macros, `static` on anything private, header guards named after the file, Doxygen in the imperative mood, `self` as a pointer (and `const` when the function only reads), Yoda comparisons, braces on every `if`, fixed-width types with `U` suffixes, `const` on by-value parameters in the definition but not the prototype, and `volatile` on anything shared with an ISR.
- **"State the consequence" means the consequence.** Naming a defect earns half the marks; saying what it does to the running program earns the other half. "This is wrong" scores nothing.
- **Where a question asks for a stated number of defects**, listing more is not penalised, but only that many are marked, so the strongest answers go first.

Marking

Every solution is written to be marked by somebody who has read the chapters and does not otherwise write embedded C for a living, so each carries the reasoning rather than the answer alone. Marks are shown per part, and the arithmetic in the timer, ADC and baud-rate questions is worked through step by step rather than asserted.

Three conventions worth agreeing before a paper is marked:

- **Method carries the marks.** A correct diagnosis with a clumsy fix is worth more than a correct fix with no diagnosis. Several questions feed the next part, so an error should be followed through rather than penalised twice.
- **The named traps are worth full marks on their own.** A number of parts exist entirely to see whether a candidate walks into a specific mistake: reading a button from `PORTx`, a `1U << 9U` written to an 8-bit register, a plain `=` where `|=` was needed, an ISR-shared flag without `volatile`, `EEPE` before `EEMPE`, a watchdog pet written as a register access, `^=` on

`PINx`, a `gpio_t` passed by value, an interface field that is not first in its struct. Where a solution flags one of these, a candidate who walks into it loses those marks and no others.

- **The discussion parts are not decoration.** "State why" is where the book's actual content is, and a paper marked only on whether the code would compile would pass a candidate who has understood none of it.

APPENDIX D

Self-Assessment Paper A

Time: 3 hours. **Closed book.** No compiler and no datasheet. The register reference at the back of this paper is the only material you may use. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all ten chapters, so **it is meant to be taken once you have finished the book**. The model answers are in Appendix F.

Rubric

C11 on an ATmega328P (Arduino Uno/Nano) at 16 MHz is assumed throughout, built with `avr-gcc` and `-std=c11`. Standard and AVR headers (`<stdint.h>`, `<stdbool.h>`, `<avr/io.h>`, `<avr/interrupt.h>`, `<util/delay.h>`) may be assumed included unless a question asks you to name them.

Register and bit names are not memory-tested. Every register and bit this paper refers to is listed in the register reference at the back (page 290), with what it is; consult it freely. What is marked is knowing which register to reach for and *when* — the ordering rules, timing rules and formulas that govern their use are deliberately left out of it, because those are the parts being examined. Misspelling a bit costs nothing; reaching for the wrong register, or the right one at the wrong moment, costs everything.

Code is marked on semantics, not on syntax. A missing semicolon, a forgotten `#include` or a brace in the wrong column costs nothing. A missing `volatile` costs everything.

Where a question says "in the course's style", it means the conventions used in the chapters:

- `snake_case` for variables, functions and constants; `UPPER_CASE` for macros and enum constants.
- `static` on anything that is not part of a module's public API.
- A header guard named after the file in `UPPER_CASE`, `.` replaced by `_`, trailing `_` (`driver/gpio.h` → `DRIVER_GPIO_H_`).
- A Doxygen comment in the imperative mood above every non-trivial function.
- `self` as the first parameter of any function operating on a driver struct, always a pointer, and `const` when the function only reads through it.

- Yoda notation on comparisons against a constant, and braces on every `if`, even one-liners.
- `const` on pass-by-value parameters in a function *definition*, dropped from the prototype.
- Fixed-width types from `<stdint.h>`, and `U` suffixes on unsigned literals.
- `volatile` on anything shared between an ISR and the rest of the program.

Where a question says "state the consequence", naming the defect earns half the marks and saying what it does to the running program earns the other half. "This is wrong" scores nothing.

Where a question asks you to find defects, the number of defects is stated. Listing more than the stated number is not penalised, but only the stated number is marked, so put your strongest answers first.

Marks

Question	1	2	3	4	5	6	7	8
Marks	12	12	12	13	13	13	12	13

Question 1 The language underneath

12 MARKS

(a) A duty-cycle helper is defined as follows.

```
#define PWM_PERIOD_MS 10U
#define DUTY_PERCENT(on_ms) (on_ms * 100U / PWM_PERIOD_MS)

const uint16_t duty = DUTY_PERCENT(2U + 3U);
```

Write the exact text the preprocessor substitutes, give the value `duty` ends up with, give the value the author intended, and give the corrected macro definition. Then name one hazard a function-like macro has that no correct parenthesisation can remove. *(3 marks)*

(b) The program below allocates a receive buffer and releases it again.

```
static void release(uint8_t* buf)
{
    free(buf);
    buf = NULL;
}

int main(void)
{
    uint8_t* data = (uint8_t*)(malloc(BUF_LEN));
    if (NULL == data) { return 1; }
```

```
    release(data);

    if (NULL != data) { data[0U] = 1U; }
    return 0;
}
```

State what `data` holds after `release()` returns and why, name what the pointer has become, and state what the write on the following line is. Then give the corrected signature, body and call site. *(3 marks)*

(c) The keyword `static` appears in this book in two entirely different roles. State both, naming what each one changes. Then state why this book writes pin numbers and bit positions as `#define LED1 1U` rather than `static const uint8_t led1 = 1U;`, giving two distinct reasons. *(3 marks)*

(d) A software-PWM routine computes its on-time from a 10-bit ADC reading.

```
#define ADC_MAX 1023U
#define PWM_PERIOD_MS 10U

const uint16_t adc_val = adc_read(POT1);              // Returns 512 for this ↩
    ↪ reading.
const uint8_t on_ms    = (uint8_t)(adc_val / ADC_MAX * PWM_PERIOD_MS);
```

Give the exact value of `on_ms` and explain how it got there. State the only reading for which the expression produces anything else, and what it produces. Then give two different corrections, and name the trap that the first of them introduces on this target. *(3 marks)*

Question 2 Registers, and a program that does nothing

12 MARKS

Two LEDs sit on D9 and D10. A button sits on D13, wired through an external pull-down, so the pin reads high while the button is held. LED1 is meant to follow the button; LED2 is meant to stay dark.

```
#include <avr/io.h>

/** GPIO pins. */
#define LED1 9U  // D9  -> PORTB1.
#define LED2 10U // D10 -> PORTB2.
#define BTN1 13U // D13 -> PORTB5.

/**
 * @brief Set up system.
```

```
 */
static void setup(void)
{
    // Configure LED1 and LED2 as outputs.
    DDRB = (1U << LED1);
    DDRB = (1U << LED2);

    // Configure BTN1 as input with its internal pull-up enabled.
    PORTB = (1U << BTN1);
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        // Enable LED1 while BTN1 is held, disable it otherwise.
        if (PORTB & (1U << BTN1)) { PORTB |= (1U << LED1); }
        else { PORTB &= (1U << LED1); }
    }
    return 0;
}
```

(a) The program compiles and does nothing at all. It contains **four** defects. Identify each one, state the consequence, and give the corrected line. *(6 marks)*

(b) Give the single C expression for each of the following on a register `reg` and a bit position `n`: set the bit, clear the bit, test the bit, and toggle the bit. Then state why this book toggles an output by writing to `PINx` rather than by `PORTx ^= (1U << n)`, and what that buys. *(3 marks)*

(c) State what `F_CPU` is for, what must be true about *where* it is defined, and what the `UL` suffix does on this target. Then state why `_delay_ms(ms)` cannot be called with a variable, and what the book's `delay_ms()` helper does about it. *(3 marks)*

Question 3 An interrupt that never fires

12 MARKS

BTN1 is on D2 (INT0), wired through an external pull-down, so a press drives the pin high. LED1 is on D9. Each press-down is meant to toggle LED1 and report the new state over UART.

```
#include <stdbool.h>

#include <avr/interrupt.h>
#include <avr/io.h>

#include "driver/serial.h"

/** GPIO pins. */
#define LED1 1U // D9 -> PORTB1.

/** LED1 state (true = lit, false = dark). */
static bool led1_state = false;

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
    DDRB = (1U << LED1);

    // Trigger an interrupt on BTN1 press-down (INT0, D2).
    EICRA = (1U << ISC01);
    EIMSK = (1U << INT0);

    // Initialize serial driver.
    serial_init();
}

/**
 * @brief Toggle the LED1 state and report it on button press.
 */
ISR(INT0_vect)
{
    led1_state = !led1_state;
    serial_print("BTN1 pressed!");
}

/**
```

```
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    setup();

    while (1)
    {
        if (led1_state) { PORTB |= (1U << LED1); }
        else { PORTB &= ~(1U << LED1); }
    }
    return 0;
}
```

(a) The program contains **four** defects. Identify each one and state the consequence. Then state which single one of the four must be fixed before any of the other three can be observed at all, and why. *(6 marks)*

(b) The button is moved from D2 to D7. INT0 is no longer available. Give the three register writes and the ISR signature needed to interrupt on D7 instead, and state **two** behavioural differences between what you have written and the INT0 version it replaces. *(3 marks)*

(c) State what `volatile` tells the compiler and why omitting it produces a bug that appears only in some builds. Then name one variable in this book that is shared with an ISR and still does not need `volatile`, and say why. *(3 marks)*

Question 4 Converting and counting 13 MARKS

(a) A colleague's ADC routine returns plausible-looking numbers that do not track the potentiometer.

```
static uint16_t adc_read(const uint8_t channel)
{
    ADMUX  = channel;
    ADCSRA = (1U << ADEN) | (1U << ADSC);
    ADCSRA = (1U << ADIF);

    const uint8_t high = ADCH;
    const uint8_t low  = ADCL;
    return (uint16_t)(high << 8U) | low;
}
```

It contains **four** defects. Identify each one and state the consequence. Then give the single line that replaces the last three lines of the function entirely. *(5 marks)*

(b) A 16 MHz ATmega328P is configured with a ÷1024 prescaler.

1. Give the tick period in milliseconds, and the time between two Timer 0 overflows.
2. Give the number of Timer 0 overflows needed for a 750 ms timeout, and the resulting error.
3. Give the value written to `OCR1A` for a 40 ms periodic interrupt on Timer 1 in CTC mode, showing the arithmetic, and name the vector its handler must use.
4. State why the 40 ms period cannot be produced by Timer 0 in CTC mode at this prescaler.

(5 marks)

(c) The overflow handler below is meant to toggle an LED once per timeout.

```
ISR(TIMER0_OVF_vect)
{
    static uint8_t ovf_count = 0U;

    if (OVF_MAX <= ++ovf_count) { LED1_TOGGLE; }
}
```

State exactly what the LED does after the first timeout, and why. Give the corrected handler. Then state the one thing about Timer 2 that differs from Timers 0 and 1 in a way that silently breaks a copy-pasted configuration. *(3 marks)*

Question 5 Talking to a PC

13 MARKS

(a) State the formula that produces the value written to `UBRR0`. Compute it for 19200 baud on a 16 MHz clock, showing the arithmetic and the rounding. Give the baud rate the chip actually produces with that value and the resulting error, and state why an error of that size is harmless. Then state what a terminal opened at 9600 baud shows while the chip transmits at 19200. *(4 marks)*

(b) The serial code below compiles and produces nothing usable.

```
#define BAUD 9600U // 9600 bps with a 16 MHz clock.
#define MSG_LEN 50U

static void serial_init(void)
{
    UCSR0B = (1U << TXEN0);
    UCSR0C = (1U << UCSZ00) | (1U << UCSZ01);
    UBRR0  = BAUD;
```

```
}

static void serial_write_byte(const uint8_t byte)
{
    UDR0 = byte;
}

void serial_print(const char* text)
{
    for (uint16_t i = 0U; i <= MSG_LEN; ++i)
    {
        serial_write_byte((uint8_t)(text[i]));
    }
}

ISR(PCINT0_vect)
{
    if (BTN1_PRESSED) { serial_print("BTN1 pressed!"); }
}
```

It contains **four** defects. Identify each one and state the consequence. *(5 marks)*

(c) The program is extended to receive commands from the PC as well as send to it.

Name the bit that must be added to `UCSR0B`, name the flag in `UCSR0A` that says a byte has arrived, and state what clears that flag. Write `serial_read_byte()` in the course's style. Then state the failure mode of that function and what Chapter 6's `read()` adds to avoid it, including what each of its three possible return values means. *(4 marks)*

Question 6 Remembering, and recovering

13 MARKS

(a) The EEPROM write below never writes anything.

```
static void eeprom_write_byte(const uint16_t addr, const uint8_t byte)
{
    EEAR = addr;
    EEDR = byte;

    EECR |= (1U << EEPE);
    EECR |= (1U << EEMPE);
}

int main(void)
{
```

```
    setup();

    while (1)
    {
        const uint8_t led_state = LED1_ENABLED ? EEPROM_LED1_ON : EEPROM_LED1_OFF;
        eeprom_write_byte(EEPROM_LED1_ADDR, led_state);
    }
    return 0;
}
```

It contains **four** defects, one of which is in `main()` rather than in the function. Identify each one and state the consequence, then give the corrected function. One of the four produces a fault that appears only occasionally; say which, and why that is worse than one that never works at all. *(5 marks)*

(b) The watchdog code below is meant to arm a 1024 ms watchdog and to pet it once per iteration of `main()`'s loop. It does neither.

```
static void watchdog_init(void)
{
    WDTCSR = (1U << WDE) | (uint8_t)(WATCHDOG_TIMEOUT_1024MS);
}

static void watchdog_reset(void)
{
    WDTCSR |= (1U << WDE);
}
```

Identify the defect in each function and state the consequence. Give both functions corrected. For the second, state why it cannot be written as a register access at all, and what the book writes instead of `asm("wdr")`, with the reason. *(4 marks)*

(c) A device with a watchdog reboots once because of a genuine hang. From then on, a colleague's attempt to disable the watchdog from `setup()` is ignored, and the board keeps rebooting, many times a second, forever.

Name the flag responsible, name the register it lives in, and explain the mechanism. State where the fix goes and why its position matters. Then state what `WDE` and `WDIE` each select, and what the chip does on timeout when **both** are set. *(4 marks)*

Question 7 One driver, many instances — 12 MARKS

(a) A program grows from one LED to six. Contrast what has to change under the `#define LED1_ON PORTB |= (1U << LED1)` approach with what has to change under a `gpio_t` driver. Then state why the ADC, UART, EEPROM and watchdog drivers in this book are *correct* to have no struct at all, and what specifically would have to be true of a peripheral before one is warranted. *(3 marks)*

(b) The GPIO driver below compiles and is used across several files.

```
typedef struct
{
    volatile uint8_t* ddrx;
    volatile uint8_t* portx;
    volatile uint8_t* pinx;
    const uint8_t pin;
} gpio_t;

void gpio_write(gpio_t self, const bool state)
{
    if (state) { *self.portx |= (1U << self.pin); }
    else { *self.portx &= ~(1U << self.pin); }
}

bool gpio_read(gpio_t* self)
{
    return (*self->portx & (1U << self->pin));
}

void gpio_toggle(gpio_t* self)
{
    *self->pinx ^= (1U << self->pin);
}
```

It contains **four** defects against the course's own rules. Identify each one, state the consequence, and give the corrected signature or line. For `gpio_toggle()`, describe precisely what happens to the whole port when it is called on a pin that currently reads high. *(6 marks)*

(c) The driver is split into `driver/gpio.h` and `driver/gpio.c`. State what goes in each. `sensor.h` earlier in the book declared only a function; `gpio.h` must also declare `gpio_t` itself. State why. Then state why `gpio_init()`'s `mode` parameter is written `const gpio_mode_t mode` in the source file but plain `gpio_mode_t mode` in the header. *(3 marks)*

Question 8 Hiding things, and choosing at runtime

13 MARKS

(a) A driver header contains only this, plus prototypes.

```
typedef struct gpio gpio_t;
```

State what this declares and what it deliberately does not. State what a caller holding a `gpio_t*` can and cannot do with it. Then state why the `gpio_init(gpio_t* self, ...)` form has to be replaced by `gpio_create(...)` returning a pointer once the struct is opaque, and name the invariant that opacity protects on `timer_t` but that was never really at risk on `gpio_t`. *(4 marks)*

(b) A colleague adds function pointers to a still-public `gpio_t`.

```
typedef struct gpio
{
    volatile uint8_t* ddrx;
    volatile uint8_t* portx;
    volatile uint8_t* pinx;
    uint8_t pin;

    void (*write)(gpio_t* self, bool state);
    bool (*read)(const gpio_t* self);
} gpio_t;
```

The file does not compile. State what the compiler reports and why, and give the corrected member declarations. Then state what marking `gpio_write()` `static` in `gpio.c` does prevent, and what it does **not** prevent. *(4 marks)*

(c) An interface and one of its two implementations are written as follows.

```
typedef struct gpio_interface
{
    const gpio_vtable_t* vptr;
} gpio_interface_t;

typedef struct gpio_impl
{
    volatile uint8_t* ddrx;
    volatile uint8_t* portx;
    volatile uint8_t* pinx;
    uint8_t pin;
    gpio_interface_t itf;
} gpio_impl_t;
```

```
gpio_interface_t* gpio_atmega328p_new(const uint8_t pin, const gpio_mode_t mode)
{
    gpio_impl_t* impl = (gpio_impl_t*)(malloc(sizeof(gpio_impl_t)));
    if (NULL == impl) { return NULL; }
    // ... configure impl->ddrx / portx / pinx / pin from pin and mode ...
    impl->itf.vptr = gpio_vptr_get_instance();
    return (gpio_interface_t*)(impl);
}
```

The code compiles without a warning. State exactly what happens on the first call to `led->vptr->toggle(led)`, and why. Give the one-line change that fixes it and the rule that makes the fix work. State why the compiler said nothing. (*3 marks*)

(d) State why every instance can share one `static const gpio_vtable_t`, why `del()` takes `gpio_interface_t**` rather than `gpio_interface_t*`, and what a `gpio_stub_t` implementation makes possible that `gpio_impl_t` alone does not. (*2 marks*)

Register Reference

Every ATmega328P register and bit this paper refers to is listed here, with what it is. Nothing below is worth marks by itself: the paper tests which register to reach for and when, not whether you can recall its name. Where a bit's **position** matters it is given; where a timing rule, an ordering rule or a formula matters it is not, because that is the part being examined.

The target is an Arduino Uno/Nano at 16 MHz.

Board pins and ports

Board pin	Port bits
D0–D7	`PORTD0`–`PORTD7`
D8–D13	`PORTB0`–`PORTB5`
A0–A5	`PORTC0`–`PORTC5`

Digital I/O — one set per port, `x` being `B`, `C` or `D`

Register	What it is
`DDRx`	Data direction, one bit per pin: `1` = output, `0` = input.
`PORTx`	On an output pin, the level driven onto it. On an input pin, the internal pull-up enable.
`PINx`	Reads the actual logic level on each pin. Writing a `1` to a bit toggles the matching `PORTx` bit, whatever `DDRx` says; bits written `0` are unaffected.

Interrupts

Register / name	What it is
`SREG`	Status register. Bit `I` is the global interrupt enable; `sei()` sets it, `cli()` clears it.
`EICRA`	External interrupt edge select: `ISC01:ISC00` for INT0, `ISC11:ISC10` for INT1.
`EIMSK`	External interrupt enable: bits `INT0`, `INT1`.
`INT0_vect` / `INT1_vect`	External interrupt vectors, for D2 and D3 respectively.
`PCICR`	Pin change interrupt enable per port: `PCIE0` (B), `PCIE1` (C), `PCIE2` (D).
`PCMSK0` / `PCMSK1` / `PCMSK2`	Pin change mask for ports B, C and D. The bit position matches the pin's own bit position within its port.
`PCINT0_vect` / `PCINT1_vect` / `PCINT2_vect`	Pin change interrupt vectors, for ports B, C and D.

`ISCn1:ISCn0` encoding: `00` low level, `01` any logical change, `10` falling edge, `11` rising edge.

ADC

Register / name	What it is
`ADMUX`	Bits 3:0 select the input channel. `REFS1:REFS0` (bits 7:6) select the reference: `00` = the external `AREF` pin, `01` = `AVCC`.
`ADCSRA`	`ADEN` enables the ADC, `ADSC` starts a conversion, `ADIF` is the conversion-complete flag (cleared by writing a `1` to it), `ADPS2:ADPS0` set the prescaler.
`ADCL` / `ADCH`	Low and high halves of the 10-bit result.
`ADC`	Macro combining `ADCL` and `ADCH` into the full 10-bit value.

`ADPS2:ADPS0` encoding: `000` and `001` both divide by 2, and each value above that doubles the divisor, up to `111` = ÷128.

Timers — `n` is `0`, `1` or `2`

Register / name	What it is
`TCCRnB`	Clock select `CSn2:CSn0` (the prescaler). `WGM12`, in `TCCR1B`, selects CTC mode on Timer 1.
`TCNTn`	The counter itself.
`OCRnA`	Compare value. In CTC mode the timer counts from 0 up to it, then clears.
`TIMSKn`	`TOIEn` enables the overflow interrupt; `OCIEnA` enables the compare-match interrupt.
`TIMERn_OVF_vect`	Overflow vector.
`TIMERn_COMPA_vect`	Compare-match vector.

Timer 0 and Timer 2 are 8-bit; Timer 1 is 16-bit.

`CSn2:CSn0` encoding differs between timers:

Bits	Timer 0 / Timer 1	Timer 2
`001`	÷1	÷1
`010`	÷8	÷8
`011`	÷64	÷32
`100`	÷256	÷64
`101`	÷1024	÷128
`110`	–	÷256
`111`	–	÷1024

UART

Register / name	What it is
`UCSR0A`	`UDRE0` is set when `UDR0` is free to accept a new byte. `RXC0` is set when a fully received byte is waiting in `UDR0`.
`UCSR0B`	`TXEN0` enables the transmitter, `RXEN0` enables the receiver.
`UCSR0C`	Frame format. `UCSZ01` and `UCSZ00` set together give 8 data bits.
`UBRR0`	Baud rate divisor. Only the low 12 bits are used.
`UDR0`	Data register: written to transmit, read to receive. Reading it clears `RXC0`.

EEPROM

Register / name	What it is
`EEAR`	Address to read from or write to, 0–1023.
`EEDR`	Data register: holds the byte to be written, or receives the byte just read.
`EECR`	`EERE` starts a read. `EEMPE` is the master write enable, a safety interlock rather than a data bit. `EEPE` starts a write and stays set until that write completes.

The ATmega328P has 1024 bytes of EEPROM. A byte write takes a few milliseconds, and a cell tolerates roughly 100,000 write cycles.

Watchdog

Register / name	What it is
`MCUSR`	MCU status register. `WDRF` is set by a watchdog-caused reset.
`WDTCSR`	Watchdog control and status. Bit 0 `WDP0`, bit 1 `WDP1`, bit 2 `WDP2`, bit 3 `WDE`, bit 4 `WDCE`, bit 5 `WDP3`, bit 6 `WDIE`, bit 7 `WDIF`.
`WDT_vect`	Watchdog interrupt vector.

`WDE` is the watchdog enable, `WDIE` the watchdog interrupt enable, and `WDCE` the change enable used when reconfiguring the watchdog. `WDP3:WDP0` select the timeout: `0000` is 16 ms, and each step doubles it, up to `1001` = 8192 ms.

APPENDIX E

Self-Assessment Paper B

Time: 3 hours. **Closed book.** No compiler and no datasheet. The register reference at the back of this paper is the only material you may use. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all ten chapters, so **it is meant to be taken once you have finished the book**. The model answers are in Appendix G.

Rubric

C11 on an ATmega328P (Arduino Uno/Nano) at 16 MHz is assumed throughout, built with `avr-gcc` and `-std=c11`. Standard and AVR headers (`<stdint.h>`, `<stdbool.h>`, `<avr/io.h>`, `<avr/interrupt.h>`, `<util/delay.h>`) may be assumed included unless a question asks you to name them.

Register and bit names are not memory-tested. Every register and bit this paper refers to is listed in the register reference at the back (page 301), with what it is; consult it freely. What is marked is knowing which register to reach for and *when* — the ordering rules, timing rules and formulas that govern their use are deliberately left out of it, because those are the parts being examined. Misspelling a bit costs nothing; reaching for the wrong register, or the right one at the wrong moment, costs everything.

Code is marked on semantics, not on syntax. A missing semicolon, a forgotten `#include` or a brace in the wrong column costs nothing. A missing `volatile` costs everything.

Where a question says "in the book's style", it means the conventions used in the chapters:

- `snake_case` for variables, functions and constants; `UPPER_CASE` for macros and enum constants.
- `static` on anything that is not part of a module's public API.
- A header guard named after the file in `UPPER_CASE`, `.` replaced by `_`, trailing `_` (`driver/gpio.h` → `DRIVER_GPIO_H_`).
- A Doxygen comment in the imperative mood above every non-trivial function.
- `self` as the first parameter of any function operating on a driver struct, always a pointer, and `const` when the function only reads through it.

- Yoda notation on comparisons against a constant, and braces on every `if`, even one-liners.
- `const` on pass-by-value parameters in a function *definition*, dropped from the prototype.
- Fixed-width types from `<stdint.h>`, and `U` suffixes on unsigned literals.
- `volatile` on anything shared between an ISR and the rest of the program.

Where a question asks you to write code, Doxygen comments are marked only where the question says so. Everywhere else, write the code.

Where a question says "state the consequence", naming the defect earns half the marks and saying what it does to the running program earns the other half. "This is wrong" scores nothing.

Marks

Question	1	2	3	4	5	6	7	8
Marks	12	12	12	13	13	13	12	13

Question 1 Saying what you mean

12 MARKS

(a) A blink module supports four patterns: off, slow, fast, and a two-pulse heartbeat. Write, in the book's style, an enumeration `led_pattern_t` naming all four, and a function

```
bool led_pattern_valid(led_pattern_t pattern);
```

that returns `true` only for a named member. State three concrete advantages an `enum` has over a block of `#defines` for this, and then state the one property a `#define` has that a `const uint8_t` does not, which is why this book still writes its pin numbers as macros. *(4 marks)*

(b) Write a function-like macro `TICKS_NEEDED(timeout_ms, tick_ms)` that yields the number of ticks in a timeout. State the two parenthesisation rules it must obey, showing for one of them a call that goes wrong if the rule is broken. Then name the third macro hazard, which parentheses cannot fix, and state whether your macro is exposed to it, with the reason. *(3 marks)*

(c) A host-side test harness grows a buffer as data arrives.

```
typedef struct
{
    uint8_t* data;
    size_t size;
} buffer_t;
```

Write `buffer_resize(buffer_t* self, size_t size)` returning `0` on success and `1` on failure. It must leave `self` completely untouched if the allocation fails, and must handle a requested size of zero. State the specific mistake that a naive one-line `self->data = realloc(self->data, size);` makes, and what it costs. (*3 marks*)

(d) A driver is split into `driver/serial.h` and `driver/serial.c`. Give the header guard for that header, in the book's convention, and say what it is for. Then state the ordering convention this book uses inside a `.c` file for its public functions and its `static` helpers, and the reason. (*2 marks*)

Question 2 Straight to the registers

12 MARKS

(a) Three LEDs are wired to D8, D9 and D10, and a button to D12 through an external pull-down, so the pin reads high while the button is held.

Write a complete single-file program, in the book's style, that lights exactly one LED at a time in the order D8 → D9 → D10, advancing every 150 ms, for as long as the button is held, and turns all three off the moment it is released. Use macros for the pins and for the GPIO operations, and a `delay_ms()` helper. (*5 marks*)

(b) Give the single C expression for each of the following on a register `reg` and a bit position `n`: set the bit, clear the bit, test the bit, toggle it via `PORTx`, and toggle it atomically. Then state the difference between `DDRB = (1U << 3U);` and `DDRB |= (1U << 3U);`, and give one situation where each is the right choice. (*4 marks*)

(c) `PORTx` means one thing on an output pin and something else entirely on an input pin. State both, and state which register must be read to get a pin's actual level.

The button circuit in §2.1.4 uses an external 10 kΩ pull-down, yet the code still writes `PORTB = (1U << BTN1);`. State what that write does, what the pin reads when idle, and why the write is harmless in practice but not guaranteed to be. (*3 marks*)

Question 3 Reacting instead of polling

12 MARKS

(a) A button is wired to D3 through an external pull-down, so a press drives the pin high. An LED is on D10. Every press-down must toggle the LED, and the toggle must be immune to another interrupt changing a different bit of the same port at the same moment.

Write `setup()` and the interrupt handler, in the book's style, giving every register write and the exact vector name. State which edge you selected and why. (*5 marks*)

(b) The button is moved from D3 to A2. State everything that changes: the enable bit, the mask register, the bit written into it, and the vector. Then state **two** things the handler must now do that the D3 version did not, and why each is forced on you rather than optional. (*4 marks*)

(c) State the rule this book applies to decide what belongs inside an ISR and what belongs in `main()`'s loop. Name two specific calls from the book's own drivers that must never appear in an ISR, and give the concrete cost of each. Then write the three-line pattern an ISR and `main()` use to communicate instead, and say what `volatile` is doing in it. *(3 marks)*

Question 4 Reading the world, and keeping time

13 MARKS

(a) Write `adc_read(uint8_t channel)` in full, in the book's style, returning a 10-bit result. For each register you write, state in one sentence what that write is for. State what the ÷128 prescaler is chosen to achieve and what goes wrong without it. *(4 marks)*

(b) A 16 MHz ATmega328P is configured with a ÷1024 prescaler, and a 250 ms periodic event is required.

1. Give the tick period and the Timer 0 overflow period.
2. Give the `OCR1A` value for a 250 ms interrupt on Timer 1 in CTC mode, showing the arithmetic, and write the three register assignments plus the vector name.
3. Give the number of Timer 0 overflows the same 250 ms needs in Normal Mode, the actual period that produces, and the error as a percentage.
4. State why Timer 0 cannot reach 250 ms without counting overflows in software at any prescaler available to it.

(5 marks)

(c) A potentiometer on A0 sets the brightness of an LED on D9 by software PWM over a 10 ms period.

Write `run_pwm_cycle()`, in the book's style, computing the on-time and off-time from a fresh ADC reading. State why the book adds `+ 0.5F` before the cast, what this whole approach costs the rest of the program, and what replaces it. *(4 marks)*

Question 5 A channel to the host

13 MARKS

(a) Write `driver/serial.h` in full, in the book's style: header guard, includes, and a documented prototype for each of `serial_init()`, `serial_print()` and `serial_write()` (the buffer-and-length form). Doxygen comments are marked here.

Then state which functions of the driver belong in the header and which must be `static` in `serial.c`, with the reason, and state why a `const` on a by-value parameter is dropped from the prototype but kept in the definition. *(5 marks)*

(b) Write `serial_write_byte()` and `serial_print()`. State what `UDRE0` signals, and why the wait on it comes before the write to `UDR0` rather than after. Then state what `sprintf()` is for in this context, and the one caveat §6.1.5 attaches to it on AVR. *(4 marks)*

(c) A program must report an ADC reading over UART once a second while still responding to a debounced button on a pin-change interrupt.

Describe where the `sprintf()` and the `serial_print()` calls go, and why they cannot go where the event is detected. Write the flag pattern that connects the two, and state exactly what `volatile` is doing. Then state why the `read()` of §6.1.7 takes a `timeout_ms` parameter, and what each of its three possible return values means. *(4 marks)*

Question 6 Surviving a power cut, and a hang

13 MARKS

(a) Write `eeprom_write_byte(uint16_t addr, uint8_t byte)` and `eeprom_read_byte(uint16_t addr)` in full, in the book's style. For the write, explain each of the following in one sentence: the loop on `EEPE` at the top, the order of `EEMPE` and `EEPE`, and the `cli()`/`sei()` pair. State why the read needs no equivalent of the last of those. *(5 marks)*

(b) Write the enumeration `watchdog_timeout_t` covering all ten timeout settings, and `watchdog_init(watchdog_timeout_t timeout)`.

Explain why eight of the ten members can take their default values while the last two must be given `32` and `33` explicitly. Then explain what the `WDCE` write immediately before the real one is for, and what happens if the second write comes too late. *(4 marks)*

(c) Four values are candidates for EEPROM storage:

1. a calibration offset written once during manufacture;
2. a live ADC reading sampled at 100 Hz;
3. an on/off state a user toggles with a button;
4. a boot counter incremented on every reset.

For each, state whether EEPROM is appropriate and why, quantifying your answer for the one that is clearly wrong. Then state where in `main()` the watchdog reset belongs, and how the timeout should be chosen relative to the loop it is protecting. *(4 marks)*

Question 7 A driver that scales

12 MARKS

(a) Write `driver/gpio.h`, and the body of `gpio_init()`, for a GPIO driver in which the caller passes a single Arduino pin number rather than register pointers:

```
bool gpio_init(gpio_t* self, uint8_t pin, gpio_mode_t mode);
```

- Pins 0–7 map to `PORTD0`–`PORTD7`; pins 8–13 map to `PORTB0`–`PORTB5`.
- `gpio_init()` derives and stores the register pointers and the bit position itself.
- Three modes are supported: input, input with pull-up, and output.
- Any pin outside 0–13 returns `false` without touching a single register.

Doxygen comments are marked on the header. (*6 marks*)

(b) In the original driver, `gpio_t`'s `pin` field was declared `const uint8_t pin;`. State why it can no longer be `const` in your version, and what specifically a `const` struct member permits and forbids. Then state why `gpio_read()` takes `const gpio_t* self` while `gpio_write()` takes `gpio_t* self`, and what would change if either took `gpio_t self` by value instead. (*3 marks*)

(c) Two calls to `gpio_init()` with the same pin number would silently hand out two drivers for one piece of hardware.

Write the pin-reservation table that prevents it: the declaration in `gpio.c`, the check and the claim inside `gpio_init()`, and the release inside `gpio_free()`. State why the table must be `static`, and what breaks if it is not. (*3 marks*)

Question 8 Hiding the implementation

13 MARKS

(a) The GPIO driver's struct currently sits, fields and all, in `gpio.h`.

Write the opaque version of that header: what replaces the full struct definition, and what has to happen to the struct itself. Then state the three things this takes away from calling code, and for each, what the caller must do instead. (*4 marks*)

(b) Write a `button_t` struct holding a GPIO and a press callback, together with

```
static void button_check(const button_t* self, bool edge);
```

which invokes the callback when a press has been detected. State in words what `void (*on_press)(void);` declares, write the line that assigns a function to it, and write the line that calls through it. Then state why `button_check()` is better for never mentioning the LED it ultimately toggles. (*4 marks*)

(c) Given the interface

```
typedef struct gpio_interface gpio_interface_t;

typedef struct gpio_vtable
{
    void (*del)(gpio_interface_t** self);
    bool (*read)(const gpio_interface_t* self);
    void (*write)(gpio_interface_t* self, bool value);
    void (*toggle)(gpio_interface_t* self);
} gpio_vtable_t;

typedef struct gpio_interface
{
```

```
    const gpio_vtable_t* vptr;
} gpio_interface_t;
```

write a second implementation, `gpio_stub_t`, that touches no hardware at all: the struct, its four vtable functions, the vtable itself, and `gpio_stub_new()`.

State the one rule your struct's field order must obey and the language guarantee that makes the whole scheme work. State why a single shared vtable is enough for every instance ever created. Then state what having a stub makes possible that a hardware-only driver does not. *(5 marks)*

Register Reference

Every ATmega328P register and bit this paper refers to is listed here, with what it is. Nothing below is worth marks by itself: the paper tests which register to reach for and when, not whether you can recall its name. Where a bit's **position** matters it is given; where a timing rule, an ordering rule or a formula matters it is not, because that is the part being examined.

The target is an Arduino Uno/Nano at 16 MHz.

Board pins and ports

Board pin	Port bits
D0–D7	`PORTD0`–`PORTD7`
D8–D13	`PORTB0`–`PORTB5`
A0–A5	`PORTC0`–`PORTC5`

Digital I/O — one set per port, `x` being `B`, `C` or `D`

Register	What it is
`DDRx`	Data direction, one bit per pin: `1` = output, `0` = input.
`PORTx`	On an output pin, the level driven onto it. On an input pin, the internal pull-up enable.
`PINx`	Reads the actual logic level on each pin. Writing a `1` to a bit toggles the matching `PORTx` bit, whatever `DDRx` says; bits written `0` are unaffected.

Interrupts

Register / name	What it is
`SREG`	Status register. Bit `I` is the global interrupt enable; `sei()` sets it, `cli()` clears it.
`EICRA`	External interrupt edge select: `ISC01:ISC00` for INT0, `ISC11:ISC10` for INT1.
`EIMSK`	External interrupt enable: bits `INT0`, `INT1`.
`INT0_vect` / `INT1_vect`	External interrupt vectors, for D2 and D3 respectively.
`PCICR`	Pin change interrupt enable per port: `PCIE0` (B), `PCIE1` (C), `PCIE2` (D).
`PCMSK0` / `PCMSK1` / `PCMSK2`	Pin change mask for ports B, C and D. The bit position matches the pin's own bit position within its port.
`PCINT0_vect` / `PCINT1_vect` / `PCINT2_vect`	Pin change interrupt vectors, for ports B, C and D.

`ISCn1:ISCn0` encoding: `00` low level, `01` any logical change, `10` falling edge, `11` rising edge.

ADC

Register / name	What it is
`ADMUX`	Bits 3:0 select the input channel. `REFS1:REFS0` (bits 7:6) select the reference: `00` = the external `AREF` pin, `01` = `AVCC`.
`ADCSRA`	`ADEN` enables the ADC, `ADSC` starts a conversion, `ADIF` is the conversion-complete flag (cleared by writing a `1` to it), `ADPS2:ADPS0` set the prescaler.
`ADCL` / `ADCH`	Low and high halves of the 10-bit result.
`ADC`	Macro combining `ADCL` and `ADCH` into the full 10-bit value.

`ADPS2:ADPS0` encoding: `000` and `001` both divide by 2, and each value above that doubles the divisor, up to `111` = ÷128.

Timers — `n` is `0`, `1` or `2`

Register / name	What it is
`TCCRnB`	Clock select `CSn2:CSn0` (the prescaler). `WGM12`, in `TCCR1B`, selects CTC mode on Timer 1.
`TCNTn`	The counter itself.
`OCRnA`	Compare value. In CTC mode the timer counts from 0 up to it, then clears.
`TIMSKn`	`TOIEn` enables the overflow interrupt; `OCIEnA` enables the compare-match interrupt.
`TIMERn_OVF_vect`	Overflow vector.
`TIMERn_COMPA_vect`	Compare-match vector.

Timer 0 and Timer 2 are 8-bit; Timer 1 is 16-bit.

`CSn2:CSn0` encoding differs between timers:

Bits	Timer 0 / Timer 1	Timer 2
`001`	÷1	÷1
`010`	÷8	÷8
`011`	÷64	÷32
`100`	÷256	÷64
`101`	÷1024	÷128
`110`	–	÷256
`111`	–	÷1024

UART

Register / name	What it is
`UCSR0A`	`UDRE0` is set when `UDR0` is free to accept a new byte. `RXC0` is set when a fully received byte is waiting in `UDR0`.
`UCSR0B`	`TXEN0` enables the transmitter, `RXEN0` enables the receiver.
`UCSR0C`	Frame format. `UCSZ01` and `UCSZ00` set together give 8 data bits.
`UBRR0`	Baud rate divisor. Only the low 12 bits are used.
`UDR0`	Data register: written to transmit, read to receive. Reading it clears `RXC0`.

EEPROM

Register / name	What it is
`EEAR`	Address to read from or write to, 0–1023.
`EEDR`	Data register: holds the byte to be written, or receives the byte just read.
`EECR`	`EERE` starts a read. `EEMPE` is the master write enable, a safety interlock rather than a data bit. `EEPE` starts a write and stays set until that write completes.

The ATmega328P has 1024 bytes of EEPROM. A byte write takes a few milliseconds, and a cell tolerates roughly 100,000 write cycles.

Watchdog

Register / name	What it is
`MCUSR`	MCU status register. `WDRF` is set by a watchdog-caused reset.
`WDTCSR`	Watchdog control and status. Bit 0 `WDP0`, bit 1 `WDP1`, bit 2 `WDP2`, bit 3 `WDE`, bit 4 `WDCE`, bit 5 `WDP3`, bit 6 `WDIE`, bit 7 `WDIF`.
`WDT_vect`	Watchdog interrupt vector.

`WDE` is the watchdog enable, `WDIE` the watchdog interrupt enable, and `WDCE` the change enable used when reconfiguring them. `WDP3:WDP0` select the timeout: `0000` is 16 ms, and each step doubles it, up to `1001` = 8192 ms.

APPENDIX F

Model Answers to Paper A

Marks are shown per part. Method carries them: a correct diagnosis with a clumsy fix is worth more than a correct fix with no diagnosis, and several later parts consume earlier answers, so an error should be followed through rather than penalised twice.

Code is marked on semantics. Missing semicolons, missing includes and misplaced braces cost nothing.

The candidate had the register reference at the back of the paper. Every register and bit name, every bit *position* where one matters, and the `ISCn1:ISCn0`, `ADPS2:ADPS0` and `CSn2:CSn0` encodings were in front of them. What was deliberately withheld is the part being examined: the ordering rules, the timing rules and the formulas. So a misspelled bit costs nothing, but a candidate who cannot say which of `EEMPE` and `EEPE` comes first, or derive an encoding from bit positions the reference gave them, has not answered the question.

Question 1 The language underneath

12 MARKS

(a)

3 MARKS

The substituted text. The preprocessor replaces the parameter with the argument's text, verbatim, before the compiler ever runs:

```
const uint16_t duty = (2U + 3U * 100U / 10U);
```

(*1 mark*)

The value. `*` and `/` bind tighter than `+`, so this is `2U + ((3U * 100U) / 10U)` = `2 + 30` = **32**. The author intended `((2U + 3U) * 100U) / 10U` = **50**. (*1 mark*)

The corrected definition. Every use of a parameter gets its own parentheses, and so does the whole body:

```
#define DUTY_PERCENT(on_ms) (((on_ms) * 100U) / PWM_PERIOD_MS)
```

(*0.5 marks*)

The hazard parentheses cannot remove: multiple evaluation. A macro parameter is substituted at every point its name appears in the body, so an argument with a side effect hap-

pens once per appearance. `SQUARE(x) ((x) * (x))` is fully parenthesised and `SQUARE(i++)` still modifies `i` twice, with nothing to order the two, which is undefined behaviour. A real function evaluates its argument exactly once. `DUTY_PERCENT` names `on_ms` only once, so it escapes this particular time — but that is a property of this body, not a guarantee, and it changes the day somebody adds a second use. *(0.5 marks)*

Credit as an alternative third answer: a macro has no type, so nothing checks what is passed in, and the error message names the expansion rather than the macro.

(b) 3 MARKS

What `data` holds: the same address it held before. Everything in C is passed by value, including pointers. `release()` received a *copy* of `data`; `free(buf)` correctly released the block, and `buf = NULL;` then nulled the copy, which ceased to exist at the closing brace. The caller's `data` was never reachable from inside the function. *(1 mark)*

What it has become: a dangling pointer — a pointer still holding the address of a block that has been returned to the heap. *(0.5 marks)*

What the write is. `NULL != data` is true, so the guard passes and `data[0U] = 1U;` writes through a freed pointer. That is a use-after-free, which is **undefined behaviour**, not a crash you can rely on: the block may have been handed to a later `malloc()`, so the write silently corrupts somebody else's data instead of failing anywhere near the mistake. *(0.5 marks)*

The correction is the pointer-to-pointer idiom: to change the caller's pointer, the function needs that pointer's address.

```
/**
 * @brief Free a dynamically allocated buffer and null out the caller's pointer.
 *
 * @param[in,out] buf Address of the pointer to the buffer to free.
 */
static void release(uint8_t** buf)
{
    if ((NULL == buf) || (NULL == *buf)) { return; }
    free(*buf);
    *buf = NULL;
}
```

```
release(&data);
```

(1 mark)

(c) 3 MARKS

`static` at file scope: internal linkage. The name is not visible outside its own `.c` file and is not emitted for the linker to resolve. It is the closest thing C has to "private", and because the compiler can see every call site it can inline and optimise more aggressively. This book marks every helper in a `main.c` `static` for exactly that reason. *(1 mark)*

`static` on a local variable: static storage duration. The variable's *scope* is unchanged — it is still only nameable inside its block — but its *lifetime* becomes the whole program, so it keeps its value between calls and is initialised once. `static uint8_t ovf_count = 0U;` inside `ISR(TIMER0_OVF_vect)` is the book's example. *(1 mark)*

Why `#define` for pin numbers, any two of:

- **A `const` variable in C is not a constant expression.** Its value cannot change at runtime, but the language still refuses it anywhere a *constant expression* is required: the size of a file-scope array, a `case` label, a `#if`. A macro works in all of them; an `enum` constant works in the first two but not in `#if`, because the preprocessor runs before the compiler and knows nothing about enums. This is the real reason embedded C keeps reaching for `#define` and not merely habit.
- **Code generation.** With `LED1` a literal, `PORTB |= (1U << LED1)` folds at compile time into a single bit-set instruction. With `LED1` a `const uint8_t` variable, the shift amount is, as far as the language is concerned, only known at runtime, so nothing stops the compiler emitting an actual shift.
- The macro is also usable from a `#if` in the preprocessor, before there is any such thing as a variable.

(1 mark for any two)

A candidate who also notes the cost — a macro has no type, no scope, and is invisible to a debugger by name — has understood the trade-off rather than just the rule. Credit it, but not in place of one of the two above.

(d) 3 MARKS

The value is 0. Both operands of `/` are unsigned integers, so the division is integer division and truncates: `512U / 1023U` is `0`, not `0.5005`. `0 * 10U` is `0`, and the cast to `uint8_t` changes nothing. The LED is never lit. *(1 mark)*

The only reading that behaves differently is 1023, the top of the ADC's range: `1023U / 1023U` is `1`, so `on_ms` becomes `10` — full brightness. The pot therefore does nothing at all across its entire travel and then snaps to fully lit at the very end of it. *(0.5 marks)*

Correction 1 — multiply before dividing:

```
const uint8_t on_ms = (uint8_t)((adc_val * PWM_PERIOD_MS) / ADC_MAX);
```

`512 * 10 = 5120`, `5120 / 1023 = 5`. *(0.5 marks)*

The trap it introduces: silent overflow in the intermediate product. The multiplication is carried out in the promoted type, and on AVR `int`/`unsigned int` is **16 bits**. `1023 * 10`

is 10230 and fits; change `PWM_PERIOD_MS` to `100U` and `1023 * 100` is 102300, which wraps to 36764 with no warning and no runtime symptom other than a wrong duty cycle at the top of the range. Widening one operand — `((uint32_t)(adc_val) * PWM_PERIOD_MS) / ADC_MAX` — removes it. *(0.5 marks)*

Correction 2 — §4.1.4's, in floating point:

```
const float duty_cycle = (float)(adc_val) / ADC_MAX;
const uint8_t on_ms    = (uint8_t)(duty_cycle * PWM_PERIOD_MS + 0.5F);
```

The cast on the numerator forces the division to be done in `float`; `+ 0.5F` before the truncating cast rounds to nearest rather than always down. `0.5005 * 10 + 0.5 = 5.505` → `5`. *(0.5 marks)*

Question 2 Registers, and a program that does nothing

12 MARKS

(a)

6 MARKS

(1.5 marks per defect: 0.5 for naming it, 0.5 for the consequence, 0.5 for the corrected line.)

1. Board pin numbers are used where port bit positions are required. D9 is `PORTB1`, D10 is `PORTB2` and D13 is `PORTB5` — the comments say so — but the macros hold `9U`, `10U` and `13U`. `1U` is an `unsigned int`, 16 bits on this target, so `1U << 9U` is `512` (`0x0200`); assigning it to the 8-bit `DDRB` keeps only the low byte, which is `0x00`. The same goes for `1U << 10U` (1024 → `0x00`) and `1U << 13U` (8192 → `0x00`). **Every register write in the program writes zero.** Neither LED is ever configured as an output, the pull-up is never enabled, and `(1U << BTN1)` is a zero mask, so the button test can never be true. This alone accounts for the program doing nothing.

The compiler does notice the three plain assignments: avr-gcc warns `large integer implicitly truncated to unsigned type` (`-Woverflow`, on by default), which the demos' `-Werror` would turn into a failed build. The test, the `|=` and the `&=` compile silently.

```
#define LED1 1U // D9  -> PORTB1.
#define LED2 2U // D10 -> PORTB2.
#define BTN1 5U // D13 -> PORTB5.
```

2. `DDRB` is assigned twice, not accumulated. `=` replaces the whole register. The second write drops LED1's direction bit, so once defect 1 is fixed LED1 is still an input and never lights.

```
DDRB = (1U << LED1) | (1U << LED2);
```

(`DDRB |= (1U << LED2);` on the second line earns the same marks.)

3. The button is read from `PORTB` instead of `PINB`. On an input pin `PORTx` is the pull-up setting, not the pin level, and `setup()` has just set that bit to `1`. The condition is therefore permanently true regardless of the button, so LED1 would be permanently lit rather than following anything. `PINx` is the register that reports the actual logic level on the pins.

```
if (PINB & (1U << BTN1)) { PORTB |= (1U << LED1); }
```

4. The clear is missing its `~`. `PORTB &= (1U << LED1)` ANDs the whole port with a mask in which only LED1 is set. Every *other* bit is cleared — LED2 goes dark and BTN1's pull-up is switched off — while LED1 itself keeps whatever value it already had, so the branch that is supposed to turn LED1 off never turns anything off. The clear-bit idiom is AND with the *negated* mask.

```
else { PORTB &= ~(1U << LED1); }
```

A candidate who also objects to enabling the internal pull-up at all, against an external pull-down, is right: the two form a divider that holds the idle pin near 1.1 V with a typical pull-up, and only a typical one keeps it safely below the 1.5 V a `0` needs (§2.1.4). Credit it, but not in place of one of the four; the book's own programs enable it too, and it is not why this one does nothing.

(b) 3 MARKS

Operation	Expression
Set bit `n`	`reg |= (1U << n);`
Clear bit `n`	`reg &= ~(1U << n);`
Test bit `n`	`(reg & (1U << n))`
Toggle bit `n`	`reg ^= (1U << n);`

(2 marks for all four; 1 mark for two or three)

Why the book toggles through `PINx`. Writing a `1` to a `PINxn` bit is a documented feature of this chip: it toggles the corresponding `PORTxn` bit, whatever `DDRxn` says, and bits written as `0` leave their counterpart alone. So `PINB = (1U << LED1);` is a **pure write** — no read at all.

`PORTB ^= (1U << LED1);` is a read-modify-write: read `PORTB`, flip one bit, write the byte back. If an interrupt fires between the read and the write and changes a different bit of `PORTB`, that change is inside the byte the interrupt wrote but not inside the byte the main program is about to write back, so it is silently overwritten. The `PINx` write has no read and therefore no window: the toggle is atomic no matter what else is touching the port. *(1 mark)*

(c) 3 MARKS

`F_CPU` is the CPU clock frequency in Hz. `<util/delay.h>` computes its busy-loop iteration counts from it **at compile time**, so it must be defined *before* that header is included; otherwise the header has already been expanded against its own fallback and every delay in the program is wrong by whatever ratio. (*1 mark*)

The `UL` suffix makes the literal an `unsigned long`, explicitly. `int` is only 16 bits on AVR, far too small for 16,000,000. The value would not be truncated without the suffix: C gives an unsuffixed decimal constant the first of `int`, `long` and `long long` that can hold it, so `16000000` would quietly become a 32-bit signed `long`. `UL` states the type rather than leaving it to that rule, so every calculation built on `F_CPU`, a delay's loop count or a baud-rate divisor, is done in 32-bit unsigned arithmetic by design rather than by accident. (*1 mark*)

Why `_delay_ms()` cannot take a variable. The same compile-time computation: `_delay_ms()` turns its argument into a loop count while compiling, so the argument has to be a constant expression. A `uint16_t` variable is not one.

The book's helper works around it by calling `_delay_ms()` with a *constant* and looping:

```
static void delay_ms(const uint16_t ms)
{
    for (uint16_t i = 0U; i < ms; ++i)
    {
        _delay_ms(TICK_1MS);
    }
}
```

(*1 mark*)

Question 3 An interrupt that never fires 12 MARKS

(a) 6 MARKS

(*1.25 marks per defect; 1 mark for the final part.*)

1. `sei()` is never called. Every peripheral interrupt is gated by the global `I` bit in `SREG`, which is clear out of reset. `EIMSK` arms INT0 at the peripheral, but with `I` clear the core never takes the vector, so `ISR(INT0_vect)` never runs. `led1_state` stays `false` forever and the LED never lights. Add `sei();` as the last line of `setup()`.

2. The wrong edge is selected. `EICRA = (1U << ISC01);` sets `ISC01:ISC00` to `10`, which is the **falling** edge. The button is wired through an external pull-down, so a press drives the pin *high* and a release drives it low — the handler therefore runs on release, not on press-down. For a rising edge, both bits are set:

```
EICRA = (1U << ISC00) | (1U << ISC01);
```

3. `led1_state` is not `volatile`. It is written only inside the ISR and read only inside `main()`'s loop. Nothing in that loop, as far as the compiler can see, ever changes it, and the compiler has no idea an interrupt exists — so it is entitled to load the variable once before the loop and test the same register value on every iteration thereafter. The loop then runs forever on a stale `false`. Declare it `static volatile bool led1_state = false;`.

4. `serial_print()` is called from inside the ISR. `serial_write_byte()` blocks on `UDRE0` for each byte. At 9600 baud a 15-byte message (13 characters plus `\r\n`) is 150 bit-times, about **15.6 ms**, and for nearly all of that time every other interrupt on the chip is held off — including the debounce timer a program like this depends on. An ISR should flip a flag and return; the transmission belongs in `main()`'s loop:

```
ISR(INT0_vect)
{
    led1_state = !led1_state;
    led1_event = true;   // static volatile bool, read and cleared in main().
}
```

Which one must be fixed first: the missing `sei()`. Until the `I` bit is set the handler never executes at all, so the wrong edge, the stale read and the blocking transmission have no opportunity to show themselves. Every one of them is still a defect — they are simply unobservable while the interrupt is disabled, which is what makes fixing `sei()` alone so misleading: three latent faults arrive at once. *(1 mark)*

(b) 3 MARKS

D7 is `PORTD7`, which is not an external-interrupt pin, so it needs a **pin change interrupt** on port D:

```
#define BTN1 7U // D7 -> PORTD7.

PCICR  = (1U << PCIE2);  // Enable pin change interrupts on port D.
PCMSK2 = (1U << BTN1);   // Specifically D7 / PORTD7.
sei();                   // Enable interrupts globally.

ISR(PCINT2_vect) { /* ... */ }
```

(1.5 marks: 0.5 for `PCICR`/`PCIE2`, 0.5 for `PCMSK2` with the right bit, 0.5 for the vector. The bit position written into `PCMSKn` is the pin's own bit position within its port; the global `PCINT23` number is only for looking the pin up in the datasheet.)

Two behavioural differences, both of them forced rather than optional:

1. **Pin change interrupts have no edge selection.** There is no `EICRA` equivalent; the interrupt fires on **both** edges, press and release. If only one is wanted, the handler must read the

pin's actual level itself and return without acting otherwise — the filtering moves from hardware into software.

2. **The vector is shared by the whole port.** `PCINT2_vect` is the handler for every enabled PCI pin on port D, and nothing in the call says which pin changed. With more than one enabled, the handler has to work it out, typically by keeping a previous-`PIND` byte and XOR-ing it against the current read to see which bit flipped.

(*1.5 marks, 0.75 each*)

(c) 3 MARKS

What `volatile` says. That the object may be read or written by something the compiler cannot see, so every access in the source must become a real memory access in the output: no caching the value in a register across statements, no eliding a write that appears redundant, no reordering accesses to it relative to each other. (*1 mark*)

Why the bug is build-dependent. Nothing about omitting `volatile` is wrong in a way the compiler must diagnose; it merely *permits* an optimisation. At `-O0` the compiler reloads from memory anyway, so the program works and the omission is invisible. Turn on `-Os` or `-O2` — which is what a release build does — and the load is hoisted out of the loop. Same source, same board, same hardware; the fault appears only when the build changes, which is why it is so unpleasant to track down after the fact. (*1 mark*)

One that does not need it. `static uint8_t ovf_count`, declared *inside* `ISR(TIMER0_OVF_vect)`. It is function-local: nothing outside that one handler can name it, so the compiler can see every read and write to it and there is no invisible second party. §5.1.4 marks it `volatile` as a habit and says so explicitly; the moment anything outside the ISR needs to read it, the qualifier stops being decoration. (*1 mark*)

Credit equally `led_state` in §5.1.5's Timer 1 example, which is genuinely shared: the button's ISR writes it and the timer's ISR reads it. On the AVR an ISR runs to completion with interrupts disabled, so nothing can change `led_state` in the middle of either handler, and each reads it afresh every time it runs; the chapter keeps `volatile` on it by the book's rule, and it becomes load-bearing the moment `main()`'s loop reads it too. `led1_lit`, touched only by the timer's ISR, is the same case as `ovf_count`.

Question 4 Converting and counting 13 MARKS

(a) 5 MARKS

(*1 mark per defect, 1 mark for the replacement line.*)

1. No voltage reference is selected. `ADMUX = channel;` leaves `REFS1:REFS0` at `00`, which selects the external `AREF` pin. On an Uno/Nano nothing drives `AREF`, so the converter is measuring against a floating reference: the results are plausible-looking numbers that drift and bear no fixed relationship to the input. Setting `REFS0` selects `AVCC`, the chip's own supply.

```
ADMUX = (1U << REFS0) | channel;
```

2. No prescaler is configured. `ADPS2:0` is left at `000`, a division factor of 2, so the ADC clock runs at 8 MHz. The successive-approximation hardware needs to be below roughly 200 kHz for full 10-bit accuracy; at 8 MHz the low bits are noise. At 16 MHz, ÷128 gives 125 kHz:

```
ADCSRA = (1U << ADEN) | (1U << ADSC) | (1U << ADPS0) | (1U << ADPS1) | (1U << ↪
    ↪ ADPS2);
```

3. The conversion is never waited for. `ADCSRA = (1U << ADIF);` executes immediately after the start, so the result registers are read while the conversion is still in flight; what comes back belongs to a previous conversion or to nothing. Worse, that plain assignment also writes `0` to `ADEN`, switching the ADC off, which *aborts* the conversion it just started. The flag must be polled first and only then cleared:

```
while (0U == (ADCSRA & (1U << ADIF))) {}
ADCSRA = (1U << ADIF);
```

4. `ADCH` is read before `ADCL`, and the shift precedes the widening cast. Reading `ADCL` first locks both result registers against update until `ADCH` has been read too, so the pair is guaranteed to come from the same conversion. Reading `ADCH` first leaves that lock unarmed: a conversion completing between the two reads tears the result, and the two halves end up belonging to different samples.

In this function the wrong order does something worse, and it fits the symptom. The last read is `ADCL`, so the function returns with the result registers still locked, and a conversion that completes while they are locked is lost: neither register is updated. The next call then reads the previous conversion's result again, and re-arms the lock as it goes. The numbers stay plausible and stop tracking the potentiometer.

Separately, `high` is a `uint8_t`, so `high << 8U` promotes it to a 16-bit *signed* `int` and shifts there; the `(uint16_t)` cast is applied to the result, too late to have protected anything. The cast belongs on the operand:

```
const uint8_t low  = ADCL;
const uint8_t high = ADCH;
return ((uint16_t)(high) << 8U) | (uint16_t)(low);
```

`ADCH` never uses more than its bottom two bits, so this particular shift happens to be safe either way — but the habit is what keeps it safe unconditionally, and a candidate who says so has answered better than one who claims a bug that cannot fire here.

The one line that replaces the last three:

```
return ADC;
```

`ADC` is a macro that already combines `ADCL` and `ADCH` into the full 10-bit value, in the correct order. Nothing is assembled by hand. (*1 mark*)

(b)

5 MARKS

1. Tick period and overflow period. (*1 mark*)

```
tick period = 1024 / 16,000,000 s = 64 us = 0.064 ms
Timer 0 overflows every 256 ticks = 256 * 0.064 = 16.384 ms
```

2. Overflows for 750 ms. (*1 mark*)

```
750 / 16.384 = 45.78  ->  46 overflows
46 * 16.384 = 753.664 ms, i.e. about +0.49 %
```

(*The book's macro rounds with* `(uint8_t)(TIMEOUT_MS / OVF_PERIOD_MS + 0.5F)`*, which gives the same 46. A candidate who truncates to 45 and says so, giving* −1.7 %*, has done the arithmetic correctly and should lose at most half a mark.*)

3. `OCR1A` for 40 ms on Timer 1 in CTC mode. (*2 marks*)

```
ticks needed = 40 / 0.064 = 625
OCR1A counts 0 up to its value, so reaching it takes (OCR1A + 1) ticks:
OCR1A = 625 - 1 = 624
```

Vector: `ISR(TIMER1_COMPA_vect)`.

Credit the configuration if it is given: `TCCR1B = (1U << CS10) | (1U << CS12) | (1U << WGM12);` *for the ÷1024 prescaler plus CTC mode,* `OCR1A = 624U;`, `TIMSK1 = (1U << OCIE1A);`, `sei();`.

4. Why Timer 0 cannot. (*1 mark*)

625 ticks does not fit an 8-bit counter, whose compare register stops at 255. Timer 0 can only reach 40 ms by counting overflow interrupts in software — which is precisely why the Normal Mode example in §5.1.4 does it that way, and why Timer 1's 16-bit range is what lets CTC hit an arbitrary period directly.

(c)

3 MARKS

What the LED does. `ovf_count` is never cleared, so once it reaches `OVF_MAX` the condition `OVF_MAX <= ++ovf_count` is true on **every** subsequent overflow. The LED therefore toggles once per overflow — every 16.384 ms, about 30 Hz — which reads as a blur or a steady half-brightness rather than a blink. It carries on until `ovf_count` wraps past 255 back to 0, at which point there is one quiet interval of `OVF_MAX` overflows before the whole thing repeats. (*1 mark*)

Corrected:

```
ISR(TIMER0_OVF_vect)
{
    static uint8_t ovf_count = 0U;

    // Increment the overflow counter, toggle LED1 and clear the counter on ↩
    ↪ timeout.
    if (OVF_MAX <= ++ovf_count)
    {
        LED1_TOGGLE;
        ovf_count = 0U;
    }
}
```

(*1 mark*)

Timer 2's difference: its prescaler bit patterns are not the same as Timers 0 and 1. `101` selects ÷1024 on Timers 0 and 1, but ÷128 on Timer 2; ÷1024 on Timer 2 is `111`. Every register and bit *name* exists on both (`CS20`/`CS22` next to `CS00`/`CS02`), so a configuration copied from one timer to the other compiles cleanly and produces a timer running eight times too fast, with nothing to point at. (*1 mark*)

*The register reference gives both encodings side by side, so finding the discrepancy is a table comparison rather than a feat of recall. The mark is for the second half — why it fails **silently**, i.e. that `CS20`/`CS22` exist just as `CS00`/`CS02` do, so nothing in the copied line is even suspicious. A candidate who only reports "the prescaler bits differ" earns half of it.*

Question 5 Talking to a PC

13 MARKS

(a)

4 MARKS

The formula. (*0.5 marks*)

```
UBRR0 = F_CPU / (16 * baud_rate) - 1
```

For 19200 baud at 16 MHz. (*1.5 marks*)

```
16,000,000 / (16 * 19200) = 16,000,000 / 307,200 = 52.083
52.083 - 1 = 51.083  ->  UBRR0 = 51
```

The actual baud rate and the error. (*1 mark*)

```
16,000,000 / (16 * (51 + 1)) = 16,000,000 / 832 = 19230.8 baud
error = +0.16 %
```

Why that is harmless. UART is asynchronous but re-synchronises on every start bit. The receiver only has to stay in step for the length of one frame — ten bit-times for 8-N-1 — before the next start bit re-aligns it. A 0.16 % rate mismatch has accumulated 1.6 % of a single bit period by the stop bit, against a budget of roughly half a bit; the usual tolerance is a couple of percent. Errors never accumulate across bytes. (*0.5 marks*)

A terminal at 9600 while the chip sends 19200 does not go silent — the line is genuinely active. It samples at half the true rate, so each of its bit slots spans two transmitted bits, start bits are detected in the wrong places, and the stop bit frequently is not where it should be. The result is a stream of garbage characters and framing errors. (*0.5 marks*)

(b)

5 MARKS

(*1.25 marks per defect.*)

1. The baud *rate* is written into `UBRR0` instead of the value computed from it. `UBRR0` takes the divisor from part (a)'s formula, which for 9600 baud at 16 MHz is `103`, not `9600`. Only the low 12 bits of the register pair are used, so `9600` (`0x2580`) is truncated to `0x580` = 1408, giving `16,000,000 / (16 * 1409)` ≈ **710 baud**. Even without the truncation, `UBRR0 = 9600` would produce about 104 baud. Either way the port is nowhere near 9600 and a terminal opened there sees nothing but garbage.

```
#define BAUD 103U // UBRR value for 9600 bps with a 16 MHz clock.
```

2. `serial_write_byte()` never waits on `UDRE0`. `UDR0` can take a new byte only while `UDRE0` says it is free, and a byte written to it while `UDRE0` is clear is ignored by the transmitter. The loop in `serial_print()` hands bytes over a few CPU cycles apart, while at 9600 baud each one takes over a millisecond to send, so only the first byte or two of each message reach the line and the rest are silently dropped. The wait belongs before the write, so that every byte finds the register free:

```
static void serial_write_byte(const uint8_t byte)
{
    // Wait until the transmit buffer is ready for a new byte.
    while (0U == (UCSR0A & (1U << UDRE0))) {}

    // Place the new byte in the transmit data register.
    UDR0 = byte;
}
```

3. `serial_print()` ignores the string's terminator and reads past the end of it. A C string has no length field; it ends at the `'\0'`. This loop instead transmits a fixed `MSG_LEN + 1` bytes — 51, because of the `<=` — regardless of the string it was given. For the 13-character literal at the call site it walks 38 bytes past the end and transmits whatever happens to follow it in

memory. It also has no `NULL` check, though it is an exported function with call sites outside this file: exactly the case where the book's own rule says the check stops being decorative.

```
void serial_print(const char* text)
{
    // Terminate if the text pointer is NULL.
    if (NULL == text) { return; }

    // Transmit each character one by one.
    for (uint16_t i = 0U; text[i]; ++i)
    {
        serial_write_byte((uint8_t)(text[i]));
    }
    serial_write_byte('\r');
    serial_write_byte('\n');
}
```

4. The transmission happens inside the ISR. Once `serial_write_byte()` waits on `UDRE0` as it must, it blocks once per byte. At 9600 baud, 15 bytes is 150 bit-times, about **15.6 ms**, and the ISR holds off every other interrupt for nearly the whole of it — including the debounce timer this pattern normally depends on, so presses arriving during the message are lost. The handler should flag the event and return; `main()`'s loop does the printing.

```
ISR(PCINT0_vect)
{
    if (BTN1_PRESSED) { btn1_event = true; }   // static volatile bool.
}
```

(c) 4 MARKS

The enable bit is `RXEN0`, in `UCSR0B`, alongside `TXEN0`; set both to have transmit and receive at once: `UCSR0B = (1U << TXEN0) | (1U << RXEN0);` *(0.5 marks)*

The flag is `RXC0`, in `UCSR0A`. It is `UDRE0`'s counterpart: clear while nothing has arrived, set the moment a fully received byte is sitting in `UDR0`. **Reading `UDR0` clears it**, and readies the register for the next incoming byte — there is nothing to write back by hand. *(1 mark)*

```
/**
 * @brief Receive a single byte over UART, blocking until one arrives.
 *
 * @return Byte received.
 */
static char serial_read_byte(void)
{
```

```
    // Wait until a byte has been received.
    while (0U == (UCSR0A & (1U << RXC0))) {}

    // Return the received byte.
    return UDR0;
}
```

(*1 mark*)

The failure mode: it blocks indefinitely. If the sender never speaks, the program sits in that loop forever and does nothing else — no button, no timer work, no main loop. With a watchdog armed, the chip simply reboots instead, which at least makes the hang visible but does not make it correct. (*0.5 marks*)

What `read()` adds. A `timeout_ms` parameter, and a buffer rather than a single byte. Non-zero, it gives up after roughly that many milliseconds even if the buffer never filled, pacing itself one millisecond at a time with `delay_ms(1U)`; `0` keeps the old block-until-done behaviour deliberately. Its three return values are: **`-1`** for a bad call (`NULL` buffer or `buflen == 0`), a **count below `buflen`** when the timeout elapsed first, and **`buflen`** itself when the buffer filled completely. (*1 mark*)

Question 6 Remembering, and recovering — 13 MARKS

(a) — 5 MARKS

(*1 mark per defect, 1 mark for the corrected function.*)

1. There is no wait for a previous write to finish. `EEPE` stays set for the several milliseconds an EEPROM write takes. Loading a new address and data while one is still in progress either loses the new write or disturbs the one under way. The first line of the function must be `while (EECR & (1U << EEPE)) {}`.

2. `EEPE` is set before `EEMPE`. This is why nothing is ever written. `EEMPE` is not a data bit; it is a safety interlock that arms the write, and `EEPE` must follow it, not precede it. Setting `EEPE` with the interlock unarmed does nothing at all, and the later `EEMPE` write arms an interlock that is never used. The function silently succeeds and stores nothing.

3. Interrupts are not disabled around the sequence. The datasheet requires `EEPE` to be set within **four CPU cycles** of `EEMPE`. Four cycles is nothing; any interrupt taken in that window pushes the second write outside it and the write silently does not happen. `cli()` before the pair and `sei()` after is not optional.

4. `main()` writes on every iteration of the loop. Each write blocks for roughly 3.4 ms, so the loop manages about 290 writes a second. Against the cell's ~100,000-cycle endurance, that address is worn out in **under six minutes** — and for those six minutes the program

is doing nothing else, because the write blocks. The value should be written only when it changes, which is what the `led1_event` flag in §7.1.4's demo is for.

Corrected:

```
/**
 * @brief Write a byte to EEPROM at the given address.
 *
 * @param[in] addr EEPROM address to write to (0-1023).
 * @param[in] byte Byte to write.
 */
static void eeprom_write_byte(const uint16_t addr, const uint8_t byte)
{
    // Wait for any earlier write to finish.
    while (EECR & (1U << EEPE)) {}

    // Set destination address and data.
    EEAR = addr;
    EEDR = byte;

    // Perform write operation.
    cli();
    EECR |= (1U << EEMPE);
    EECR |= (1U << EEPE);
    sei();
}
```

The intermittent one is defect 3, the missing `cli()`/`sei()`.

Why that is worse than one that never works. A write that never works is caught by the first test that reads the value back. A write that works except when an interrupt lands inside a four-cycle window works on the bench, works in review, and fails in the field at a rate that depends on how busy the interrupts happen to be — so the failure rate changes when unrelated code is added. Each individual failure looks like "it didn't save that one time", which is indistinguishable from a power cut at an unlucky moment, so the bug is easy to explain away and hard to reproduce. (*Included in the marks above; a candidate who identifies defect 3 but not why it is worse loses half a mark.*)

(b) 4 MARKS

`watchdog_init()`: the timed sequence is missing. (*1 mark*)

Switching the watchdog *on* is always allowed: a write that sets `WDE` takes effect with no unlock. What is protected, exactly as EEPROM's `EEPE` is, is clearing `WDE` and changing the prescaler bits, and for a sharper reason: the interlock exists so that runaway code cannot accidentally switch off or reconfigure the one thing meant to catch it. `WDCE` must be set together with `WDE` first, and the real configuration written within four cycles. Without the

unlock, the write enables the watchdog but the prescaler bits keep their reset value, `0000`: the watchdog runs with a 16 ms timeout instead of 1024 ms, and nothing reports a problem.

`watchdog_reset()`: petting the watchdog is not a register write at all. *(1 mark)*

`WDE` is the enable bit. Setting an already-set bit changes nothing and does not restart the countdown. The countdown is restarted by a dedicated CPU instruction, `WDR`, which no assignment to any register produces. As written, the watchdog is never pet. With the 16 ms watchdog that `watchdog_init()` did switch on, the chip resets 16 ms after every boot, in an endless boot loop; with `watchdog_init()` corrected, it resets every 1024 ms instead.

Corrected: *(1 mark)*

```
/**
 * @brief Enable the watchdog timer with the given timeout.
 *
 * @param[in] timeout Timeout to arm the watchdog with.
 */
static void watchdog_init(const watchdog_timeout_t timeout)
{
    cli();
    WDTCSR |= (1U << WDCE) | (1U << WDE);      // Unlock the timed sequence.
    WDTCSR = (1U << WDE) | (uint8_t)(timeout); // Enable the watchdog with the ↩
    ↪ given timeout.
    sei();
}

/**
 * @brief Reset the watchdog timer, restarting its countdown.
 */
static void watchdog_reset(void)
{
    __asm__("wdr");
}
```

Why it cannot be a register access, and why `__asm__`. *(1 mark)*

There is no bit anywhere whose write emits `WDR`, so no C statement produces it. The only way to place a specific machine instruction in the compiler's output is **inline assembly**, which drops the literal instruction straight into the generated code. The plain spelling `asm` is a GNU extension, recognised only under a GNU dialect (`-std=gnu11`) or GCC's default mode; this book builds with `-std=c11`, strict ISO C, where `asm` is not a keyword and the line fails to compile. `__asm__` is recognised regardless of `-std=`, which is why the book uses that spelling.

(c) 4 MARKS

The flag is `WDRF`, the Watchdog Reset Flag, in `MCUSR`. (*0.5 marks*)

The mechanism. A watchdog-caused reset sets `WDRF`, and while `WDRF` is set, `WDE` **reads back as set no matter what is written to it**. Every attempt to clear `WDE` — or to disable the watchdog by any other route — therefore does nothing at all, silently, and the watchdog stays armed and fires again one timeout later. After a watchdog reset that timeout is the shortest, 16 ms, because the reset cleared the prescaler bits, which is why the board resets many times a second. That is the boot loop the colleague is seeing: the chip resets, `WDRF` is set again by that very reset, and the disable is refused again on the next boot. (*1.5 marks*)

It is deliberate. It stops a chip that has just failed and reset from immediately switching off the safety net that caught it, which is precisely the failure mode a watchdog exists for.

The fix and why its position matters. (*1 mark*)

```
MCUSR &= ~(1U << WDRF); // Clear the flag; WDE cannot be cleared while it is set.
```

It must be the **first** line of `setup()`, unconditionally, before anything else touches `WDTCSR`. Anything in between that tries to switch the watchdog off is silently ignored, so a clear placed later does not undo those writes — they are simply lost. The watchdog must then be configured, with the timeout actually wanted, or switched off, straight after the clear and still before anything slow: after a watchdog reset it is running with that 16 ms timeout, and a `setup()` that first spends 16 ms on something else (printing one line at 9600 baud takes that long) is reset before it gets there. That is why the book's `watchdog_init()` clears `WDRF` and then configures the watchdog, and is called first thing in `setup()`. Clearing the flag unconditionally costs nothing and needs no knowledge of why the chip reset; a program that wants to know reads `MCUSR` into a variable first.

`WDE` and `WDIE`. (*1 mark*)

- **`WDE` alone — System Reset Mode.** A timeout resets the chip.
- **`WDIE` alone — Interrupt Mode.** A timeout runs `ISR(WDT_vect)` and resets nothing.
- **Both set.** The timeout fires `ISR(WDT_vect)` *and* hardware clears `WDIE` on the way in, leaving `WDE` armed. That gives exactly **one warning**: the handler gets a chance to pet the watchdog, log the near-miss or shut something down safely, and if the *next* timeout is also missed the chip resets for real.

Question 7 One driver, many instances

12 MARKS

(a)

3 MARKS

The macro approach. Each LED needs its own pair of macros, `LED1_ON/LED1_OFF`, each with a port and a bit position baked into its text. A sixth LED means a sixth copied-and-renamed pair, plus another line in every function that acts on "all the LEDs". There is nothing to iterate over, because the substitution happens before the compiler exists — a loop counter cannot select a macro. (*1 mark*)

The `gpio_t` approach. The functions do not know or care which GPIO they were handed; the port pointers and the bit live in the instance. A sixth LED is one more entry in a `gpio_t leds[]` array, and every existing loop over that array picks it up with no new code at all. (*1 mark*)

Why the ADC, UART, EEPROM and watchdog drivers are right to have no struct. The chip has exactly one of each. There is no second instance to distinguish, so there is nothing for `self` to select between and nothing to bundle; a struct would be ceremony around state that exists once. (`adc_read()` already takes a channel number as an ordinary parameter, so it was never tied to a single input in the first place.)

What warrants one: the moment a program can plausibly want *more than one* of the thing, each with its own configuration and its own state. That is true of LEDs, buttons and software timers, and false of the chip's single EEPROM. (*1 mark*)

(b)

6 MARKS

(*1.5 marks per defect: 0.5 for naming it, 0.5 for the consequence, 0.5 for the correction.*)

1. `gpio_write()` takes `gpio_t` by value. Every call copies the whole struct — three pointers and a byte — onto the stack of an 8-bit MCU with 2 KB of RAM, for nothing. The deeper problem is what it means: the function is operating on a *copy*. It happens to work only because it writes *through* the copied pointers rather than to `self`'s own fields; the day it needs to change a field (a cached direction, a reservation flag, a callback) the change is discarded at the closing brace and the caller never sees it. The book's convention is a pointer, always.

```
void gpio_write(gpio_t* self, const bool state)
```

2. `gpio_read()` reads `portx` instead of `pinx`, and is not `const`-correct. On an input pin `PORTx` is the pull-up setting, not the pin level. The function therefore returns whatever the pull-up was configured to, and never changes when the button is actually pressed — "the button is stuck" with no bad instruction anywhere. `PINx` is the register that reports the live logic level. And since the function only ever reads through `self`, its parameter should say so.

```
bool gpio_read(const gpio_t* self)
{
    return (NULL != self) ? (*self->pinx & (1U << self->pin)) : false;
}
```

3. `gpio_toggle()` uses `^=` on `PINx`. Writing a `1` to a `PINxn` bit toggles `PORTxn`; bits written as `0` are left alone. `^=` is a read-modify-write, so it first *reads* `PINx` — the live level of **every** pin on the port — flips one bit in that byte, and writes the whole thing back. Every other pin on the port that currently reads high is written as `1`, and is therefore toggled too. Meanwhile the target bit is written as `level XOR 1`: if the pin currently reads high it is written as `0`, so the one pin that was asked for **does not toggle at all**. The correct form is a plain write — no read, no window, one bit:

```
*self->pinx = (1U << self->pin);
```

4. No `NULL` checks, and no direction check. These are exported functions of a reusable driver, called from files the author cannot audit: the case where the book's own rule says a pointer check does real work rather than demonstrating a habit. A `NULL self` here does not fault cleanly — address 0 on AVR is the register file, so `*self->portx` reads a garbage pointer out of the register file and writes somewhere real. `gpio_write()` and `gpio_toggle()` should also refuse a pin that is not configured as an output, which is what §9.1.2's versions do.

```
if (NULL == self) { return; }
const bool output = (*self->ddrx & (1U << self->pin));
if (!output) { return; }
```

(c) 3 MARKS

What goes where. `gpio.h` is the public interface: the include guard, the `gpio_t` and `gpio_mode_t` type definitions, and a documented prototype for every function callers are allowed to use. `gpio.c` includes its own header, after the standard and AVR headers, then defines those same functions in the order the header declares them; anything not in the header is `static` and lives at the bottom of the file. *(1 mark)*

Why `gpio_t` has to be in the header, unlike anything in `sensor.h`. Callers of `sensor.h` only ever received a `double` back; they never named a type belonging to the module. Callers of `gpio.h` must **declare their own instances** — `gpio_t led1 = {&DDRB, &PORTB, &PINB, LED1_PIN};` — and to declare an object the compiler has to know its size and layout. A forward declaration alone would leave `gpio_t` an incomplete type, which can be pointed at but never instantiated. That is exactly the trade-off Chapter 10 revisits: hiding the fields costs the caller the ability to declare one, which is why the opaque version has to replace `gpio_init()` with an allocating `gpio_create()`. *(1 mark)*

Why the `const` differs between the two files. A top-level `const` on a pass-by-value parameter constrains only what the function body may do with its own local copy. It is not part of the function's type and cannot affect how any caller calls it, so in the prototype it says nothing and is omitted; in the definition it says something real about the body, so it is kept. (*1 mark*)

Question 8 Hiding things, and choosing at runtime 13 MARKS

(a) 4 MARKS

What it declares. That a type `struct gpio` exists, and that `gpio_t` is another name for it. It deliberately does **not** declare the fields, so in every file that sees only this header `gpio_t` is an **incomplete type**. (*1 mark*)

What a caller can do: declare, copy, store, pass and compare `gpio_t*` variables, check them against `NULL`, and hand them to the driver's functions.

What a caller cannot do: declare a `gpio_t` object, take `sizeof(gpio_t)`, put one in an array, dereference the pointer, or read or write a single field. All of those need the layout, and the layout is only in `gpio.c`. (*1 mark*)

Why `gpio_init(gpio_t* self, ...)` cannot survive. That signature puts the storage in the caller's hands: the caller declares the object and passes its address. Declaring the object needs a complete type, which is exactly what opacity removes. So the allocation has to move to the only file that knows the size — `gpio.c` — and the function becomes `gpio_create(...)`, allocating with `malloc()` and returning the pointer, or `NULL` if the allocation failed. (*1 mark*)

The invariant. `timer_t` carries an elapsed-time count that `timer_running()` and `timer_reset()` depend on, a running/stopped state, and a reserved hardware timer/counter. With public fields any file can set the elapsed count directly, skip `timer_init()` altogether, or leave the reservation table and the instance disagreeing about which hardware timer is claimed — all of which the driver's own functions then reason about incorrectly. `gpio_t` had nothing comparable: its fields are a port address and a bit position, and setting them by hand is unhelpful rather than invariant-breaking, which is why §10.1.1 calls the exposure "mostly harmless" there and not on `timer_t`. (*1 mark*)

(b) 4 MARKS

What the compiler reports: `unknown type name 'gpio_t'`, on both function-pointer members. (*0.5 marks*)

Why. `typedef struct gpio { ... } gpio_t;` introduces the name `gpio_t` only when the whole declaration finishes, at the final `;` after the closing brace. Inside the braces the struct is still being defined and that name does not exist yet. The *tag* `struct gpio`, by contrast, is in scope from the moment `struct gpio {` is seen, which is what lets a member refer to a pointer to its own struct type — the same rule a linked list's `struct node* next` relies on. (*1.5 marks*)

Corrected:

```
    /** Set state of the given GPIO. Only supported for outputs. */
    void (*write)(struct gpio* self, bool state);

    /** Read state of the given GPIO. */
    bool (*read)(const struct gpio* self);
```

Everywhere *outside* the struct, `gpio_t` remains the name to use; only these member declarations, written before their own type has a name, need the tag. (*1 mark*)

What `static` on `gpio_write()` prevents, and what it does not. (*1 mark*)

It gives the function internal linkage: no other translation unit can call it **by name**, and the symbol is not emitted for the linker, so `gpio_write(&led1, true);` in `main.c` fails to link.

It does **not** hide the data. `gpio_t`'s fields are all still public: `led1.ddrx`, `led1.pin` and `led1.write` are readable and writable by any file that includes the header. It does not even hide the function itself in any meaningful sense — its *address* is deliberately published in `led1.write` by `gpio_init()`, so a caller can call through it, copy it, or overwrite it with something else entirely. Hiding functions and hiding fields are independent techniques; `static` does only the first, and the opaque struct is what does the second.

(c) 3 MARKS

What happens: the program jumps to a garbage address on the very first call. (*1.5 marks*)

`itf` is not the first member of `gpio_impl_t`. The cast `(gpio_interface_t*)(impl)` yields a pointer to the *start* of the object, which is `ddrx`, not `itf`. Reading `self->vptr` therefore reads the first bytes of `ddrx` — the address of a hardware register, `0x0024` for `DDRB` and so on — and treats that number as a `const gpio_vtable_t*`. `self->vptr->toggle` then loads a function pointer from inside the I/O register area and calls whatever value happens to be sitting there. On AVR that lands in the middle of nothing and typically ends in a reset.

The initialisation was not skipped: `impl->itf.vptr = gpio_vptr_get_instance();` did run and did store the correct table — at an offset nobody ever reads.

The fix and the rule. (*1 mark*)

```
typedef struct gpio_impl
{
    gpio_interface_t itf; // Must be first.
    volatile uint8_t* ddrx;
    volatile uint8_t* portx;
    volatile uint8_t* pinx;
    uint8_t pin;
} gpio_impl_t;
```

C guarantees that a struct has no padding before its first member, so a pointer to the struct and a pointer to its first member hold the same address. That is what makes the cast safe in

both directions without adjusting the pointer — and it is exactly the layout a C++ compiler generates for single inheritance, where an object's address *is* its base class's address.

Why the compiler said nothing. (*0.5 marks*)

The conversion was written as an explicit cast, which is the programmer asserting that it is intended. The compiler has no way to know `gpio_impl_t` was *meant* to begin with a `gpio_interface_t`; it takes the assertion at face value and emits no diagnostic.

(d)

2 MARKS

One shared vtable. The table contains nothing instance-specific — four function addresses, identical for every `gpio_impl_t` ever created. All per-instance state lives in the concrete struct's own fields, which each function reaches by casting `self` back. So a single `static const gpio_vtable_t` inside the accessor is enough, and every instance's `itf.vptr` points at that same address: one pointer per object rather than four. (`const` does not put the table in flash on the ATmega328P: avr-gcc copies read-only data into RAM at startup unless it is marked `PROGMEM`, so the single shared table costs its 8 bytes of RAM once, not once per instance.) (*1 mark*)

Why `del()` takes `gpio_interface_t**`. So that it can `free()` the instance *and* null out the caller's own pointer in a single call — the same pointer-to-pointer idiom as Question 1(b). Given only a `gpio_interface_t*`, it could release the memory but would have to leave the caller holding a dangling pointer, ready for a second `del()` or a use-after-free through that variable. (*0.5 marks*)

What a stub makes possible. It satisfies the same interface with a `bool state` in place of registers, so anything written against `gpio_interface_t*` can be exercised on a host with no chip attached: unit tests, simulated inputs, or a third logging implementation dropped in without touching a line of the code under test. With only `gpio_impl_t`, every run needs real hardware. (*0.5 marks*)

APPENDIX G

Model Answers to Paper B

Marks are shown per part. Method carries them: a correct diagnosis with a clumsy fix is worth more than a correct fix with no diagnosis, and several later parts consume earlier answers, so an error should be followed through rather than penalised twice.

Code is marked on semantics. Missing semicolons, missing includes and misplaced braces cost nothing. The questions are in Appendix E.

The candidate had the register reference at the back of the paper. Every register and bit name, every bit *position* where one matters, and the `ISCn1:ISCn0`, `ADPS2:ADPS0` and `CSn2:CSn0` encodings were in front of them. What was deliberately withheld is the part being examined: the ordering rules, the timing rules and the formulas. So a misspelled bit costs nothing, but a candidate who cannot say which of `EEMPE` and `EEPE` comes first, or derive an encoding from bit positions the reference gave them, has not answered the question.

Where a question asks for code "in the book's style", the conventions in the rubric are marked: `snake_case`, `static` on anything private, `self` as a pointer, Yoda comparisons, braces on every `if`, fixed-width types, and `volatile` on anything shared with an ISR. Doxygen comments are marked only where the question says so.

Question 1 Saying what you mean

12 MARKS

(a)

4 MARKS

```
/**
 * @brief Enumeration of LED blink patterns.
 */
typedef enum
{
    LED_PATTERN_OFF,       ///< LED is off.
    LED_PATTERN_SLOW,      ///< Slow blink.
    LED_PATTERN_FAST,      ///< Fast blink.
    LED_PATTERN_HEARTBEAT, ///< Two short pulses, then a pause.
} led_pattern_t;
```

```c
/**
 * @brief Check whether the given pattern is a named member of led_pattern_t.
 *
 * @param[in] pattern Pattern to check.
 *
 * @return True if the pattern is valid, false otherwise.
 */
bool led_pattern_valid(const led_pattern_t pattern)
{
    switch (pattern)
    {
        case LED_PATTERN_OFF:
        case LED_PATTERN_SLOW:
        case LED_PATTERN_FAST:
        case LED_PATTERN_HEARTBEAT:
            return true;
        default:
            return false;
    }
}
```

(*2 marks. The prototype in a header drops the top-level* `const`*:* `bool led_pattern_valid(led_pattern_t pattern);` *— credit a candidate who says so. A range check* `(LED_PATTERN_HEARTBEAT >= pattern)` *also earns the marks, and a candidate who notes that it only works because the members happen to be contiguous, so it silently stops being correct the day one takes an explicit value, has answered better.*)

Three advantages over a block of `#defines`, any three of:

- **The type survives to runtime.** A debugger shows `LED_PATTERN_FAST` by name; a macro was erased by the preprocessor before there was a program to debug.
- **The compiler can help with `switch`.** Some compilers warn when a `switch` on an enumeration does not handle every named case. Nothing comparable exists for a set of macros, which are just integers by the time the compiler sees them.
- **Scope and namespacing.** An enumerator is an ordinary identifier obeying the usual scoping rules. A macro is live text substitution from its `#define` to the end of the translation unit and every file that includes it, colliding with anything of the same name anywhere.
- **The values maintain themselves.** Members count up from 0 in declaration order, so adding or reordering one does not mean renumbering a block of literals by hand.

(*1 mark*)

The property `#define` has that `const uint8_t` does not: it is a compile-time constant expression. A `const` variable in C cannot change at runtime, but the language still refuses it anywhere a *constant expression* is required — an array bound, a `case` label, a `#if`. A macro

works in all of them, and folds into the generated code, so `PORTB |= (1U << LED1)` becomes a single bit-set instruction rather than a runtime shift. (An `enum` constant has this property too, except in `#if`, which the preprocessor evaluates before any enum exists; that is why the book reaches for an enum wherever a named set fits and for a macro where a bare number is needed.) *(1 mark)*

(b) 3 MARKS

```
#define TICKS_NEEDED(timeout_ms, tick_ms) ((timeout_ms) / (tick_ms))
```

(1 mark)

The two rules. *(1 mark)*

1. **Parenthesise the whole body**, so the macro's meaning does not depend on what surrounds it at the call site. Substitution happens before precedence gets a chance to group anything.
2. **Parenthesise every use of every parameter**, so an argument that is itself an expression is grouped as the caller wrote it.

A call that goes wrong when rule 2 is broken:

```
#define TICKS_NEEDED(timeout_ms, tick_ms) (timeout_ms / tick_ms) // Parameters ↩
    ↪ unparenthesised.

TICKS_NEEDED(TIMEOUT_MS + MARGIN_MS, 4U)
// Expands to: (TIMEOUT_MS + MARGIN_MS / 4U)
// Only the margin is divided. The timeout is added to it whole. No warning.
```

(*Credit a rule-1 example instead, e.g. a body ending in* `+ 2U` *used as* `2U * TICKS_NEEDED(...)`.)

The third hazard: multiple evaluation. A parameter's text is substituted at *every* point its name appears in the body, so an argument with a side effect happens once per appearance, in an unspecified order (and for an argument like `i++`, two unsequenced changes to one variable are undefined behaviour). A real function evaluates each argument exactly once.

This macro is not exposed to it, because each parameter appears exactly once in the body. That is a property of this particular body, not a guarantee the macro mechanism offers: the moment somebody adds a second use of `timeout_ms` — a clamp, a rounding term — `TICKS_NEEDED(read_ms(), 4U)` starts calling `read_ms()` twice, and no amount of parenthesising prevents it. *(1 mark)*

(c) 3 MARKS

```c
/**
 * @brief Grow or shrink a buffer's backing storage.
 *
 * @param[in,out] self Pointer to the buffer to resize.
 * @param[in] size New size in bytes.
 *
 * @return 0 on success, 1 on failure (self is left untouched).
 */
static int buffer_resize(buffer_t* self, const size_t size)
{
    if (NULL == self) { return 1; }

    // A size of zero means "release everything and go back to empty".
    if (0U == size)
    {
        free(self->data);
        self->data = NULL;
        self->size = 0U;
        return 0;
    }

    // Capture the result in a temporary: a failed realloc() must not overwrite ↩
    ↪ self->data.
    uint8_t* copy = (uint8_t*)(realloc(self->data, size));
    if (NULL == copy) { return 1; }

    self->data = copy;
    self->size = size;
    return 0;
}
```

(*2 marks: 0.5 for the `NULL` check, 0.5 for the zero-size branch, 1 for the temporary and the failure path. `realloc(NULL, size)` behaves as `malloc(size)`, so the first call needs no special case — credit a candidate who says so.*)

The mistake in `self->data = realloc(self->data, size);`. On failure `realloc()` returns `NULL` and **leaves the original block allocated and entirely valid** — that is the contract. The assignment has just overwritten the only pointer to it. The old block is therefore leaked with no way to reach it again, the buffer's contents are gone, and `self->size` now describes a null pointer, so the next read through it is a null dereference rather than a clean failure. Capturing into a temporary and only committing on success costs one local variable and loses nothing. (*1 mark*)

(d)

2 MARKS

```c
#ifndef DRIVER_SERIAL_H_
#define DRIVER_SERIAL_H_

/* ... declarations ... */

#endif /* DRIVER_SERIAL_H_ */
```

What it is for. A header can be reached more than once in a single translation unit — included directly and pulled in again through another header — and the compiler would then see the same declarations twice, which is an error for anything that cannot be redeclared. The guard makes the second and later inclusions expand to nothing.

The convention is the file name in `UPPER_CASE` with `.` replaced by `_` and a trailing `_`, qualified by its directory where it has one (`driver/serial.h` → `DRIVER_SERIAL_H_`), chosen so the macro is unlikely to collide with any other in the program. `#pragma once` does the same job in one line and is widely supported, but is not part of the C standard, so the traditional guard is the portable choice. *(1 mark)*

The ordering convention inside a `.c` file: the `#includes` first, then forward declarations of the file's own `static` helpers, then the definitions of the public functions in the order the header declares them, and the `static` helpers' own definitions at the bottom.

The reason: a reader opening the file meets the module's public API first and only descends into private implementation detail if they keep scrolling. The forward declarations at the top are what allow that ordering without the compiler complaining about a call to a function it has not seen yet. *(1 mark)*

Question 2 Straight to the registers

12 MARKS

(a)

5 MARKS

```c
#define F_CPU 16000000UL // CPU frequency in Hz.

#include <stdint.h>

#include <avr/io.h>
#include <util/delay.h>

/** GPIO pins. */
#define LED1 0U // D8  -> PORTB0.
#define LED2 1U // D9  -> PORTB1.
#define LED3 2U // D10 -> PORTB2.
```

```c
#define BTN1 4U // D12 -> PORTB4.

/** Bit mask covering every LED. */
#define LED_MASK ((1U << LED1) | (1U << LED2) | (1U << LED3))

/** GPIO operations. */
#define LED_ON(led) PORTB |= (1U << (led))  // Enable the given LED.
#define LEDS_OFF PORTB &= ~LED_MASK         // Disable every LED.
#define BTN1_PRESSED (PINB & (1U << BTN1))  // High if BTN1 is pressed, low ↩
    ↪ otherwise.

/** Time parameters. */
#define TICK_1MS 1U       // Generate a 1 ms tick in delay_ms().
#define STEP_TIME_MS 150U // Time each LED stays lit, in ms.

/** Number of LEDs taking part in the chase. */
#define LED_COUNT 3U

/** Bit positions of the LEDs, in the order they are lit. */
static const uint8_t leds[LED_COUNT] = {LED1, LED2, LED3};

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 - LED3 as outputs. BTN1 needs no configuration: it is an ↩
    ↪ input
    // out of reset, and its external pull-down defines the idle level.
    DDRB = LED_MASK;
}

/**
 * @brief Generate delay.
 *
 * @param[in] ms Delay duration in ms.
 */
static void delay_ms(const uint16_t ms)
{
    for (uint16_t i = 0U; i < ms; ++i)
    {
        _delay_ms(TICK_1MS);
    }
}
```

```
/**
 * @brief Light a single LED for one step of the chase.
 *
 * @param[in] index Index of the LED to light.
 */
static void run_chase_step(const uint8_t index)
{
    LEDS_OFF;
    LED_ON(leds[index]);
    delay_ms(STEP_TIME_MS);
}

/**
 * @brief Application entry point.
 *
 * @return 0 on termination of the program (should never occur).
 */
int main(void)
{
    uint8_t index = 0U;
    setup();

    while (1)
    {
        // Advance the chase while BTN1 is held, disable every LED otherwise.
        if (BTN1_PRESSED)
        {
            run_chase_step(index);
            if (LED_COUNT <= ++index) { index = 0U; }
        }
        else
        {
            LEDS_OFF;
            index = 0U;
        }
    }
    return 0;
}
```

(5 marks, distributed: 1 for the correct port bit positions — D8/D9/D10 are `PORTB0`/`PORTB1`/`PORTB2` and D12 is `PORTB4`, not 8/9/10/12; 1 for `F_CPU` defined before `<util/delay.h>` and the `delay_ms()` loop; 1 for reading the button from `PINB`; 1 for `LEDS_OFF`

clearing with ~ and leaving the rest of the port alone; 1 for the chase advancing and resetting correctly.)

Do not deduct for enabling BTN1's internal pull-up the way Chapter 2's examples do; part (*c*) *is about why that is harmless only in practice.*

Worth noting in a strong answer: `run_chase_step()` blocks for the full 150 ms, so a release is only noticed at the top of the next iteration — up to 150 ms late. That is the same blocking limitation `_delay_ms()` carries throughout Chapters 2–4, and exactly what Chapter 5's timers exist to remove. A candidate who flags it has understood the program rather than only written it.

A per-LED macro family (`LED1_ON`, `LED2_ON`, …) *and a hand-written three-way* `switch` *is also a correct answer and loses nothing. It is, however, precisely the pattern Chapter 9 is about replacing.*

(b) 4 MARKS

Operation	Expression
Set bit `n`	`reg |= (1U << n);`
Clear bit `n`	`reg &= ~(1U << n);`
Test bit `n`	`(reg & (1U << n))`
Toggle bit `n` via `PORTx`	`PORTB ^= (1U << n);`
Toggle bit `n` atomically	`PINB = (1U << n);`

(*2.5 marks; 0.5 each*)

`DDRB = (1U << 3U);` replaces the whole register. Bit 3 becomes an output and **every other pin on port B is forced back to input**, whatever it had been configured as. `DDRB |= (1U << 3U);` sets bit 3 and leaves the other seven exactly as they were. (*0.5 marks*)

When each is right. (*1 mark*)

- `=` when the line is establishing the whole port from a known state in one place — the single configuration line in `setup()` that declares every direction at once. It is also a plain store rather than a read-modify-write, so it is one instruction and cannot be torn by an interrupt.
- `|=` whenever some other part of the program may already have configured a bit that must not be disturbed: anything running after `setup()`, and anything inside a driver that owns one pin and knows nothing about the rest of the port. A driver that used `=` would silently deconfigure every other peripheral sharing its port.

(c) 3 MARKS

`PORTx` on an output pin sets the level driven onto the pin: `1` drives it high, `0` drives it low.

`PORTx` on an input pin does something else entirely: it enables (`1`) or disables (`0`) the pin's **internal pull-up resistor**. It does not read anything and it does not drive the pin.

The pin's actual level is read from `PINx`, never from `PORTx`. Reading `PINB` returns the real logic level present on each of port B's pins, whatever their direction. *(1.5 marks)*

The circuit of §2.1.4. One leg of the button goes to 5 V; the other goes to D13 and, through a 10 kΩ resistor, to ground — an **external pull-down**. So the pin reads `0` when idle and `1` while the button is held, the opposite way round from the internal-pull-up convention.

`PORTB = (1U << BTN1);` enables the AVR's internal pull-up on that pin anyway, and the pull-up and the external pull-down now form a voltage divider. It is **harmless in practice**: with the pull-up's typical value of about 35 kΩ the idle pin sits at $5\,\text{V} \times 10/(10+35) \approx 1.1\,\text{V}$, below the $0.3 \times 5\,\text{V} = 1.5\,\text{V}$ under which a pin is guaranteed to read `0`, so the reading is unchanged. It is **not guaranteed**, though: the datasheet only promises 20–50 kΩ, and at 20 kΩ the idle pin rises to $5\,\text{V} \times 10/30 \approx 1.7\,\text{V}$, where the reading is undefined. The write is there so that `PORTx`'s input-mode behaviour is exercised somewhere visible, not because this particular circuit needs it — a candidate who says the circuit would work identically without that line, and that a design of their own should not enable a pull-up against an external pull-down, is right. *(1.5 marks)*

Question 3 Reacting instead of polling 12 MARKS

(a) 5 MARKS

D3 is `PORTD3`, which is **INT1**. D10 is `PORTB2`.

```
#include <avr/interrupt.h>
#include <avr/io.h>

/** GPIO pins. */
#define LED1 2U // D10 -> PORTB2.

/** GPIO operations. */
#define LED1_TOGGLE PINB = (1U << LED1) // Toggle LED1 atomically.

/**
 * @brief Set up system.
 */
static void setup(void)
{
    // Configure LED1 as output.
```

```
    DDRB = (1U << LED1);

    // Enable rising edge interrupt on INT1 (D3) => a button press causes an ↩
    ↪ interrupt.
    EICRA = (1U << ISC10) | (1U << ISC11);
    EIMSK = (1U << INT1);

    // Enable interrupts globally.
    sei();
}

/**
 * @brief Toggle LED1 on button press.
 */
ISR(INT1_vect) { LED1_TOGGLE; }
```

(3.5 marks: 1 for `EICRA` with both `ISC1x` bits, 0.5 for `EIMSK`, 0.5 for `sei()`, 0.5 for `DDRB`, 1 for the vector name `INT1_vect`.)

Which edge, and why: the rising edge, `ISC11:ISC10 = 11`. The external pull-down holds the pin low when idle and the press drives it high, so the low→high transition *is* the press-down. Selecting the falling edge (`10`) would give a handler that fires on release instead — the same program, one press out of step. *(1 mark)*

Why the toggle is written to `PINx`. Writing a `1` to a `PINxn` bit toggles the corresponding `PORTxn` bit, and bits written as `0` are left alone. It is a **pure write**: no read is involved, so there is no window in which another interrupt's change to a different bit of `PORTB` could be read, ignored and then overwritten. `PORTB ^= (1U << LED1);` is a read-modify-write and has exactly that window, which is the requirement the question asked for. *(0.5 marks)*

D3 itself needs no register write here: it is an input out of reset, and the external pull-down already defines the idle level. Leaving the internal pull-up off is the right choice — enabling it would fight the divider for no benefit. Credit a candidate who says so; do not penalise one who writes `PORTD &= ~(1U << 3U);` for explicitness.

(b)

4 MARKS

A2 is `PORTC2`, which has no external-interrupt hardware, so it needs a **pin change interrupt** on port C:

Enable bit in `PCICR`	`PCIE1` → `PCICR = (1U << PCIE1);`
Mask register	`PCMSK1`
Bit written into it	bit 2 → `PCMSK1 = (1U << 2U);`
Vector	`ISR(PCINT1_vect)`

The bit written into `PCMSKn` is the pin's **own bit position within its port**. The global `PCINTx` number for this pin is 10 (`PORTCx + 8`), but that number is only for looking the pin up in the datasheet — `PCMSK1 = (1U << PCINT10)` sets the same bit as `(1U << 2U)`.

`EICRA` and `EIMSK` disappear entirely; `sei()` is still required. (*2 marks*)

Two things the handler must now do, both forced rather than optional:

1. **Filter the edge itself.** Pin change interrupts have no edge-select register — there is no `EICRA` equivalent — and fire on **both** edges, press and release. To act only on press-down, the handler must read the pin's actual level and return otherwise: `if (PINC & (1U << BTN1)) { ... }`. The filtering has moved out of hardware and into software because the hardware simply does not offer it.
2. **Work out which pin changed**, if more than one PCI pin on port C is ever enabled. The vector is per-**port**, not per-pin, and nothing in the call says which pin fired it. The usual technique is to keep a previous-`PINC` byte and XOR it against the current read to find the bit that flipped. Again forced: one vector genuinely serves the whole port.

(*2 marks, 1 each*)

(c) 3 MARKS

The rule. An ISR does the minimum needed to *record* that the event happened — flip a pin, set a flag, update one small piece of state — and returns. Everything else belongs in `main()`'s loop. The reason is that you do not control when an ISR fires, and for as long as one is running every other interrupt on the chip is held off; a slow handler therefore delays events it has nothing to do with. (*1 mark*)

Two calls that must never appear in an ISR, any two of:

- **`serial_print()`** — blocks on `UDRE0` once per byte. At 9600 baud a 15-byte line is about **15.6 ms** of held-off interrupts, including the debounce timer the button itself depends on.
- **`eeprom_write()`** — blocks on `EEPE` for roughly **3.4 ms per byte**, and carries its own `cli()`/`sei()` pair, which inside an ISR re-enables interrupts the handler was entered with disabled.
- **`delay_ms()` / `_delay_ms()`** — a busy-wait, by construction; whatever duration was asked for is spent with everything else blocked.

(*1 mark*)

The pattern. (*1 mark*)

```
/** Flags that a BTN1 event has occurred. */
static volatile bool btn1_event = false;

ISR(PCINT0_vect)
{
    if (BTN1_PRESSED) { btn1_event = true; }
}
```

```
// In main()'s loop:
if (btn1_event)
{
    serial_print("BTN1 pressed!");
    btn1_event = false;
}
```

What `volatile` is doing: the flag is set inside the ISR and read (and cleared) inside `main()`'s loop, and the compiler cannot see that an interrupt exists. Without `volatile` it is entitled to assume nothing in the loop changes `btn1_event`, load it once into a register before the loop, and test that stale `false` on every iteration forever. `volatile` forces every read back to memory.

Question 4 Reading the world, and keeping time — 13 MARKS

(a) — 4 MARKS

```
/**
 * @brief Read and return a 10-bit ADC conversion result.
 *
 * @param[in] channel ADC channel to read.
 *
 * @return Conversion result (0-1023).
 */
static uint16_t adc_read(const uint8_t channel)
{
    // Use AVCC as the voltage reference and select the ADC channel to read.
    ADMUX = (1U << REFS0) | channel;

    // Enable the ADC, start a conversion, use a prescaler of 128.
    ADCSRA = (1U << ADEN) | (1U << ADSC) | (1U << ADPS0) | (1U << ADPS1) | (1U << ↩
    ↪ ADPS2);

    // Wait for the conversion to complete.
    while (0U == (ADCSRA & (1U << ADIF))) {}

    // Clear the ADC interrupt flag for the next conversion, then return the ↩
    ↪ result.
    ADCSRA = (1U << ADIF);
    return ADC;
}
```

(*2.5 marks*)

What each write is for. (*1 mark*)

- **`ADMUX`** — `REFS0` (with `REFS1` clear) selects `AVCC`, the chip's own 5 V supply, as the voltage reference; the low four bits select which input channel is connected to the converter. Leaving both `REFS` bits clear would select the external `AREF` pin, which nothing drives on this board.
- **`ADCSRA`, first write** — `ADEN` powers the converter on, `ADSC` starts a single conversion, and `ADPS2:0 = 111` divides the CPU clock by 128 to produce the ADC's own clock.
- **The `ADIF` poll** — a conversion is not instantaneous; the result registers are not valid until the hardware sets the flag.
- **`ADCSRA = (1U << ADIF);`** — `ADIF` is cleared by writing a 1 back to it, readying it for the next conversion.
- **`return ADC;`** — the `ADC` macro combines `ADCL` and `ADCH` into the full 10-bit value, reading them in the order that guarantees both halves come from the same conversion.

The ÷128 prescaler. The successive-approximation hardware needs a clock in the region of 50–200 kHz for full 10-bit accuracy. 16 MHz ÷ 128 = **125 kHz**, comfortably inside that window. Without any prescaler bits, `ADPS2:0 = 000` selects a division factor of 2, i.e. 8 MHz — about forty times too fast. The sample-and-hold and the comparator never settle, so the low bits of every result are noise: readings that look plausible, are not repeatable, and get quietly worse as the source impedance rises. (*0.5 marks*)

(b)

5 MARKS

1. Tick and overflow periods. (*1 mark*)

```
tick period = 1024 / 16,000,000 s = 64 us = 0.064 ms
Timer 0 overflow = 256 ticks = 16.384 ms
```

2. Timer 1, CTC, 250 ms. (*2 marks*)

```
ticks needed = 250 / 0.064 = 3906.25  ->  3906
OCR1A counts 0 up to its value, so reaching it takes (OCR1A + 1) ticks:
OCR1A = 3906 - 1 = 3905
check: 3906 * 0.064 = 249.984 ms
```

```
TCCR1B = (1U << CS10) | (1U << CS12) | (1U << WGM12); // Prescaler 1024, CTC mode.
OCR1A  = 3905U;                                      // 250 ms period.
TIMSK1 = (1U << OCIE1A);                             // Enable the compare-match ↪
    ↪ interrupt.
```

Vector: `ISR(TIMER1_COMPA_vect)`. `sei()` is still needed.

3. Timer 0, Normal Mode, 250 ms. (*1 mark*)

```
250 / 16.384 = 15.259  ->  15 overflows
15 * 16.384 = 245.76 ms
error = (245.76 - 250) / 250 = -1.70 %
```

(*A candidate who rounds up to 16 overflows — 262.144 ms, +4.9 % — has applied a defensible rule and should lose at most half a mark, provided the error is computed for the number actually chosen.*)

4. Why Timer 0 cannot reach 250 ms directly. (*1 mark*)

Timer 0 is 8-bit: its counter and its compare register both stop at 255. At its slowest available prescaler, ÷1024, one full count is $256 \times 0.064 = \mathbf{16.384\ ms}$, and there is no larger divisor on that timer. 250 ms is over fifteen times that, so no combination of prescaler and compare value reaches it in a single hardware period — the overflows have to be counted in software. Timer 1's 16-bit range is exactly what removes the need, which is why CTC on Timer 1 hits the period directly.

(c) 4 MARKS

```c
/**
 * @brief Run a single PWM cycle.
 */
static void run_pwm_cycle(void)
{
    // Read the ADC and compute the duty cycle.
    const uint16_t adc_val = adc_read(POT1);
    const float duty_cycle = (float)(adc_val) / ADC_MAX;

    // Compute the PWM durations from the duty cycle, rounded to the nearest ↩
    ↪ millisecond.
    const uint8_t on_ms  = (uint8_t)(duty_cycle * PWM_PERIOD_MS + 0.5F);
    const uint8_t off_ms = PWM_PERIOD_MS - on_ms;

    // Enable LED1 for the active duration, disable it for the rest of the period.
    LED1_ON;
    delay_ms(on_ms);
    LED1_OFF;
    delay_ms(off_ms);
}
```

(*2 marks. The cast on the numerator is load-bearing: without it,* `adc_val / ADC_MAX` *is integer division and yields 0 for every reading below 1023. A candidate who writes* `(adc_val * PWM_PERIOD_MS) / ADC_MAX` *in integer arithmetic instead is also correct, and better on this target, provided they note that the product must not overflow 16 bits.*)

Why `+ 0.5F`. The cast to `uint8_t` truncates towards zero. Adding a half before it turns truncation into round-to-nearest. Without it, a duty cycle of 0.99 gives `9.9` → `9`: the LED reaches full brightness only at a reading of exactly 1023, and the entire scale is biased low by up to one millisecond in ten — a systematic error, not a rounding one. (*1 mark*)

What it costs, and what replaces it. `delay_ms()` is built on `_delay_ms()`, which busy-waits. The routine therefore blocks for the whole 10 ms period, every period, so the CPU is doing nothing else for 100 % of the time — no button polling, no other work — and only interrupts still get through. Chapter 5's timers are the way out: a hardware timer counts on its own and raises an interrupt when a period has passed, so the pin can be switched from the handler and `main()`'s loop is free in between instead of spinning inside `_delay_ms()`. (The timers can also generate PWM entirely in hardware, which the book does not cover.) (*1 mark*)

Question 5 A channel to the host

13 MARKS

(a)

5 MARKS

```c
/**
 * @file UART driver.
 */
#ifndef DRIVER_SERIAL_H_
#define DRIVER_SERIAL_H_

#include <stdint.h>

/**
 * @brief Initialize UART with a 9600 bps baud rate.
 */
void serial_init(void);

/**
 * @brief Transmit text over UART.
 *
 * @param[in] text Text to transmit. Ignored if NULL.
 */
void serial_print(const char* text);

/**
 * @brief Transmit a buffer of arbitrary bytes over UART.
 *
 * @param[in] data Buffer to transmit. Ignored if NULL.
 * @param[in] size Number of bytes to transmit.
 */
```

```
void serial_write(const uint8_t* data, uint16_t size);

#endif /* DRIVER_SERIAL_H_ */
```

(*3 marks: 1 for the guard and its naming, 1 for the three prototypes with correct types, 1 for Doxygen in the imperative mood documenting every parameter.*)

Which functions belong where. The three above are the module's public API and must have external linkage: `main.c` calls them and the linker has to resolve them. `serial_write_byte()` is not among them — it is an implementation detail that `serial_print()` and `serial_write()` are both built on, and no caller has any business poking a single byte into `UDR0` behind the driver's back. It should be `static` in `serial.c`, which keeps it out of the symbol table, prevents a name collision with anything else in the program, and lets the compiler inline it into its two call sites. (*1 mark*)

Why the `const` differs between prototype and definition. A **top-level** `const` on a pass-by-value parameter — `uint16_t size` — constrains only what the function body may do with its own local copy. It is not part of the function's type and cannot affect any caller, so it carries no information in the prototype and is omitted there; in the definition it says something real about the body, so it is kept.

`const char* text` is **not** that case and must not be confused with it. That `const` qualifies the *pointee*, not the parameter: it is part of the type, it is a genuine promise to the caller that the function will not write through the pointer, and it belongs in both files. Dropping it from the prototype would change what the function accepts. (*1 mark*)

(b)

4 MARKS

```
/**
 * @brief Transmit a single byte over UART, blocking until the data register is ↩
    ↪ free.
 *
 * @param[in] byte Byte to transmit.
 */
static void serial_write_byte(const uint8_t byte)
{
    // Wait until the transmit buffer is ready for a new byte.
    while (0U == (UCSR0A & (1U << UDRE0))) {}

    // Place the new byte in the transmit data register.
    UDR0 = byte;
}

/**
 * @brief Transmit text over UART.
```

```
 *
 * @param[in] text Text to transmit.
 */
void serial_print(const char* text)
{
    // Terminate if the text pointer is NULL.
    if (NULL == text) { return; }

    // Transmit each character one by one.
    for (uint16_t i = 0U; text[i]; ++i)
    {
        serial_write_byte((uint8_t)(text[i]));
    }
    serial_write_byte('\r');
    serial_write_byte('\n');
}
```

(*2 marks: 1 each, with the loop terminating on the string's* `'\0'` *rather than a fixed length, and the* `NULL` *check present because this is an exported function.*)

What `UDRE0` signals. USART Data Register Empty: it is set when `UDR0` is free to accept a new byte, which happens as soon as the previous byte has been moved out into the shift register — not when transmission has finished. (*0.5 marks*)

Why the wait comes first. The flag describes whether the register is ready *now*, so it has to be consulted before the write it guards. Waiting afterwards means the very first byte is written unconditionally, into a register that may still hold an untransmitted byte, and from then on each wait is one step out of phase with the write it was supposed to protect. The order is not a stylistic choice. (*0.5 marks*)

`sprintf()`. It formats into a caller-supplied buffer exactly the way `printf()` formats to a terminal, so numbers can be mixed into text and the result handed to `serial_print()` as an ordinary string — the whole reason the UART is useful for anything beyond fixed messages.

The caveat: AVR's default `sprintf()` frequently has floating-point formatting disabled, or available only by linking a larger variant that costs flash the device may not have. `%f` then prints nothing useful with no diagnostic. Prefer integers — scale a reading to hundredths and print the two parts separately — unless float support has been confirmed in the toolchain. (*1 mark*)

(c) 4 MARKS

Where the calls go: both in `main()`'s loop. (*1 mark*)

They cannot go where the event is detected — inside the pin-change ISR or inside the one-second timer's ISR — because `serial_print()` blocks byte-by-byte on `UDRE0`. A short line at 9600 baud is 15–20 ms with every other interrupt on the chip held off, including the debounce timer the button depends on, so presses arriving during a report would simply be

lost. `sprintf()` is not much better: it is comparatively slow and takes a fair amount of stack, neither of which belongs in a handler that can fire at any moment.

The pattern. *(2 marks)*

```
/** Flags that a report is due. */
static volatile bool report_due = false;

/** Flags that a BTN1 event has occurred. */
static volatile bool btn1_event = false;

ISR(TIMER1_COMPA_vect) { report_due = true; }                 // Fires once a ↩
    ↪ second.
ISR(PCINT0_vect)       { if (BTN1_PRESSED) { btn1_event = true; } }

int main(void)
{
    setup();

    while (1)
    {
        // Report the current ADC reading once a second.
        if (report_due)
        {
            char msg[MSG_LEN] = {'\0'};
            sprintf(msg, "ADC: %u", adc_read(POT1));
            serial_print(msg);
            report_due = false;
        }

        // Handle a button press, if one occurred.
        if (btn1_event)
        {
            // ...
            btn1_event = false;
        }
    }
    return 0;
}
```

What `volatile` is doing: each flag is set inside an ISR and read (and cleared) inside `main()`'s loop. The compiler cannot see that interrupts exist, so without `volatile` it is entitled to load each flag once into a register before the loop and test that stale `false` forever. The program then compiles cleanly, works at `-O0`, and stops responding the moment optimisation is turned up.

`read()`'s `timeout_ms`, and its return values. *(1 mark)*

Without a timeout, a receive blocks until the requested number of bytes has arrived. A sender that goes quiet — unplugged, crashed, or simply not talking yet — hangs the entire program, and with a watchdog armed the chip reboots instead of continuing. `timeout_ms` bounds the wait: non-zero gives up after roughly that many milliseconds, pacing itself one millisecond at a time; `0` deliberately keeps the old block-until-done behaviour for callers that want it.

The return value distinguishes three outcomes:

- **`-1`** — a bad call: a `NULL` buffer, or `buflen == 0`.
- **a count below `buflen`** — the timeout elapsed first; that many bytes did arrive.
- **`buflen`** — the buffer filled completely.

Question 6 Surviving a power cut, and a hang

13 MARKS

(a)

5 MARKS

```
/**
 * @brief Write a byte to EEPROM at the given address.
 *
 * @param[in] addr EEPROM address to write to (0-1023).
 * @param[in] byte Byte to write.
 */
static void eeprom_write_byte(const uint16_t addr, const uint8_t byte)
{
    // Wait for any earlier write to finish.
    while (EECR & (1U << EEPE)) {}

    // Set destination address and data.
    EEAR = addr;
    EEDR = byte;

    // Perform write operation.
    cli();
    EECR |= (1U << EEMPE);
    EECR |= (1U << EEPE);
    sei();
}

/**
 * @brief Read a byte from EEPROM at the given address.
 *
```

```
 * @param[in] addr EEPROM address to read from (0-1023).
 *
 * @return Byte stored at that address.
 */
static uint8_t eeprom_read_byte(const uint16_t addr)
{
    // Wait for any earlier write to finish.
    while (EECR & (1U << EEPE)) {}

    // Set the address to read from.
    EEAR = addr;

    // Start the read operation.
    EECR |= (1U << EERE);

    // Return the byte stored at the given address.
    return EEDR;
}
```

(*2 marks, 1 each*)

The loop on `EEPE` at the top. `EEPE` stays set for the several milliseconds an EEPROM write takes. Loading a new address and data while a write is still in progress either loses the new write or disturbs the one already under way, so the function's first act is to establish that the peripheral is free. It is the same "wait for the peripheral" pattern as `ADIF` in Chapter 4 and `UDRE0` in Chapter 6. (*1 mark*)

The order of `EEMPE` and `EEPE`. `EEMPE` is not a data bit; it is a **safety interlock** that arms the write, and `EEPE` must follow it within four CPU cycles. In the other order, `EEPE` is set with nothing armed and does nothing at all, and the subsequent `EEMPE` arms an interlock that is never used — a function that silently writes nothing. (*1 mark*)

The `cli()`/`sei()` pair. Four CPU cycles is a handful of instructions. An interrupt taken between the two writes pushes `EEPE` outside the window and the write silently does not happen. Interrupts are therefore disabled for exactly those two instructions and re-enabled immediately after — the shortest possible critical section. (*0.5 marks*)

Why the read needs no equivalent. There is no timed sequence on the read side. `EERE` starts the read on its own: no interlock bit to satisfy, no four-cycle deadline, and therefore nothing for an interrupt to break. The read still waits on `EEPE` first, but for the other reason — the peripheral must be free before a new address is loaded. (*0.5 marks*)

(b) 4 MARKS

```
/**
 * @brief Enumeration of watchdog timeouts.
 */
typedef enum
{
    WATCHDOG_TIMEOUT_16MS,        ///< 16 ms timeout.
    WATCHDOG_TIMEOUT_32MS,        ///< 32 ms timeout.
    WATCHDOG_TIMEOUT_64MS,        ///< 64 ms timeout.
    WATCHDOG_TIMEOUT_128MS,       ///< 128 ms timeout.
    WATCHDOG_TIMEOUT_256MS,       ///< 256 ms timeout.
    WATCHDOG_TIMEOUT_512MS,       ///< 512 ms timeout.
    WATCHDOG_TIMEOUT_1024MS,      ///< 1024 ms timeout.
    WATCHDOG_TIMEOUT_2048MS,      ///< 2048 ms timeout.
    WATCHDOG_TIMEOUT_4096MS = 32, ///< 4096 ms timeout.
    WATCHDOG_TIMEOUT_8192MS = 33, ///< 8192 ms timeout.
} watchdog_timeout_t;

/**
 * @brief Enable the watchdog timer with the given timeout.
 *
 * @param[in] timeout Timeout to arm the watchdog with.
 */
static void watchdog_init(const watchdog_timeout_t timeout)
{
    // Clear the reset flag before anything else touches the watchdog.
    MCUSR &= ~(1U << WDRF);

    cli();
    WDTCSR |= (1U << WDCE) | (1U << WDE);      // Unlock the timed sequence.
    WDTCSR = (1U << WDE) | (uint8_t)(timeout); // Enable the watchdog with the ↩
    ↪ given timeout.
    sei();
}
```

(2 marks: 1 for the enum, 1 for the function with the `WDRF` clear, its unlock and its `cli()`/`sei()`. `WDRF` comes first because `WDE` cannot be cleared while it is set, and `watchdog_init()` is called first in `setup()`: after a watchdog reset the watchdog is still running, at its 16 ms timeout.)

Why eight members can default and two cannot. The enumerator's value is written straight into `WDTCSR`'s prescaler bits, so it has to be the register encoding, not an index. `WDP2`–`WDP0` occupy contiguous bits 2–0 of `WDTCSR`, so the first eight timeouts, 16 ms to 2048 ms, encode as exactly 0–7 — which is what an `enum` counting up in declaration order already produces for free. `WDP3`, needed for 4096 ms and 8192 ms, does **not** sit next to them: it lives

at bit 5. Those two settings therefore encode as `1 << 5` = **32** and `(1 << 5) | (1 << 0)` = **33**, values no automatic counting would ever produce, so they must be spelled out. The payoff is that a `watchdog_timeout_t` can be cast straight to `uint8_t` and written to the register with every bit already in the right place. *(1 mark)*

*The register reference gives `WDTCSR`'s bit positions, so 32 and 33 are meant to be **derived** here, not recalled. A candidate who reproduces the two numbers without being able to say where they come from has copied §8.1.2; one who derives them from `WDP3` sitting at bit 5 has understood the encoding, and should be credited even if they misremember which timeout is which.*

What the `WDCE` write is for. It is the unlock half of a timed sequence, exactly like EEPROM's `EEMPE`, but for a sharper reason: it exists so that runaway code cannot accidentally switch off the one mechanism meant to catch it. Setting `WDCE` and `WDE` together opens a four-cycle window in which `WDE` and the prescaler bits may actually be changed. *(0.5 marks)*

If the second write comes too late. The hardware closes the window — clearing `WDCE` after four cycles — and the write is simply ignored. There is no error and no diagnostic; the watchdog keeps whatever configuration it had. After a power-on reset that means disabled, so the program looks exactly like a program with no watchdog in it; after a watchdog reset it means still running at 16 ms, so the chip resets again and again. That is why the `cli()`/`sei()` pair around the sequence is not optional. *(0.5 marks)*

(c) 4 MARKS

(0.5 marks per value, plus 0.5 for quantifying case 2; 1.5 marks for the watchdog part.)

1. Calibration offset written once at manufacture — yes, this is what EEPROM is for. One write in the life of the device, a value that must survive power loss, and no other place to keep it. Endurance and write time are both irrelevant at a rate of one.

2. Live ADC reading sampled at 100 Hz — no, badly wrong, for two independent reasons.

- **Endurance.** A cell tolerates roughly 100,000 write cycles. At 100 writes a second that is `100,000 / 100 = 1000` seconds — **under 17 minutes** before that address stops reliably holding data. Permanently.
- **Time.** Each write blocks for about 3.4 ms, so 100 writes a second means 340 ms out of every second spent inside the write loop: a third of the CPU, doing nothing, forever.

And the value does not need to survive a reset in the first place — a live reading is by definition current, and belongs in RAM.

3. On/off state toggled by a user button — yes, with one condition. It must be written *when it changes*, not once per iteration of `main()`'s loop. A human presses a button a few times a second at most and a few thousand times over the device's life, comfortably inside the endurance budget, and surviving a reset is the entire point of storing it. This is exactly the book's own demo (§7.1.4).

4. Boot counter incremented on every reset — yes, with a caveat worth knowing. Resets are rare, so the write rate is fine. But the endurance ceiling now translates directly into a device lifetime of about **100,000 reboots**, after which the counter's address is worn out. If that number is ever likely to be approached, spread the count across several addresses rather than hammering one.

Where the watchdog reset belongs. As the **first** statement of `main()`'s loop, on every iteration, unconditionally — not inside an `if`, and not inside a subroutine that a hang could skip. Petting it only when work happened defeats the purpose: a loop spinning without making progress would still be petting it happily. *(1 mark)*

How to choose the timeout. Comfortably longer than the slowest *normal* iteration of the loop it is protecting, worst case included — an EEPROM write of several milliseconds, a full serial message of tens, the longest burst of interrupts — with margin on top, so a healthy program never trips it. But not so long that a genuine hang goes unnoticed for an unacceptable time. The book's 1024 ms against a loop that normally completes in microseconds is generous on the first count and still fast enough on the second. *(0.5 marks)*

Question 7 A driver that scales

12 MARKS

(a)

6 MARKS

```c
/**
 * @file GPIO driver.
 */
#ifndef DRIVER_GPIO_H_
#define DRIVER_GPIO_H_

#include <stdbool.h>
#include <stdint.h>

/**
 * @brief GPIO driver structure.
 */
typedef struct
{
    /** Pointer to the data direction register associated with the GPIO. */
    volatile uint8_t* ddrx;

    /** Pointer to the output register associated with the GPIO. */
    volatile uint8_t* portx;

    /** Pointer to the input register associated with the GPIO. */
```

```
    volatile uint8_t* pinx;

    /** Bit position of the GPIO pin within the associated port. */
    uint8_t pin;
} gpio_t;

/**
 * @brief Enumeration of GPIO modes.
 */
typedef enum
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/**
 * @brief Initialize GPIO.
 *
 * @param[in] self GPIO instance.
 * @param[in] pin  Digital pin to use. Must be in range [0, 13].
 * @param[in] mode GPIO mode.
 *
 * @return True on success, false if self is NULL or the pin is out of range.
 */
bool gpio_init(gpio_t* self, uint8_t pin, gpio_mode_t mode);

/**
 * @brief Set state of the given GPIO.
 *
 * @param[in] self  GPIO instance.
 * @param[in] state GPIO state (true = high, false = low).
 *
 * @note This operation is only supported for GPIOs configured as outputs.
 */
void gpio_write(gpio_t* self, bool state);

/**
 * @brief Read state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 *
 * @return GPIO state (true = high, false = low).
 */
```

```c
bool gpio_read(const gpio_t* self);

/**
 * @brief Toggle state of the given GPIO.
 *
 * @param[in] self GPIO instance.
 */
void gpio_toggle(gpio_t* self);

#endif /* DRIVER_GPIO_H_ */
```

(3 marks: 1 for the guard, the includes and the struct; 1 for `gpio_mode_t` and the prototypes with `const gpio_t*` on the reader and plain `gpio_t*` elsewhere; 1 for Doxygen on every declaration, with the top-level `const` correctly absent from the prototypes.)

```c
#define PIN_OFFSET_B 8U  // Pin offset for I/O port B.
#define GPIO_PIN_MAX 13U // Maximum GPIO pin number.

// ----------------------------------------------------------------------------
bool gpio_init(gpio_t* self, const uint8_t pin, const gpio_mode_t mode)
{
    // Check the GPIO instance, return false if invalid.
    if (NULL == self) { return false; }

    // Check the pin number, return false if out of range.
    if (GPIO_PIN_MAX < pin) { return false; }

    // Pins 0 - 7 => I/O port D, bit position = pin number.
    if (PIN_OFFSET_B > pin)
    {
        self->ddrx  = &DDRD;
        self->portx = &PORTD;
        self->pinx  = &PIND;
        self->pin   = pin;
    }
    // Pins 8 - 13 => I/O port B, bit position = pin number - 8.
    else
    {
        self->ddrx  = &DDRB;
        self->portx = &PORTB;
        self->pinx  = &PINB;
        self->pin   = pin - PIN_OFFSET_B;
    }
```

```
    // Configure GPIO mode as specified.
    switch (mode)
    {
        case GPIO_MODE_INPUT_PULLUP:
            // Configure GPIO as input with its internal pull-up enabled.
            *self->ddrx &= ~(1U << self->pin);
            *self->portx |= (1U << self->pin);
            break;
        case GPIO_MODE_OUTPUT:
            // Configure GPIO as output.
            *self->ddrx |= (1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
        default:
            // Configure GPIO as standard input.
            *self->ddrx &= ~(1U << self->pin);
            *self->portx &= ~(1U << self->pin);
            break;
    }
    return true;
}
```

*(3 marks: 1 for both guard clauses coming **before** any register write, so an out-of-range pin touches nothing, as the question required; 1 for the port mapping and the `- 8` on port B; 1 for the three modes with `|=`/`&= ~` rather than plain assignment, which would deconfigure every other pin on the port.)*

(b)

3 MARKS

Why `pin` can no longer be `const`. A `const` struct member can only be given a value by an **initializer at the point the object is declared**; it can never be assigned afterwards. In the original driver that was exactly how it arrived — `gpio_t led1 = {&DDRB, &PORTB, &PINB, 1U};` — so `const` cost nothing. In this version `gpio_init()` derives the bit position itself and writes `self->pin = pin - PIN_OFFSET_B;` to an object that already exists, which a `const` member forbids. The compiler rejects it outright: assignment of read-only member.

(1 mark)

What a `const` member permits and forbids. It permits initialisation at declaration and reading thereafter. It forbids assignment through any route, including whole-struct assignment `*self = other;`, which is why a struct with a `const` member is also awkward to keep in an array that is filled in a loop. *(0.5 marks)*

Why the two functions differ in their parameter. `gpio_read()` only ever reads through `self`, so `const gpio_t*` documents that and lets the compiler reject an accidental write. `gpio_write()` writes to the hardware through `self`'s pointers — and in any version that caches state would write to `self`'s own fields — so it cannot make that promise. *(1 mark)*

If either took `gpio_t self` by value. Every call would copy the whole struct — three pointers plus a byte, on a target with 2 KB of RAM — for no benefit. `gpio_read()` and `gpio_write()` would both still *work*, because they read and write through the copied pointers and those point at the same registers. What breaks is a function whose job is to change `self`'s own fields: `gpio_init()` would not work at all. Its entire job is to fill in the caller's instance, and every field it set would be discarded at the closing brace, leaving the caller's `gpio_t` exactly as uninitialised as before, with no diagnostic. *(0.5 marks)*

(c) 3 MARKS

```
/* In gpio.c, at file scope: */

/** Pin reservation table; bit n is set while pin n is claimed. */
static uint16_t pin_reg = 0U;
```

In `gpio_init()`, after both guard clauses and before any register is touched:

```
    // Check whether the pin is already reserved, return false if so.
    if (pin_reg & (1U << pin)) { return false; }

    // Reserve the pin.
    pin_reg |= (1U << pin);
```

Releasing needs the Arduino pin number, which the instance no longer holds — so `gpio_t` gains one more field, set alongside the rest in `gpio_init()`:

```
    /** Digital pin number the GPIO was initialized with, 0-13. */
    uint8_t pin_id;
```

```
/**
 * @brief Release a GPIO and free its reserved pin.
 *
 * @param[in] self GPIO instance.
 */
void gpio_free(gpio_t* self)
{
    // Check the GPIO instance, terminate if invalid.
    if (NULL == self) { return; }

    // Return the pin to its power-on state: input, pull-up disabled.
    *self->ddrx &= ~(1U << self->pin);
    *self->portx &= ~(1U << self->pin);
```

```
    // Release the pin so a later gpio_init() call can claim it again.
    pin_reg &= ~(1U << self->pin_id);
}
```

(*2 marks. The extra field is the trap:* `self->pin` *is the* ***bit position within the port****, not the Arduino pin number, once* `gpio_init()` *has subtracted the offset — clearing* `pin_reg` *bit* `self->pin` *would free pin 1 when the caller asked to free pin 9. Either store the original number, as above, or reconstruct it with* `self->pin + PIN_OFFSET_B` *when* `self->portx == &PORTB`*. A candidate who spots this has read their own code; award the full 2 marks without it only if everything else is complete.*)

Why the table must be `static`. At file scope `static` means **internal linkage**: the name is not emitted for the linker and no other translation unit can reach it. That makes the table the driver's private state, which is the whole point — it exists to enforce an invariant, and an invariant that any file can quietly edit is not one.

Without `static`, the symbol is exported and two things break. Any other file can write `extern uint16_t pin_reg;` and set or clear bits behind the driver's back, handing out the same pin twice or leaking a reservation. And `pin_reg` is not an unlikely name: a second global of the same name anywhere in the program collides at link time, giving either a multiple-definition error or — under the older common-symbol behaviour some toolchains still allow — one silently shared variable serving two unrelated modules. (*1 mark*)

Note that `static` *here is* ***not*** *about lifetime. A file-scope variable already lives for the whole program either way; the qualifier is doing only the linkage job. That is the distinction between its two roles, and a candidate who states it has understood the keyword rather than memorised the rule.*

Question 8 Hiding the implementation

13 MARKS

(a)

4 MARKS

```
/**
 * @file GPIO driver.
 */
#ifndef DRIVER_GPIO_H_
#define DRIVER_GPIO_H_

#include <stdbool.h>
#include <stdint.h>

/**
 * @brief Enumeration of GPIO modes.
 */
```

```c
typedef enum
{
    GPIO_MODE_INPUT,        ///< Standard GPIO input.
    GPIO_MODE_INPUT_PULLUP, ///< GPIO input with its internal pull-up enabled.
    GPIO_MODE_OUTPUT,       ///< GPIO output.
} gpio_mode_t;

/** GPIO driver structure. Incomplete type: the fields are not visible here. */
typedef struct gpio gpio_t;

/**
 * @brief Allocate and configure a new GPIO.
 *
 * @return Pointer to the new GPIO, or NULL if allocation failed.
 */
gpio_t* gpio_create(volatile uint8_t* ddrx, volatile uint8_t* portx, volatile
    ↪ uint8_t* pinx,
                    uint8_t pin, gpio_mode_t mode);

/**
 * @brief Release a GPIO and null out the caller's pointer.
 */
void gpio_delete(gpio_t** self);

void gpio_write(gpio_t* self, bool state);
bool gpio_read(const gpio_t* self);
void gpio_toggle(gpio_t* self);

#endif /* DRIVER_GPIO_H_ */
```

The full field list is replaced by the single line `typedef struct gpio gpio_t;`, and `struct gpio { ... };` moves into `gpio.c` — the only file that ever sees the layout. (*2 marks*)

The three things it takes away, and what the caller does instead. (*2 marks*)

1. **Declaring an instance.** `gpio_t led1 = {...};` needs the size and layout, and an incomplete type has neither. The caller instead calls `gpio_create(...)`, holds the returned pointer, and checks it against `NULL`.
2. **Reading or writing a field.** `led1.pin` and `led1.ddrx` no longer compile anywhere outside `gpio.c`. The caller goes through the driver's functions — which is the entire point: the code that could break the invariant is now the only code that can reach it.
3. **Owning the storage.** The caller no longer chooses where the object lives — stack, static array, member of a larger struct. The driver allocates it, and with that the caller inherits an obligation it did not have before: releasing it with `gpio_delete()`, and coping with `gpio_create()` returning `NULL` when the heap is exhausted.

(b) 4 MARKS

```c
/**
 * @brief Button structure.
 */
typedef struct
{
    /** GPIO associated with the button. */
    gpio_t* pin;

    /** Handle press events. Called from main(), never from an ISR. */
    void (*on_press)(void);
} button_t;

/**
 * @brief Check for a button press event, invoke its callback if one occurred.
 *
 * @param[in] self Button instance.
 * @param[in] edge True if a press was just detected (e.g. from a flag an ISR set).
 */
static void button_check(const button_t* self, const bool edge)
{
    const bool valid = (NULL != self) && (NULL != self->on_press);
    if (valid && edge) { self->on_press(); }
}
```

(*2 marks: 1 for the struct, 1 for* `button_check()` *checking both* `self` *and the callback before calling through it — a null function pointer called is a jump to address 0, not a diagnostic.*)

What `void (*on_press)(void);` declares. A member named `on_press` that is a **pointer to a function taking no arguments and returning `void`**. The declaration reads inside-out, and the parentheses around `*on_press` are required: `void *on_press(void);` would instead declare a member that *is* a function returning `void*`, which is not something a struct can contain.

```c
btn1.on_press = toggle_led;   // A function name without () decays to its address; no & needed.
```

```c
self->on_press();             // Calling through the pointer, written exactly like a normal call.
```

(*1 mark*)

(*Credit a candidate who cleans the syntax up with* `typedef void (*button_callback_t)(void);` *and declares the member as* `button_callback_t on_press;` *— provided they can still say what the underlying declaration means.*)

Why `button_check()` is better for never naming the LED. It depends only on the callback's **signature**, not on what the callback does. The same `button_check()` therefore serves a button that toggles an LED, one that sends a UART message, one that starts a timer and one that does nothing at all, with no edit and no recompilation of the driver. The caller decides the behaviour; the driver stays a driver, and it can be tested without an LED existing. That is the whole of what a callback buys, and it is the hand-written version of what a virtual function does in a language that has them. (*1 mark*)

(c) 5 MARKS

```c
/**
 * @brief In-memory GPIO stub; satisfies gpio_interface_t with no hardware at all.
 */
typedef struct gpio_stub
{
    /** Must be the first member. */
    gpio_interface_t itf;

    /** Simulated pin state (true = high, false = low). */
    bool state;
} gpio_stub_t;

// ----------------------------------------------------------------------------
static inline gpio_stub_t* get_impl(gpio_interface_t* self) { return (gpio_stub_t
    ↪ *)(self); }

// ----------------------------------------------------------------------------
static void gpio_del(gpio_interface_t** self)
{
    if ((NULL == self) || (NULL == *self)) { return; }
    free(*self);
    *self = NULL;
}

// ----------------------------------------------------------------------------
static bool gpio_read(const gpio_interface_t* self)
{
    const gpio_stub_t* impl = (const gpio_stub_t*)(self);
    return (NULL != impl) ? impl->state : false;
}
```

```c
// ----------------------------------------------------------------------------
static void gpio_write(gpio_interface_t* self, const bool value)
{
    gpio_stub_t* impl = get_impl(self);
    if (NULL == impl) { return; }
    impl->state = value;
}

// ----------------------------------------------------------------------------
static void gpio_toggle(gpio_interface_t* self)
{
    gpio_stub_t* impl = get_impl(self);
    if (NULL == impl) { return; }
    impl->state = !impl->state;
}

// ----------------------------------------------------------------------------
static const gpio_vtable_t* gpio_vptr_get_instance(void)
{
    static const gpio_vtable_t vtable = {
        .del = gpio_del, .read = gpio_read, .write = gpio_write, .toggle =
    ↪ gpio_toggle,
    };
    return &vtable;
}

// ----------------------------------------------------------------------------
gpio_interface_t* gpio_stub_new(void)
{
    gpio_stub_t* impl = (gpio_stub_t*)(malloc(sizeof(gpio_stub_t)));
    if (NULL == impl) { return NULL; }

    impl->itf.vptr = gpio_vptr_get_instance();
    impl->state    = false;
    return (gpio_interface_t*)(impl);
}
```

(3 marks: 1 for the struct with `itf` first and `state` standing in for the registers; 1 for the four functions casting `self` back to the concrete type and being `static`; 1 for the vtable and `gpio_stub_new()` setting `vptr` before returning, with the `malloc()` failure handled.)

The rule about field order, and the guarantee behind it. `gpio_interface_t itf;` must be the **first** member. C guarantees that a struct has no padding before its first member, so a pointer to the struct and a pointer to that member hold the same address. That is exactly what

makes `(gpio_interface_t*)(impl)` in `gpio_stub_new()` and the reverse cast in `get_impl()` correct with no pointer adjustment — and it is the same layout a C++ compiler generates for single inheritance, where an object's address *is* its base class's address. Put `itf` anywhere else and the cast silently yields a pointer to the wrong field; `self->vptr` then reads whatever happens to be at offset 0 and the first call through it jumps to a garbage address. (*1 mark*)

Why a single shared vtable is enough. The table holds four function addresses and nothing else — there is nothing instance-specific in it, because all per-instance state lives in `gpio_stub_t`'s own fields, which each function reaches by casting `self` back. One `static const gpio_vtable_t` inside the accessor therefore serves every stub ever created: each instance carries one pointer rather than four copies of the table. (`const` does not put the table in flash on the ATmega328P: avr-gcc copies read-only data into RAM at startup unless it is marked `PROGMEM`, so the single shared table costs its 8 bytes of RAM once, not once per instance.) (*0.5 marks*)

What the stub makes possible. Everything written against `gpio_interface_t*` can now be built and run with no chip attached: unit tests on a host, logic driven by simulated inputs that would otherwise need a soldering iron, and a third implementation — one that logs every `write()` and `toggle()` before updating its own `state` — dropped in without touching a line of the code under test. `main()`'s edge-detection logic cannot tell which implementation it was handed, which is the whole point of programming against the interface rather than against the driver. (*0.5 marks*)